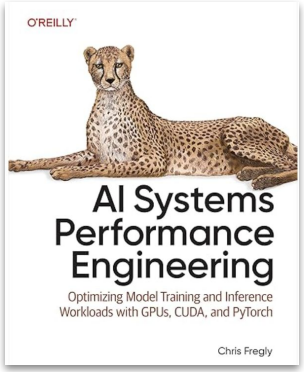
NVIDIA GTC 2026 Recap

by [Chris Fregly]



GTC Conference 2026 - Attendee and Session Breakdown

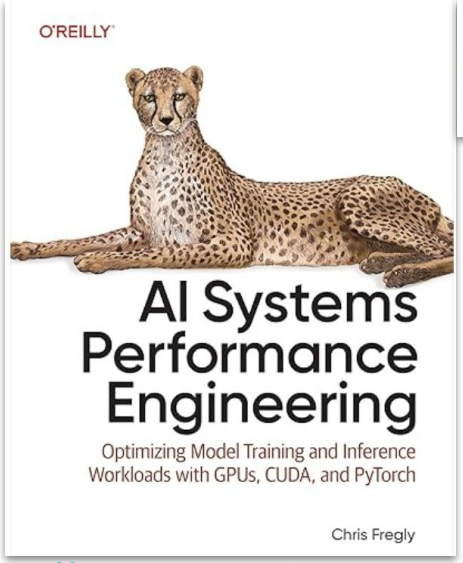
1,000 sessions, 2,000 speakers, 450 sponsors

42,000 in-person, 50,000 live-stream, 100+ robots

150 updated/new CUDA-X libraries (cuDNN, cuBLAS, cuTile, etc)

#1 O’Reilly Book - AI Systems Perf - Updated NVIDIA GTC 2026!!

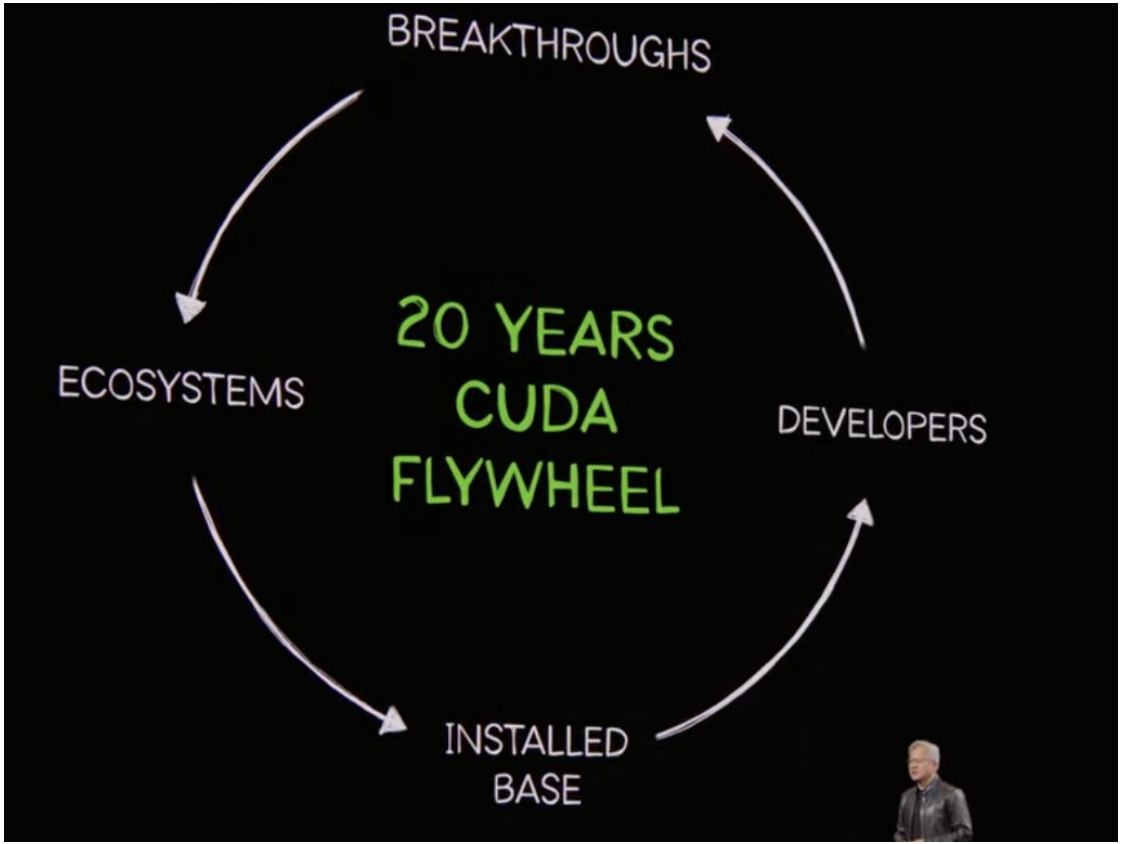
https://www.amazon.com/Systems-Performance-Engineering-Optimizing-Algorithms/dp/B0F47689K8/



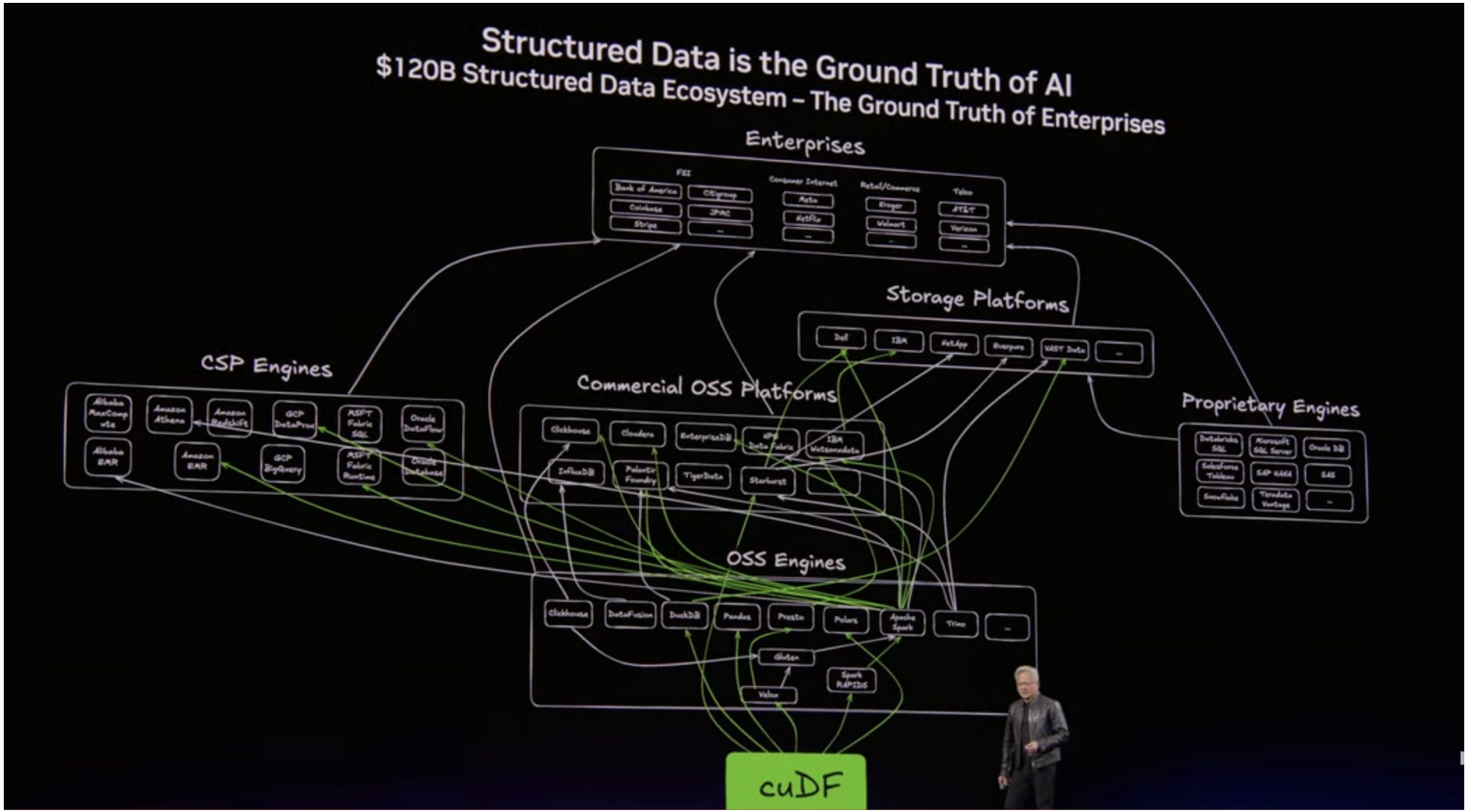
https://www.meetup.com/ai-performance-engineering/

https://github.com/cfregly/ai-performance-engineering/tree/main/code

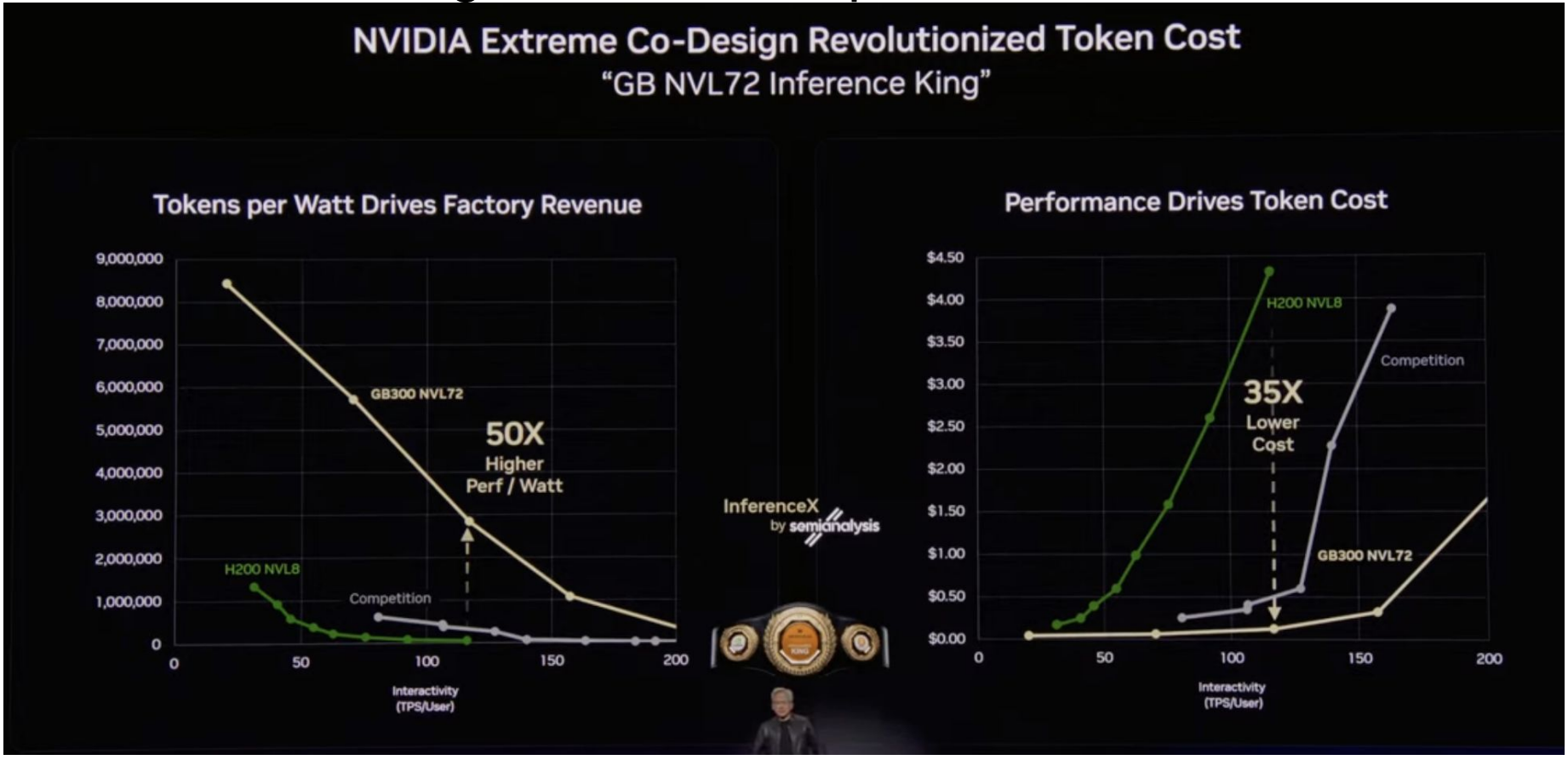
20 Year CUDA flywheel - massive install base



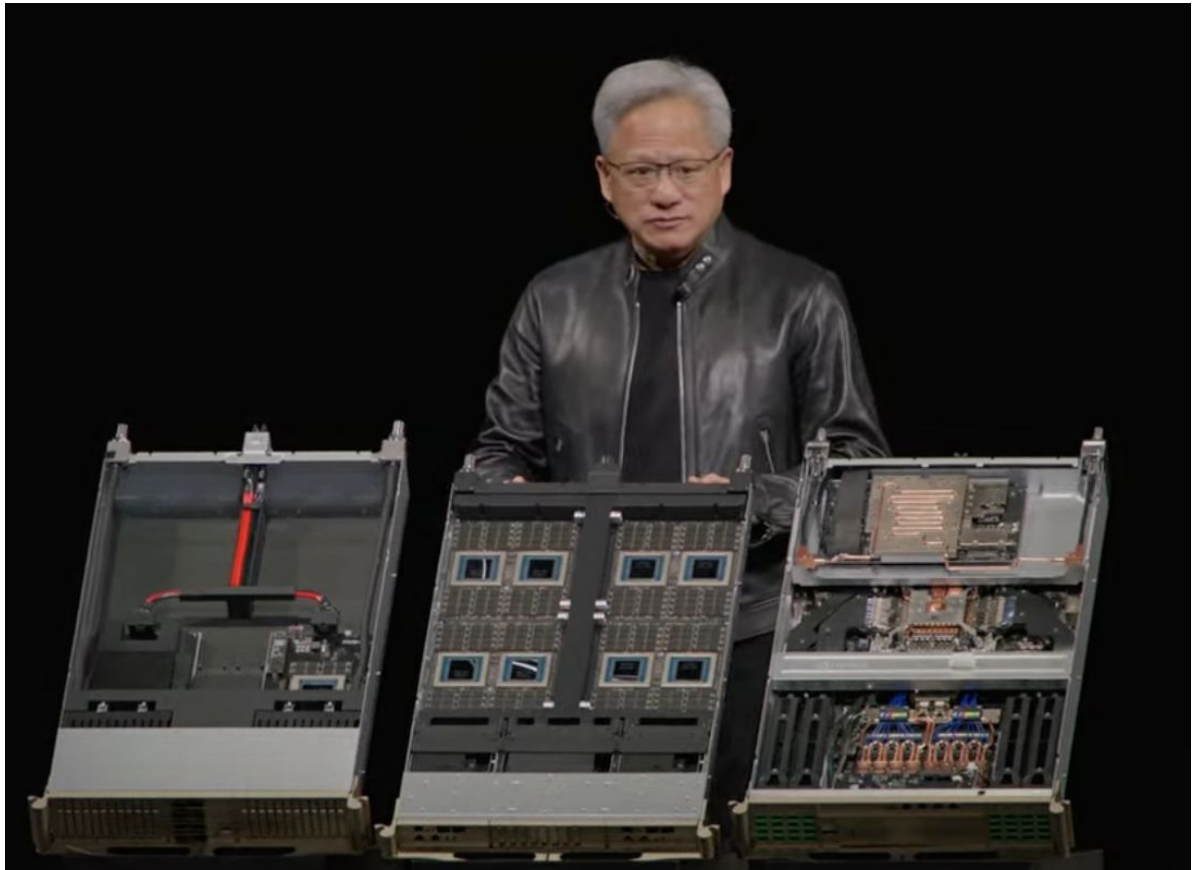
Structured data - ground truth of AI (Jensen's fave slide)



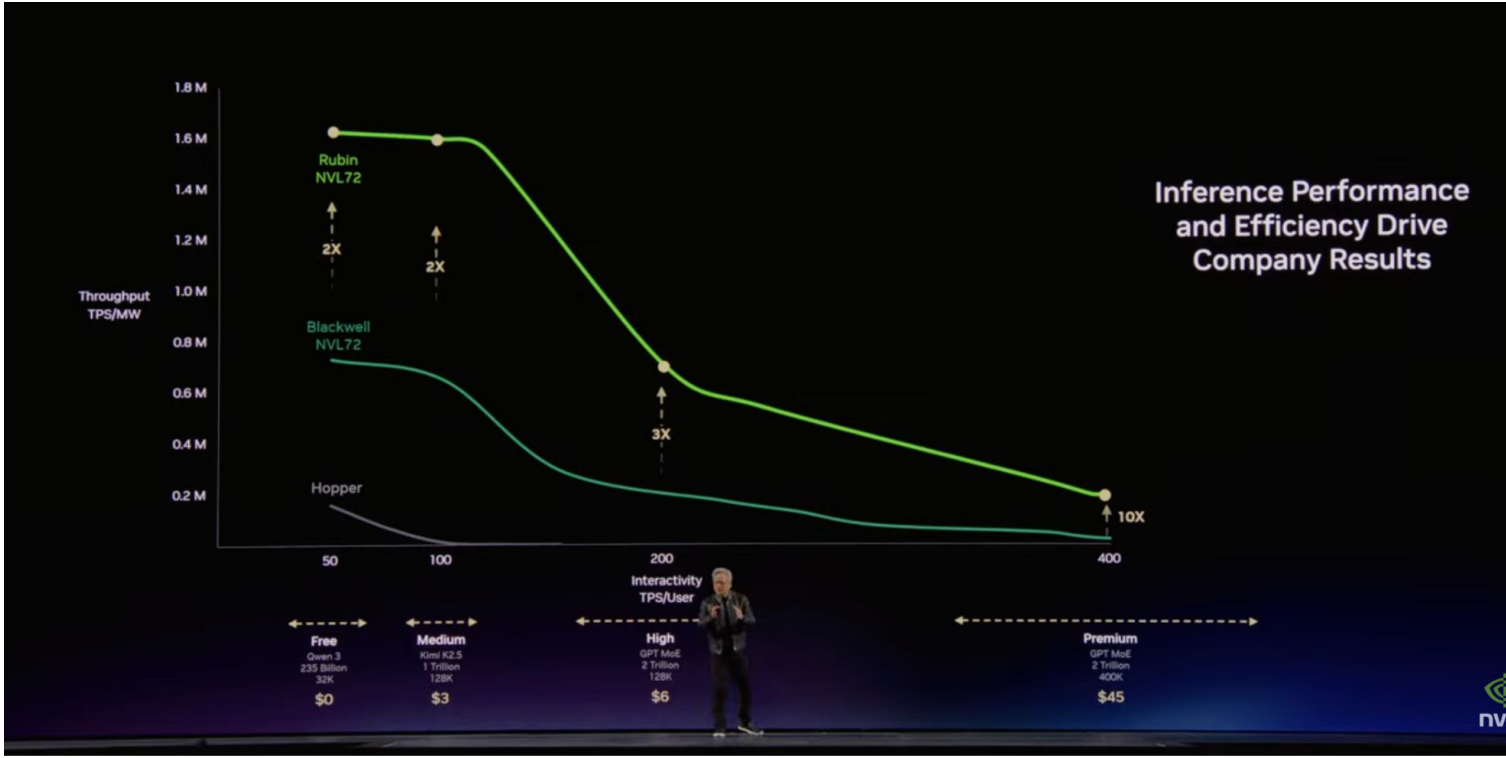
Extreme co-design is critical for performance and cost



Groq, Co-Packaged Optics (CPO), Vera-Rubin in production

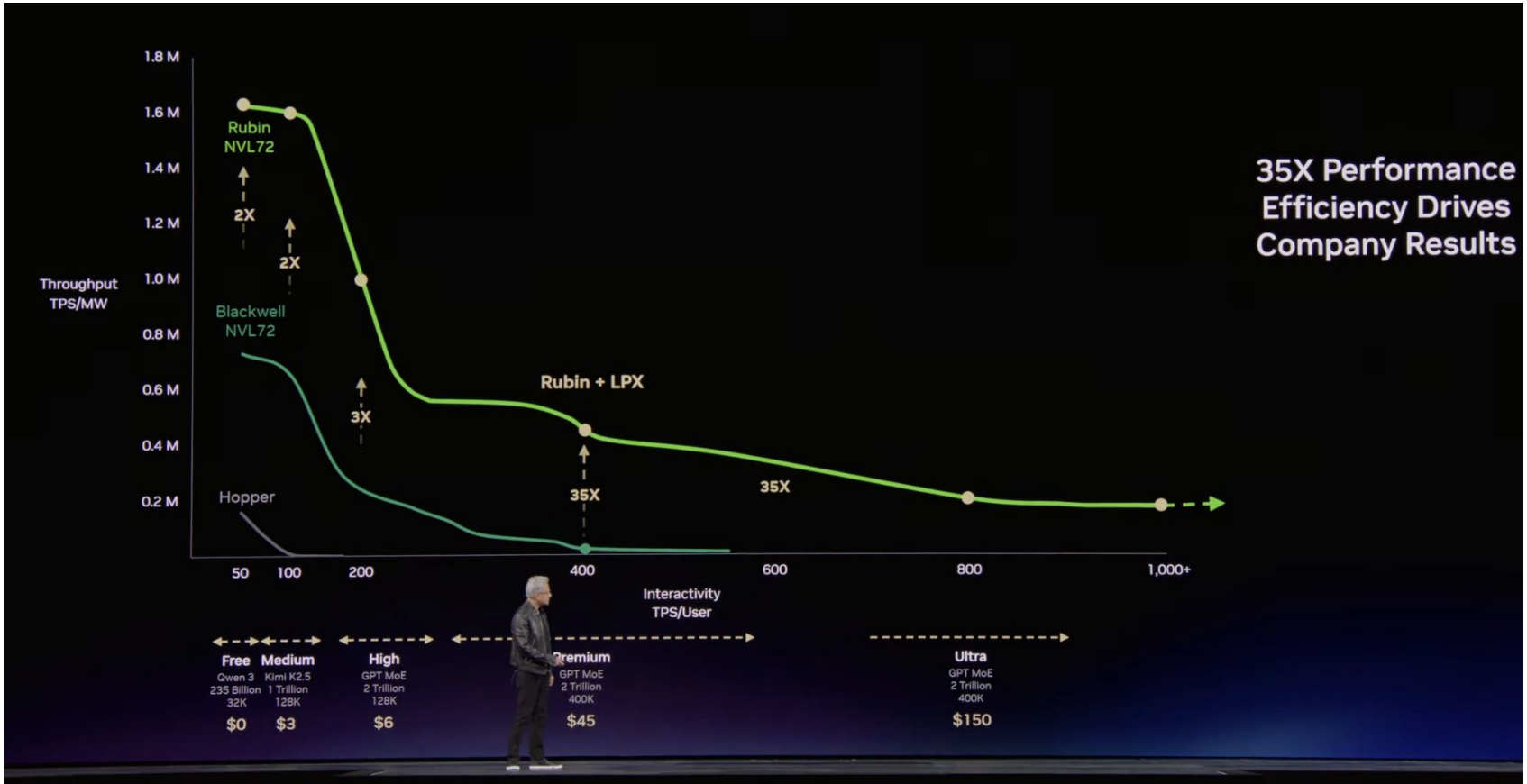


NVL72 Rubin vs. Blackwell Performance Improvement

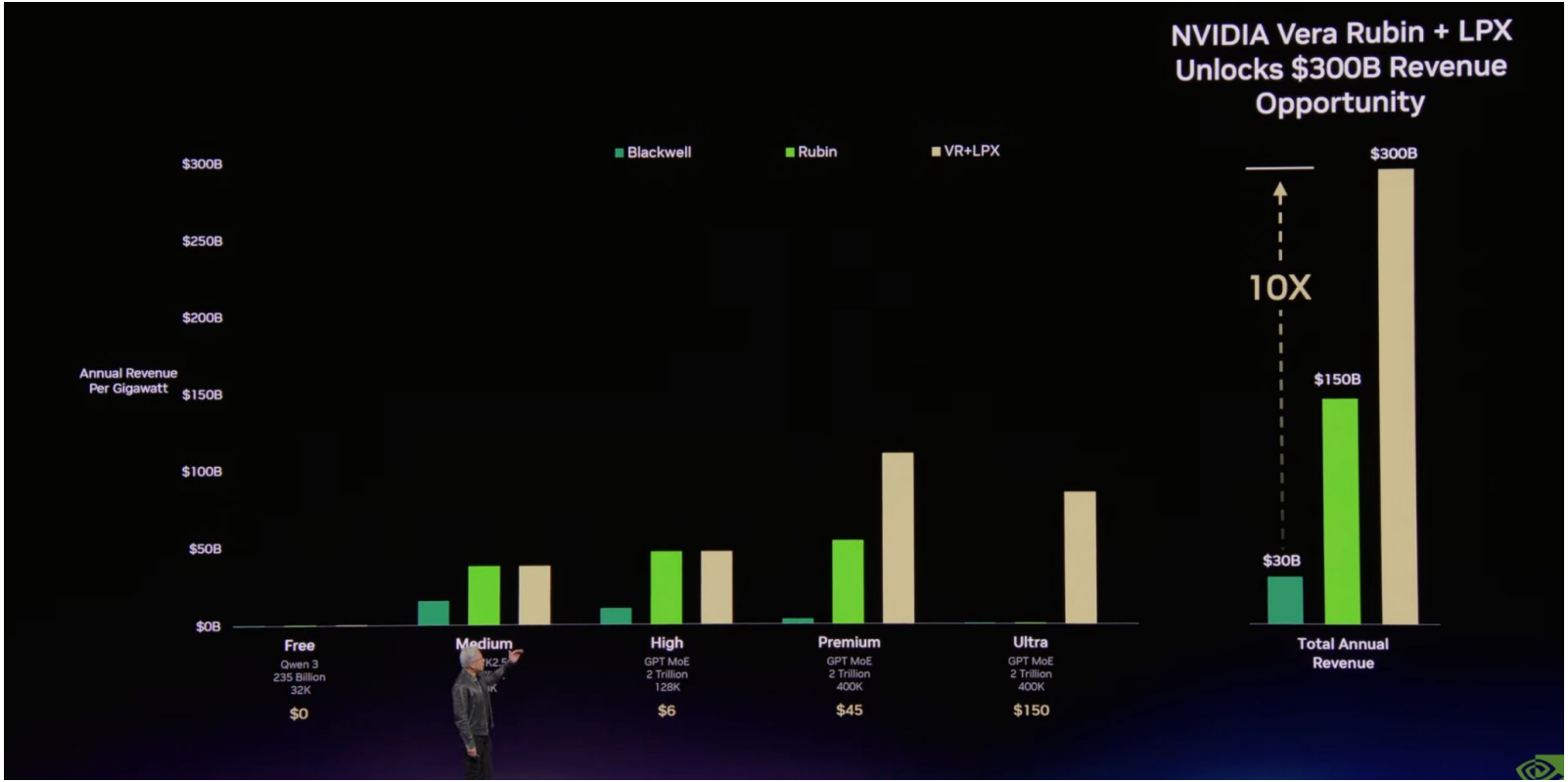


Vera-Rubin vs. Grace-Blackwell revenue opportunity ($150B)

Vera-Rubin NVL72 + Groq LPX performance improvement

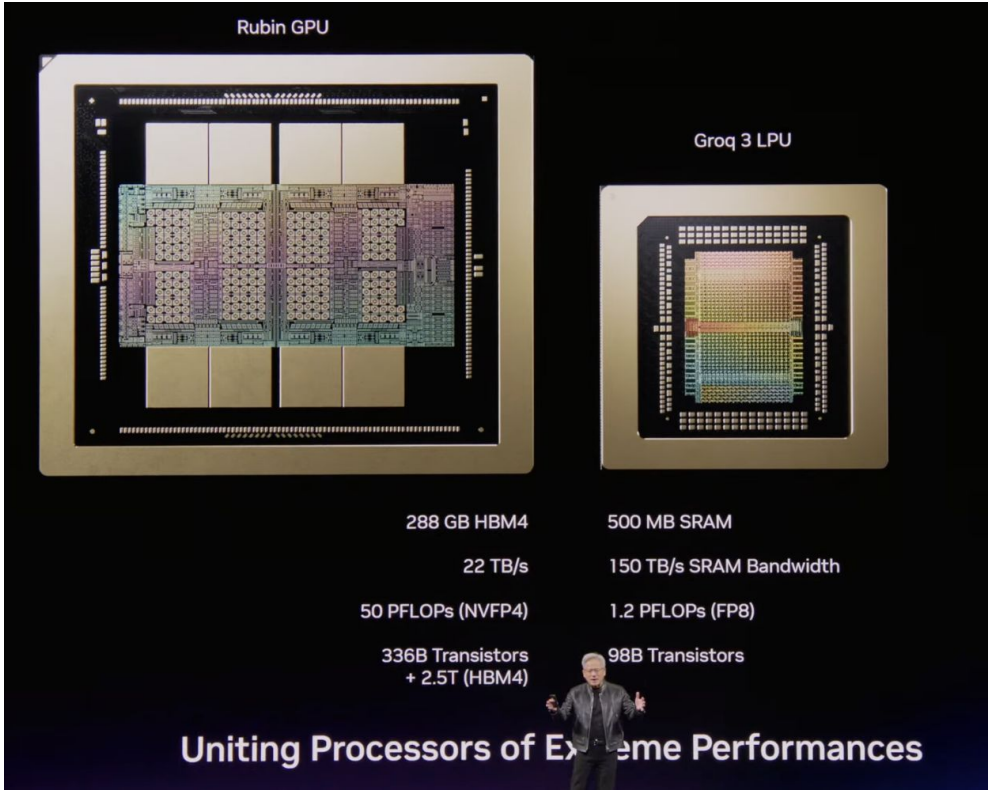


Vera-Rubin NVL72 + Groq revenue opportunity ($300B)



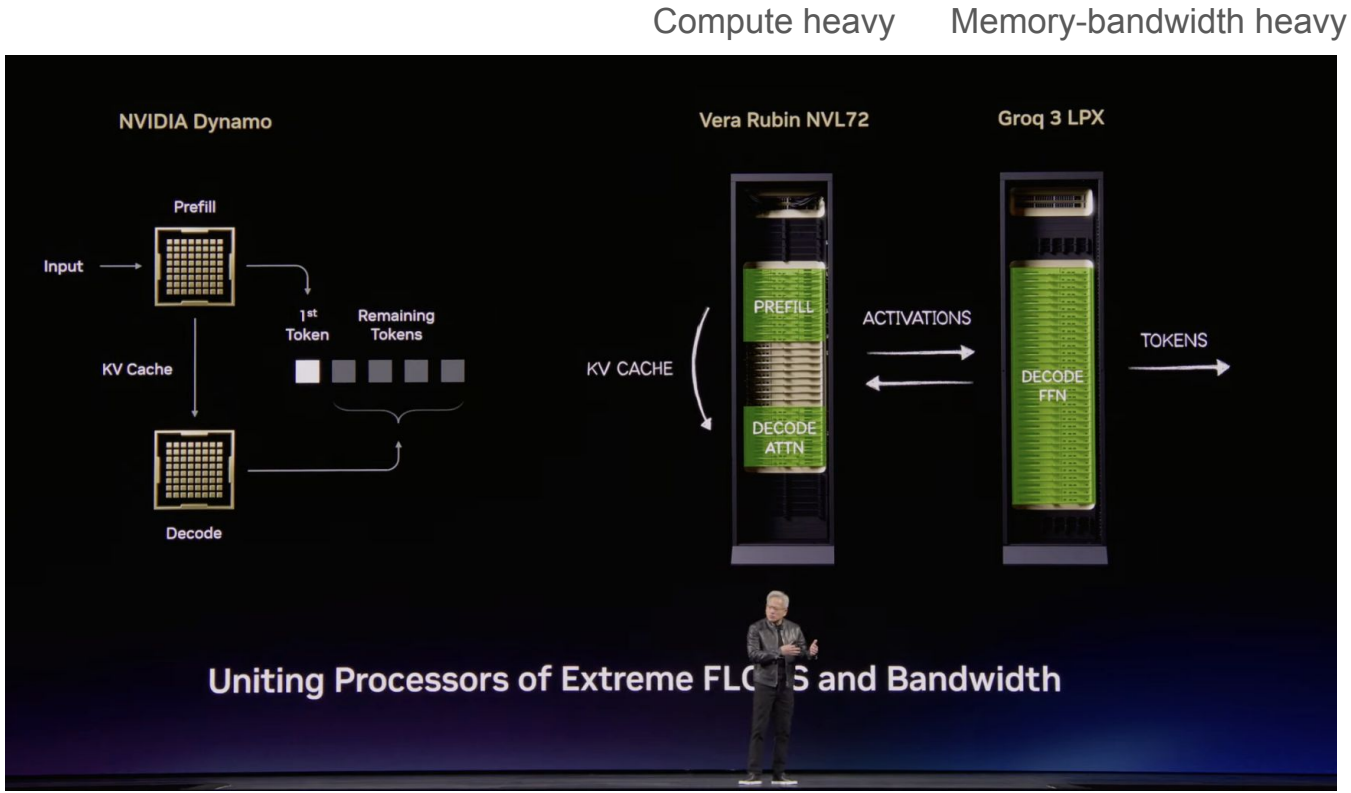
Rubin vs. Groq memory size + bandwidth differences

High throughput



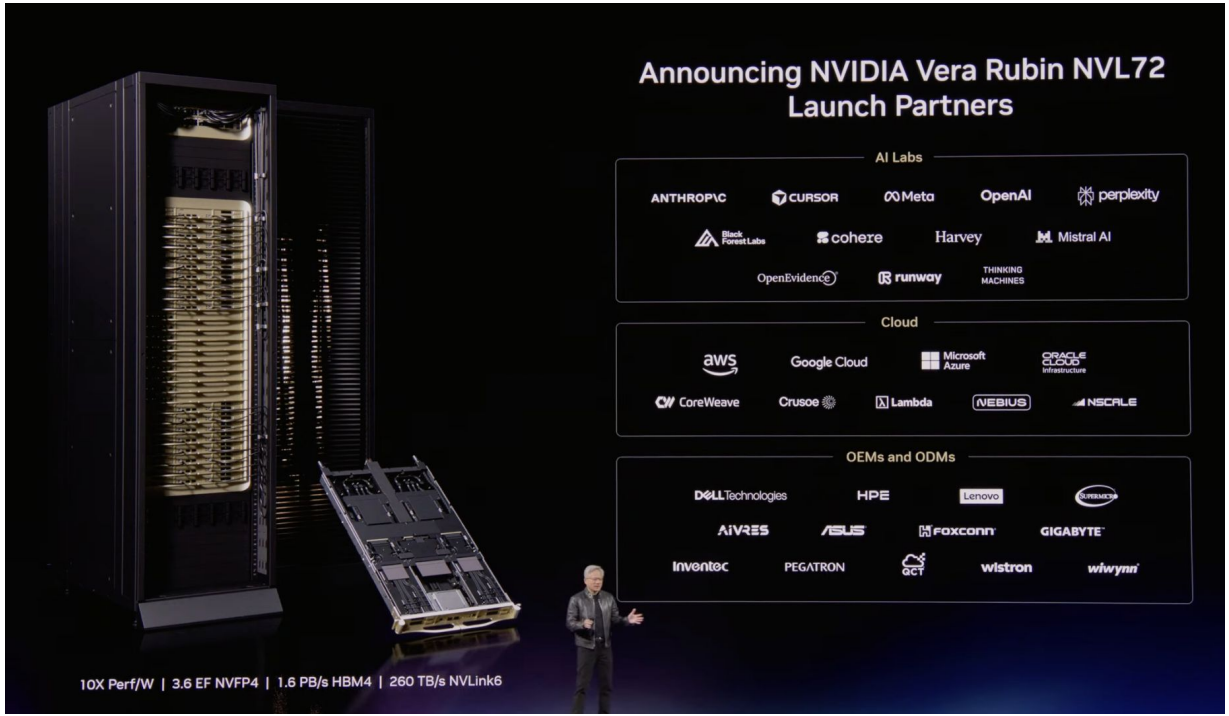
Low latency (55X memory bandwidth)

Disaggregated Prefill and Decode with Rubin and Groq



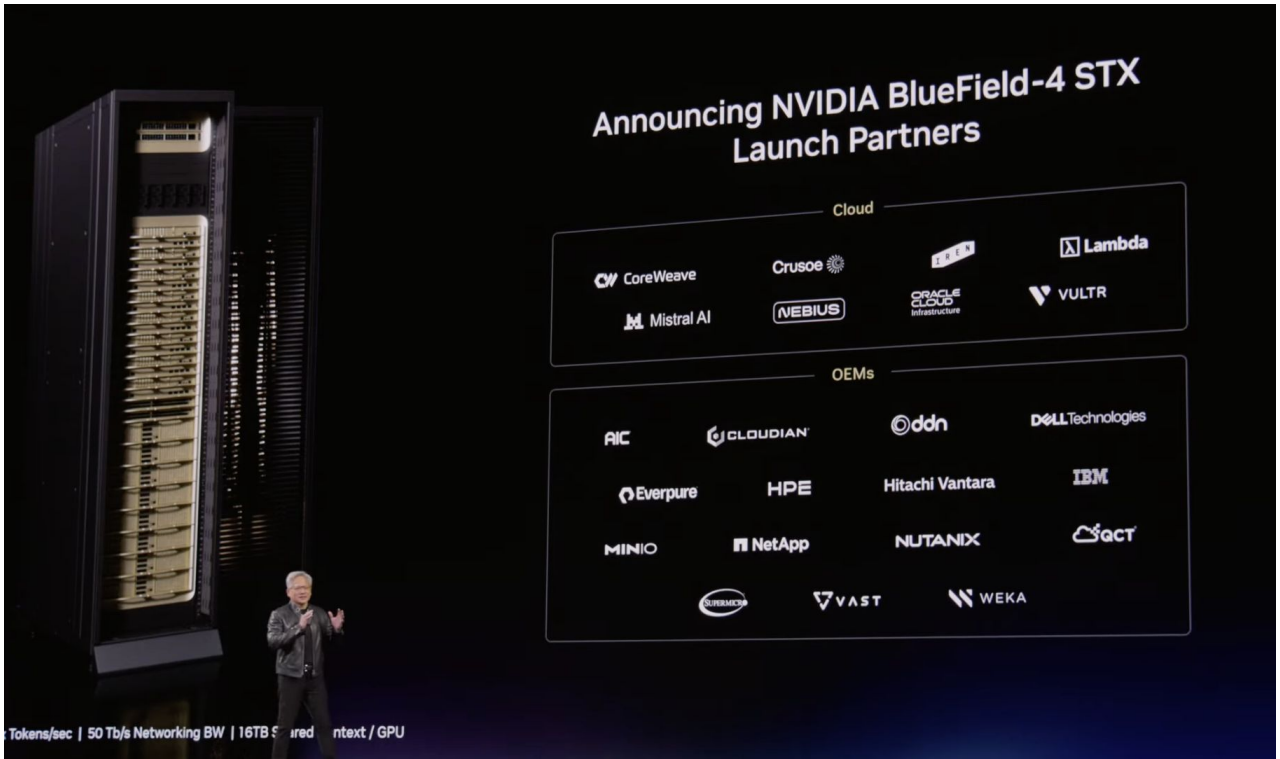
Vera-Rubin NVL72 Racks - Partners and Customers

1,000’s per week are being produced, Azure is 1st cloud to deploy VR NVL72



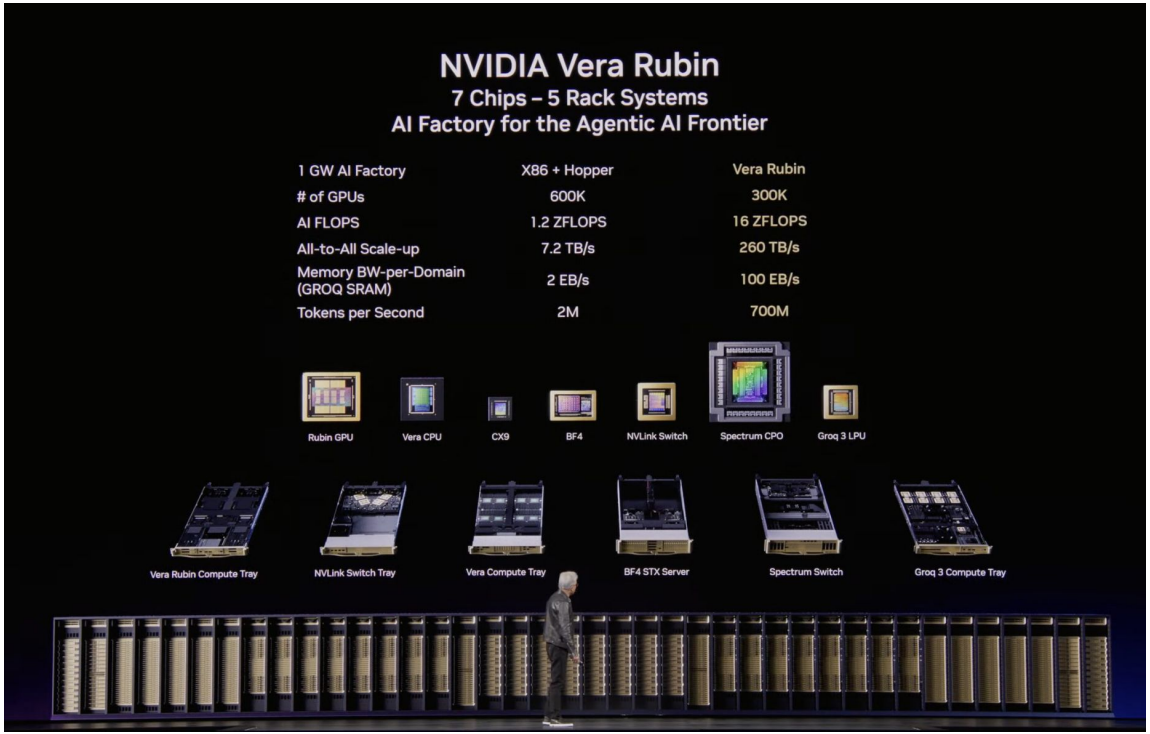
Storage Systems - Partners and Customers

KV cache offload, training/RL data, etc.

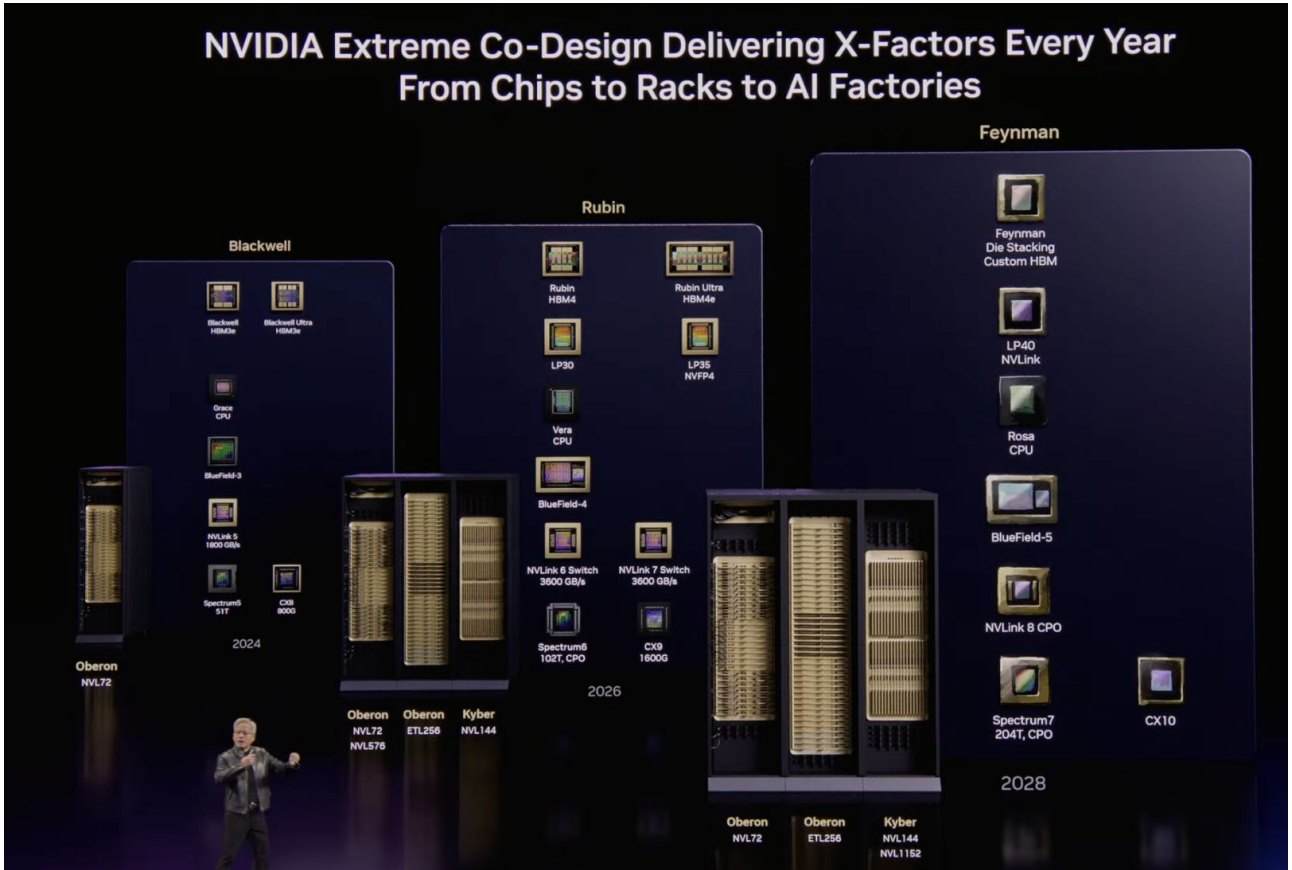


Vera-Rubin system - 7 chips, 5 racks - extreme co-design

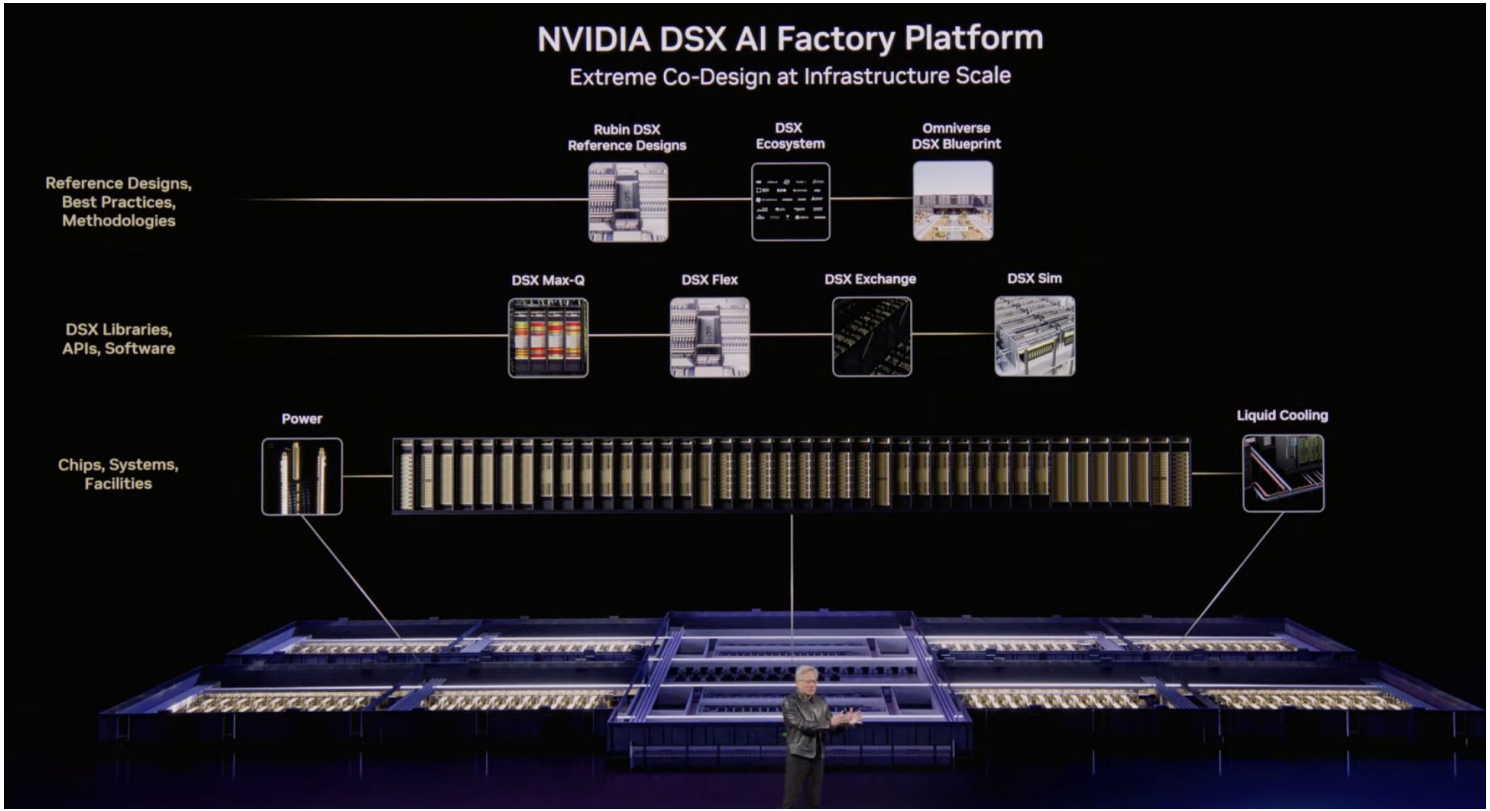
Breaking Moore's Law



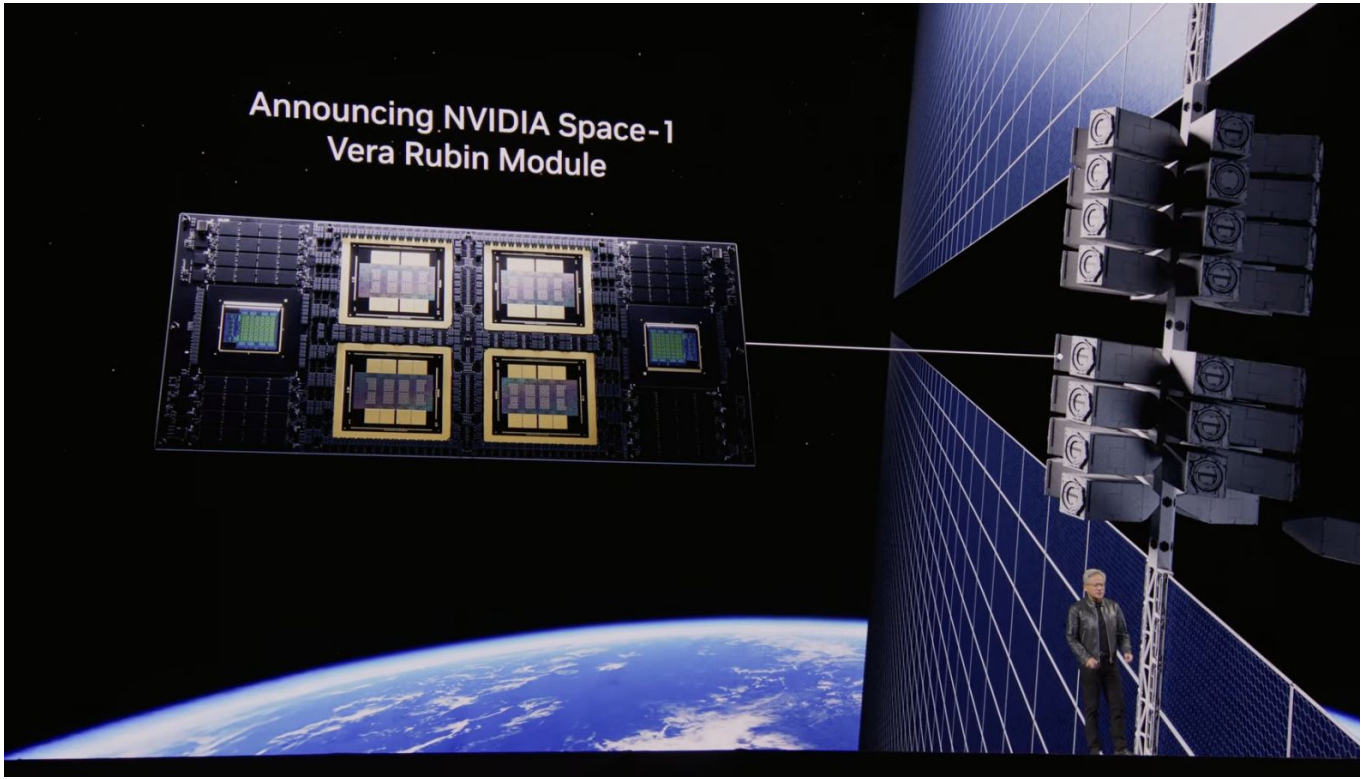
Looking ahead - Feynman (2028) - both copper and optical



NVIDIA DSX - AI Factory Platform - Design gigawatt systems

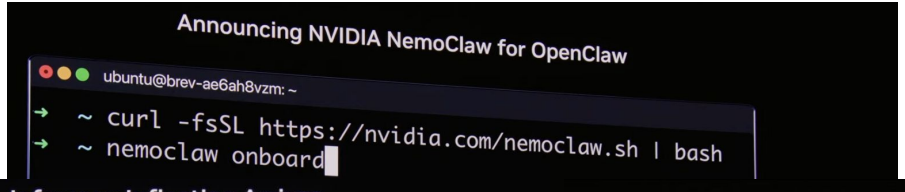


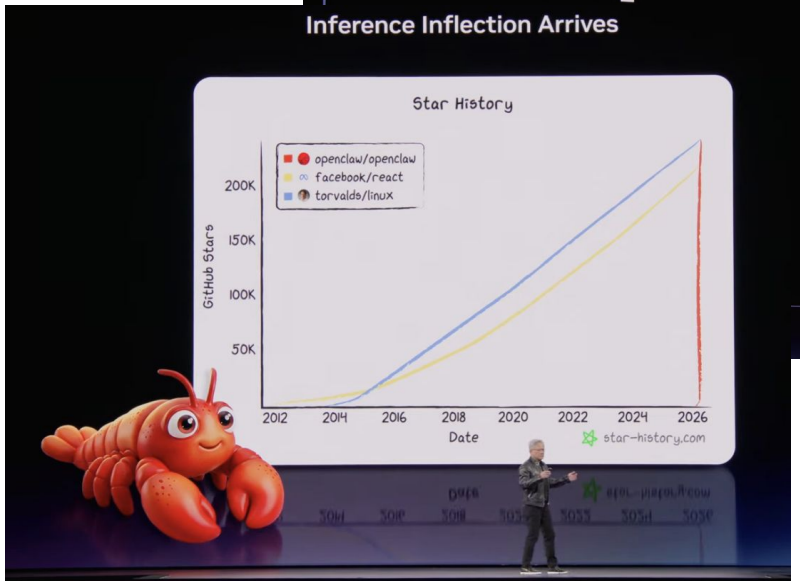
Vera Rubin Space-1 - cleared for takeoff

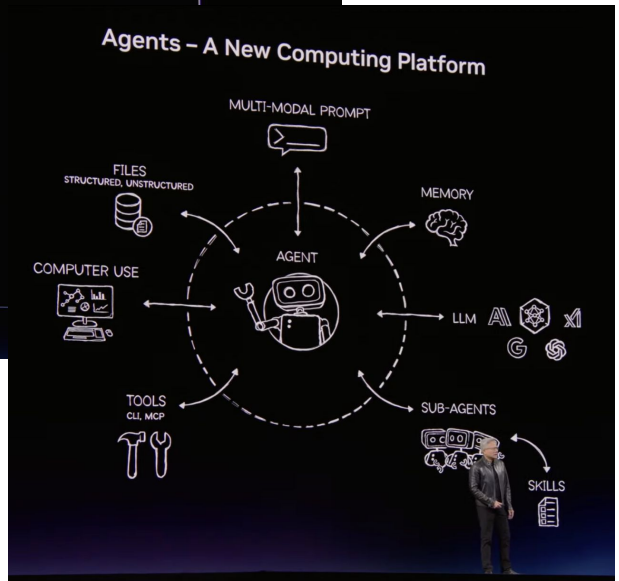


NemoClaw: “Every company needs an OpenClaw strategy”

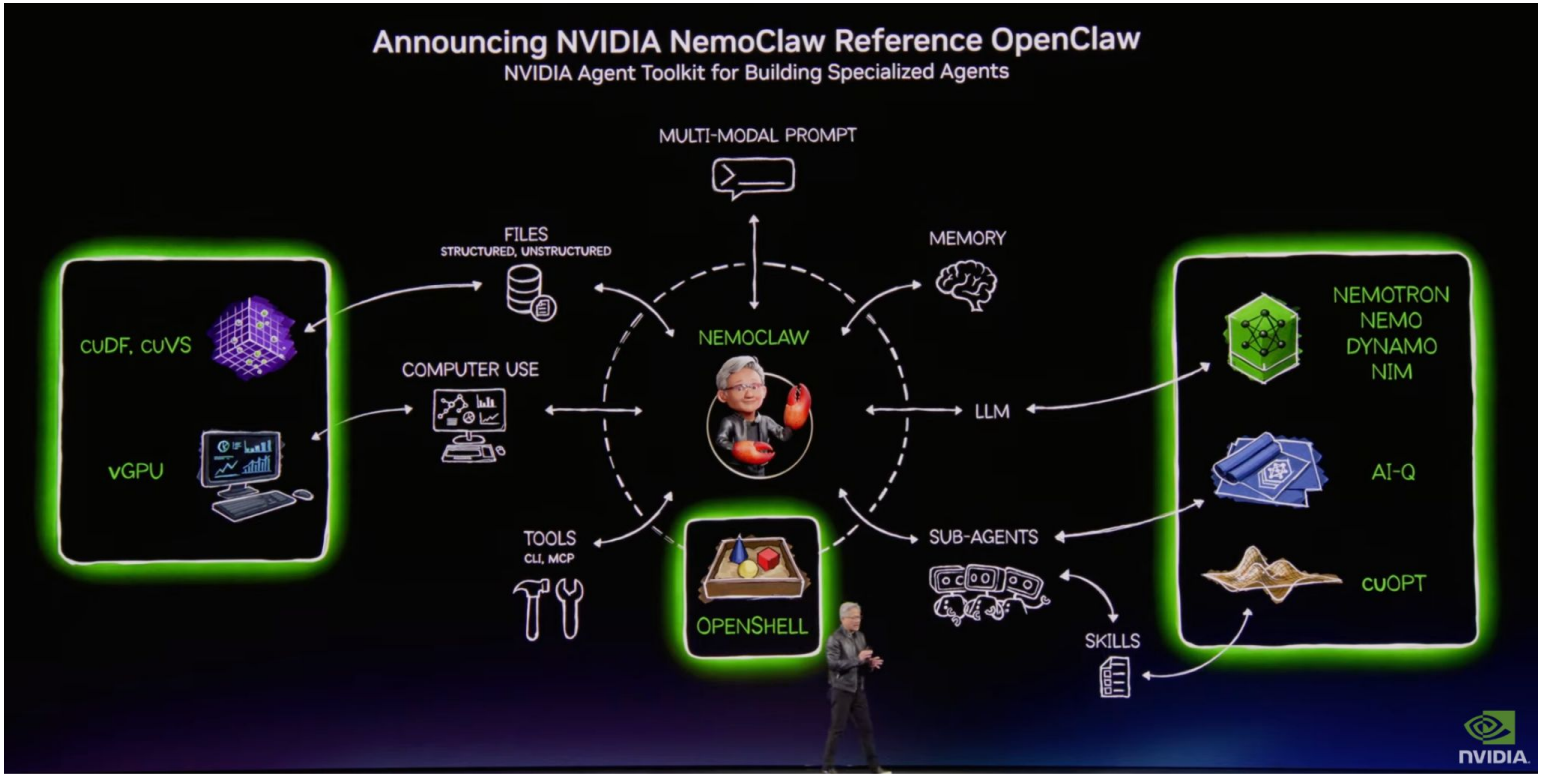
“OpenClaw is as big of a deal as HTML and Linux” --Jensen Huang, NVIDIA CEO



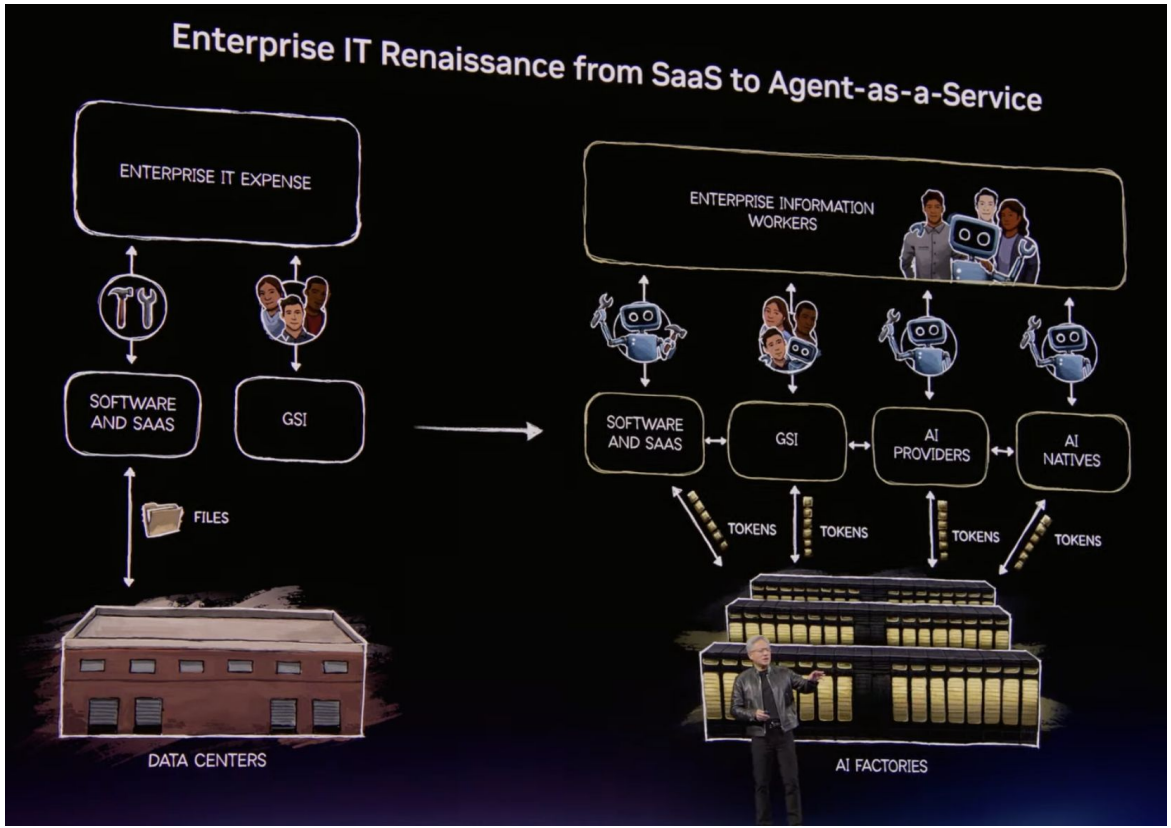




OpenClaw + OpenShell + NemoClaw = Enterprise-ready

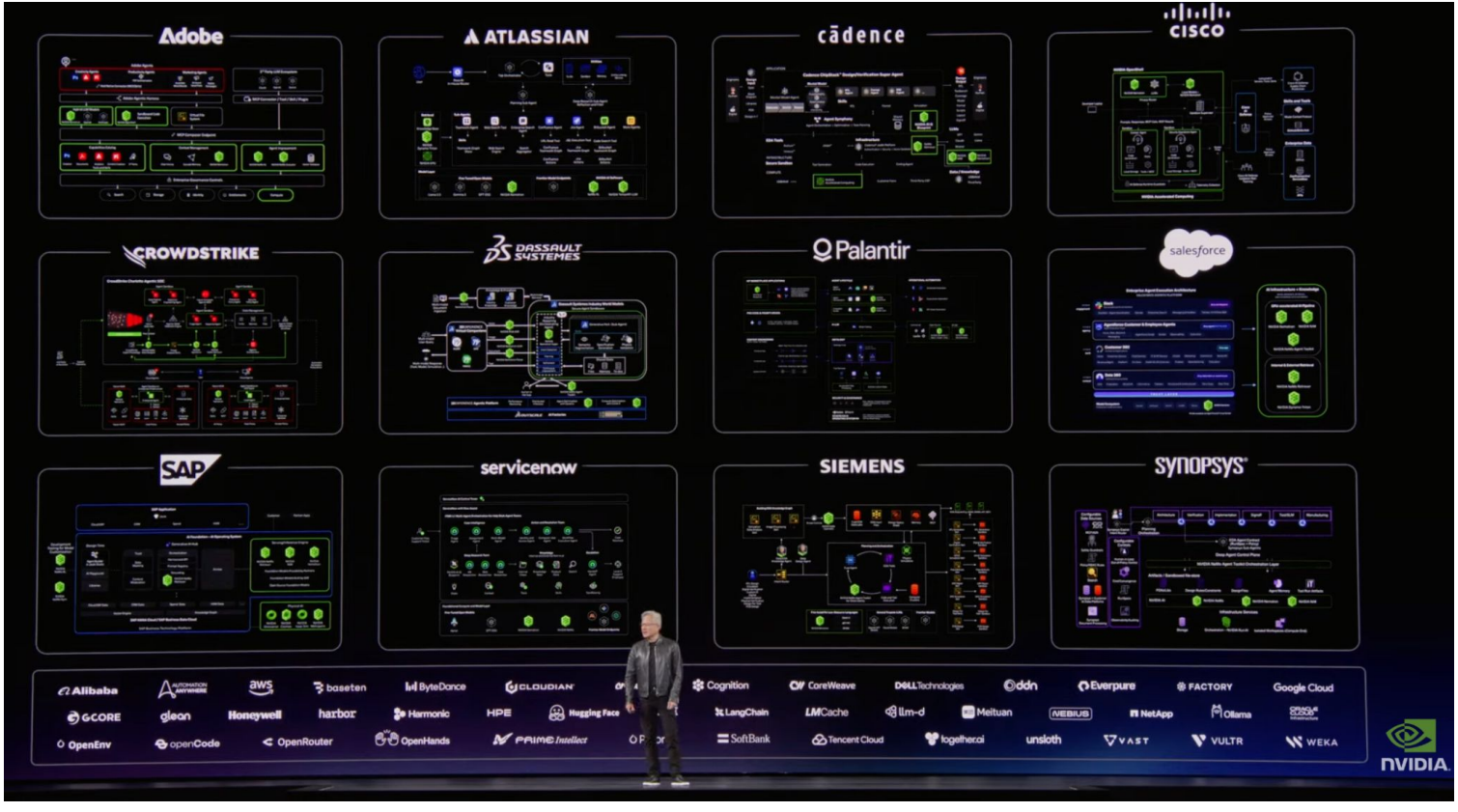


Software-as-a-Service (SaaS) => Agents-as-a-Service (AaaS)

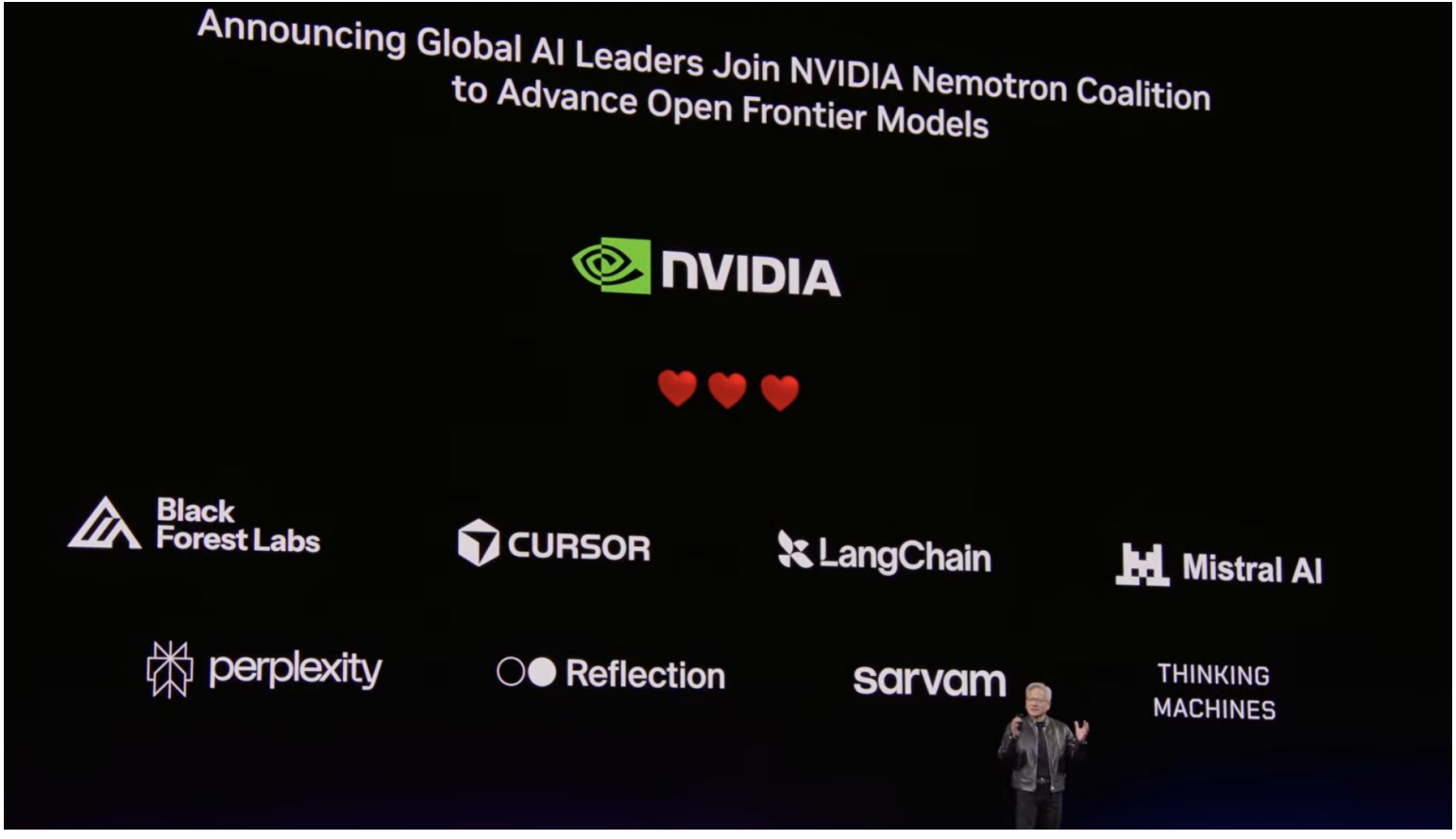


Nemo is #1 on many benchmarks including PinchBench

Implementing Nemoclaw reference architecture

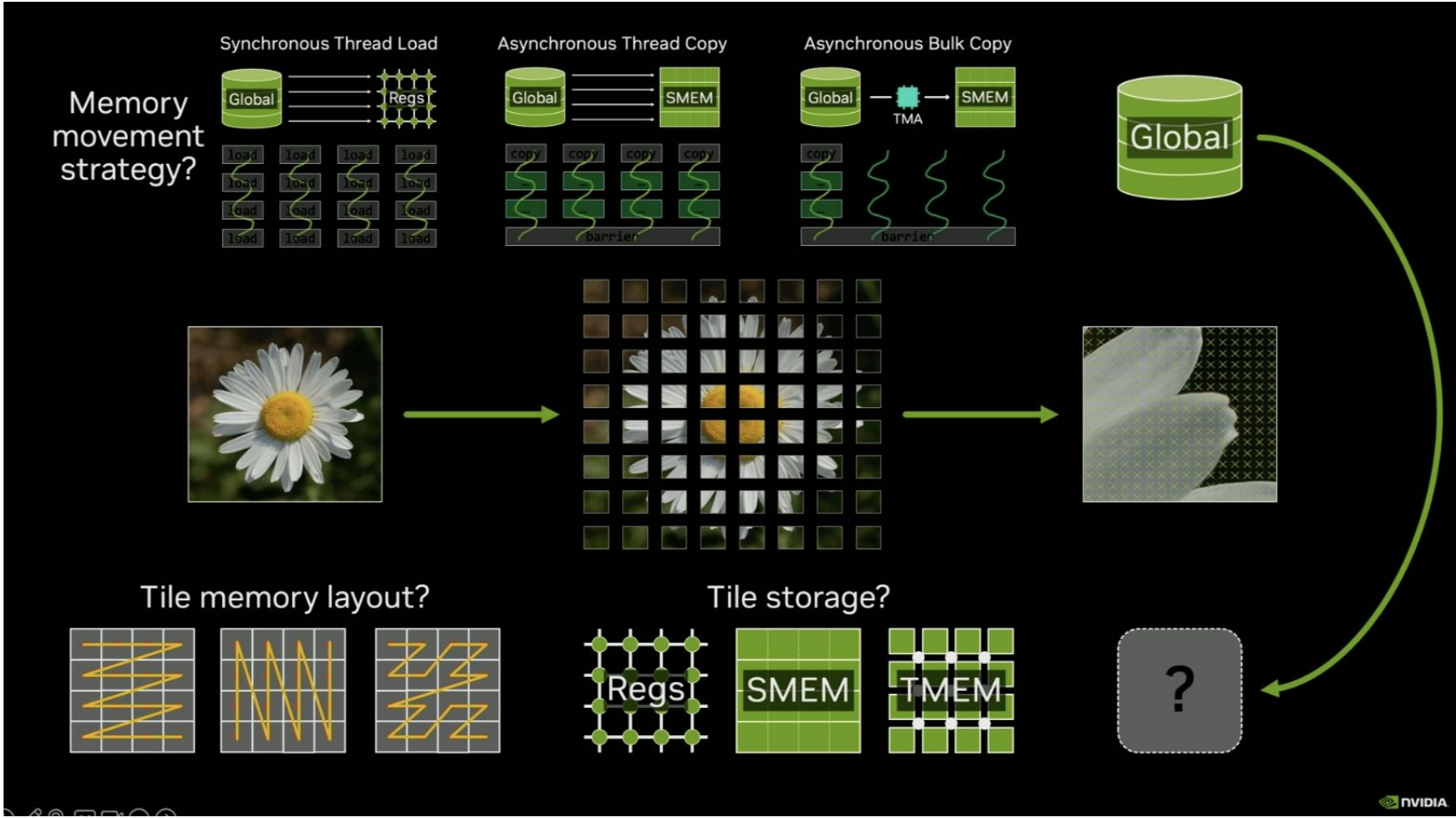


Formed “Nemotron Coalition” to advance open models

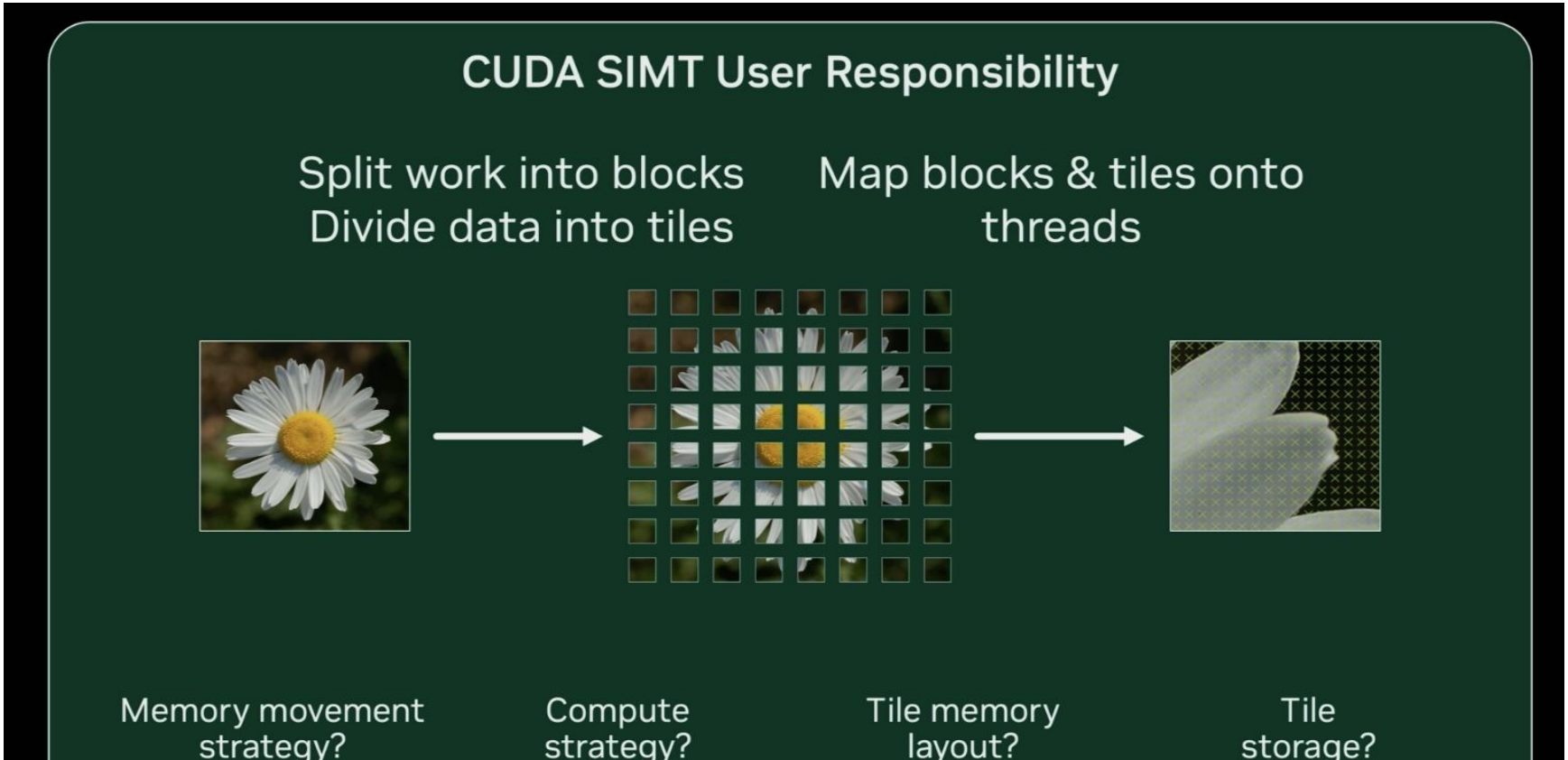


Closing montage + new country song?!

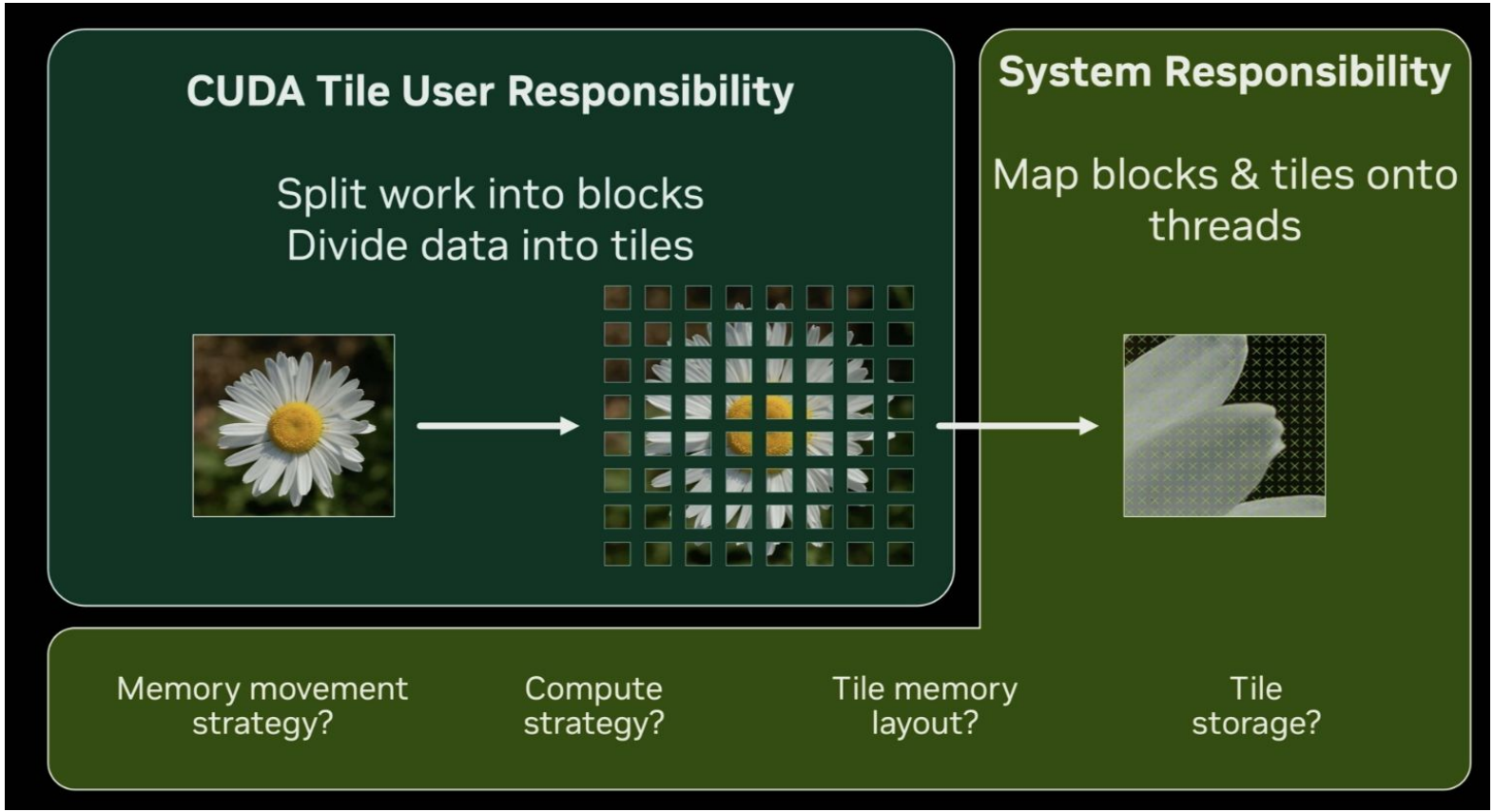
EXTRAS


Memory movement strategy?
Synchronous Thread Load
Global
Regs
load
Asynchronous Thread Copy
Global
SMEM
copy
barrier
Asynchronous Bulk Copy
Global
TMA
SMEM
copy
barrier
Global
Tile memory layout?
Tile storage?
Regs
SMEM
TMEM
?
NVIDIA

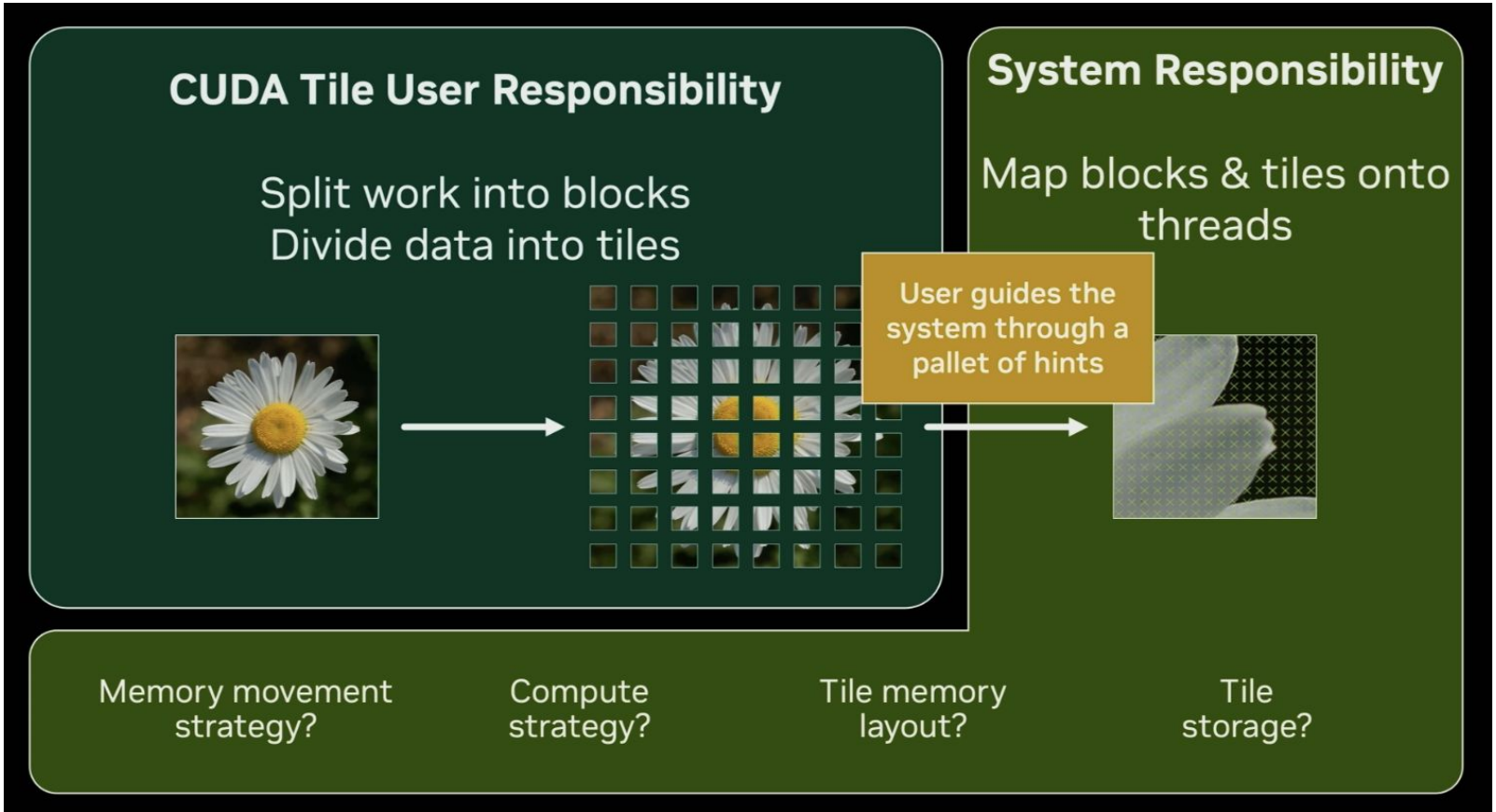
Lots of responsibility for CUDA developers right now



CUDA Tile defers a lot of that responsibility to the compiler



We should provide hints to the compiler



cuTile Python: NumPy-Like GPU Kernels

A complete softmax kernel — no TMA descriptors, no warp management

```python
import cuda.tile as ct

@ct.kernel
def softmax(I, O, r):
    c   = ct.load(I, (0, ct.bid(0)), (I.shape[0], r))
    mx  = ct.max(c, axis=0, keepdims=True)
    num = ct.exp(c - mx)
    den = ct.sum(num, axis=0, keepdims=True)
    smax = num / den
    ct.store(O, (0, ct.bid(0)), smax)
```

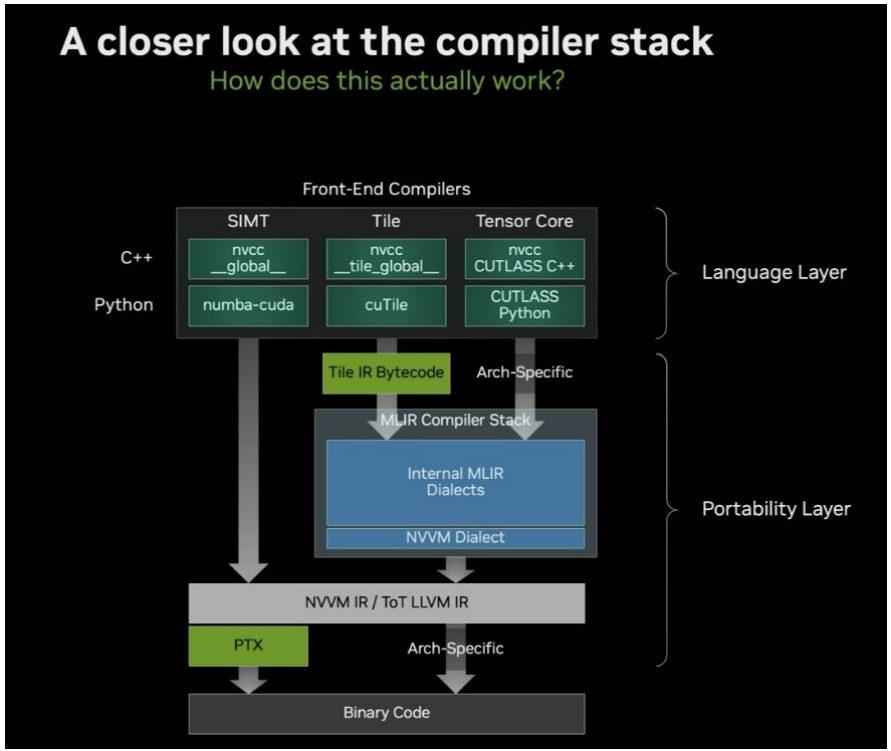
cuTile GEMM in 10 Lines

The same load → mma → store pattern we will use for MoE

```python
@ct.kernel
def matmul(A, B, C, ts: ct.Constant[tuple[int]]):
    x_bid, y_bid = ct.bid(0), ct.bid(1)
    acc = ct.zeros(shape=ts, dtype=A.dtype)

    for k in range(ct.num_tiles(A, index=1, shape=(ts[0], ts[2]))):
        a = ct.load(A, index=(x_bid, k), shape=(ts[0], ts[2]))
        b = ct.load(B, index=(k, y_bid), shape=(ts[2], ts[1]))
        acc = ct.mma(a, b, acc)

    ct.store(C, index=(x_bid, y_bid), tile=acc)
```

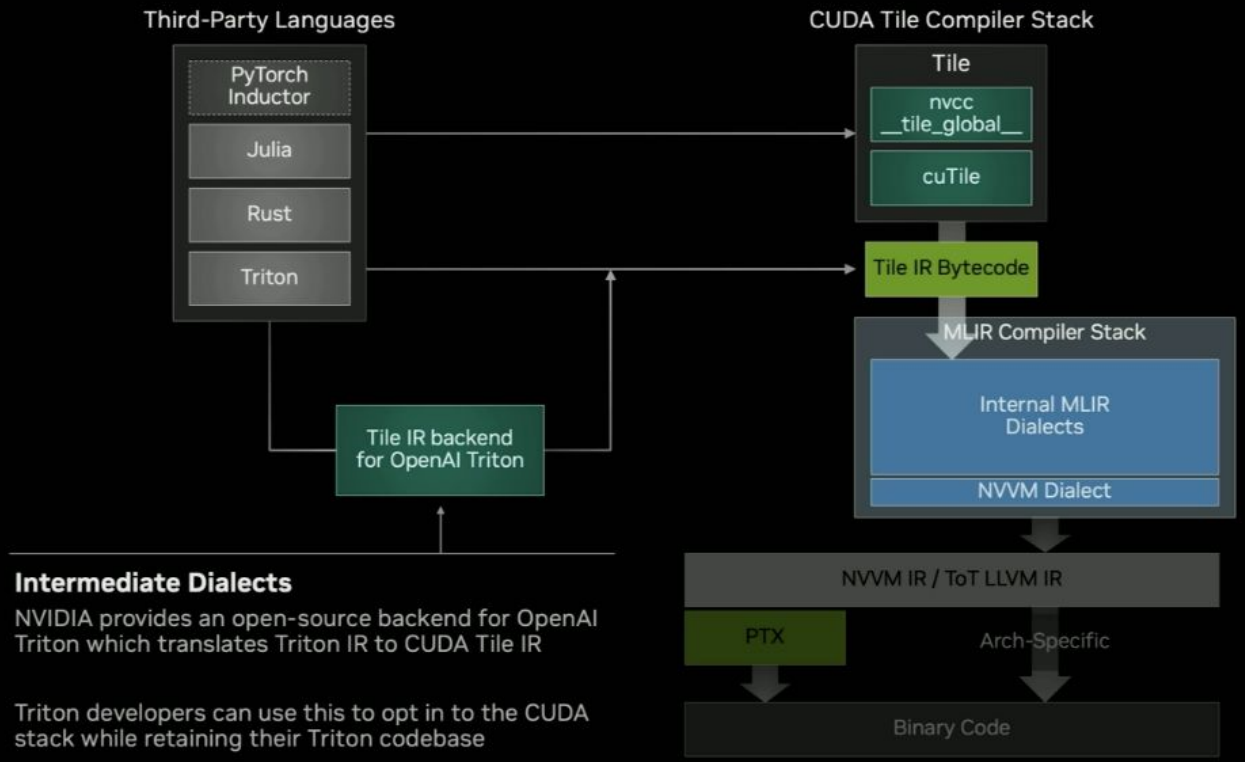
A closer look at the compiler stack

How does this actually work?



A Stack Built For Compilers, Humans and Agents

An example: the CUDA Tile portion of the stack is targeted by multiple languages at different levels
All of them are tuned equivalently



Intermediate Dialects

NVIDIA provides an open-source backend for OpenAI Triton which translates Triton IR to CUDA Tile IR

Triton developers can use this to opt in to the CUDA stack while retaining their Triton codebase

Key cuTile APIs for This Talk

The primitives that make MoE possible in clean Python

- **ct.load / ct.store** — Contiguous tile transfers (uses TMA on Blackwell)
- **ct.gather / ct.scatter** — Irregular access from non-contiguous addresses
- **ct.mma** — Tensor core matrix multiply-accumulate
- **ct.Constant[int]** — Compile-time constants (tile sizes, flags)

Optimizations and Hints

What Impacts my Kernel Performance?

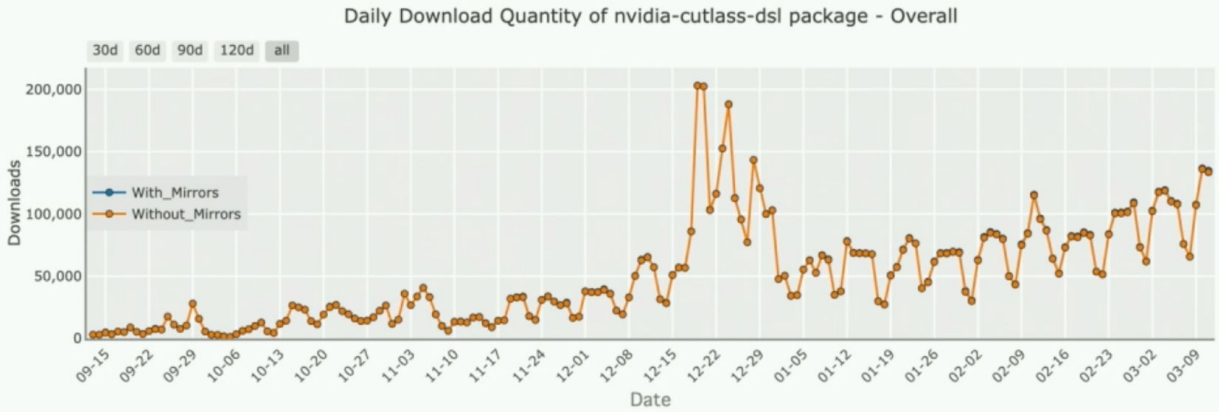
- **The algorithm – As in any kernel, the algorithm itself is the biggest determinant of performance**
  - **How efficiently am I accessing memory?**
  - **How efficiently am I processing my loaded data?**
- **Tile Size – How much data is an individual block processing?**

- **Hints to the compiler:**
  - **latency=[1, 10]** — Hint for software pipeline depth
  - **autotune_launch** — Benchmark multiple configurations automatically
  - **num_ctas=[2^n: 1, 16]** – Number of CTAs (blocks) in a CGA (cluster) (default auto)
  - **occupancy=[1, 32]** – Expected/desired number of active CTAs per SM (default auto)
  - **opt_level=[0, 3]** – Compiler effort level (default 3)

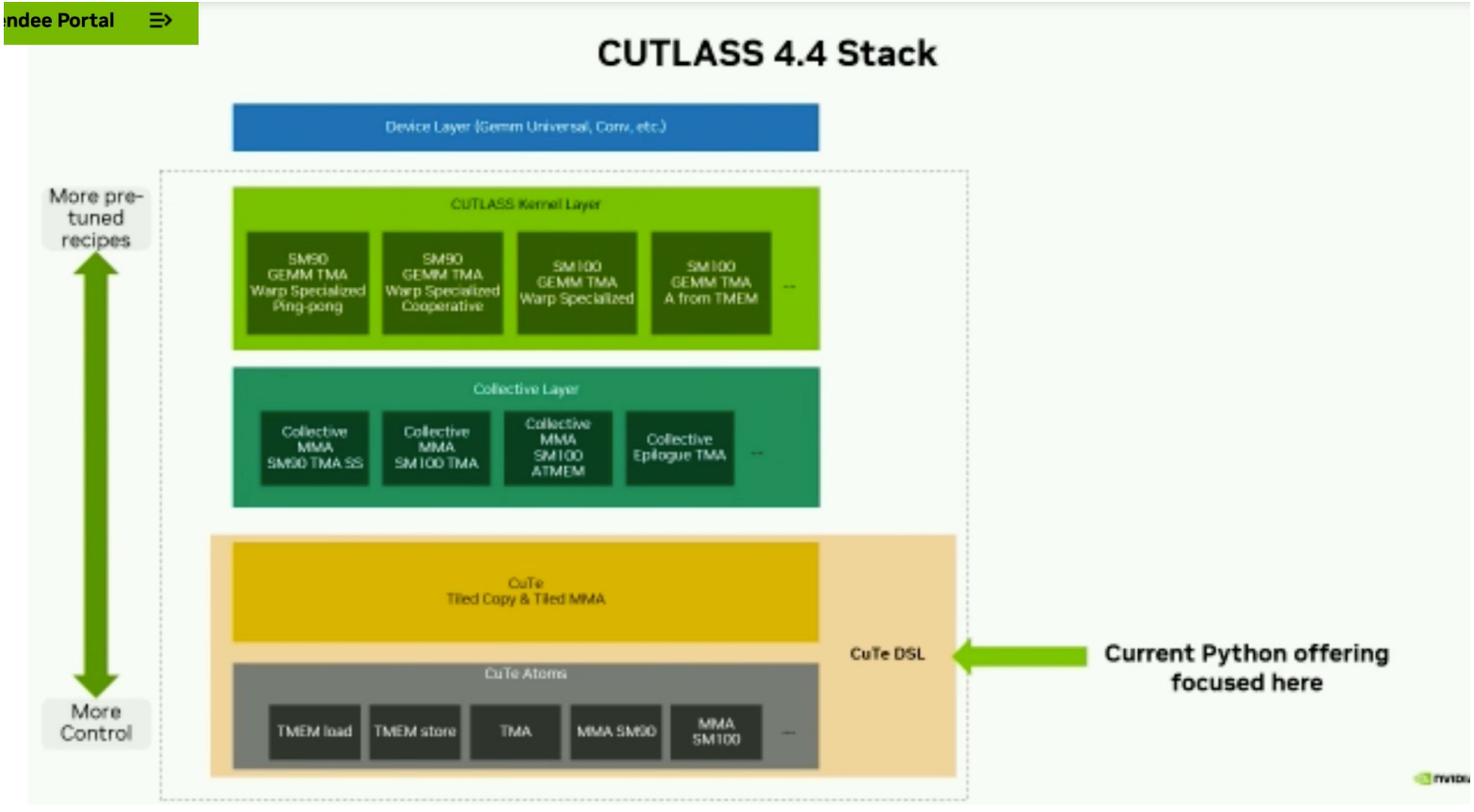
CUTLASS + Python: Low-level control, high-level use

CUTLASS Python Adoption Story

- 4.5 coming soon, will support fine grain compilation control, python linter support w/ mypy, and better source code correlation for profiling/debugging
- Peaked at **200K** downloads in a single day, and **~8.6M** downloads in total
- Used in FAv4, QuACK, vLLM, SGLang, and Sonic-MoE
- More SOL kernels (attention, MoE) are integrated into TRTLLM and FlashInfer
- Enable developers to be more efficient in authoring optimized kernels on GPU



CUTLASS Stack



CUTLASS Tutorials

Attendee Portal

Why Do We Need Tutorial GEMMs?

CUTLASS Python provides plenty of examples!

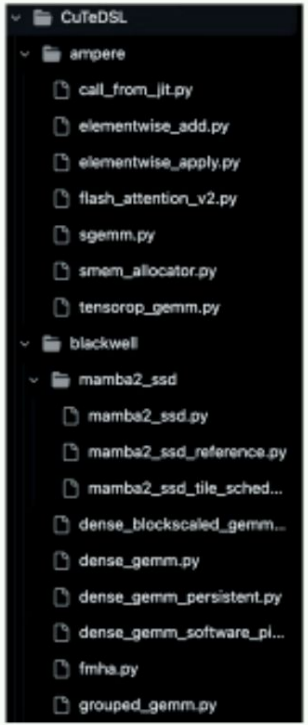
- Each of them combines multiple optimizations for peak performance

Beginners may struggle to discover the key path that leads to SOL ☹

Finding the most relevant kernel requires understanding...

- What are the key primitives for a given HW feature?
- How and when to use them to address performance issues?

Where do you start?



GEMM Algos

GEMM Algorithm

Characteristics

Parallelizable

- Each $C_{i,j}$ can be computed independently
- Allows full utilization of SMs

Data-reusable

- The row i of C uses the same data from A_i
- The column j of C uses the same data from B_j
- Opens the possibility to reduce I/O pressure

Block-friendly

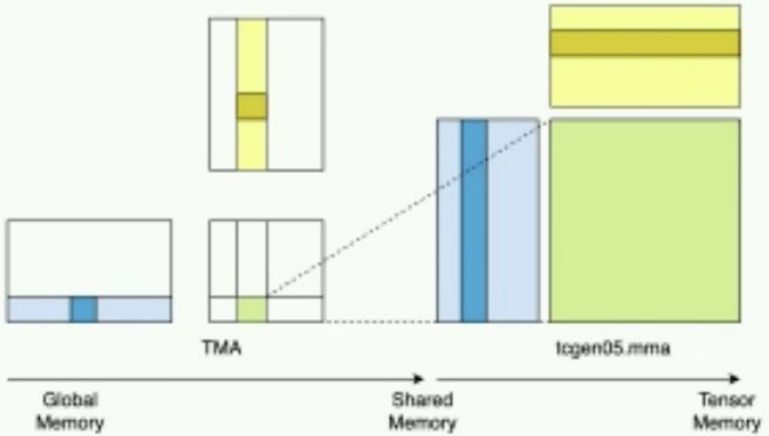
- Blocks of elements can be processed together
- Helps reduce I/O pressure for each SM
- Allows the use of MMA instructions

Bottleneck-flexible

- The bottleneck is **varied** for different problem sizes
- To reach SOL performance, need to match **strategies** according to problem sizes

Testing configuration on B200:

- Using random float as inputs
- Testing with floating clock
- Using nsight-compute to capture the execution time
- Power at 1000W



What's Covered in This Tutorial?

Optimizing GEMM step-by-step in 300 lines of Python

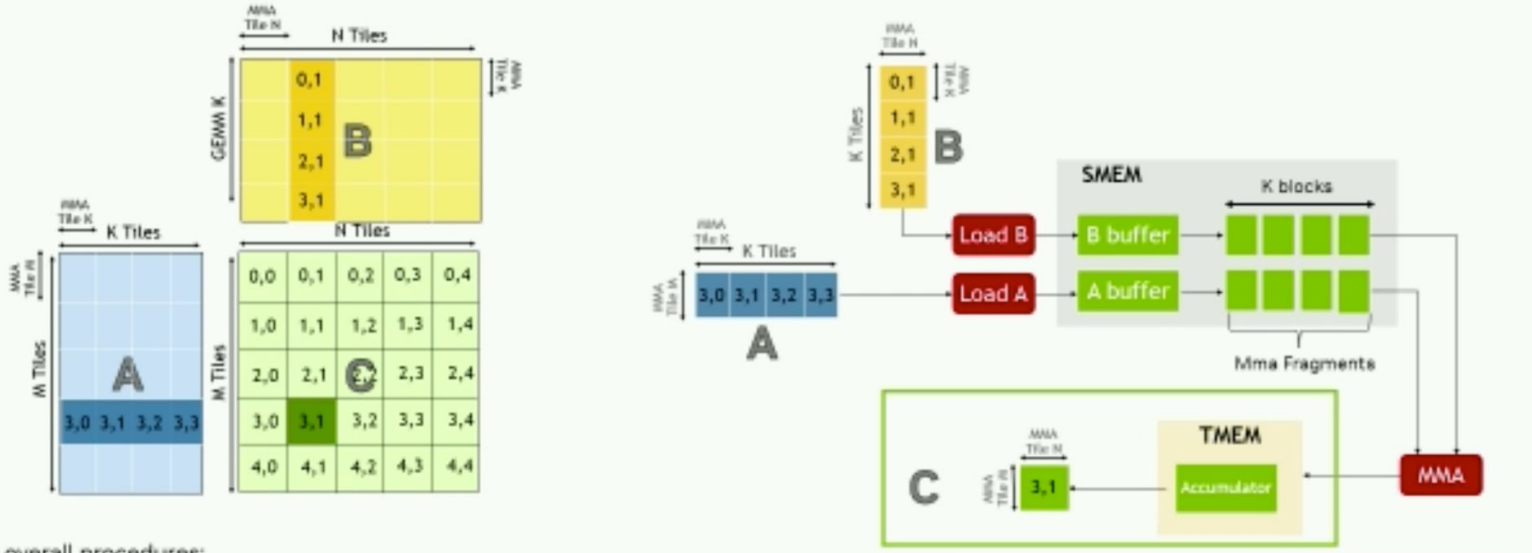
- This tutorial showcases how to:
  - Use Blackwell features via CuTe DSL
  - Analyze and improve the performance for different problem sizes

bit.ly/cute-

	Strategies Used	Cumulative Benefits
Today	Software-pipelined GEMM	Baseline implementation
	+ 2 CTA optimization	Improved latency hiding
	+ Warp-specialization and TMA store	Better epilogue phase overlap
	+ Static Scheduling persistency	Pipeline bubbles hidden among waves
	+ Preferred and fallback clusters	SMs occupancy improved
Github	+ TMA L2 prefetch	DRAM latency hidden
	+ CLC dynamic scheduling	Better workload balance for multi streams
	+ PDL	Kernel launch bubbles hidden

Tutorial 0: Blackwell MMA GEMM

Workflow of the basic GEMM



The overall procedures:

Tutorial 0: Software Pipeline

CTA tile size 128(M)x256(N)x64(K)

Hide the DRAM load latency

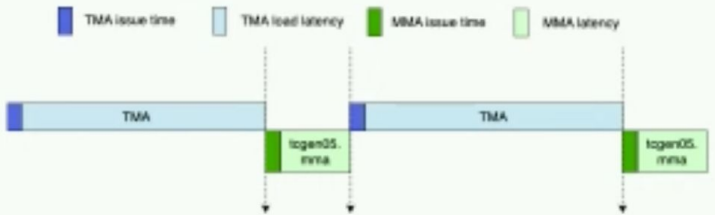
- Use multiple ab buffers for TMA load
- ab_stages **limited by SMEM capacity**

Prefetch a few stages of TMA load

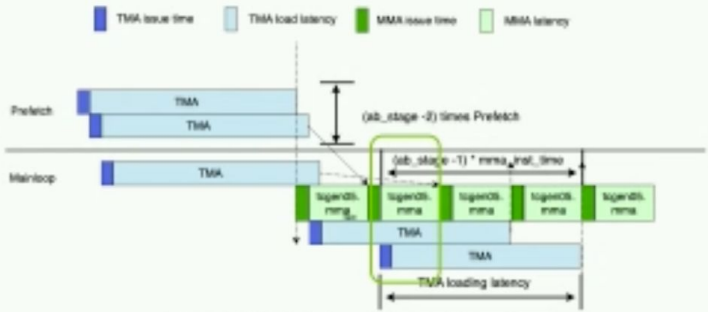
- The latency hiding ability:

(ab_stages – 1) * one_stages_mma_inst_time

```
for k_tile_idx in cutlass.range(
      num_k_tiles, prefetch_stages=ab_stages - 2
):
    # Issue TMA loads
    ab_empty = ab_producer.acquire_and_advance()
    # Load data for k_tile_idx + 2
    cute.copy(...)


    # Execute one K-block worth of MMA instructions
    ab_full = ab_consumer.wait_and_advance()
    # Compute for k_tile_idx
    cute.gemm(...)

    # Signal that the A/B buffers have been
    # consumed and are ready for the next load
    ab_full.release()
```



No software pipeline



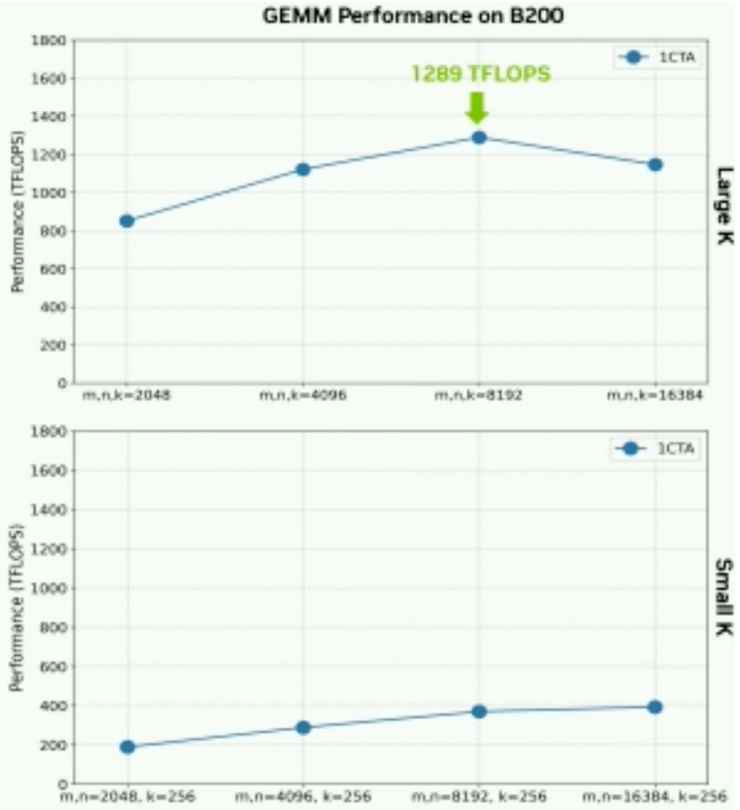
Issue TMA data load several iterations ahead of the computation

Tutorial 1: Motivation

CTA tile size 128(M)x256(N)x64(K)

Known limitations:

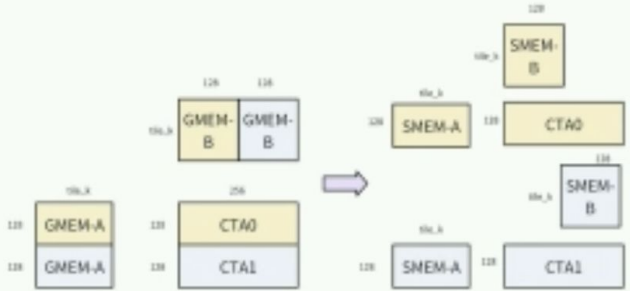
- Pipeline depth limited to 4 stages with 128x256x64 tile size
- Provide roughly (4 – 1) * 512 = **1536** cycles latency hiding
- Empirical benchmark shows latency can reach **1800** cycles
- Since **1536** < **1800** cycles, the data loading latency can not be fully hidden
- Compute throughput within a wave is only around 72%



Tutorial 1: Enabling 2CTA

CTA tile size 128(M)x256(N)x64(K)

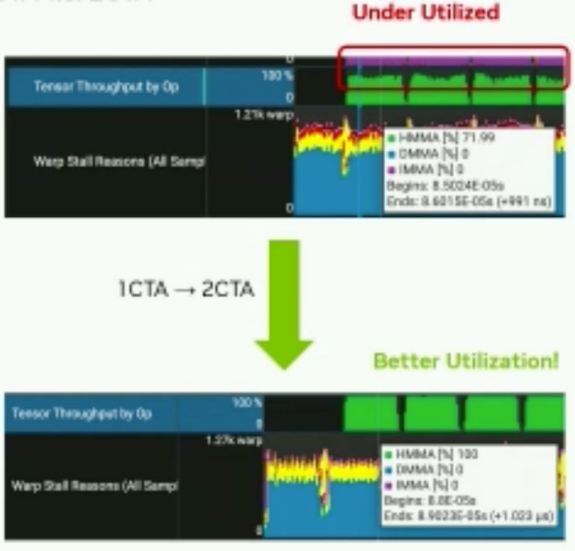
- 2SMs from a same TPC is paired together to process a larger tile size:
  - mma_tiler_mnk = **128**x256x64 → **256**x256x64
  - cta_tiler_mnk = 128x256x64
  - Cluster_shape_mnk = (1,1,1) → (2, 1, 1)
- Tiles mapping:
  - B is shared among the 2SMs
    - The SMEM space for B per SM **could be reduced by half**
- ab_stages size could be improved from **4** to **7**

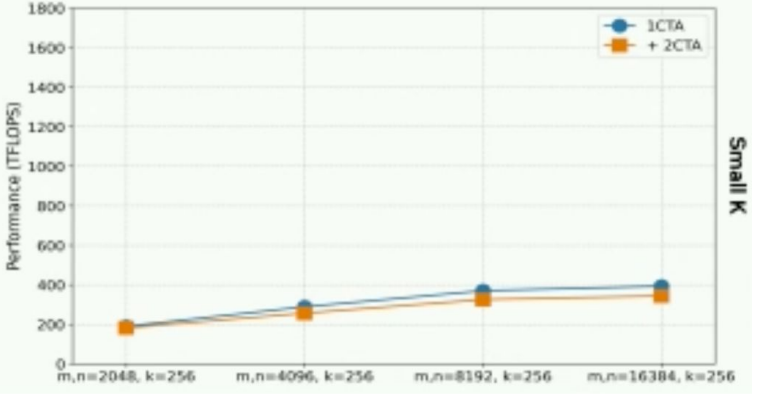


	1 CTA SMEM usage (KB)	2 CTA SMEM usage (KB)
one_stage_A	16	16
one_stage_B	32	16
one_stage_all	48	32
num_ab_stages	227 // 48 = 4	227 // 32 = 7

Tutorial 1: Performance Status

1CTA v.s. 2CTA



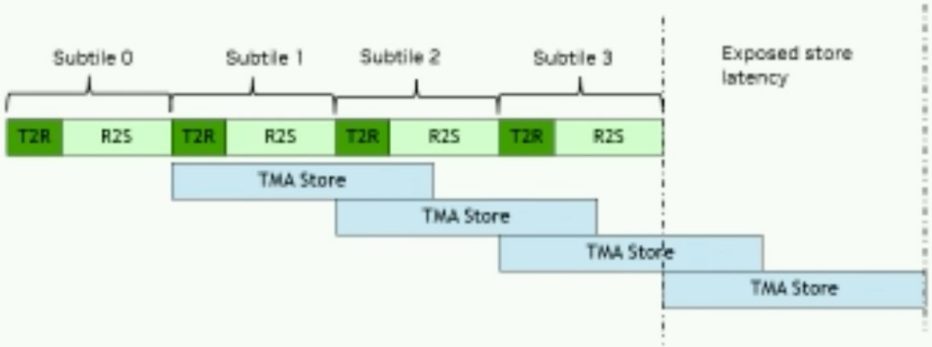


TMASTG instruction

Tutorial 2: Epilogue Optimization and Warp-Specialization

Epilogue Optimization - Using TMASTG

- In Tutorial 2, we change to use TMA Store for **better coalesced store**
  - Step 1: TMEM -> RMEM
  - Step 2: RMEM -> SMEM
  - Step 3: SMEM -> GMEM
  - Sub-tiling for R2S and TMA Store overlap to reduce SMEM and register usage:



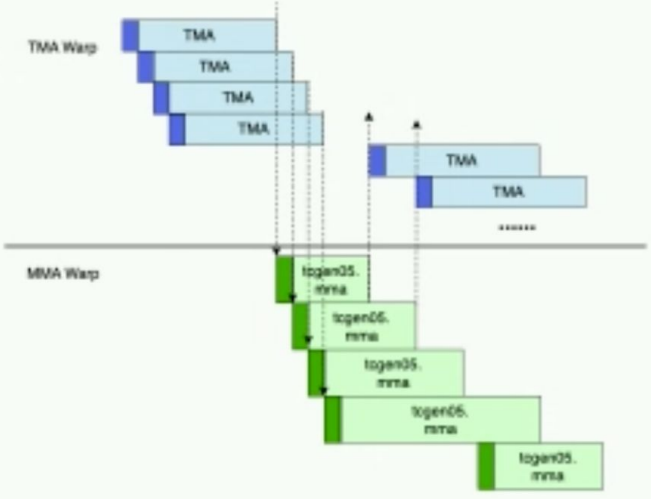

```
# Epilogue tiling loop
for subtile_idx in cutlass.range(subtile_cnt):
  # Slice
  ...
  # TMEM -> RMEM
  cute.copy(tiled_copy_t2r, tCtC, tCrC)


  # type conversion
  tCrC_out.store(tCrC.load().to(io_dtype))


  # RMEM -> SMEM
  cute.copy(tiled_copy_r2s, tCrC_out, tCsC)

  # Memory fence and barrier to ensure shared
  # memory stores are visible to TMA stores
  cute.arch.fence_view_async_shared()
  epilogue_sync_barrier.arrive_and_wait()


  # SMEM -> GMEM
  if warp_idx == epilogue_warp_ids[0]:
  cute.copy(tma_c.atom,tCsC,tCgC)
```

Warp Specialization

Tutorial 2: Epilogue Optimization and Warp-Specialization

Warp specialization optimization

- Warp Specialization can achieve similar data loading latency hiding as software pipeline optimization but provide **better instruction issue efficiency** as different instructions are distributed to different warps




```
# Warps ids for each part

epilogue_warp_ids = (0, 1, 2, 3)
mma_warp_id = 4
tma_warp_id = 5
threads_per_cta = 32 * \
    len((mma_warp_id, tma_warp_id,
*epilogue_warp_ids))
epilog_threads = 32 * len(epilogue_warp_ids)


# Execute data copy
if warp_idx == tma_warp_id:
...

# Execute math computation
if warp_idx == mma_warp_id:
    ...

# Execute epilogue, store results to GMEM
if warp_idx < epilogue_warp_ids:
...


return
```

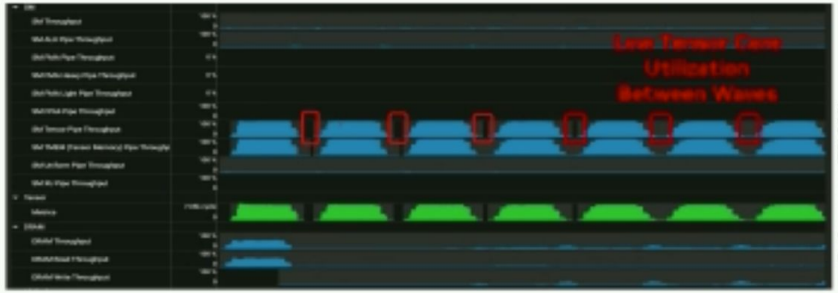
Warp Specialization + Epilogue Optimization

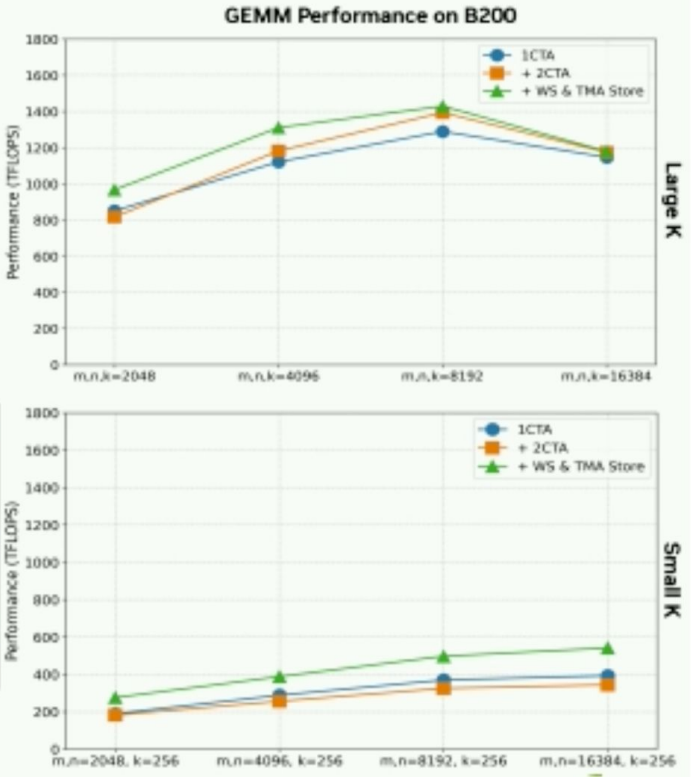
ee Portal

Tutorial 2: Performance Status

Warp specialization and epilogue optimization

- Optimization within a wave almost always beneficial...
  - For large K, main loop accounts for a very large proportion, so the speedup is not obvious
  - Small K cases always benefit
- But there is poor overlap among waves





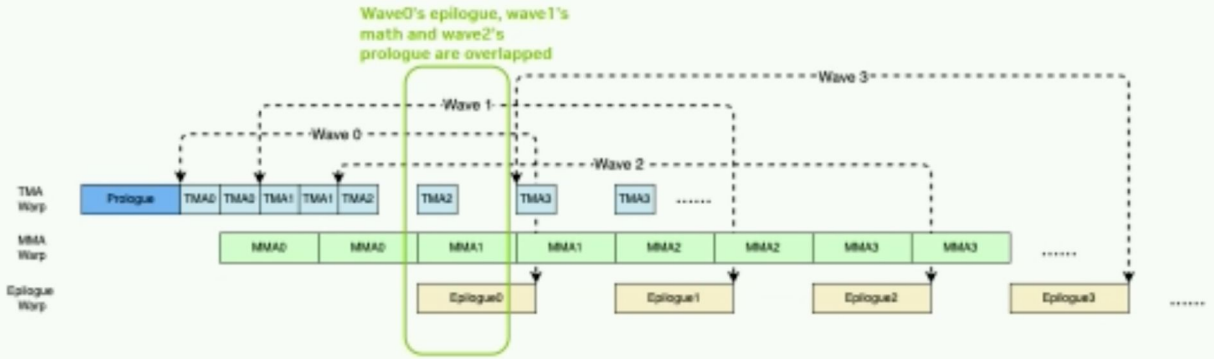
Phase Overlapping - Persistent Scheduler - Reduce bubbles among diff waves

Tutorial 3: Phase Overlapping Optimization

Static persistent scheduler

Enable Persistent thread optimization

- Save (#waves-1) prologue, only need one prologue at the beginning
- The epilogue of wave *i* – 1 could be overlapped with mma of wave *i*

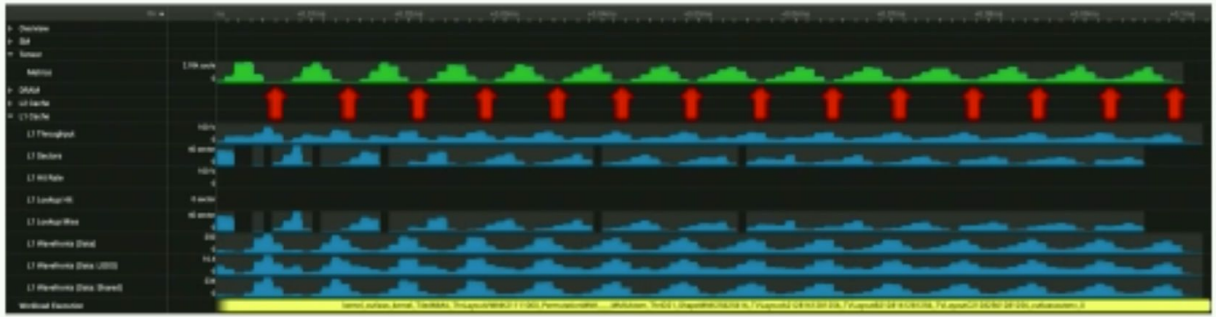


The execution timeline of warp specialize and persistent style GEMM kernel (assume perfect overlapping)

Tutorial 3: Performance Status

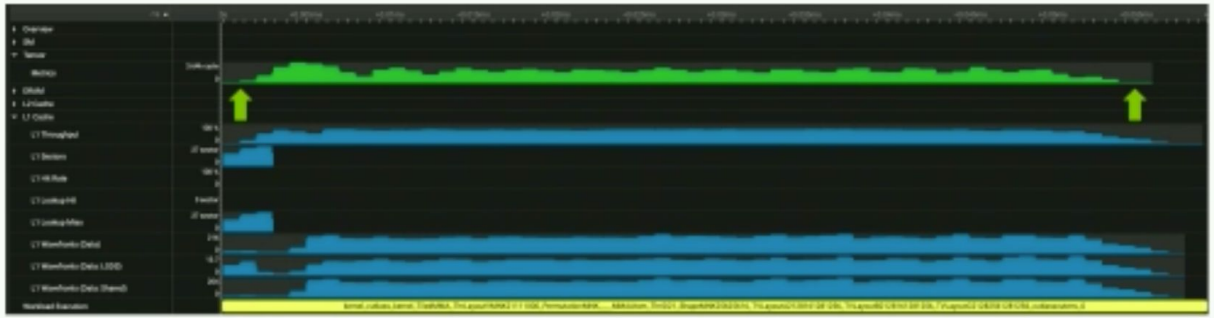
Nsight Compute views for multi-wave cases

Non-Persistent Kernel



All prologues and epilogues are exposed!

Persistent Kernel



Only the first prologue and the last epilogue are exposed

NVIDIA

Tutorial 3: Performance Status

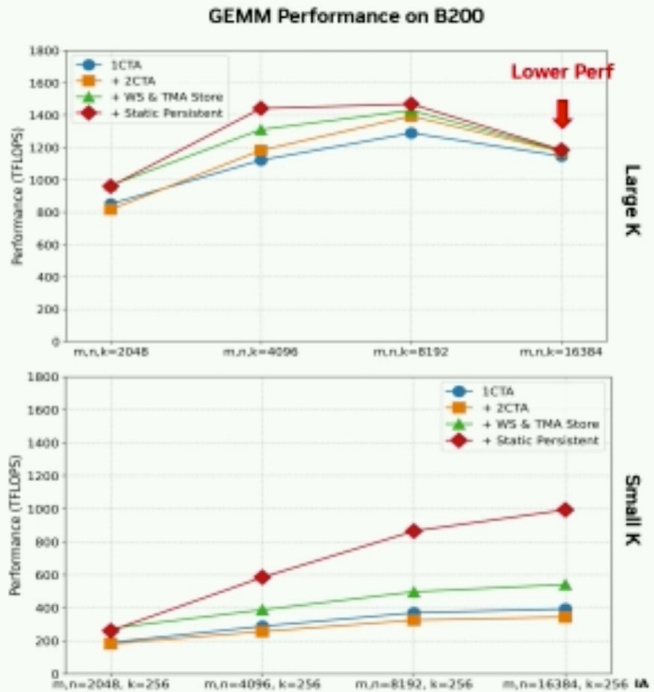
Static Persistent Kernel

Most cases benefits from Static Persistent Scheduler

We expect larger performance gains when the epilogue proportion is large, and vice versa

Why is there poor performance for 16Kx16Kx16K?

- Low L2 hit rate (32.9%) because of limited L2 size (e.g. 120 MB on B200)



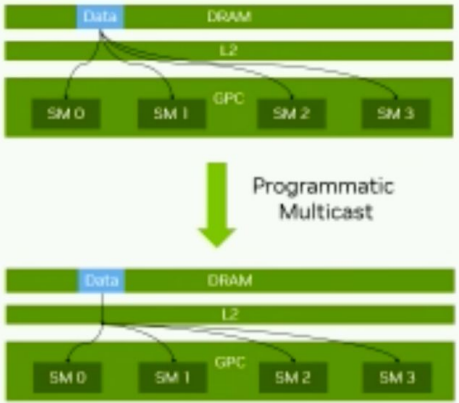
Tutorial 4: Preferred and Fallback Clusters

Larger cluster layout alleviates the situation:

- Large cluster layout → Programmatic Multicast → Reduced L2 traffic
- Large cluster layout → CTA Rasterization → High L2 Hit rate
- May result in poor SM occupancy due to quantization
  - Example: A 2x2 cluster on an 18 SM GPU would only utilize 16 SMs, leaving 2 SMs idle

Blackwell supports specify two clusters:

- Preferred (large) Cluster: Achieve higher TMA multicast
- Fallback (small) Cluster: Enhance occupancy



Programmatic Multicast: One load to store into shared memory of multiple CTAs in the same cluster, to save L2 bandwidth

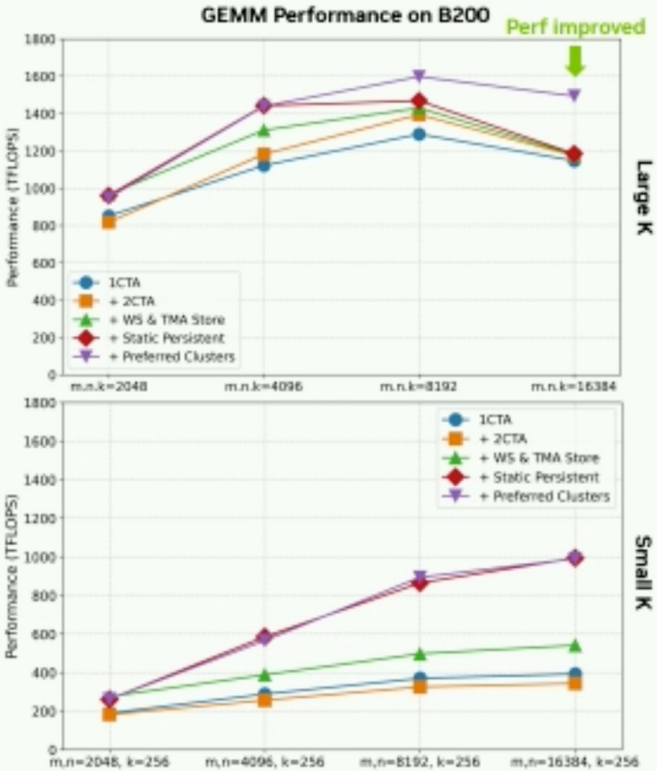
Tutorial 4: Performance Status

L2 hit rate before and after using Preferred Cluster
Large K Problem

MxNxK	2Kx2Kx2K	4Kx4Kx4K	8Kx8Kx8K	16Kx16Kx16K
w/o Preferred Cluster	97.25%	81.29%	46.17%	32.89%
w/ Preferred Cluster	97.07%	81.49%	**73.63%**	**51.13%**

Performance benefits of Preferred Cluster:

- Fully utilize the SMs while retaining the benefits of large cluster size
- Preferred and Fallback Cluster mainly helps the L2-bound cases

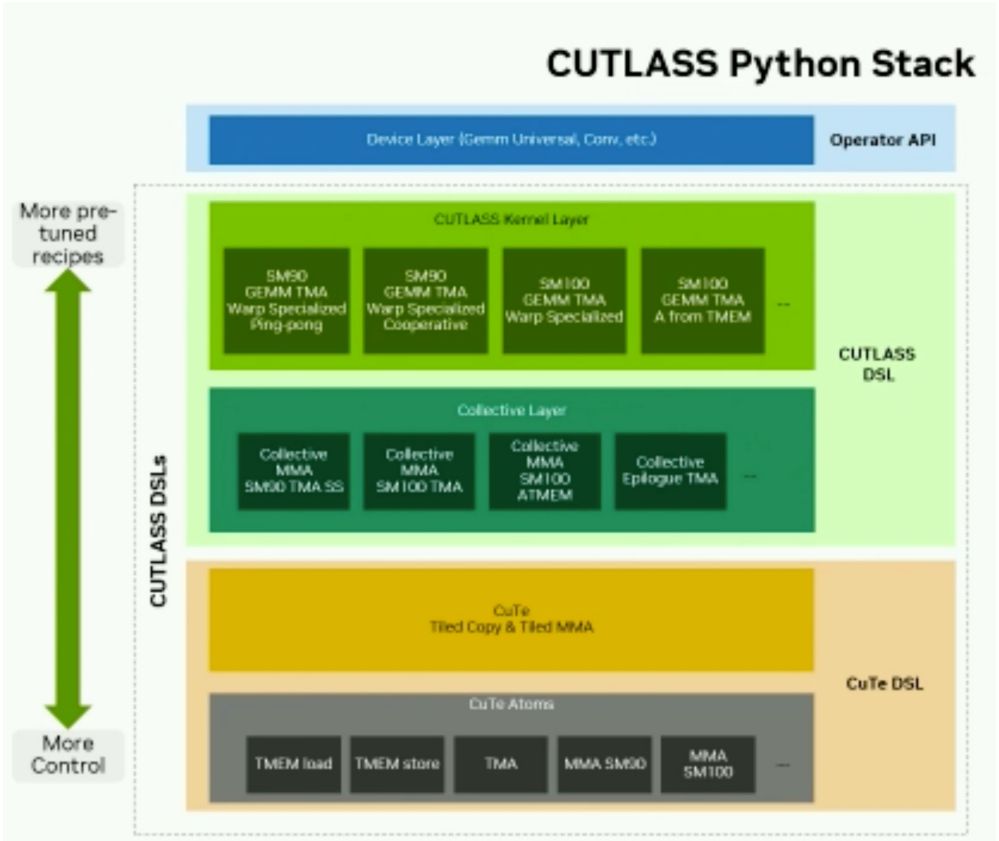


Tutorial GEMM Optimization Summary

- CUTLASS Python is a powerful tool for authoring performant kernels
- Optimization recap:
  - **2CTA + Multicast:** Reduced per-CTA SMEM allows deeper pipelining, L2 traffic is reduced due to TMA multicast
  - **Warp Specialization & Epilogue optimization:** Warp specialization provides better instruction mix and higher memory coaleasing write
  - **Persistent Scheduler:** Persistent clusters hide prologue and epilogue costs among waves
  - **Preferred & Fallback Clusters:** Balance large and small cluster shapes cluster shape for higher TMA multicast and improved occupancy

Strategies Used	Cumulative Benefits	Relative Performance Change (M=N=8192, K={8192, 256})
Software-pipelined GEMM	Baseline implementation	1.00x, 1.00x
+ 2 CTA optimization	Improved latency hiding	1.08x, 1.10x
+ Warp-specialization and TMA store	Better epilogue phase overlap	1.11x, 1.34x
+ Static Scheduling persistency	Pipeline bubbles hidden among waves	1.14x, 2.34x
+ Preferred and fallback clusters	SMs occupancy improved	1.24, 2.41x

Updated/New APIs: CuTe DSL, CUTLASS DSL



New CuTe DSL Functionality

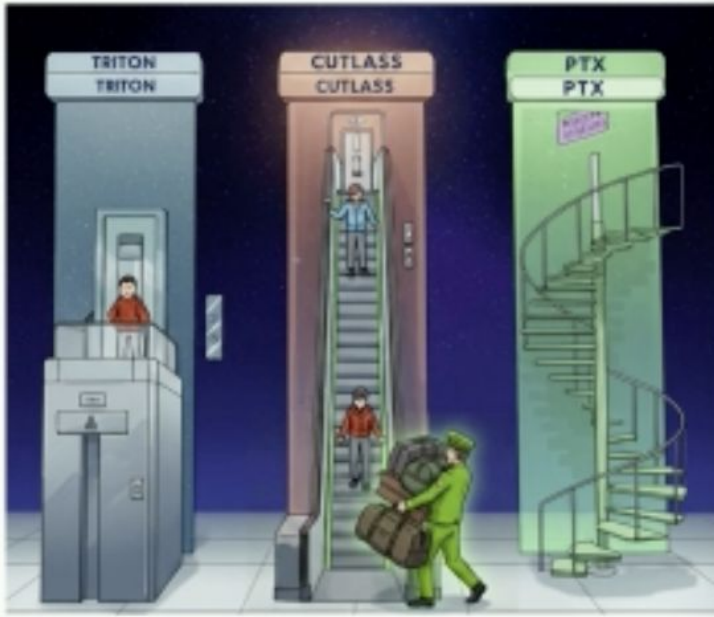
A thin abstraction layer on top of CuTe

Minimally abstract manual chores in CuTe

- Features that require global analyses, instead of local rewrites or helper functions
- Abstracts the CuTe programming model for convenience

Retain full inter-operability: No performance compromise. No representation gaps

- All abstractions compose seamlessly with CuTe



Offload heavy chores
Equal outcome
Opt-in convenience, bypass anytime

CuTe DSL Automates Chores

- Automate error-prone, tedious orchestration
- Opt-in convenience, bypass at any time for full control

Automatic TMA coord tensor transform,
TMA descriptor creation,
Outlining to host code

Automatic TMA desc update on-device when kernel ptr/shape/strides change

Automatic GMEM predication based on dynamic shapes input to the kernel

Automatic vectorization for SIMT copies

Programmatic interface for device workspace allocation in addition to compiler-managed workspace

Automatic host functions / runtime hooks generation

Fragment-free programming model for all MMA and copies

Example: Simpler TMA orchestration

CuTe DSL without extensions

```
# -- Host code -- Setup TMA load for GMEM tensor
tma_a = cpasync.make_tiled_tma_atom(
    a_op, a,
    self.a_smem_layout_staged,
    (self.mma_tiler[0] // atom_thr_size, self.mma_tiler[2]),
    internal_type=(
        cutlass.TFloat32 if a.element_type is cutlass.Float32 else None),
)
# -- Device code -- Takes TMA desc as input
@cute.kernel
def kernel(
    tma_a: cpasync.TmaInfo, ...
):

    # Prefetch TMA descriptor
    if warp_idx == tma_load_warp_id:
        cute.nvgpu.cpasync.prefetch_descriptor(tma_a.atom)


    # Tile & Partition TMA coord tensor, make fragments
    gA = cute.local_tile(mA_mkl, mma_tiler_mnk, mma_coord_mnk, proj=(1, None, 1))
    tAsA, tAgA = cpasync.tma_partition(
        tma_atom_a,
        block_in_cluster_coord_vmnk[2],
        a_cta_layout,
        cute.group_modes(sA, 0, 3),
        cute.group_modes(tCgA, 0, 3),
    )
    tAgA_slice = tAgA[(None, mma_tile_coord_mnl[0], None, mma_tile_coord_mnl[2])]


    # Issue TMA load
    cute.copy(
        tma_a.atom,
        tAgA_slice[(None, k_tile_idx)],
        tAsA[(None, handle.index)],
        tma_bar_ptr=handle.barrier,
        mcast_mask=None)
```

CuTe DSL with extensions

```
# -- Host code -- No setup needed!
```

No host TMA initialization

```
# -- Device code -- Takes GMEM tensor directly
@cute.experimental.kernel
def kernel(
    mA: cute.Tensor, ...
):
```

Kernel input is regular tensor

No TMA descriptor management

No TMA coordinate tensors

```
    # Tile the GMEM tensor directly
    gA = cute.zipped_divide(mA, tiler_mn)
    gA_tile = gA[(None, None), (cta_m, cta_n, cta_l)]
    gA_k = gA_tile[None, None, k]


    # Issue TMA load
    cute_ext.tma_load_multicast(
        gA_k,
        sA[None, None, None, idx],
        mbarrier,
    )
```

NVIDIA

CUTLASS DSL Preview

Streamlining Kernel Development

Goal: Specify algorithms in terms of tensors & tasks, without requiring all layout details

- Higher-level DSL, extending beyond Cute DSL
- Represent warp-specialized kernels with an explicit task graph
- Compiler becomes the CuTe expert enabling focus on high-level machine mapping

New division of labor:

- Programmer manages pipelines, agent structure, SMEM/TMEM layouts, and decides MMA and Copy atom
- Compiler constructs partitioning chain, manages buffers, and inserts synchronization

Coming later this year ☺

CuTE DSL GEMM Kernel

386 lines
to
34 lines!

CUTLASS DSL GEMM Kernel

```
with example_dsl.task_graph() as graph:
    tma_dep = graph.dependency([smem_a_ty, smem_b_ty])
    mma_dep = graph.dependency([tmem_acc_ty])

    @graph.agent(
        "thread", 0, 32,    # MMA agent (threads 0-31)
        producers=[mma_dep], # consumes A,B; produces accumulator
        consumers=[tma_dep],
    )
    def mma_agent(acc_queue, ab_consumer):
        accum_init = example_dsl.zeros(tmem_acc_ty)

        for k, acc, result in cutlass.for_generate(
            0, tile_count_k, 1, [accum_init]
        ):
            A, B = example_dsl.fifo_pop(ab_consumer, [smem_a_ty, smem_b_ty])
            acc = example_dsl.dot(
                A, B, acc,
                mn_type=OperationTypeEnum.SM100_MMA_128_SS,
                tiled_mma=tiled_mma,
            )
            cutlass.yield_out([acc])
        acc = result
        example_dsl.fifo_push(acc_queue, acc)

    @graph.agent(
        "thread", 32, 33,   # TMA agent (threads 32-63)
        producers=[tma_dep], #   loads A,B tiles GMEM->SMEM
    )
    def tma_agent(ab_queue):
        for k in cutlass.range(0, tile_count_k, 1, unroll=1):
            A = example_dsl.tma_load(gA_tile[None, None, k], smem_a_ty, s_mcpy=s_mcpy)
            B = example_dsl.tma_load(gB_tile[None, None, k], smem_b_ty, s_mcp=b_v_map)

            example_dsl.fifo_push(ab_queue, A, B)

    @graph.agent(
        "thread", 128, 128,  # Epilogue agent (threads 128-255)
        consumers=[mma_dep],  # TMEM->RMEM->GMEM
    )
    def epilogue_agent(acc_consumer):
        D_acc = example_dsl.fifo_pop(acc_consumer, [tmem_acc_ty])[0]
        D_rmem = example_dsl.copy(
            D_acc, rmem_d_ty,
            cp_type=OperationType.SM100_COPY_T2R,
            tiled_copy=tiled_copy_t2r,
        )

        example_dsl.store(
            gD_tile.value, D_rmem,
            cp_type=OperationType.STG,
            tiled_copy=tiled_copy_r2g,
        )
```

MMA Consumer

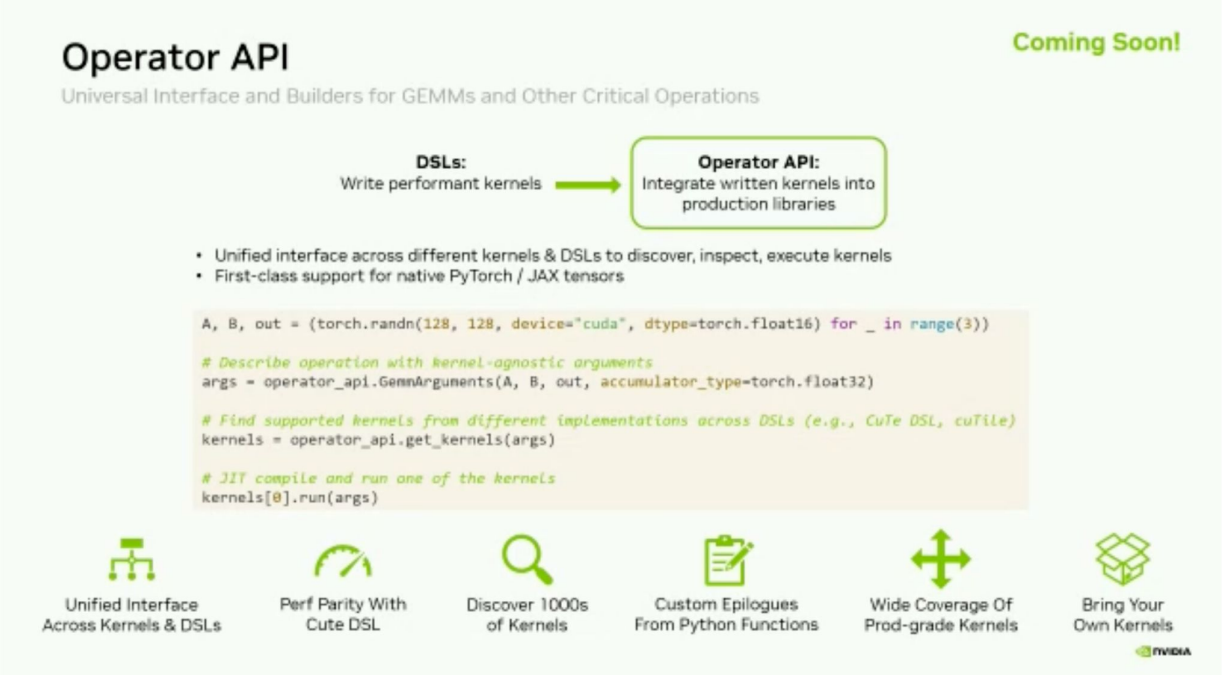
Tile Loader

Epilogue

New Operator API

Expose unified interface across kernels and DSLs to execute kernels

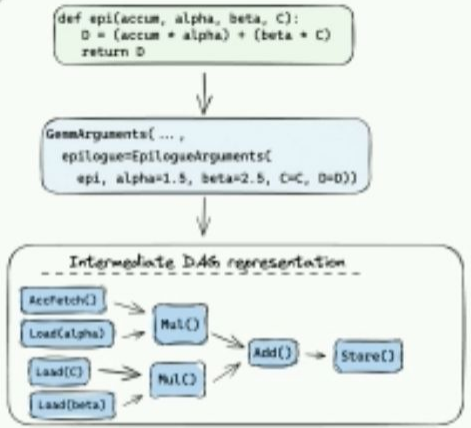
Already first-class support for PyTorch and JAX



Operator API

Universal Interface and Builders for GEMMs and Other Critical Operations

```python
# Create custom epilogue function
def my_epilogue(accum, alpha, beta, C):
    D = (accum * alpha) + (C * beta)
    return D

# Operator API traces the function
C = torch.empty(...)
epi_args = operator_api.EpilogueArguments(my_epilogue,
                  alpha=1.5, beta=2.5, C=C, D=out)

# Describe operation with kernel-agnostic arguments
args = operator_api.GemmArguments(A, B, out, accumulator_type=torch.float32,
                                  epilogue=epi_args)

# Find supported kernels from different implementations across DSLs
kernels = operator_api.get_kernels(args)

# JIT compile & run
kernels[0].run(args)
```



Unified Interface Across Kernels & DSLs

Perf Parity With Cute DSL

Discover 1000s Of Kernels

Custom Epilogues From Python Functions

Wide Coverage Of Prod-grade Kernels

Bring Your Own Kernels

NVIDIA

Operator API

Universal Interface and Builders for GEMMs and Other Critical Operations

Wide coverage of functionality, configs, and optimizations
Home for robustly-tested, optimized DSL kernels beyond examples

Initial release, available soon:

- CUTLASS DSLs & cuTile
  - Dense GEMMs (F32, F16, BF16, INT8)
  - Custom epilogue fusions (e.g., aux load/store, elementwise ops)
  - Block-scaled GEMM (MXFP8)
  - Grouped GEMM
  - **124k kernel configs** and counting...
- Bring Your Own Kernels
- CUDA Graphs
- PyTorch & JAX support
- Being integrated as a backend to **PyTorch Inductor**

Planned future releases:

- AOT support
- Kernel coverage
  - NVFP4
  - Low latency GEMMs
  - Sparsity
  - Blackwell Ultra optimizations

Unified Interface
Across Kernels & DSLs

Perf Parity With
Cute DSL

Discover 1000s
Of Kernels

Custom Epilogues
From Python Functions

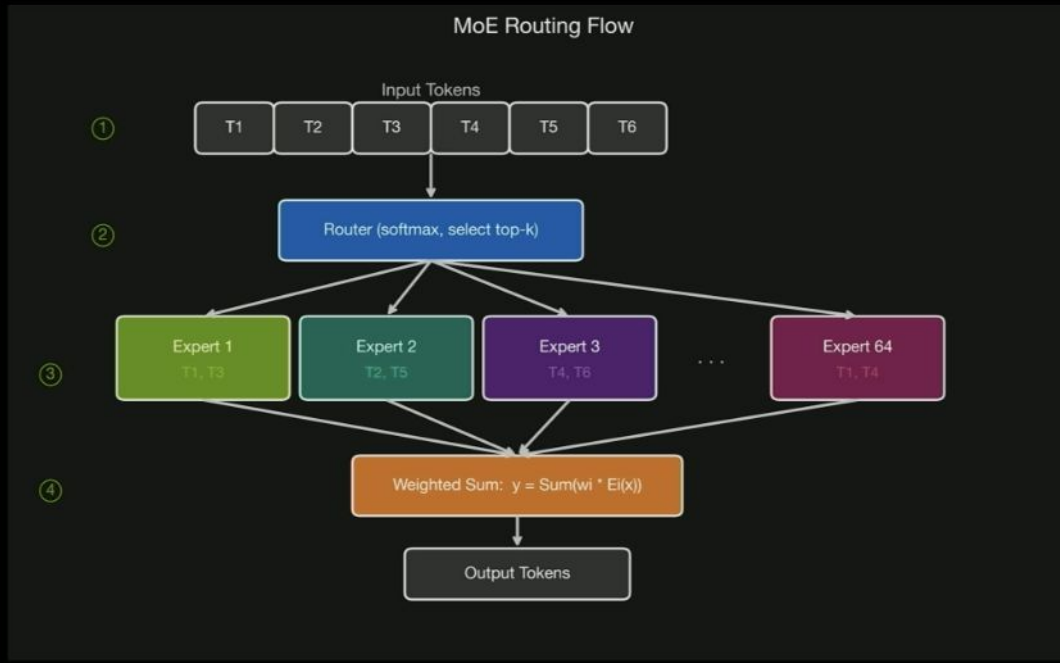
Wide Coverage Of
Prod-grade Kernels

Bring Your
Own Kernels

MoE: Routing, Scale, and the GPU Challenge

C[i] = A[token(i)] × B[expert(i)]^T — this is NOT a standard GEMM

- A **router** scores all E experts, routes each token to top-k
- Outputs combined as: $y = \Sigma\, w_i \cdot \text{Expert}_i(x)$ — lightweight router, heavy experts
- Enables trillion-parameter models without proportional compute (DeepSeek, Mixtral)
- **Irregular gather** — A-matrix rows scattered by routing, no spatial locality
- **Variable work per expert** — some get hundreds of tokens, others none
- **Scatter on output** — results written to router-determined positions



MoE vs Standard GEMM

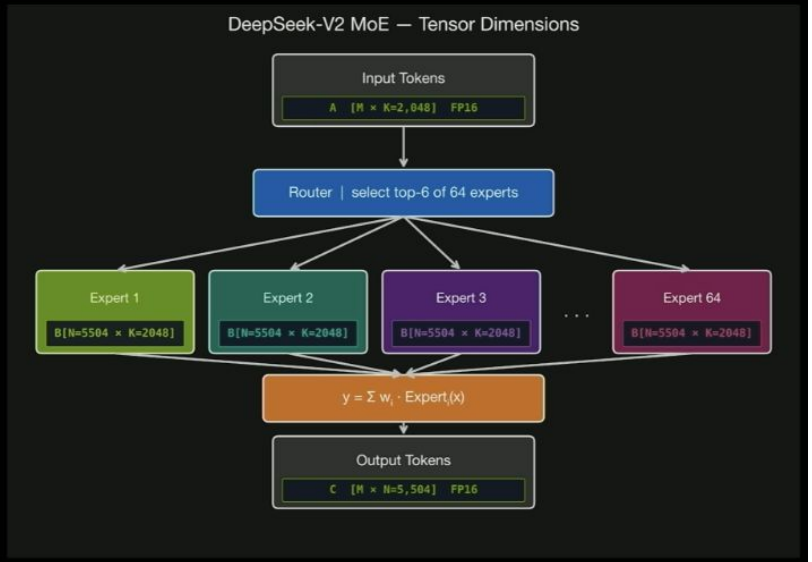
The memory access pattern is fundamentally different

- **Standard GEMM:** contiguous tiles — TMA bulk transfers for both operands
- **MoE GEMM:** A-matrix rows scattered by routing — must use ct.gather
- Only B-matrix (expert weights) can use efficient hardware TMA

Our Benchmark: DeepSeek-V2 on B200

A representative real-world MoE configuration

- **64 experts**, top-k = 6 — representative of DeepSeek-V2 configuration
- **Hidden size K = 2048**, intermediate size N = 5504, **FP16** precision
- NVIDIA B200 (Blackwell, 160 SMs, SM100 architecture)
- Workload per call scales from 0.14 to **1.1 TFLOP** at 8192 tokens



MOE TUTORIAL

Kernel Interface

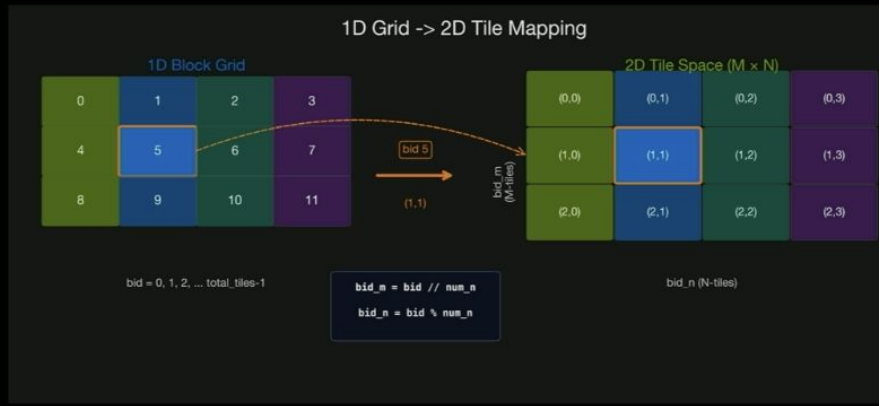
@ct.kernel with compile-time constants for tile sizes and routing configuration

```
@ct.kernel
def moe_kernel(
    a_ptr, b_ptr, c_ptr,            # Matrix pointers (flattened)
    sorted_token_ids_ptr,            # Token IDs sorted by expert
    expert_ids_ptr,                  # Expert ID per tile
    num_tokens_post_padded_ptr,      # Total padded token count
    N: ct.Constant[int],             # Output dimension (intermediate_size)
    K: ct.Constant[int],             # Input dimension (hidden_size)
    num_valid_tokens: int,           # Non-padding tokens
    TILE_SIZE_M: ct.Constant[int],   # Tile rows (e.g., 64)
    TILE_SIZE_N: ct.Constant[int],   # Tile columns (e.g., 64)
    TILE_SIZE_K: ct.Constant[int],   # Tile K-dimension (e.g., 32)
    top_k: ct.Constant[int],
    a_stride_1: ct.Constant[int],
    c_stride_1: ct.Constant[int],
):
```

Block ID Mapping

Mapping a 1D grid to (token-group, output-dimension) coordinates

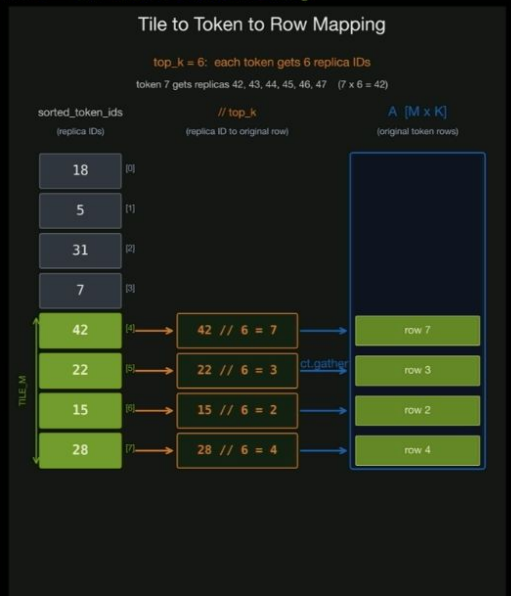
- Each CUDA block computes one output tile
- **bid_m** = which token group (M-tile)
- **bid_n** = which output dimension chunk (N-tile)
- Grid size = ceil(tokens / TILE_M) × ceil(N / TILE_N)
- With 64×64 tiles: **~68,000 blocks** for 8192 tokens
- With 128×128 tiles: **~18,000 blocks** — 4× fewer, 8× more FLOPs per tile



Gathering Token Information

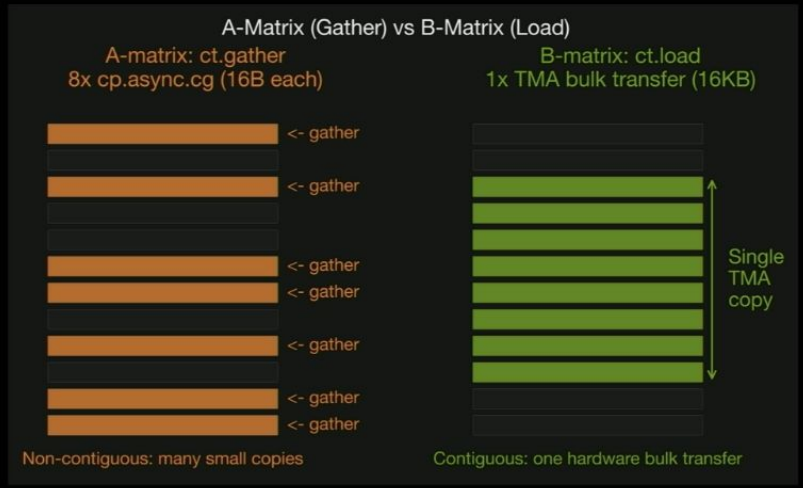
sorted_token_ids maps each tile to its token replicas — the heart of MoE routing

```
# Load token IDs for this tile
offs_token_id = bid_m * TILE_SIZE_M +
  ct.arange(TILE_SIZE_M, dtype=ct.int32)
offs_token = ct.gather(sorted_token_ids_ptr,
  offs_token_id, padding_value=0)
token_mask = offs_token < num_valid_tokens

# Which expert does this tile belong to?
off_experts = ct.load(expert_ids_ptr,
  index=(bid_m,), shape=(1,))
off_experts = off_experts.item()

# Convert token replica IDs to original row indices
row_indices = offs_token // top_k
a_row_offset = row_indices[:, None] * a_stride_1
```



The Key Asymmetry: Gather vs Load

This asymmetry drives every optimization decision in MoE kernels

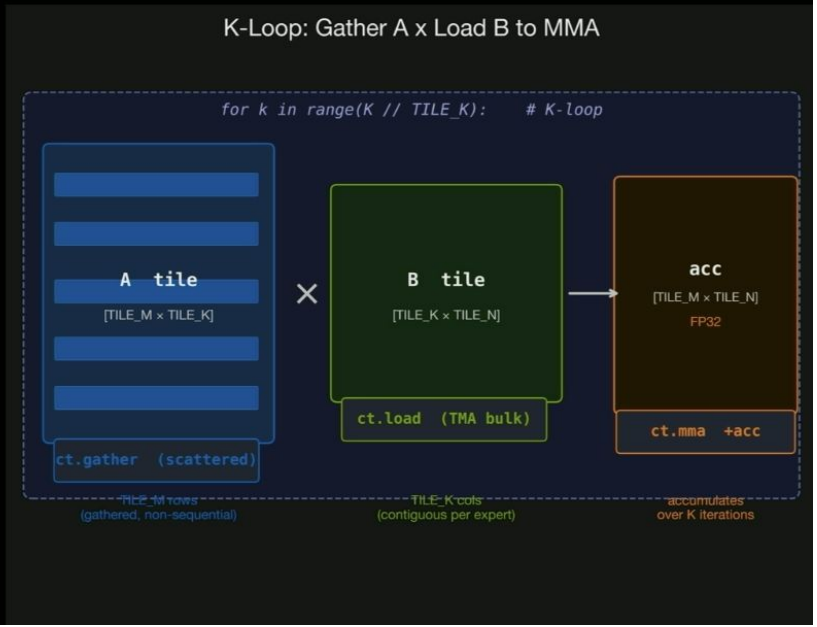


- **A-matrix:** ct.gather — 8× cp.async.cg per K-iteration (16B each, scattered)
- **B-matrix:** ct.load — 1× TMA bulk transfer per K-iteration (16KB, contiguous)

The K-Loop: Accumulating the GEMM

Iterate over K, gathering A tiles and loading B tiles for matrix multiply

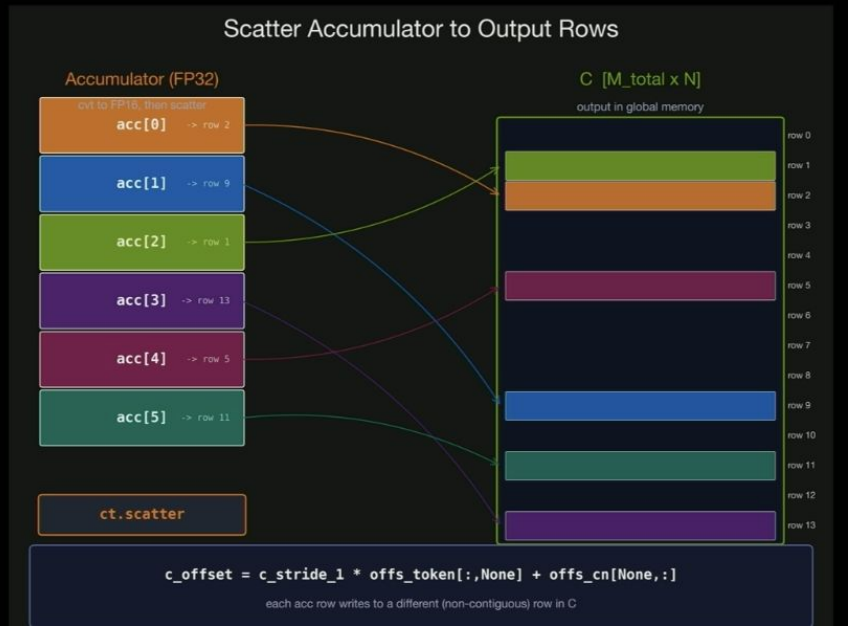
```
# FP32 accumulator for numerical stability
accumulator = ct.full((TILE_SIZE_M, TILE_SIZE_N), 0.0,
  dtype=ct.float32)

for k in range(0, ct.cdiv(K, TILE_SIZE_K)):
    # Gather scattered A-matrix rows
    col_indices = k * TILE_SIZE_K + ct.arange(TILE_SIZE_K,
  dtype=ct.int32)
    a_indices = a_row_offset + col_indices[None, :]
    a = ct.gather(a_ptr, a_indices, padding_value=0)

    # Load contiguous B-matrix tile (expert weights)
    b = ct.load(b_ptr, index=(off_experts, k, bid_n),
                shape=(1, TILE_SIZE_K, TILE_SIZE_N),
  order=(0, 2, 1))
    b = ct.reshape(b, (TILE_SIZE_K, TILE_SIZE_N))

    # Matrix multiply-accumulate on Tensor Cores
    accumulator = ct.mma(a, b, accumulator)
```



Scatter the Output

Convert dtype and scatter results to router-determined positions

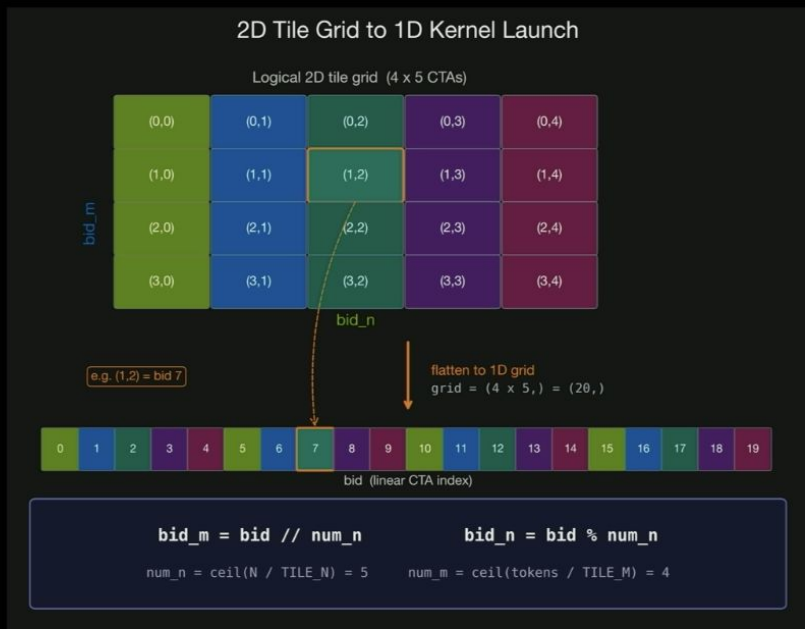
```
# Convert FP32 accumulator to output dtype (FP16)
accumulator = ct.astype(accumulator, c_ptr.dtype)

# Compute scatter offsets based on token IDs
offs_cn = bid_n * TILE_SIZE_N +
  ct.arange(TILE_SIZE_N, dtype=ct.int32)
c_offset = c_stride_1 * offs_token[:, None] +
  offs_cn[None, :]

# Scatter results to output positions
ct.scatter(c_ptr, c_offset, accumulator)
```



Launching the Kernel

Host-side setup: compute grid, flatten arrays, launch

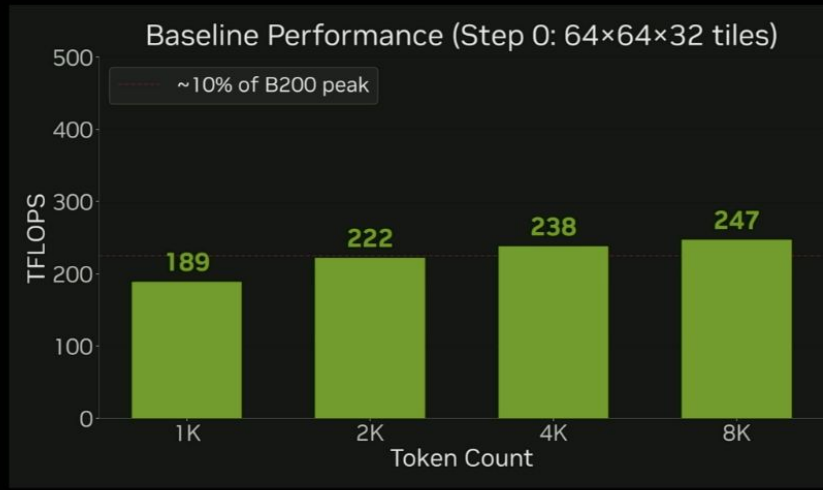
```python
from math import ceil

def launch_moe(A, B, C, topk_weights, sorted_token_ids,
               expert_ids, num_tokens_post_padded, top_k):
    TILE_M, TILE_N, TILE_K = 64, 64, 32  # Baseline tiles
    grid = (
        ceil(sorted_token_ids.shape[0] / TILE_M)
      * ceil(B.shape[1] / TILE_N),
    )

    ct.launch(
        torch.cuda.current_stream(), grid,
        fused_moe_kernel,
        (A.reshape(-1), B, C.reshape(-1), topk_weights.view(-1),
         sorted_token_ids, expert_ids, num_tokens_post_padded,
         B.shape[1], B.shape[2], top_k * A.shape[0],
         TILE_M, TILE_N, TILE_K, 0, top_k, A.shape[1], C.shape[1]),
    )
```



Baseline Performance: Step 0

Correct but unoptimized — 64×64×32 tiles

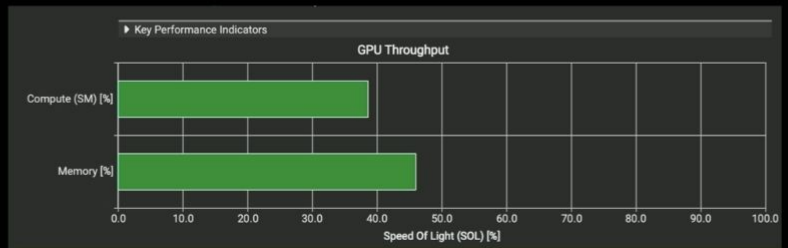
- Performance ranges from **189 to 247 TFLOPS** across token counts
- Less than **10%** of B200 tensor core peak
- Most time spent on per-tile overhead, not actual compute
- Significant room for improvement



What NCU Tells Us About the Baseline

Nsight Compute — fused_moe_kernel, step0, B200

- **11,008 blocks** × 74.4 waves/SM — tiny tiles, excessive serialization
- **Compute throughput: 37.78%** — TC pipeline fed less than half of peak
- **Issue Slots Busy: 24.44%** — SMs idle 76% of the time
- **Occupancy: 9.33%** — register pressure caps warps to 1 per scheduler
- **L1 hit rate: 0.59%** — scattered gather destroys locality



Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

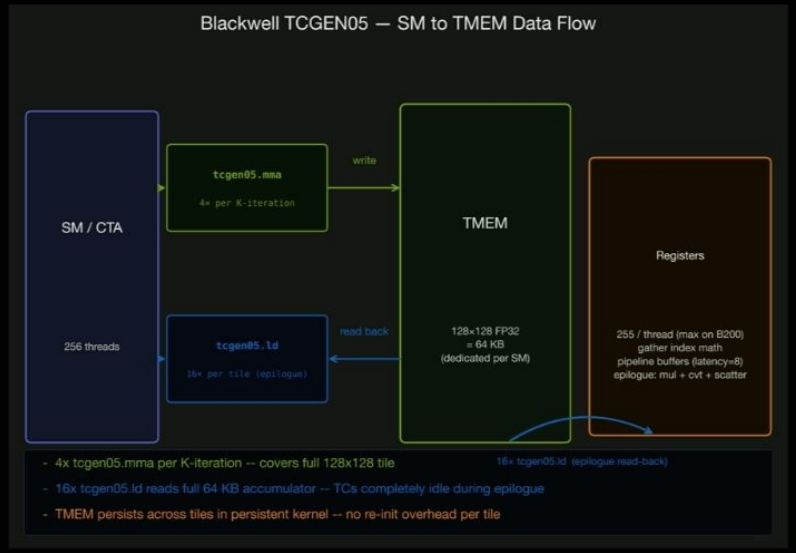
Executed Ipc Elapsed [inst/cycle]	0.98	SM Busy [%]	38.59
Executed Ipc Active [inst/cycle]	1.00	Issue Slots Busy [%]	24.44
Issued Ipc Active [inst/cycle]	1.00		

Blackwell **tcgen05** Tensor Core operation for GEMMs

Blackwell TCGEN05 Tensor Cores

5th-generation tensor cores with dedicated Tensor Memory (TMEM)

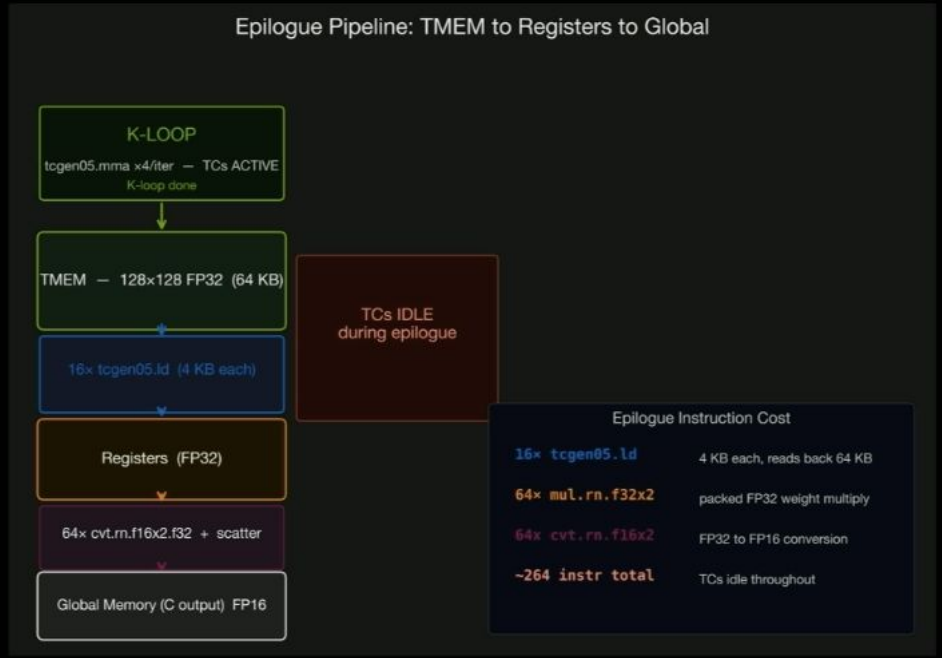
- Accumulation happens in **TMEM**, not the general register file
- 64×64 FP32 accumulator = **16 KB** TMEM — baseline uses only 25% of capacity
- 128×128 FP32 accumulator = **64 KB** TMEM — fills the entire TMEM budget
- **tcgen05.mma** instruction: 4 per K-iteration, covers the full 128×128 tile
- Frees registers for address computation and software pipeline



Reading the Accumulator: The Epilogue Cost

Moving data from TMEM back to registers for output

- After K-loop, FP32 results must leave TMEM for the epilogue
- **16× tcgen05.ld** reads the full 128×128 FP32 accumulator (4 KB each)
- Then: type conversion (FP32 → FP16) + scatter to global memory
- Tensor cores are **completely idle** during this epilogue phase



Blackwell Tensor Core

History and evolution of Tensor Core

Volta

8 threads

8 threads

Turing & Ampere

32 threads

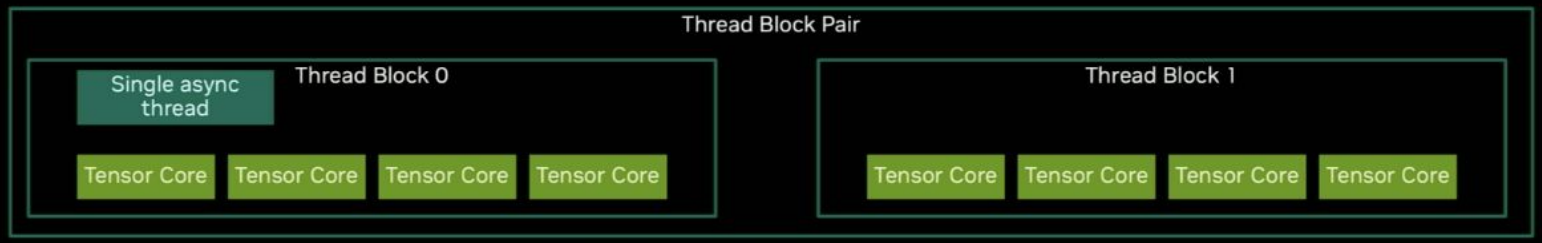
Entire warp 32 threads

Hopper

Warp 0 | Warp 1 | Warp 2 | Warp 3

Warp Group or 4 warps, entire SM

Blackwell B200/GB200

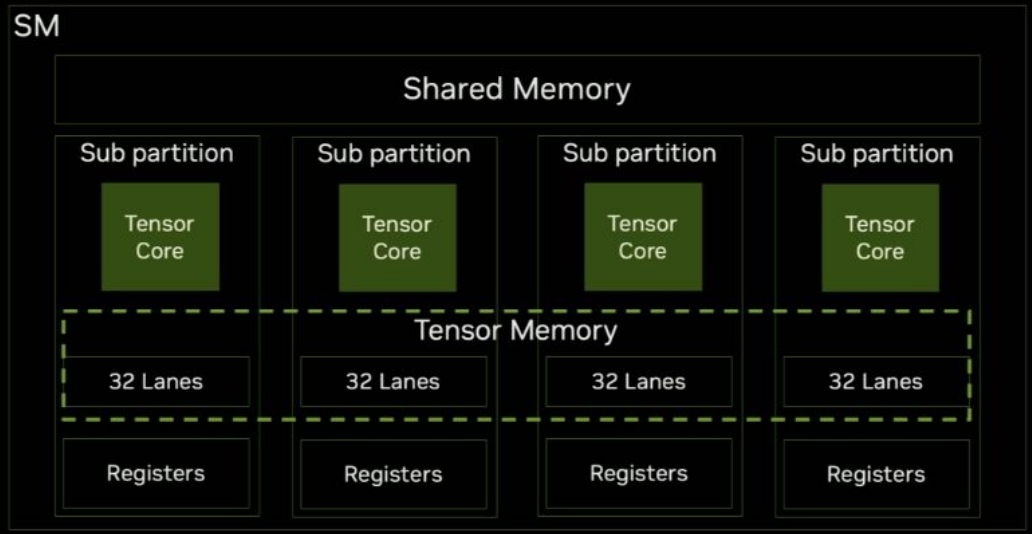


2 Thread Blocks in a Thread Block Pair

Blackwell Tensor Core

Blackwell SM architecture

- Up to 228KB of Shared Memory per SM
- 256 KB of Tensor Memory per SM for storing outputs of Tensor Core.
  - Warp synchronous load and store from registers to tensor memory.

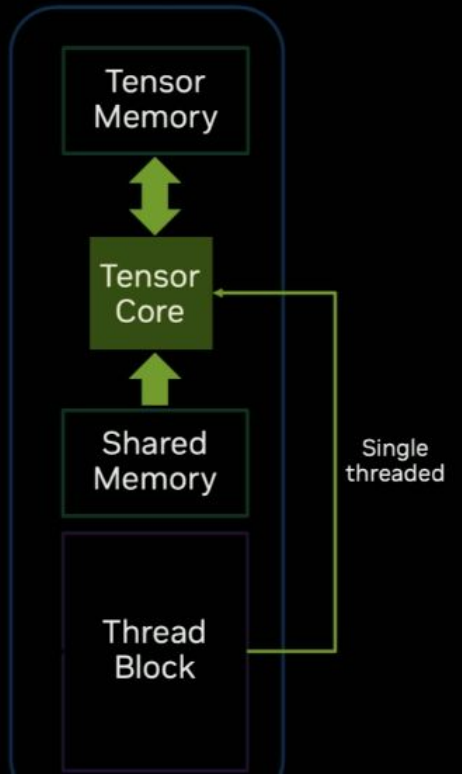


Tensor Core Cluster MMA

$D = \sum_k A_k * B_k$ 1 Thread Block Mode

Single Thread Block Mode

- Accumulator D Matrix in Tensor Memory (TMEM)
- Operand A Matrix in Tensor Memory or Shared Memory
- Operand B Matrix in Shared Memory
- A single thread submits instructions to Tensor Core

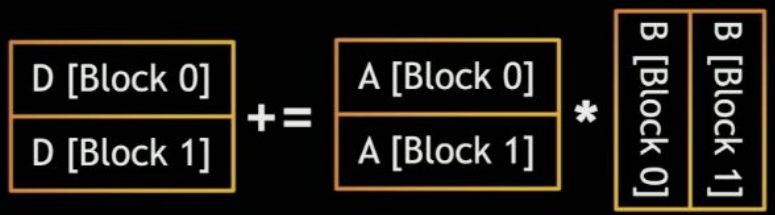


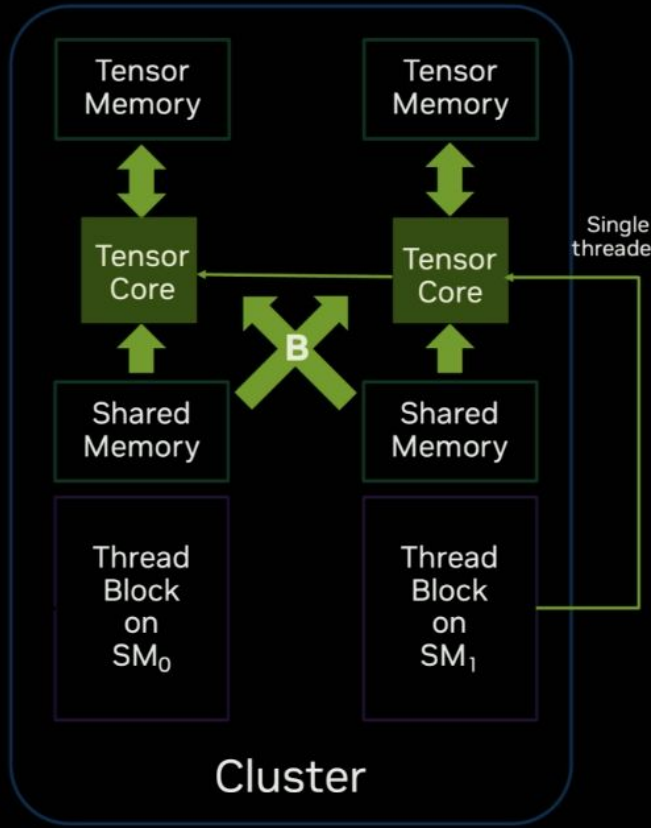
Tensor Core Cluster MMA

$D = \sum_k A_k * B_k$ Dual Thread Block Mode

2 Thread Block Mode : Paired-Thread blocks in a Cluster

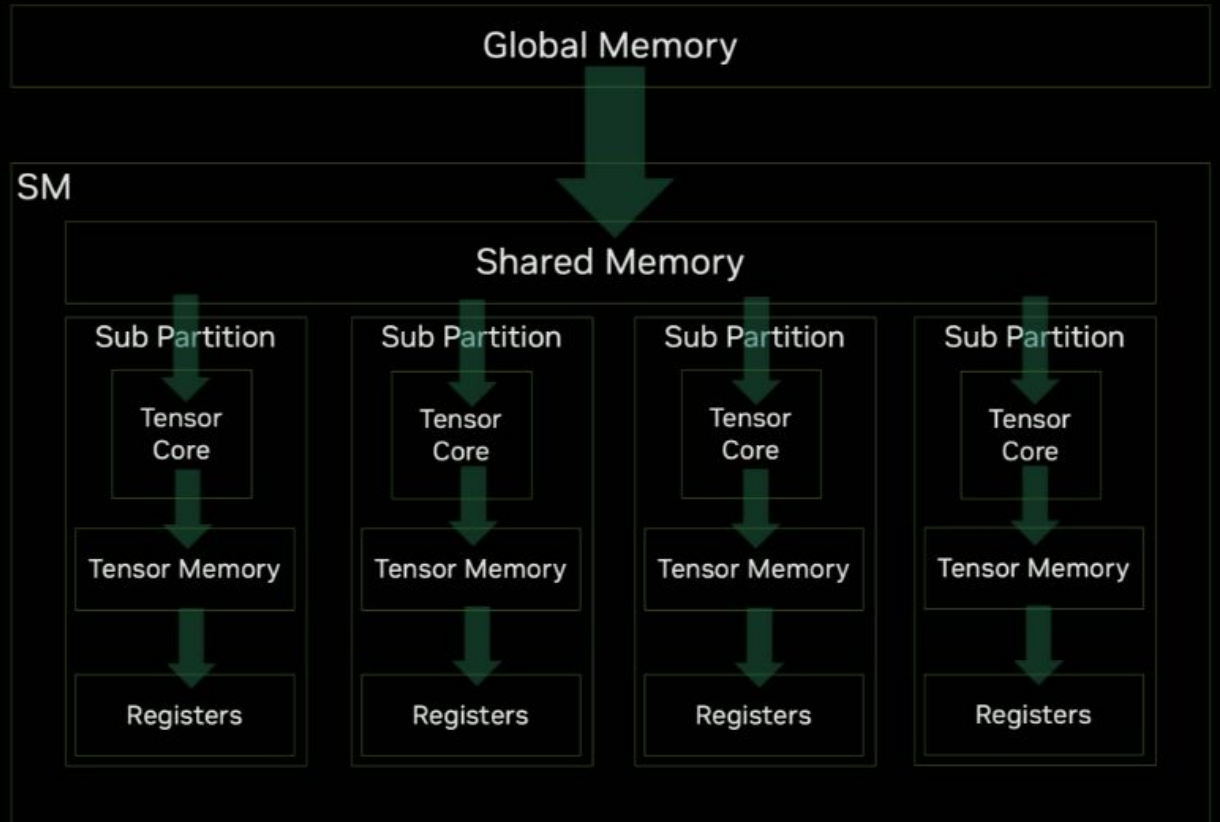
- Requires Cluster launch
- Accumulator D Matrix in TMEM, accessed locally
- Operand A Matrix in TMEM or SMEM, accessed locally
- Operand B Matrix in SMEM, accessed across **pair**
- A thread submits instructions to local Tensor Core
- … which controls the paired Tensor Core





Blackwell Architecture

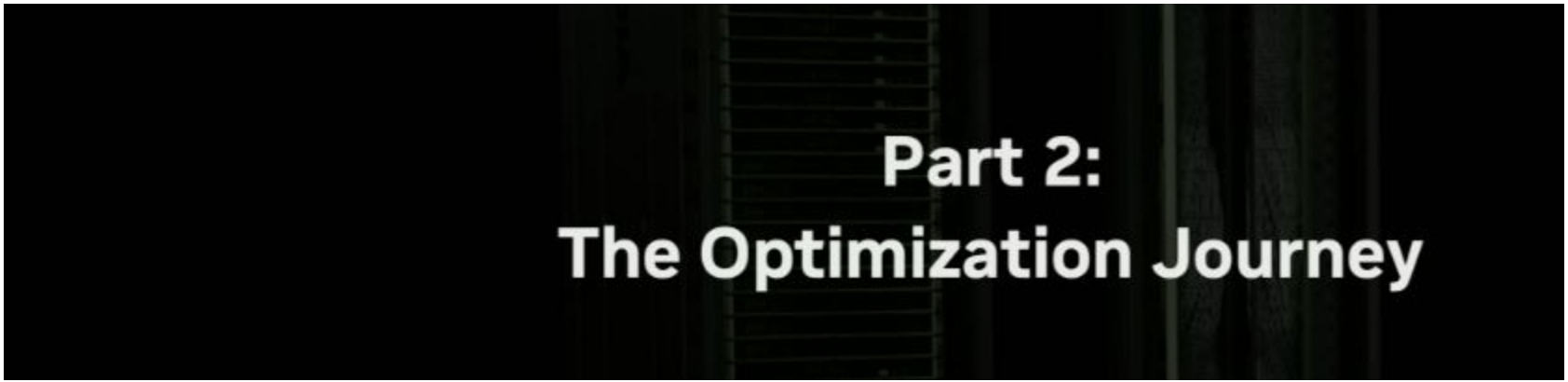
Blackwell Tensor Core Programming



Warp 0: load operands A and B into shared memory

Warp 1: Issue MMA operation. Tensor core pulls operands from shared Memory and stores output to Tensor memory

Warp 4-7: Load local TMEM tile to registers for epilogue.


Part 2:
The Optimization Journey

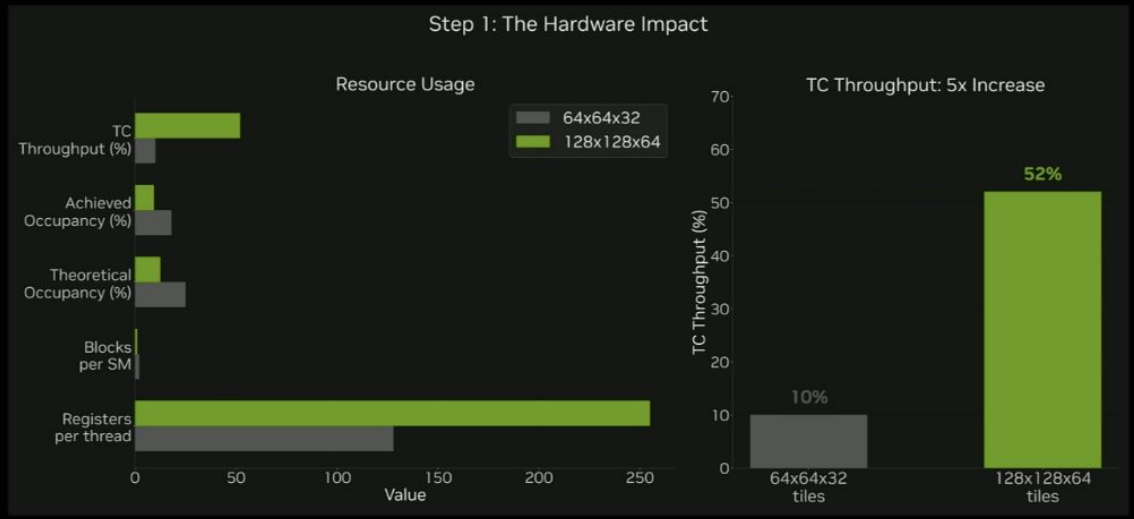
Step 1: Larger Tiles — 3.2× Speedup

The single most impactful optimization is a one-line change

- Change tile sizes from **64×64×32** to **128×128×64**
- No other code changes needed — kernel logic is identical
- **8× more FLOPs** per MMA tile (128×128×64 vs 64×64×32)
- **Grid shrinks** from ~35K to ~9.5K blocks, more work per block

Step 1: The Hardware Impact

Despite halving occupancy, tensor core throughput increases 5×

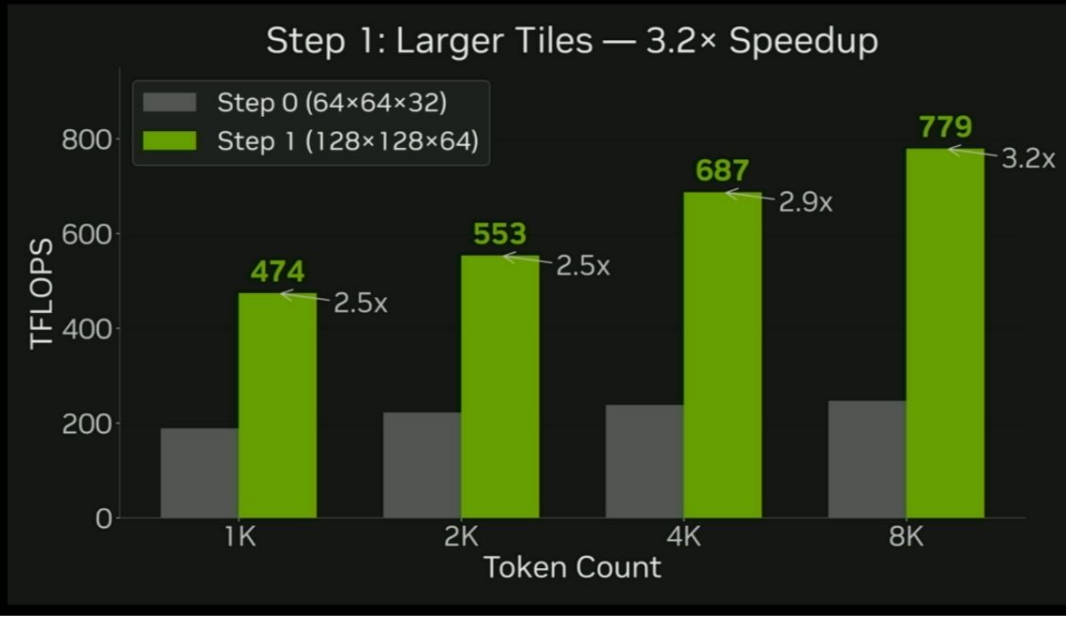


- Registers double: 128 → **255 per thread** — full register file used
- **TMEM: 16 KB → 64 KB** — baseline wastes 75% of tensor memory capacity
- Occupancy drops 25% → 12.5% — but **TC throughput jumps 10% → 52%**
- TMEM utilization: **37.8% → 41.3%** — larger tiles keep tensor cores fed
- Ref: Volkov, **Better Performance at Lower Occupancy** (GTC 2010)

Step 1 Results: 3.2× Speedup

Tile size is the foundation of GPU kernel performance — get this right first

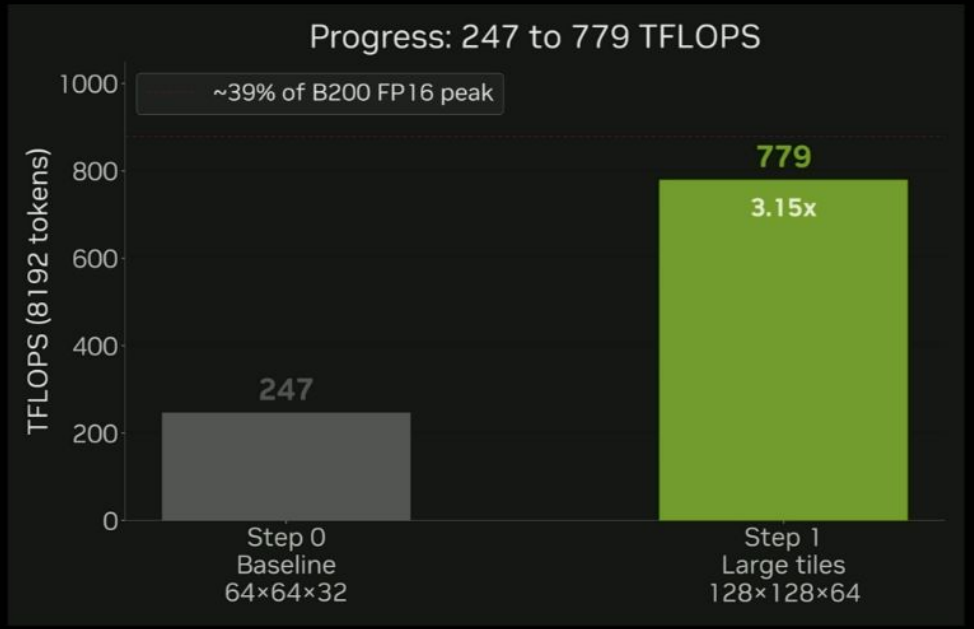
- A **3.2× speedup** from a single tile-size change
- All token counts see **>2.5× improvement**
- Tensor core throughput jumps from 10% to 52%
- Classic GPU lesson: **feeding the tensor cores matters more than occupancy**



Progress So Far

Step 0 → Step 1: 3.2× from tile size alone

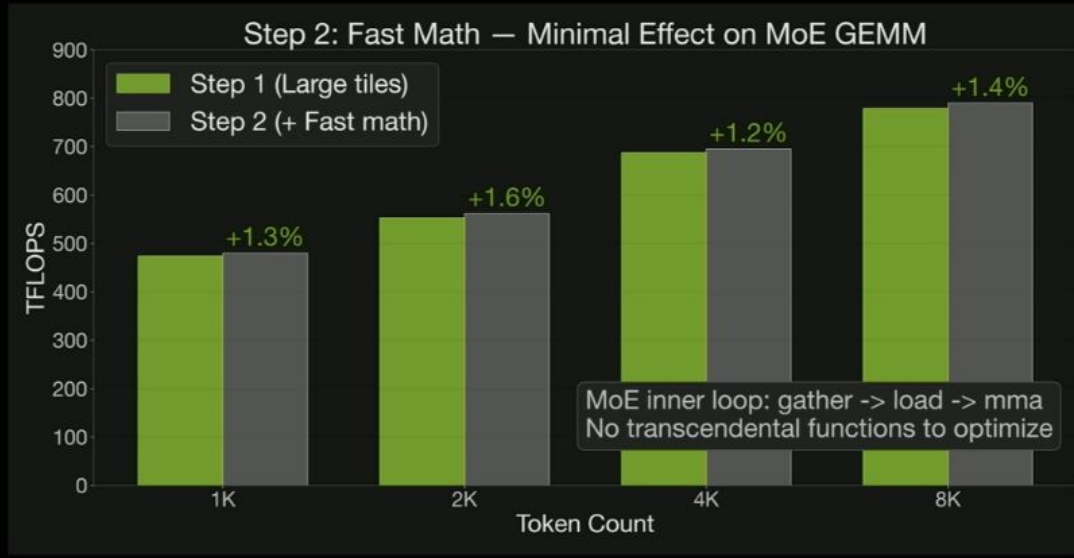
- **247 → 779 TFLOPS** at 8K tokens — tile size is the dominant factor
- One change, one huge payoff — now let's see what else we can squeeze out



Step 2: Fast Math — Not a Real Optimization for MoE

cuTile compiler always enables -fmad, -ftz, -use_fast_math — no user toggle

- cuTile **always** passes fast math flags to ptxas — not a user decision
- **fmad=true**: FMA fusion — already used by ct.mma tensor core path
- **ftz=true**: flush denormals — irrelevant for FP16 MoE workload
- **use_fast_math**: approximate exp2, rsqrt — MoE has no transcendentals
- **Flash Attention** benefits hugely (exp2, softmax) — **MoE does not**
- Result: **~1% change** — confirms fast math is a non-factor for GEMM-bound kernels



Step 3: Three Optimizations Combined

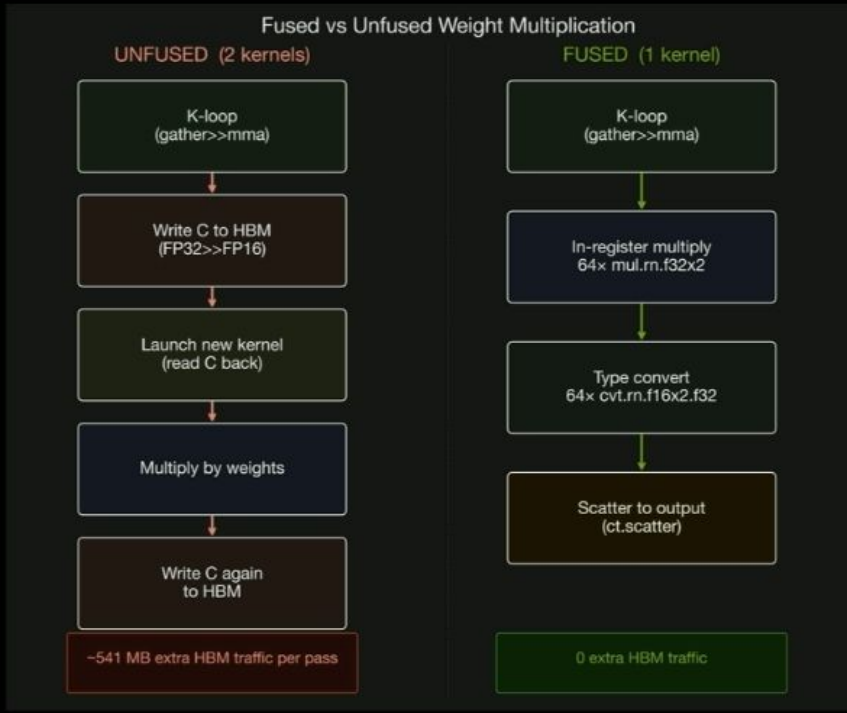
Autotuning + fused weight multiplication + latency hints

- **Fused weight multiplication** — router weights applied in-register, no separate pass
- **Latency hints** — Tell the compiler to pipeline memory loads deeper
- **GROUP_SIZE_M swizzle** — Reorder blocks for B-tile sharing
- **Autotuning** — Let the system benchmark multiple configurations
- Combined effect: +3% to +13% depending on token count

Fused Weight Multiplication

Multiply by router weights in-register — saves ~541 MB of memory traffic per pass

- After K-loop: multiply accumulator by router weight — **in-register**
- **Without fusion:** write C → new kernel → multiply → write again
- **With fusion:** 64× mul.rn.f32x2 + 64× cvt.rn.f16x2.f32 in epilogue
- Saves **~541 MB** of HBM round-trip traffic per MoE pass



Latency Hints for Deeper Pipelining

We found latency=8 through experimentation — it controls pipeline depth

```
for k in range(0, ct.cdiv(K, TILE_SIZE_K)):
    # Deeper pipeline: compiler can prefetch further ahead
    a = ct.gather(a_ptr, a_indices, padding_value=0, latency=8)

    b = ct.load(
        b_ptr,
        index=(off_experts, k, bid_n),
        shape=(1, TILE_SIZE_K, TILE_SIZE_N),
        order=(0, 2, 1),
        latency=8,
    )

# Result: compiler allocates deeper shared memory buffers
#   A-matrix: 2-stage double buffer  (8x cp.async.cg per iteration)
#   B-matrix: 3-stage triple buffer  (1x cp.async.bulk.tensor.3d)
```

Why Not Just Buffer Deeper? (latency=10)

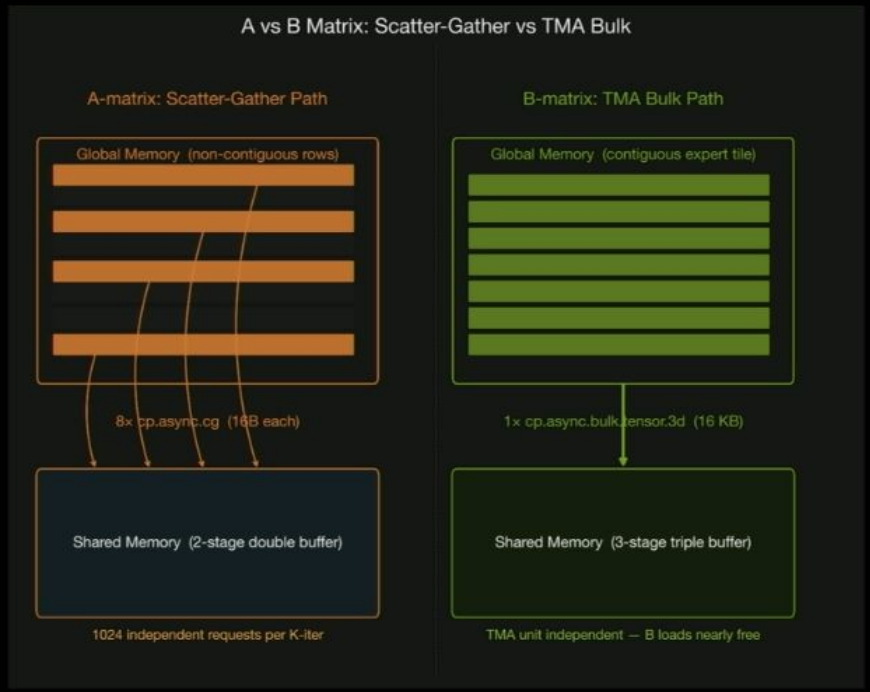
Triple buffer is already the sweet spot — deeper makes it worse

- Hypothesis: increase B pipeline from 3-stage to 4+ stages
- **latency=8 → latency=10:** compiler allocated **2× shared memory** (112 KB → 230 KB)
- **Result: 25% slower** — extra SMEM consumed resources without hiding more latency
- Root cause: **TMA throughput is the ceiling**, not pipeline depth
- 660K NANOSLEEP waits on B-data — TMA can't deliver bytes any faster
- Triple buffer is optimal: hides what it can, deeper just wastes SMEM

The A vs B Pipeline Asymmetry

MoE's core challenge: scattered A-loads vs contiguous B-loads

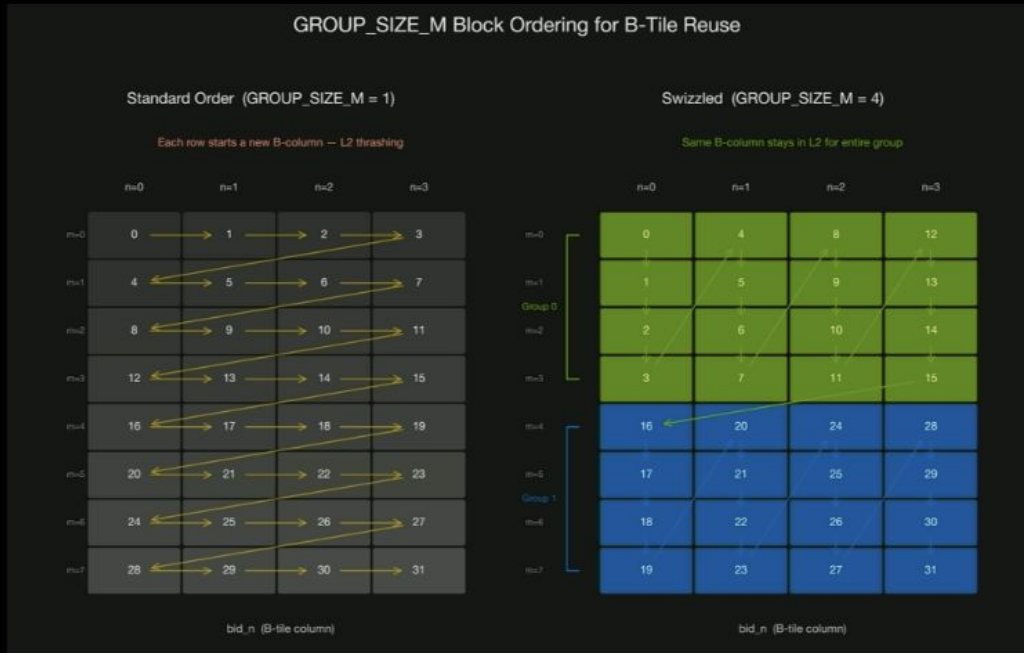
- **A-matrix:** 8× cp.async.cg per K-iteration — 16B each, non-contiguous
- **B-matrix:** 1× cp.async.bulk.tensor.3d per K-iteration — 16 KB bulk TMA
- A uses a 2-stage double buffer; B uses a 3-stage triple buffer
- TMA unit operates independently of compute — B loads nearly free



GROUP_SIZE_M Block Reordering

Group adjacent M-tiles so they share B-data — fewer redundant HBM loads

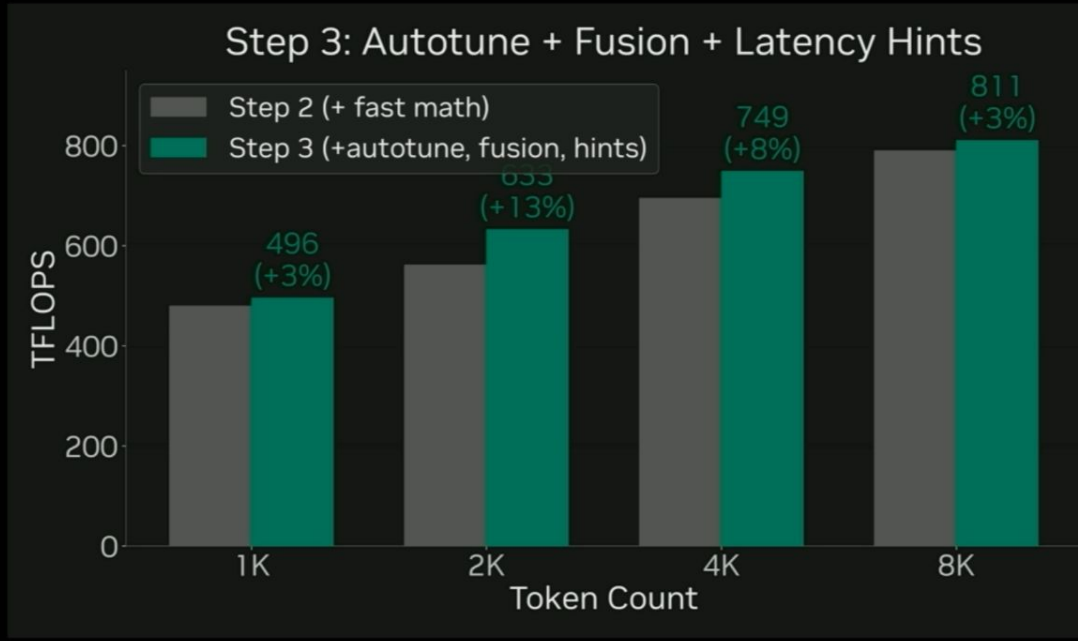
- Standard order: each block loads its B-tile independently from HBM
- **Swizzled:** GROUP_SIZE_M=4 groups 4 adjacent M-tiles that share the same B column
- Shared B-tiles stay warm in L2 while the group executes — reduces HBM traffic
- Biggest gains at **small token counts** (+8.6% at 2K tokens)



Step 3 Results

Modest but meaningful gains, especially at smaller token counts

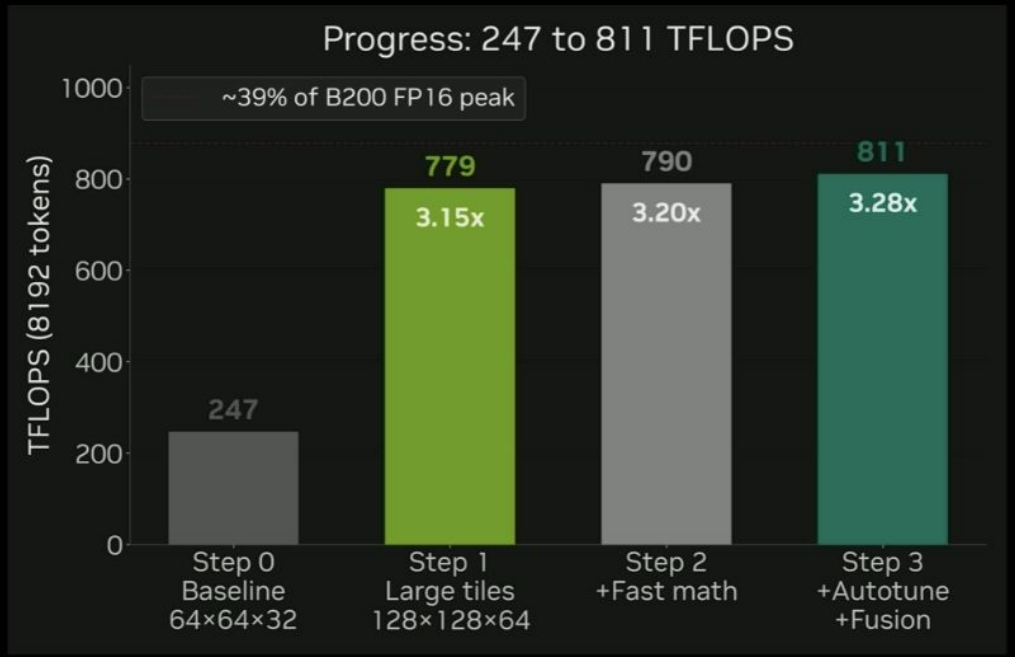
- Combined effect: **+3% to +13%** depending on token count
- Largest gains at **smaller inputs** (1K–2K tokens)
- Autotuner discovers **occupancy=2, GROUP_SIZE_M=4** for smaller inputs
- 2-CTA mode correctly rejected by autotuner



Progress So Far

Step 0 → Step 3: 3.28× with autotuning, fusion, and swizzle

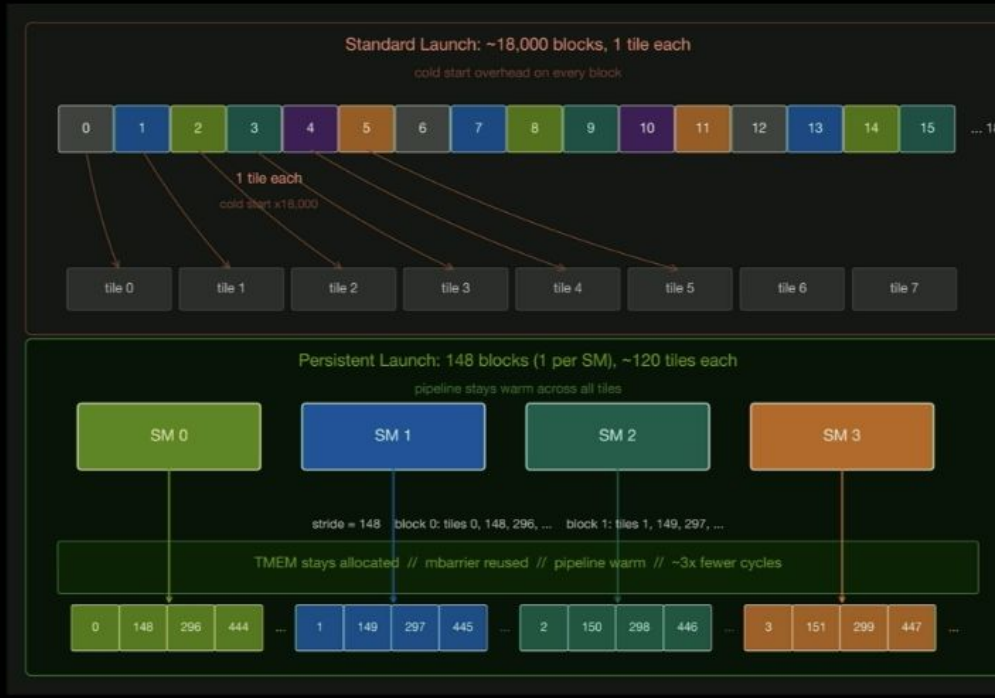
- **247 → 811 TFLOPS** at 8K tokens — steady gains from three combined optimizations
- One more step to go — persistent kernels will push past 850 TFLOPS



Step 4: Persistent Kernel — Biggest Gain After Tile Size

Launch fewer blocks, have each process multiple tiles

- Standard: one block per tile = ~18,000 blocks, each starts cold
- Persistent: **148 blocks** (one per SM), each processes **~120 tiles** at 8K tokens
- Each block strides: bid, bid+148, bid+296, ... covering all ~18,000 tiles
- Uses ct.num_blocks(0) to query actual grid size at runtime
- **+7.6%** improvement at 8K tokens (834 → 898 TFLOPS)



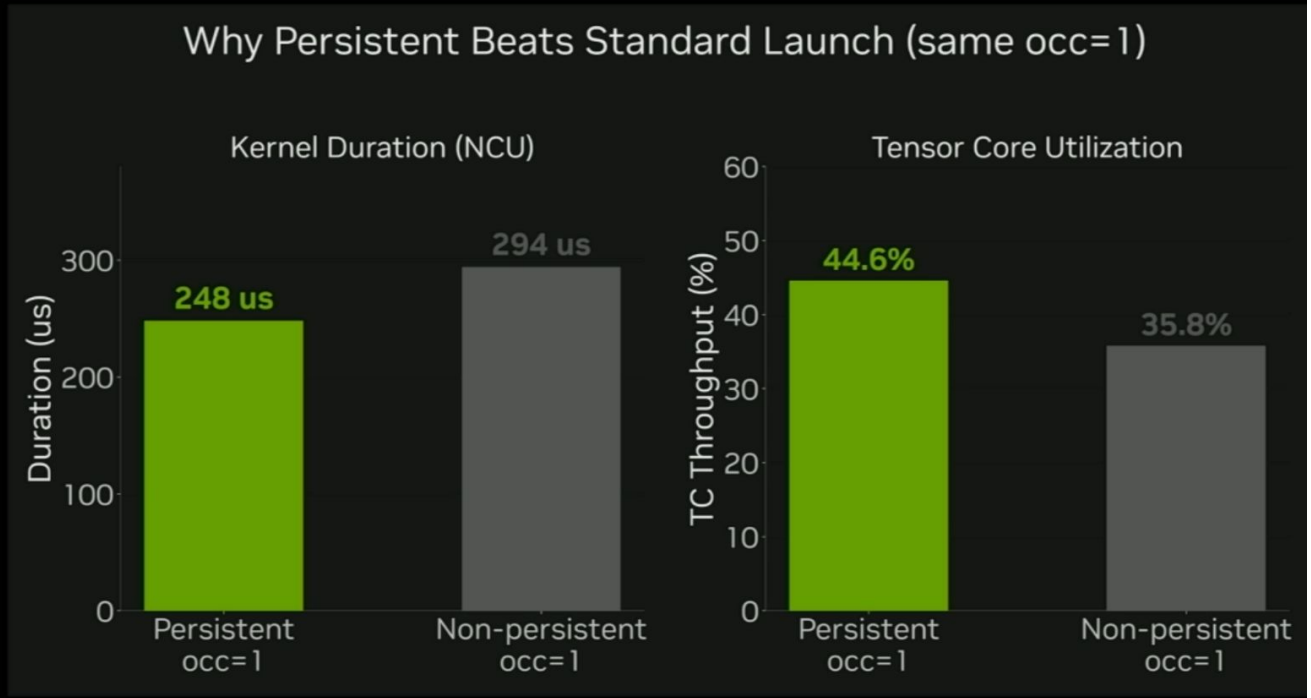
Persistent Kernel Code

A stride loop where each block processes many tiles, staying warm on the SM

```
bid = ct.bid(axis=0)
num_bid_m = ct.cdiv(EM, TILE_SIZE_M)
num_bid_n = ct.cdiv(N, TILE_SIZE_N)
upper_bound = num_bid_m * num_bid_n
num_tile_blocks = ct.num_blocks(0)  # Actual grid size (= NUM_SMs)

for current_bid in range(bid, upper_bound, num_tile_blocks):
    # Swizzled block ordering (same as step 3)
    bid_m, bid_n = swizzle(current_bid, GROUP_SIZE_M, ...)

    if bid_m * TILE_SIZE_M < num_tokens_post_padded:
        # ... same tile computation as before ...
        accumulator = ct.full(...)
        for k in range(0, ct.cdiv(K, TILE_SIZE_K)):
            a = ct.gather(a_ptr, ..., latency=8)
            b = ct.load(b_ptr, ..., latency=8)
            accumulator = ct.mma(a, b, accumulator)
        ct.scatter(c_ptr, c_offset, accumulator)
```

Why Persistent Beats Standard Launch

Same tile size, same math — persistence alone gives +25% TC utilization

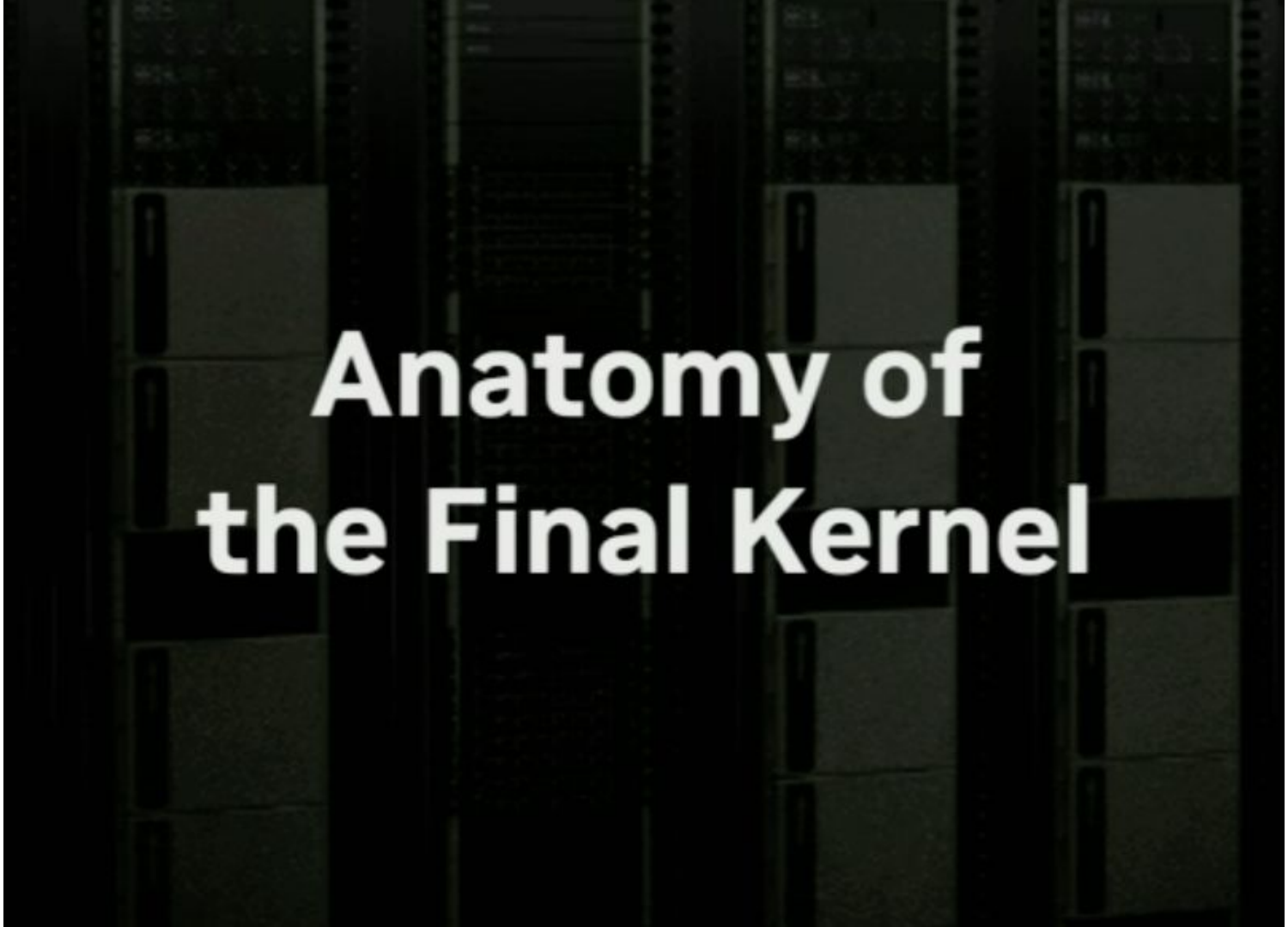


- **Persistent (148 blocks):** each block processes ~120 tiles, pipeline stays warm
- **Standard (~18,000 blocks):** cold start per block, pipeline drains between tiles
- Result: **248 us vs 294 us** (-16%), TC throughput **44.6% vs 35.8%** (+25%)

Why the Stride Loop Works

A block that stays on the SM amortizes every startup cost

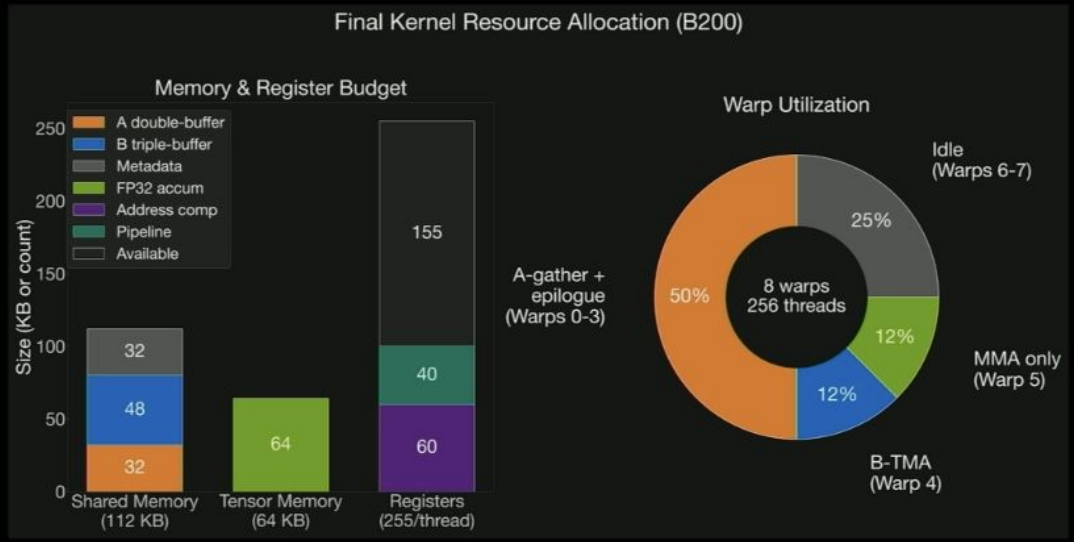
- **No cold starts:** TMEM, mbarriers, and pipeline state survive across ~120 tiles
- **Fewer blocks, more work each:** 148 blocks vs ~18,000 — scheduler overhead vanishes
- **Better cache reuse:** L2 hit rate jumps 18.2% → 25.3% — adjacent strides share B-tiles
- **Pipeline stays full:** prefetch of tile N+1 overlaps compute of tile N — no drain/refill
- **Net effect: +7.6% TFLOPS** (834 → 898 at 8K) — the cheapest optimization in the stack


Anatomy of
the Final Kernel

Final Kernel: Resource Allocation

The persistent step 4 kernel on B200

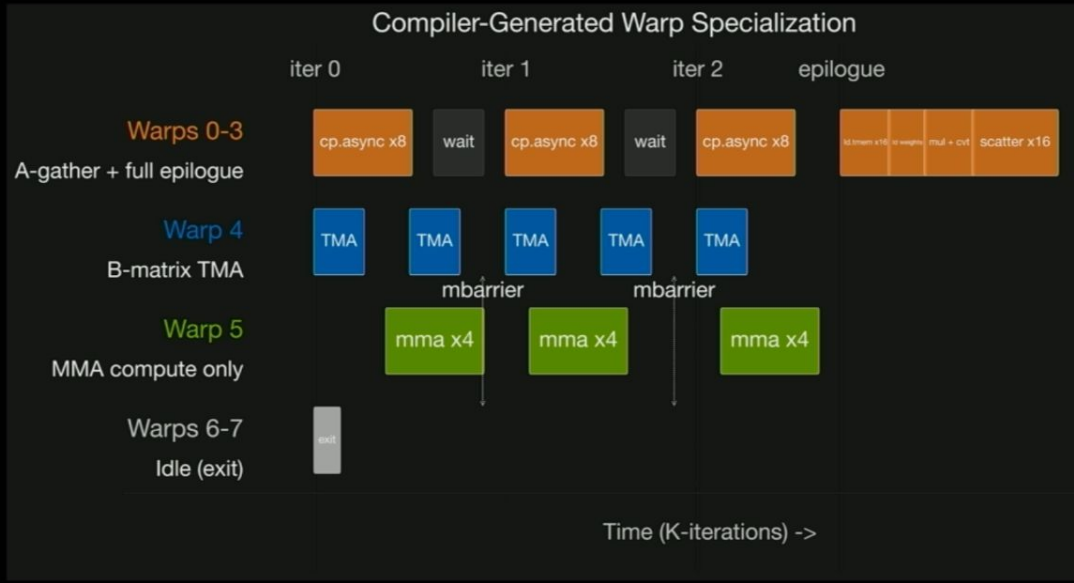
- **148 persistent blocks** — 148 SMs, ~120 tiles/block at 8K tokens, 256 threads
- **Shared memory:** 112 KB — A double-buffer (32 KB), B triple-buffer (48 KB)
- **Tensor memory:** 64 KB — 128×128 FP32 accumulator
- **Registers:** 255 per thread (maximum on SM100)



Compiler-Generated Warp Specialization

cuTile compiler automatically assigns different roles to different warps

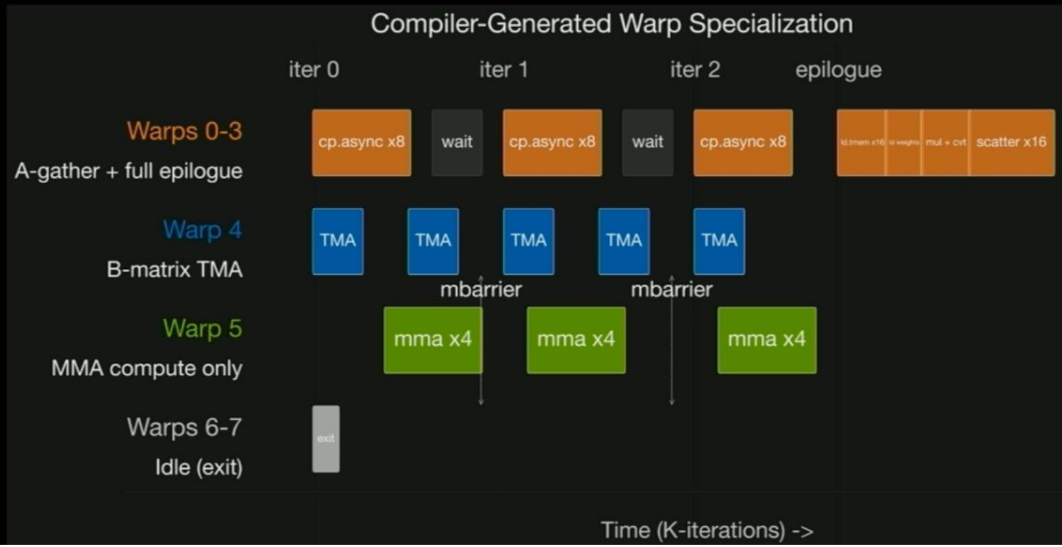
- **Warps 0–3:** A-gather (cp.async x8) + **full epilogue** (tcgen05.ld + mul + cvt + scatter)
- **Warp 4:** B-matrix TMA load (cp.async.bulk.tensor.3d, 16 KB per iter)
- **Warp 5:** MMA compute only (tcgen05.mma x4, 4 sub-tiles per K-iter)
- **Warps 6–7:** Idle (exit immediately) — 25% thread capacity wasted
- Epilogue on warps 0–3: read TMEM, load weights, mul in RF, cvt FP32→FP16, scatter
- 264 instructions between last MMA and first store — **TC idle during epilogue**



Per K-Iteration Instruction Mix

What happens 32 times per tile (K=2048 / TILE_K=64 = 32 iterations)

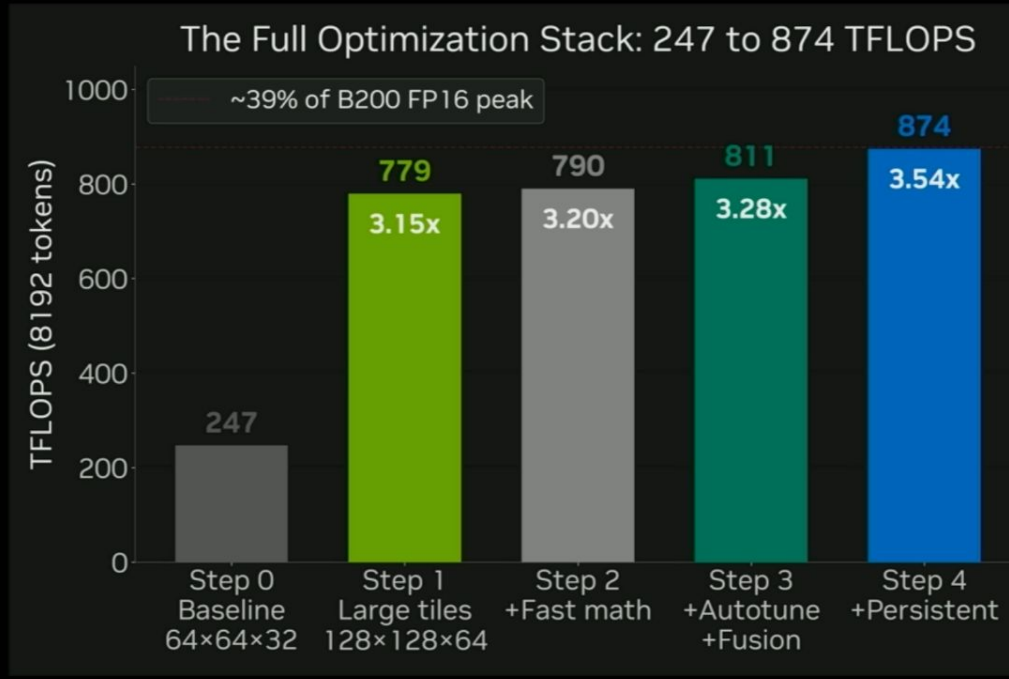
- **Memory ops dominate:** 9× memory instructions vs 4× compute per iteration
- **A-gather** is the most instruction-heavy: 8× cp.async.cg per K-iteration
- Per tile total: **288 memory + 128 compute + 224 sync** instructions
- The kernel is fundamentally **memory-bound** by A-matrix gather



The Full Optimization Stack

From 247 to 874 TFLOPS — 3.54× speedup

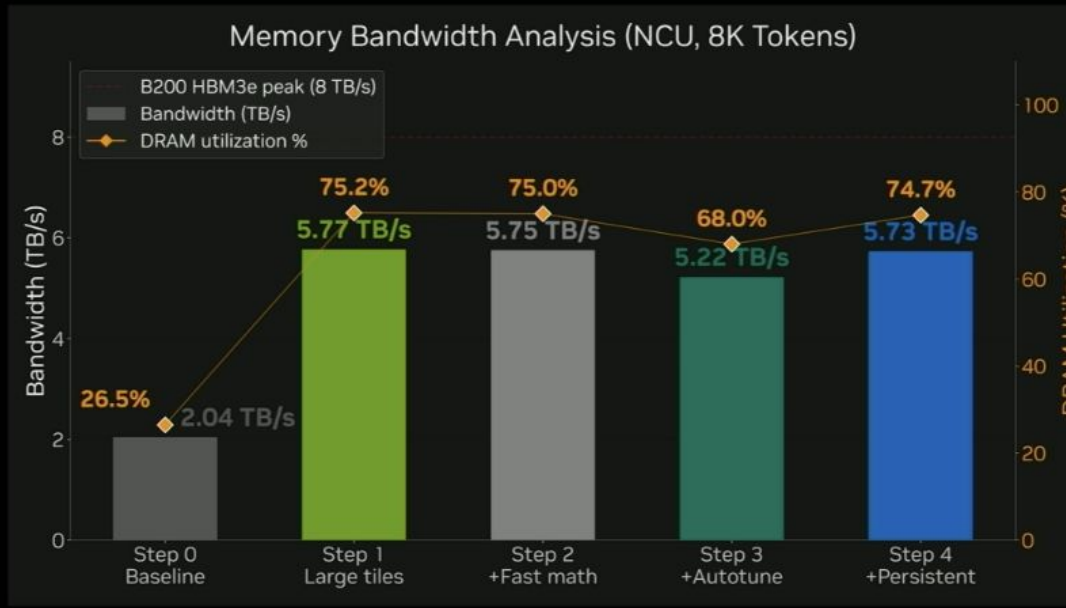
- **Tile size** is the foundation (3.2× impact) — keep tensor cores fed
- **Persistent kernel** is the second-biggest gain (+8–10%)
- **Fast math** has marginal effect on GEMM-dominated workloads
- **Autotuning + fusion** helps most at smaller token counts



Bandwidth Analysis

Bars = measured TB/s, crosses = DRAM utilization % (NCU, 8K tokens)

- **Step 0:** 2.04 TB/s (26.5%) — small tiles starve the memory bus
- **Step 1:** 5.77 TB/s (75.2%) — 2.8× BW jump from larger tiles
- **Step 3:** 5.22 TB/s (68%) — lower BW but +8% TFLOPS from doubled occupancy
- **Step 4:** 5.73 TB/s (74.7%) — best overall with highest L2 hit rate (29%)
- **Compute-bound:** ~507 FLOPs/Byte — 79.8% TC + 72% BW efficiency



What We Tried That Didn't Work

Profiling with NCU guided us away from several dead ends

- **2-CTA mode — 5× slower:** ct.gather destroys cp.async pipeline; 40.9M NANOSLEEP stalls
- **Token sorting — no effect:** 4 KB rows too large for L1 locality
- **latency=10 — 25% slower:** 112→230 KB shared memory, no throughput gain
- **TILE_N=256 — no benefit:** Doubles shared memory per tile; autotuner correctly rejects
- These negative results are **valuable** — they reveal true bottlenecks

Key Takeaways

Five lessons from optimizing MoE on Blackwell

- **Tile size is the foundation** (3.2×) — 128×128 tiles keep TCs fed
- **Persistent kernels beat standard launch** (+16%) — warm pipeline eliminates cold starts
- **Profile before optimizing** — NCU caught the 2-CTA trap (5× slower)
- **Autotuning finds what humans miss** — persistent=1, occ=1 is non-obvious but optimal
- **The compiler is your partner** — cuTile auto-specializes warps and pipelines from 10 lines of Python

Get Started with cuTile

Everything you need to write high-performance GPU kernels in Python

- **Install:** pip install cuda-tile (requires CUDA 13.1+, Blackwell GPU)
- **cuTile Python:** github.com/nvidia/cutile-python
- **TileGym:** github.com/NVIDIA/TileGym — examples, autotuner, benchmarks
- **Quickstart:** docs.nvidia.com/cuda/cutile-python/quickstart.html
- **This talk's code:** Complete kernel in TileGym repository

Problem: Memory bandwidth per number-of-SM ratio continues to increase

Maximize Memory Bandwidth on Modern GPUs: Practical Techniques, Patterns, and Worked Examples [S81666]

Background and Motivation

What is a “memory bound” kernel?

- Kernels generally have the form:
  1. Load some input data from global memory
  2. Perform some calculations
  3. Store output data back in global memory

- For a given kernel, it’s often useful to consider:
  1. t_{calc} : how long it takes to perform the calculations
  2. t_{data} : how long it takes to move data to and from the SM

- “Memory Bound”: $t_{\text{calc}} \ll t_{\text{data}}$

- “Compute Bound”: $t_{\text{calc}} \gg t_{\text{data}}$

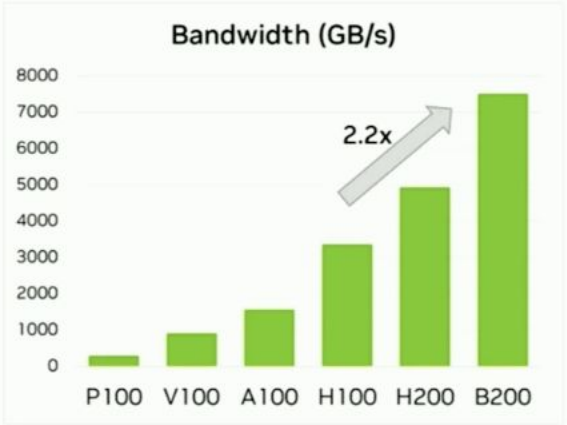
- Memory-bound kernels are **extremely** common
  - Linear Algebra* (Level 1 and Level 2 BLAS, SpMV, matrix factorization …)
  - Common parallel primitives (sort, scan, unique, …)
  - Signal Processing (filtering, convolution, FFT)
  - Physics Simulation (finite difference, finite element, CFD)

* But not dense GEMM!

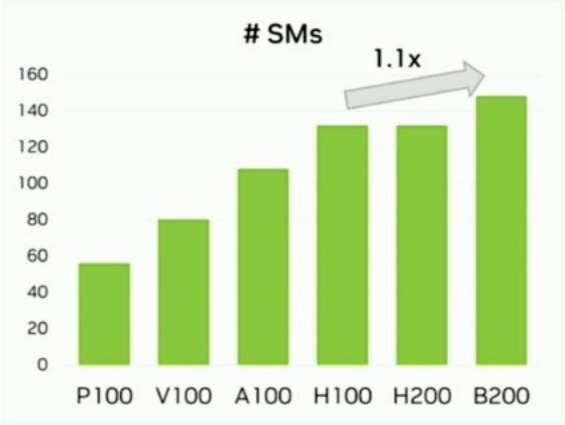
Background and Motivation

Why Should I Care?

- Memory-bound kernels can be tricky to optimize
  - Even simple kernels may underperform
  - Increased bandwidth / SM on modern GPUs



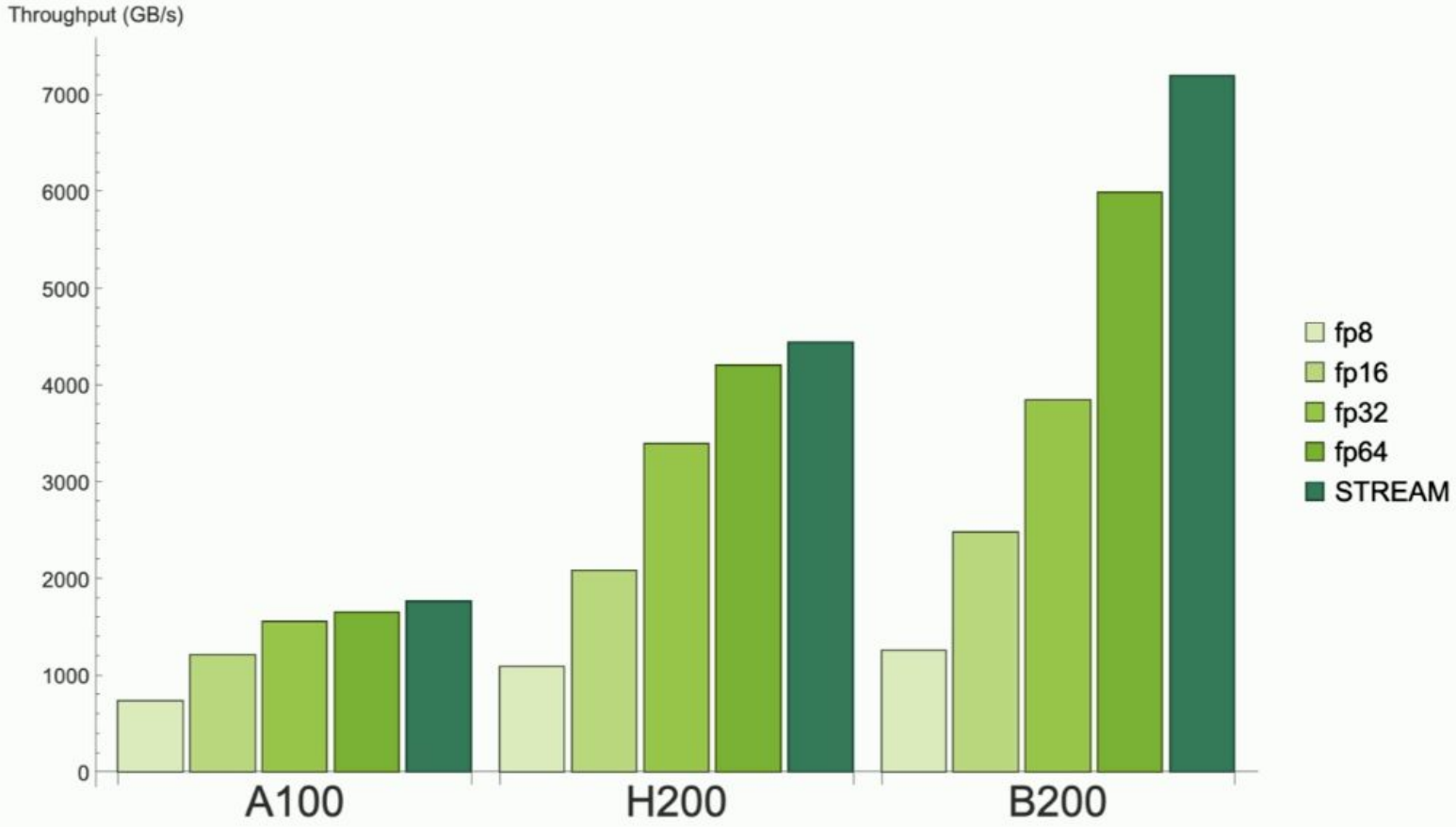
Bandwidth is increasing rapidly



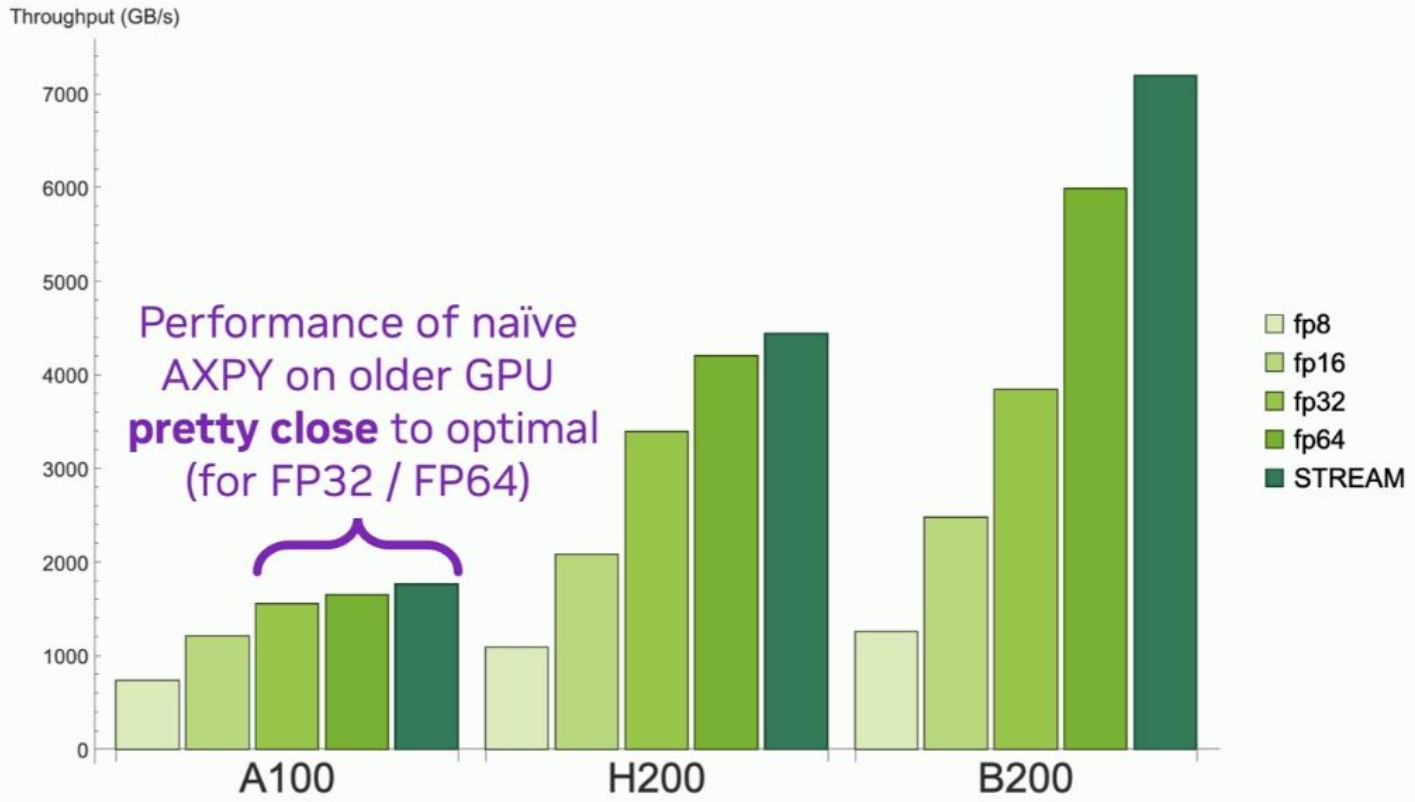
SMs is increasing slowly

Bandwidth per SM is increasing

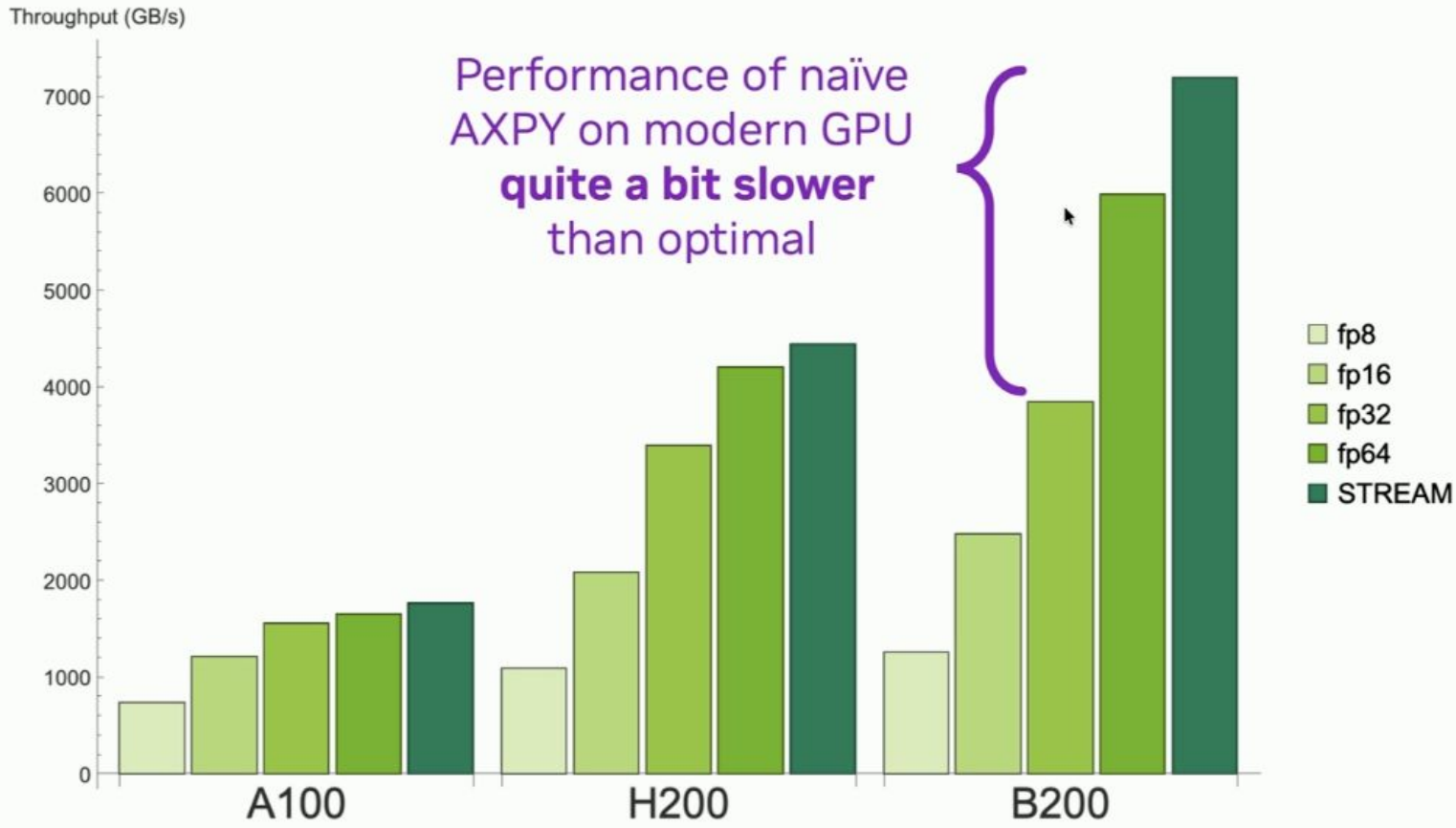
Performance of a simple AXPY Kernel



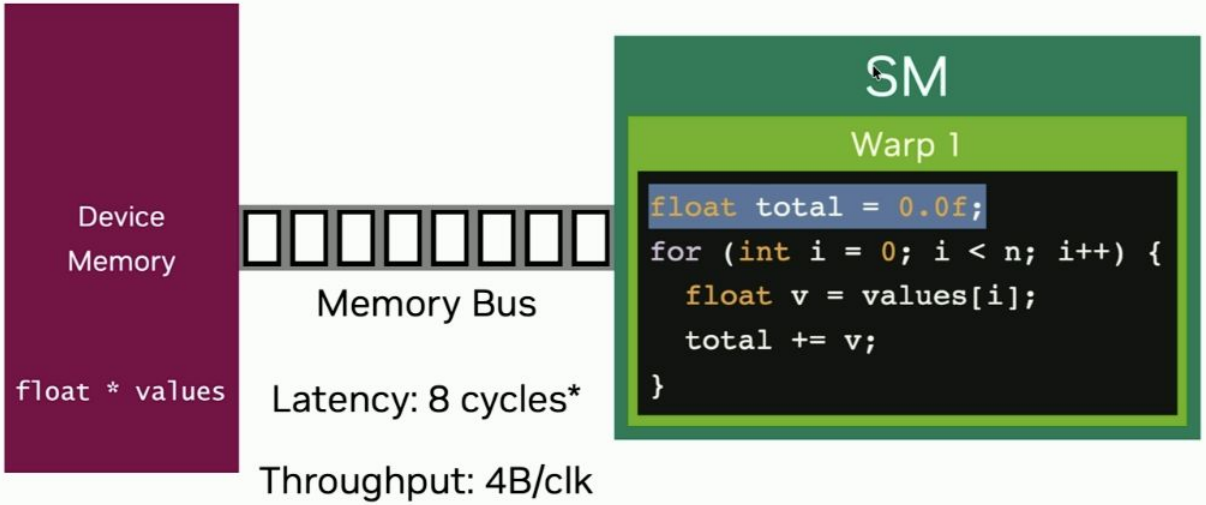
Performance of a simple AXPY Kernel



Performance of a simple AXPY Kernel

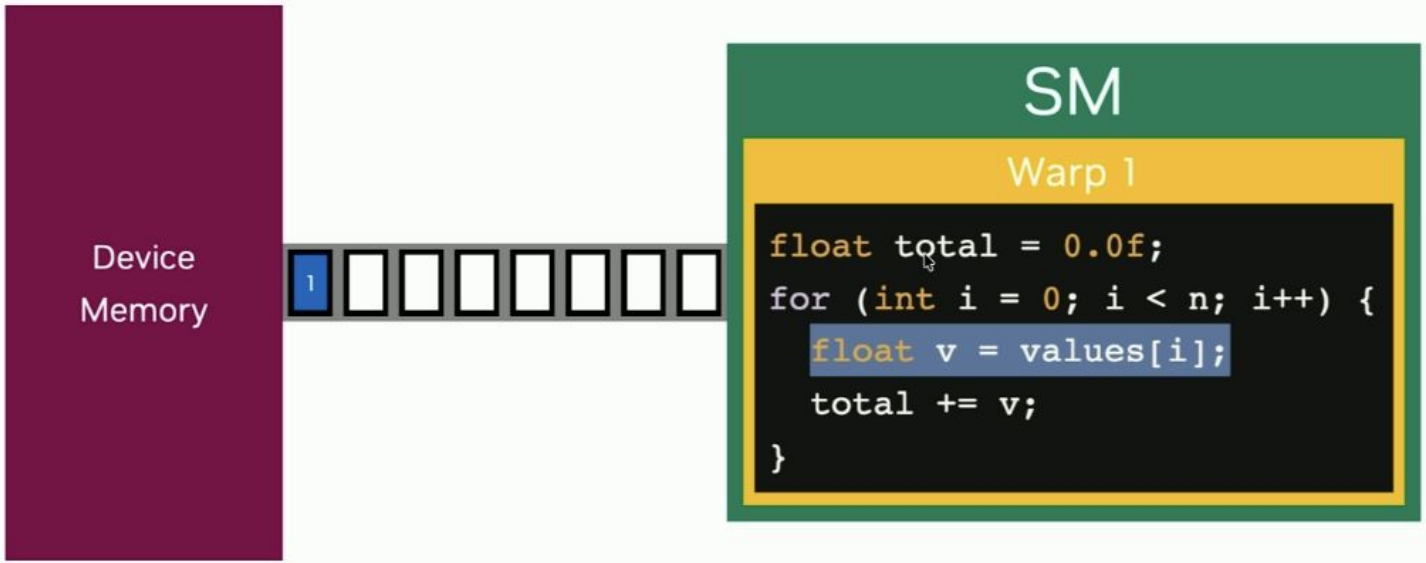


Bytes In Flight Examples



*on actual GPUs, this value is much larger (roughly 500 - 1000 cycles)

Bytes In Flight Examples



Bytes In Flight Examples

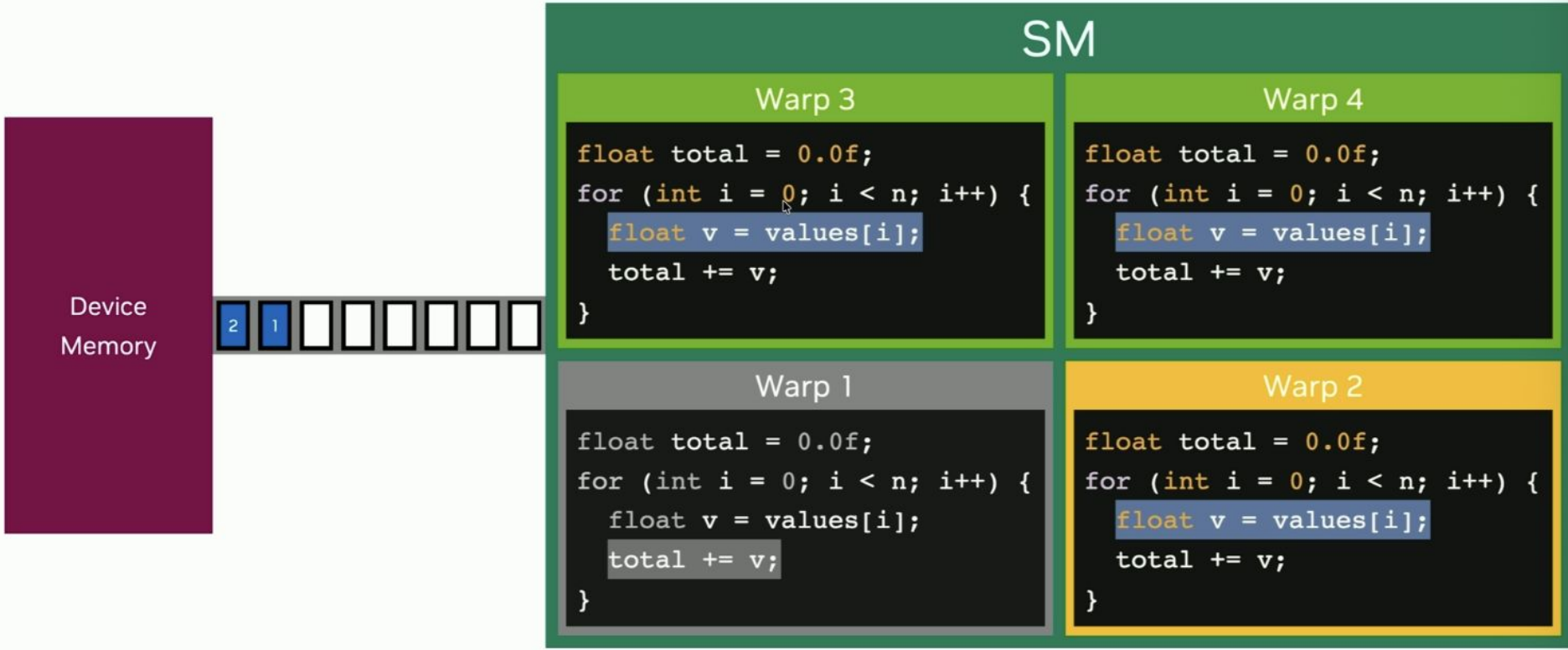
And so on …

This example has **low** bandwidth:

- **Low** occupancy (few warps / SM)
- Memory bus is **almost empty**

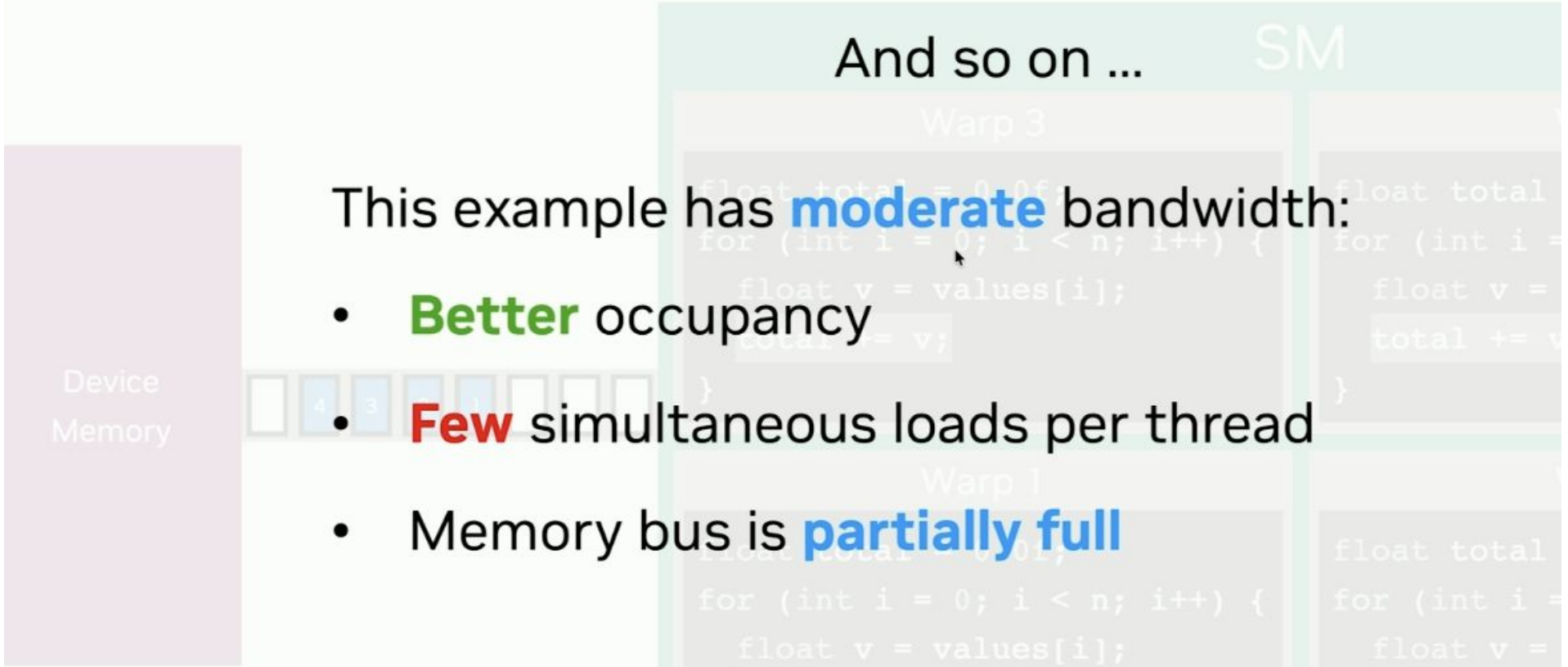
Bytes In Flight Examples

Thread-level Parallelism



Bytes In Flight Examples

Thread-level Parallelism

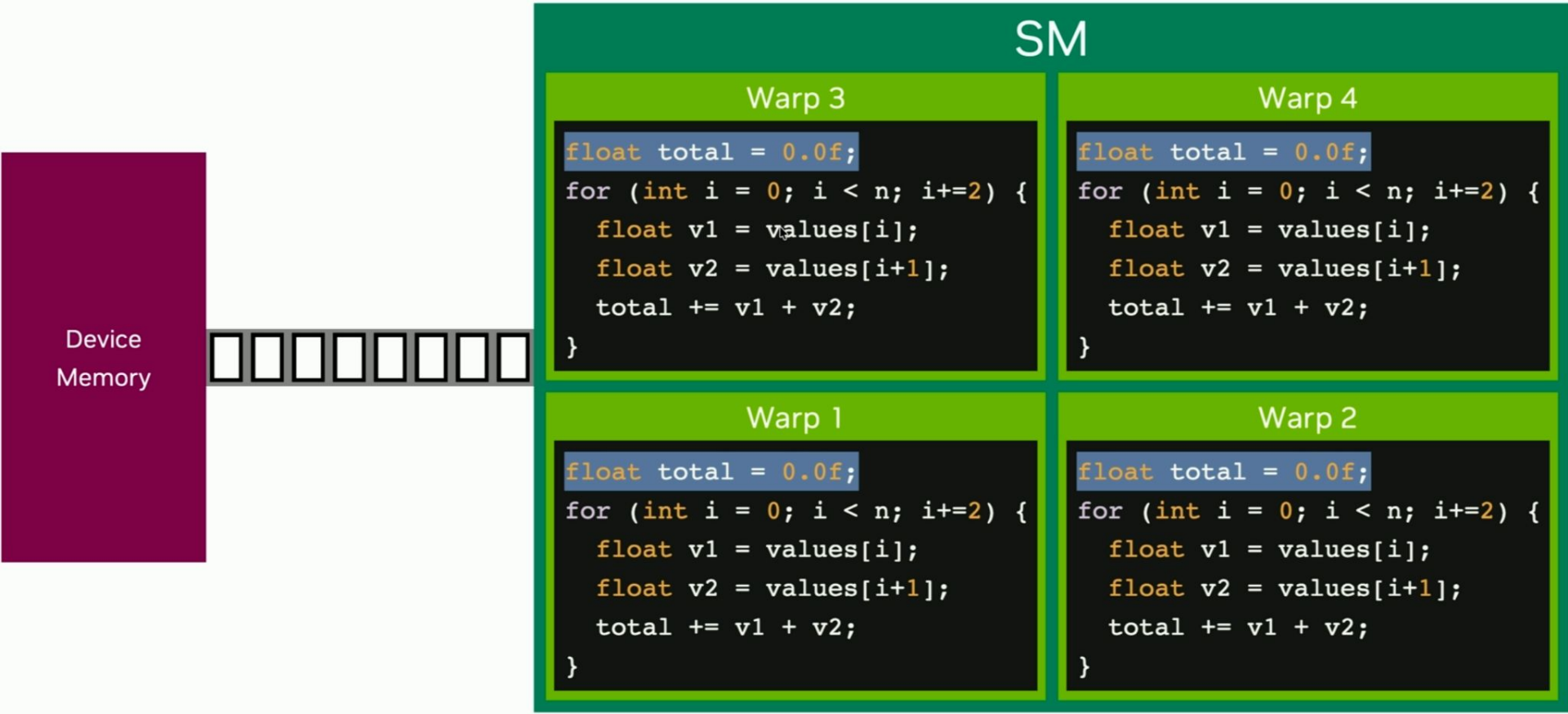


This example has **moderate** bandwidth:

- **Better** occupancy
- **Few** simultaneous loads per thread
- Memory bus is **partially full**

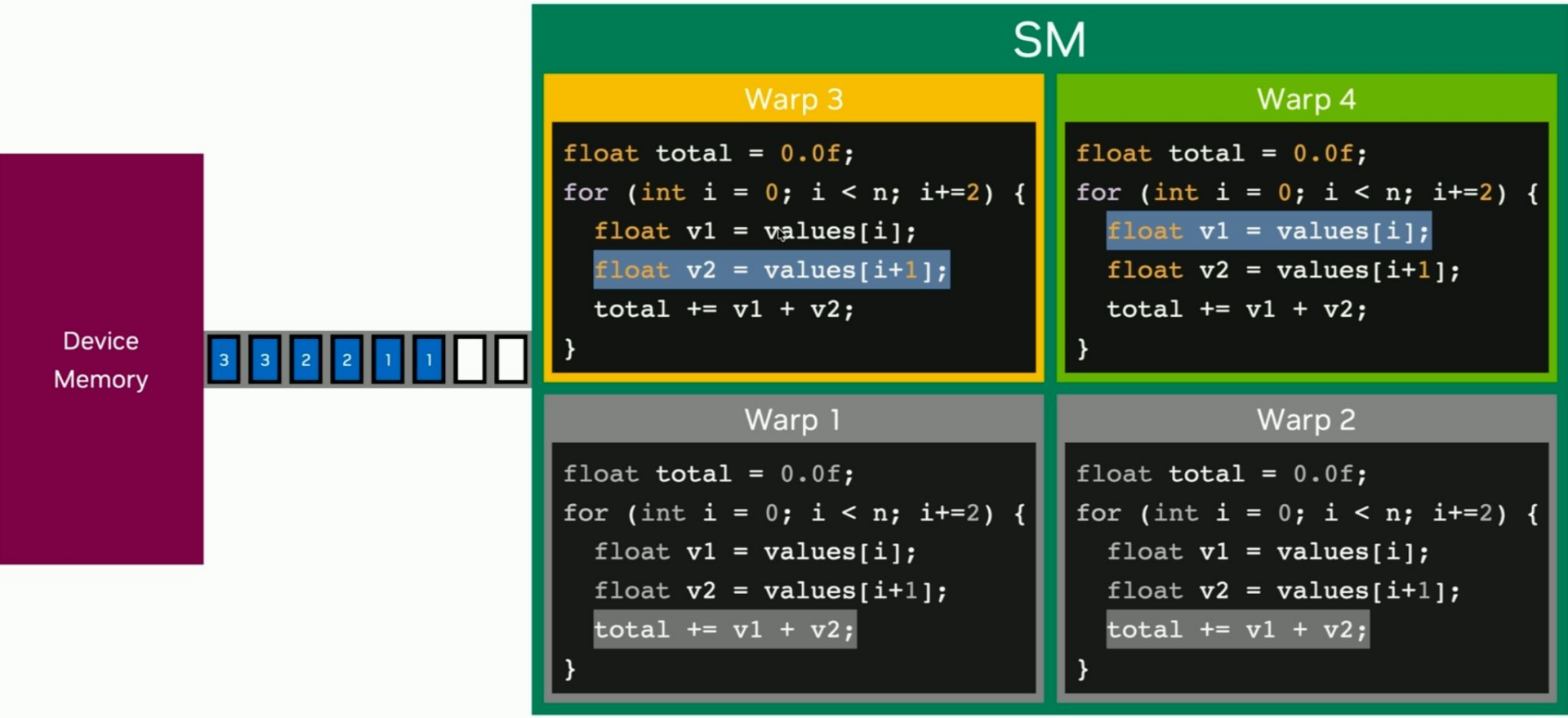
Bytes In Flight Examples

Thread-level and Instruction-Level Parallelism



Bytes In Flight Examples

Thread-level and Instruction-Level Parallelism



Bytes In Flight Examples

Thread-level and Instruction-Level Parallelism

And so on …

This example has **good** bandwidth:

- **Better** occupancy
- **Multiple** simultaneous loads per thread
- Memory bus is **occasionally full**

Bytes In Flight Examples

Recap

- Saturating the memory bus => High throughput
  - Increasing the number of resident warps helps
    - … but our original AXPY kernel already has 100% occupancy
  - Initiating more loads per thread helps
    - Our original AXPY kernel only processed 1 entry / thread
    - **Let's try this idea out on our AXPY kernel**

Process more requests per kernel

AXPY Implementation

Instruction Level Parallelism (ILP)

```
template < int k, typename scalar_t >
__global__ void axpy_ilp(scalar_t a, const scalar_t * x, scalar_t * y, int n) {

    int tid = threadIdx.x + blockIdx.x * blockDim.x;

    scalar_t x_tmp[k];
    scalar_t y_tmp[k];
    for (int i = 0; i < k; i++) {
        int j = tid + i * gridDim.x * blockDim.x;
        if (j < n) {
            x_tmp[i] = x[j];
            y_tmp[i] = y[j];
        }
    }

    for (int i = 0; i < k; i++) {
        int j = tid + i * gridDim.x * blockDim.x;
        if (j < n) {
            y[j] = a * x_tmp[i] + y_tmp[i];
        }
    }
```

Vectorized Loads

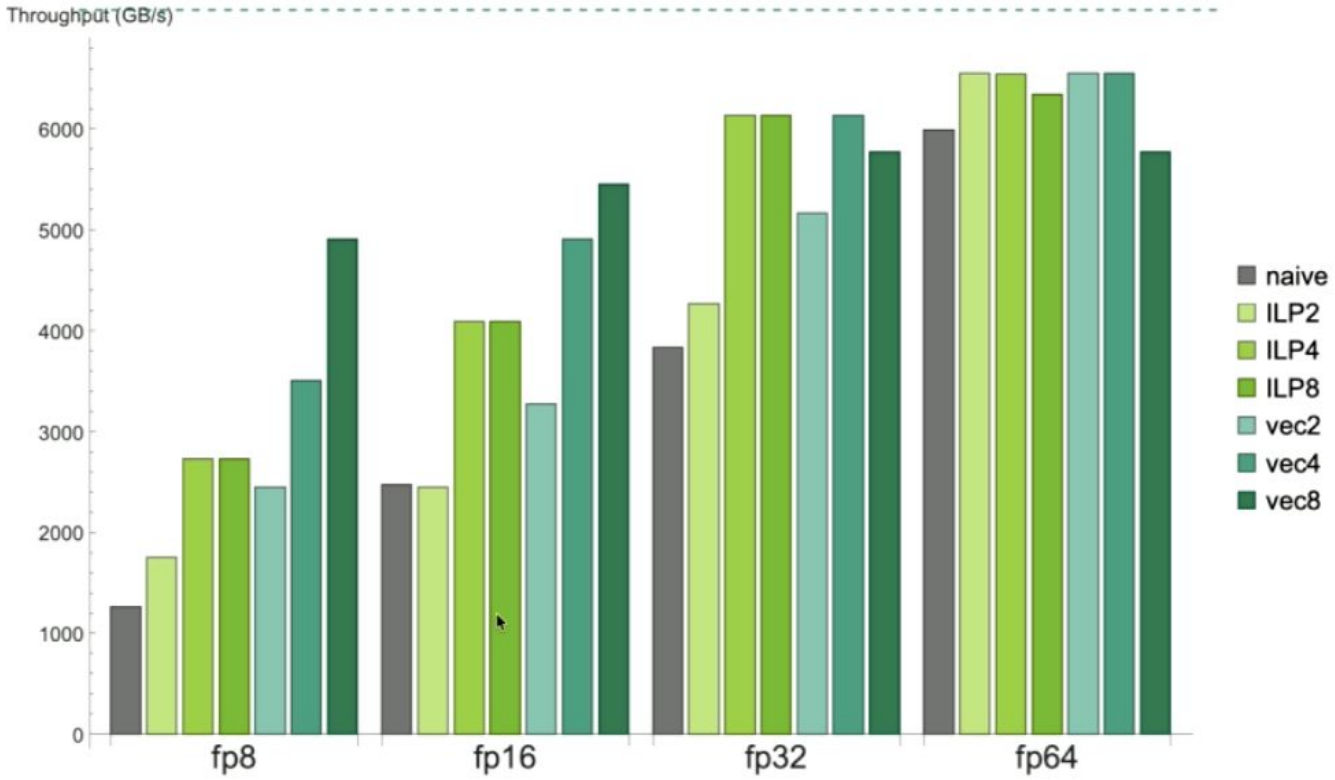
AXPY Implementation

Vectorized Loads

```
template < typename T, int n >
struct alignas(sizeof(T) * n) vec { // N.B. alignment is important!
  T values[n];
};

template < typename T, int n >
vec<T,n> operator+(const vec<T,n> & a, const vec<T,n> & b) {
  // ...
};

////////////////////////////////////////////////////////////////////////////////

template < int k, typename T >
__global__ void axpy_vectorized(T a, const vec<T,k> * x, vec<T,k> * y, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    if (k * tid < n) {
        y[tid] = a * x[tid] + y[tid]; // assumes appropriate array padding
    }
}
```

AXPY Performance

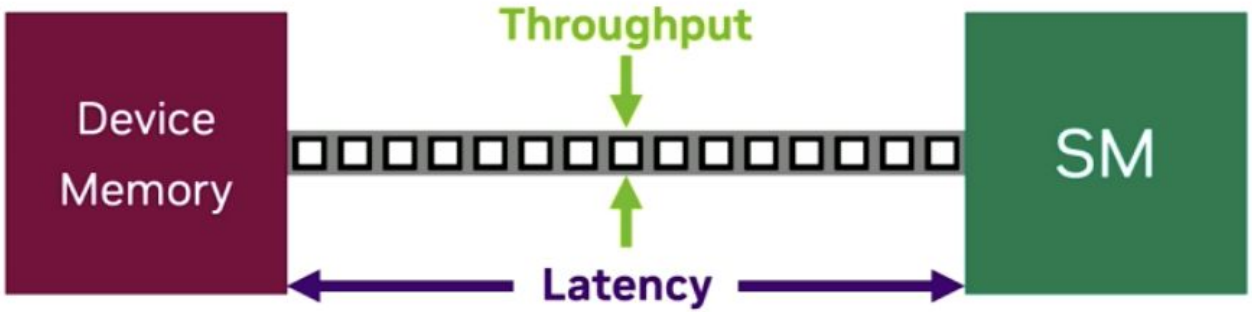
As measured on B200



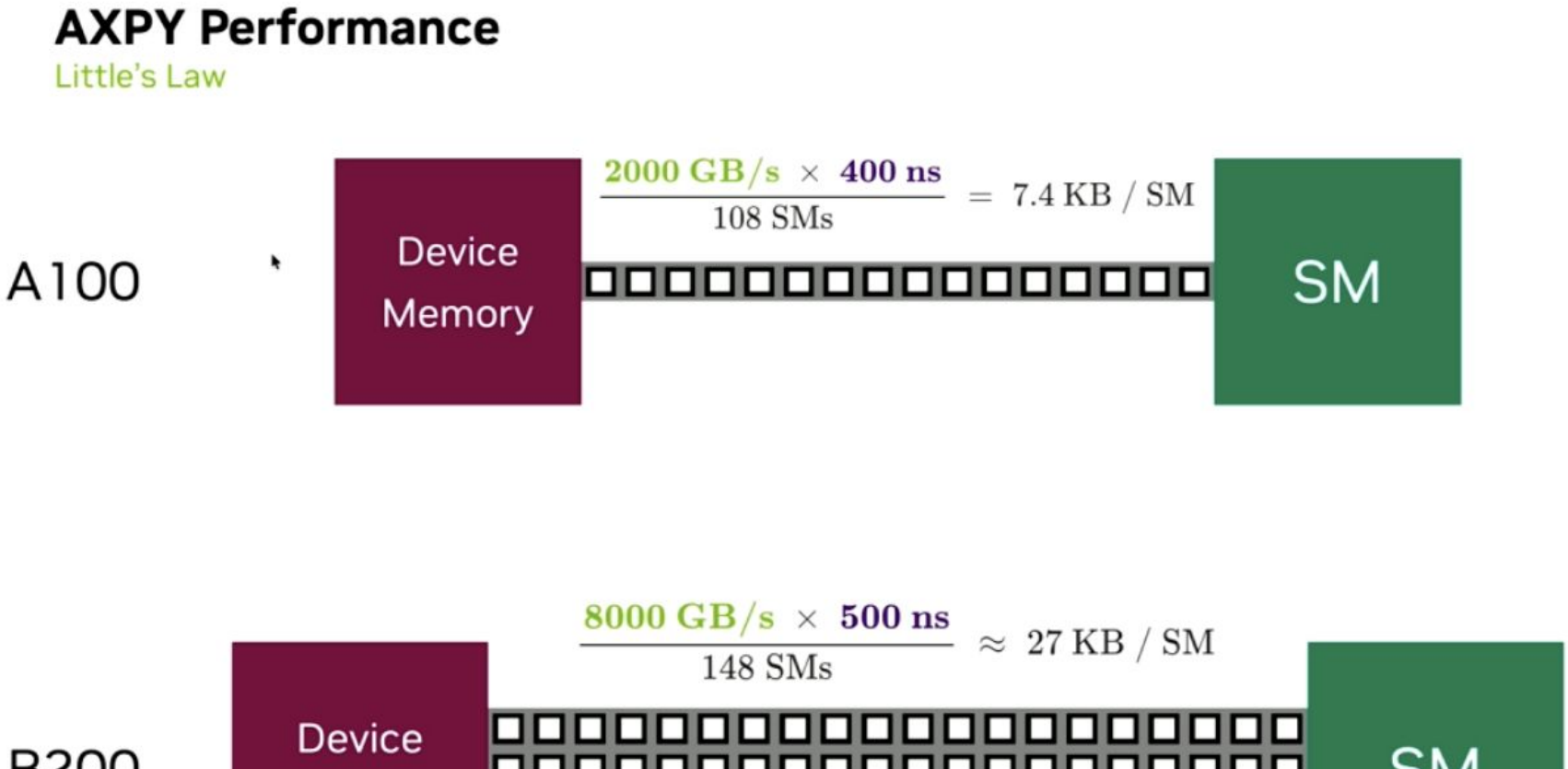
AXPY Performance

Little's Law

$$\textbf{Bytes In Flight} = \textbf{Throughput} \times \textbf{Latency}$$

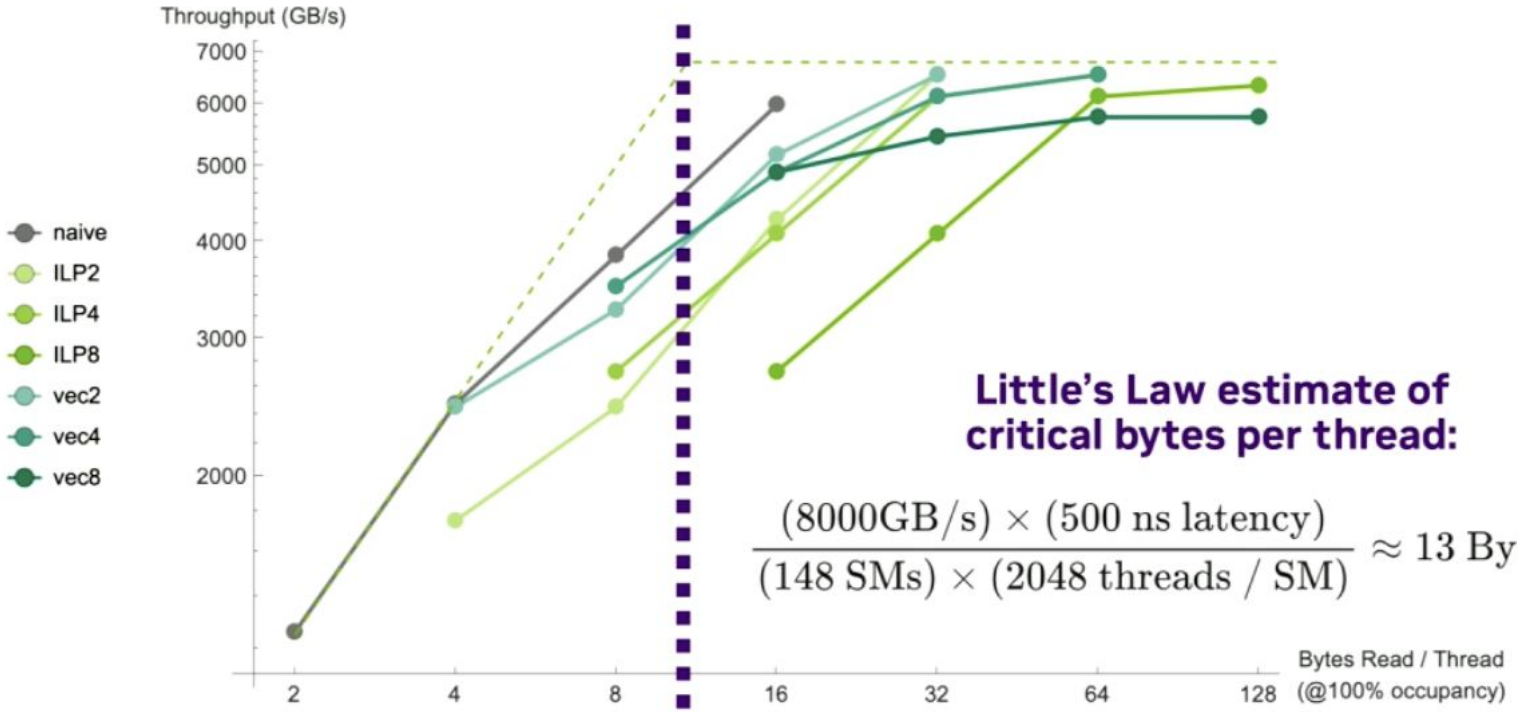


More “slots to fill” on the B200 to saturate the bus



AXPY Performance

Bytes in Flight Roofline model

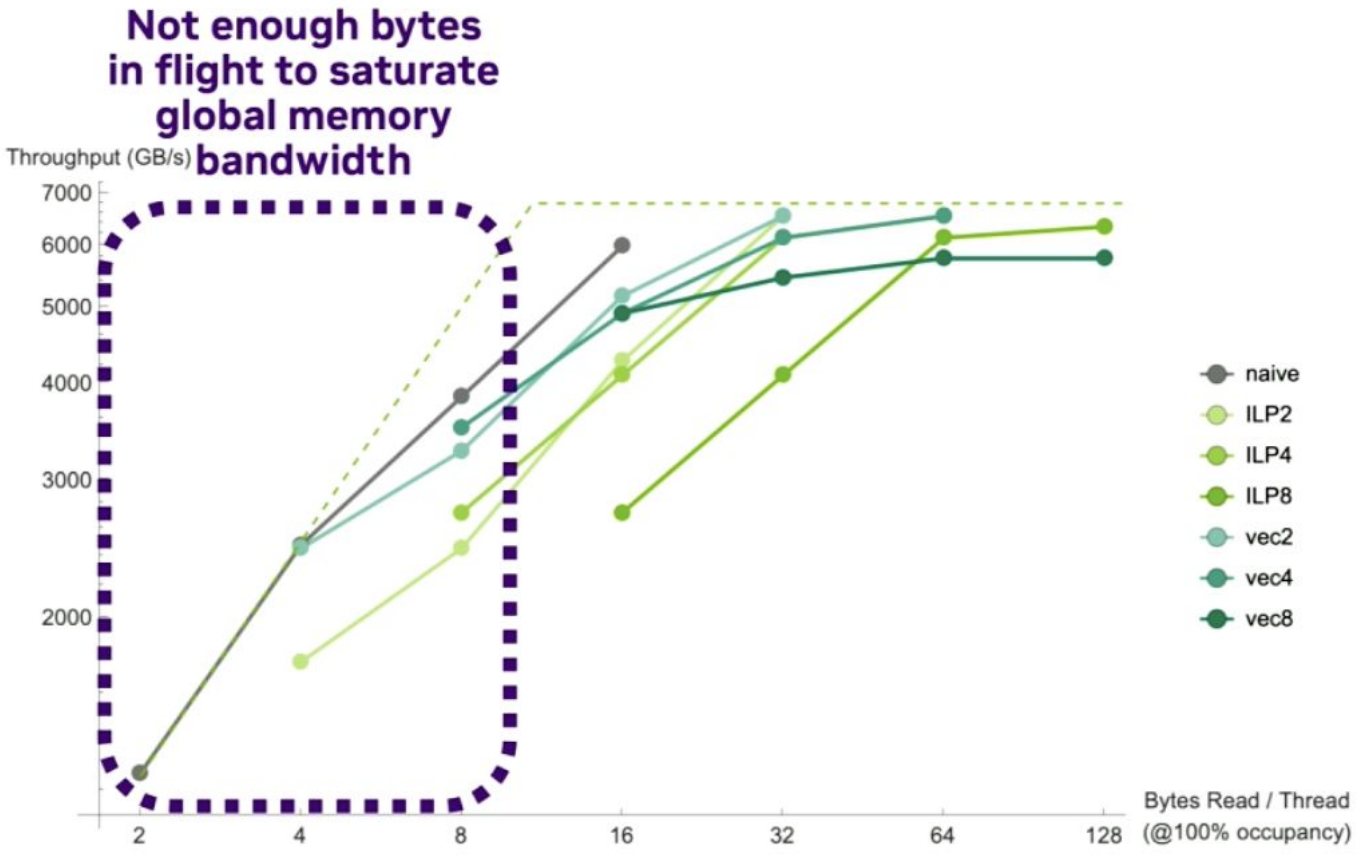


Little's Law estimate of critical bytes per thread:

$$\frac{(8000\text{GB/s}) \times (500\text{ ns latency})}{(148\text{ SMs}) \times (2048\text{ threads / SM})} \approx 13\text{ Bytes / thread}$$

AXPY Performance

Bytes in Flight Roofline model



AXPY Performance

Lessons Learned

- High memory throughput **requires** transactions to saturate the memory bus
  - Warps **stall** when reaching an instruction that depends on data that has not arrived yet
  - **bytes in flight** / SM requirements have increased for modern GPUs

- Recommendations:
  - Consider loading data **ahead-of-time**, rather than just-in-time
    - Leads to fewer stalls, and more average bytes-in-flight per thread
    - Also increases resource usage
  - Try **parameterizing the kernel** and tune to find the "sweet spot"
    - Little's law can help us get a "ballpark" estimate of required bytes / thread

Key takeaways

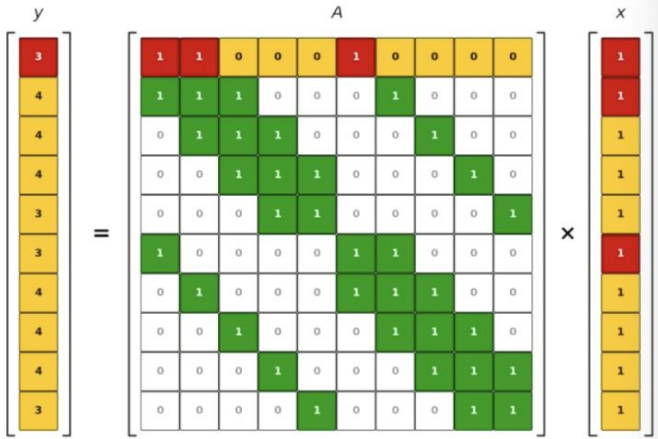
Pre-load data to keep the bus full

Don’t pre-load too much or you will reduce occupancy (SMEM and registers are precious resources)

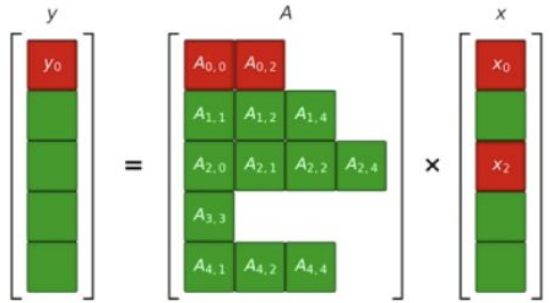
Parameterize your kernels

Matrix Vector Product

- Simple primitive operation that is pervasive in HPC
  - Used in scientific / CAE / energy / etc. simulations
- Perform product $y = Ax$
- Not intending to teach SOL matrix-vector products
  - Deep dive into the techniques



Sparse Implementation

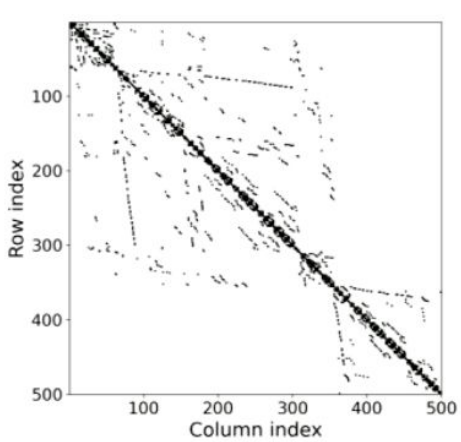


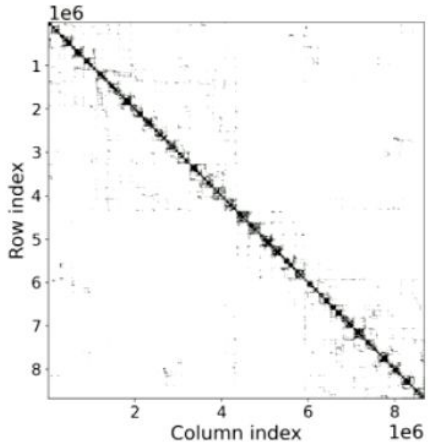
- Matrix structure determines the implementation
  - Many apps use ***Compressed Sparse Row (CSR)***

```
double sum = 0.0;
for (int i = row_offs[r]; i < row_offs[r+1]; ++i)
  sum += vals[i] * x[col_inds[i]];
y[r] = sum;
```

- Naive SpMV gives each thread a row to process
  - The rows, vals, cols arrays are accessed un-coalesced

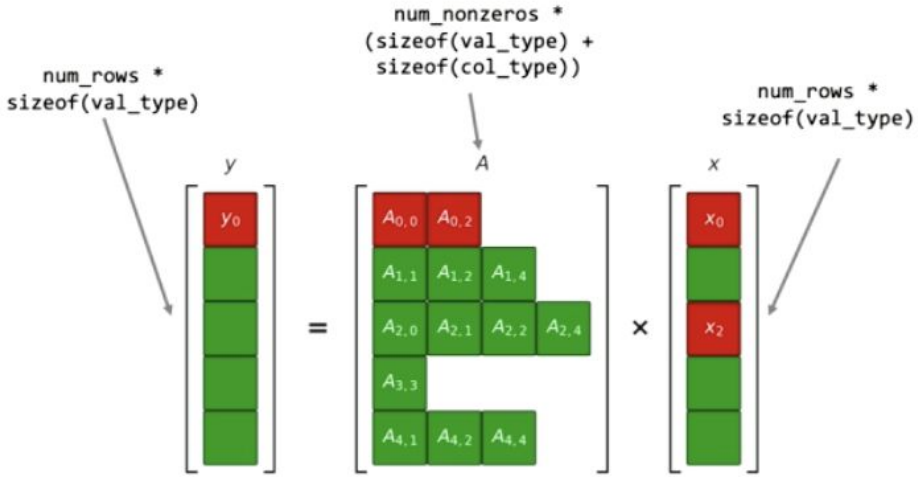
- Sparsity pattern impacts access to x
  - Impacts locality
  - Irregular sparsity introduces load balance issues

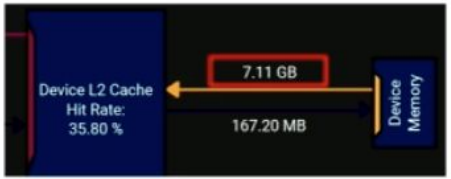




SpMV Analysis

- SpMV analytic model: effective DRAM throughput
  - Perfect caching - matrix / vectors touched once
  - Footprint is sum of bytes in **x + y + A**
- Gap between analytic and measured is DRAM over fetch
  - Sparsity with poor locality on x will increase over fetch
- Example: 27pt stencil
  - Bytes in **x + y + A** is 6.6 GB (note: FP64)
  - DRAM -> L2 traffic is 7.1 GB (right), so some over fetch occurs



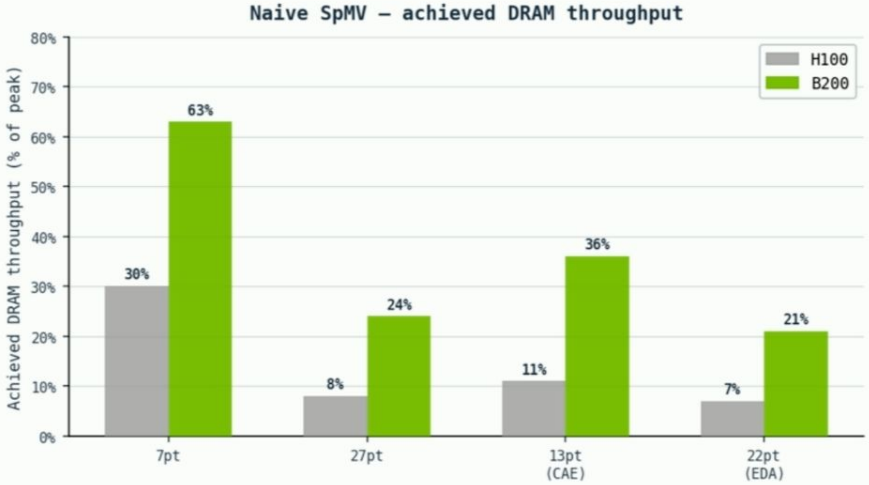


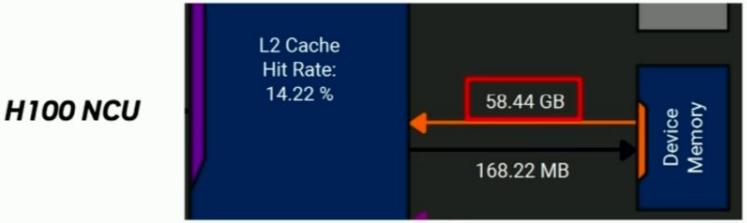
27 pt - Nsight compute (NCU) view of traffic from HBM

Always check how much data is moving from Device Memory using the Memory Workload Analysis section!

Naive SpMV Analysis

- Naive FP64 AXPY was ~80% DRAM throughput on Blackwell
- Naive SpMV example: 27pt stencil
  - H100 L2 is thrashed causing DRAM over fetch
    - Around 7x more traffic from DRAM to L2
  - B200 has minor over fetch and limited by L2 cache throughput
- For more bytes in flight, must reduce pressure on L2





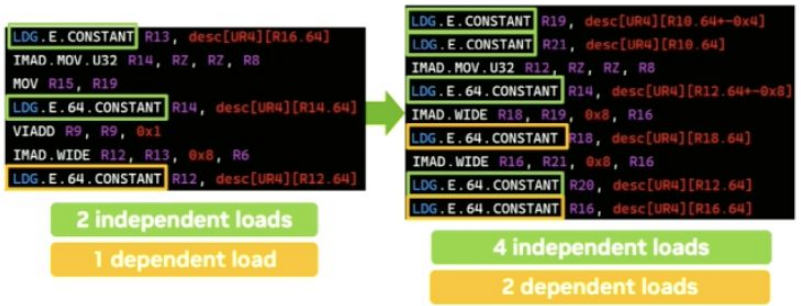
Unrolling

Unrolling

- *Use registers to increase concurrent loads per thread*
- Register utilization low with naive algorithm
  - Minimizes occupancy impact
- Optimal unrolling strategy depends on the matrix

```
  double sum = 0.0;
#pragma unroll 1
  for (int i = beg; i < end; i += unroll_len) {
                    sum += vals[i  ] * x[col_inds[i  ]];
    if (i+1 < end) sum += vals[i+1] * x[col_inds[i+1]];
  }
  y[r] = sum;
```

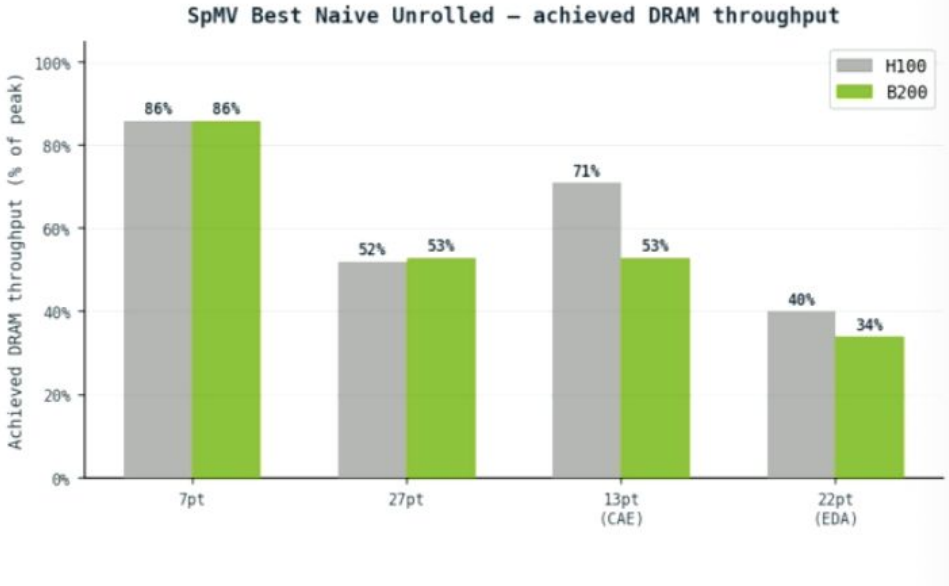
Manual inner unroll of naive implementation



SASS view of implementing 2-way unrolling
(check with SASS source view in NCU)

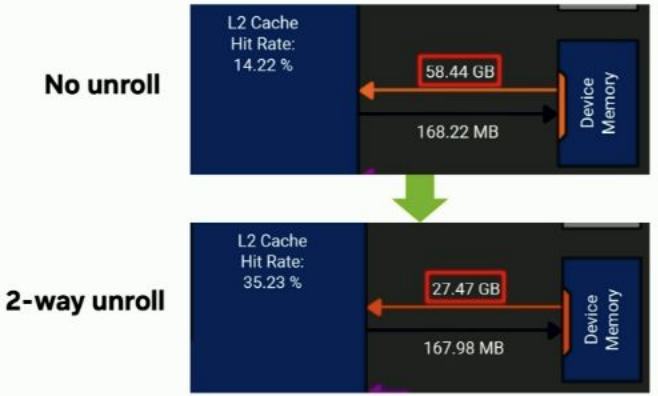
Unrolling

- The best unrolling configs (per matrix) for SpMV
- We increased bytes in flight, why are some low?
- Example: 27pt stencil
  - H100 limited by L2 (84%)
    - Cache thrashing improves, but not solved
  - B200 limited by L1 (89%)
    - 2x L2 capacity available, so unrolling shifts limiter



Unrolling

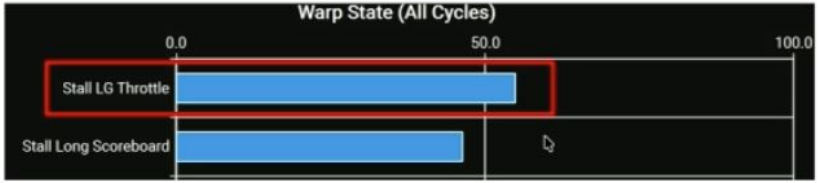
- Why did unrolling help L2 bound cases?
  - For this example, reduces L2 thrashing (right)
- Unrolling impacts temporal aspect of access pattern
- Threads in the same warp have locality on the matrix
  - As ILP increases, loads that are closer to each other in cache are scheduled closer to each other in time



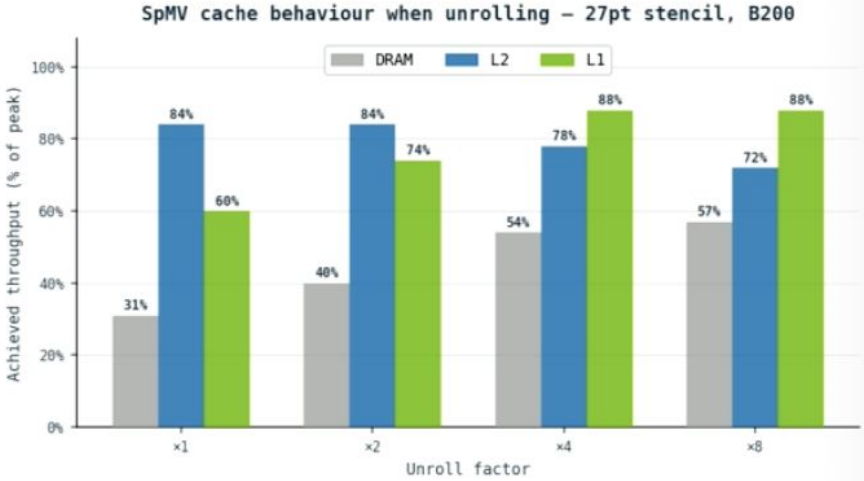
H100 Unroll - Memory Workload Analysis (NCU)
(increases L2 cache hit rate, reduces DRAM over fetch)

Global Load Throttle (B200)

- For 27 pt with 8x unroll on B200 there are stalls in NCU



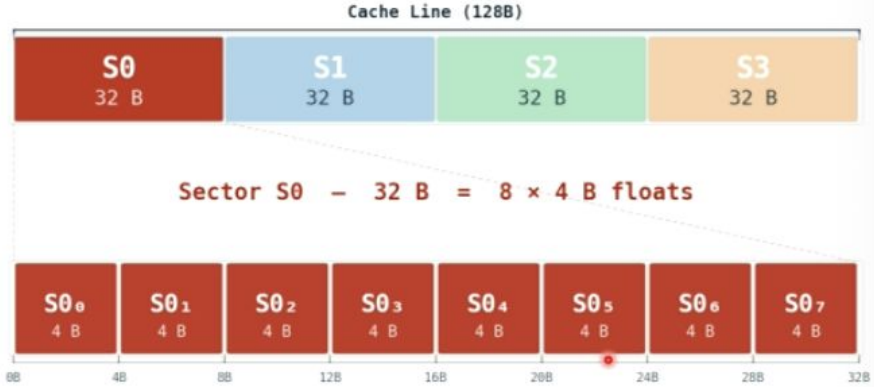
- LG throttle occurs when the L1 instruction queue is full
- Too few bytes in flight and DRAM throughput plateaus



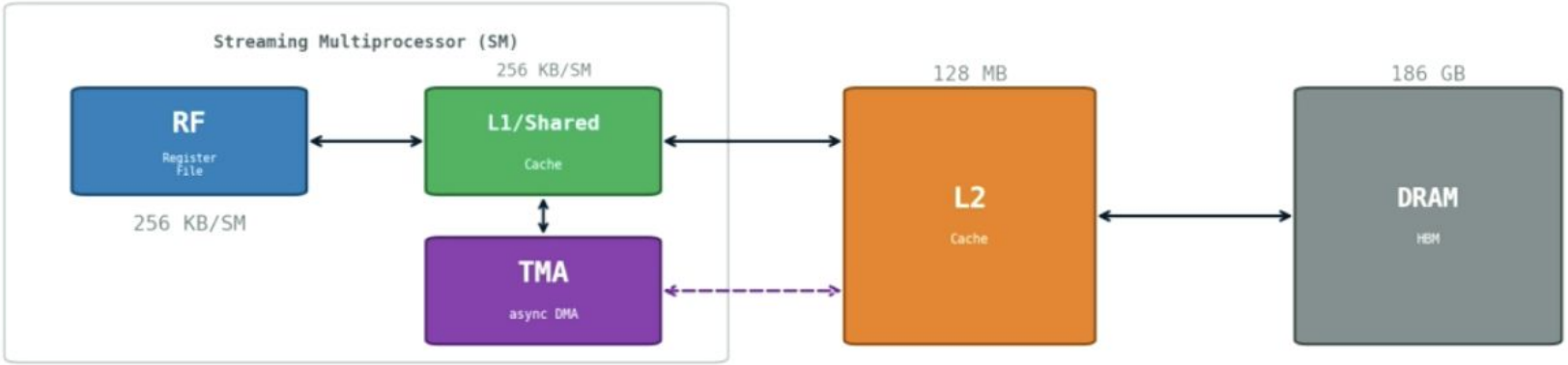
DRAM BW (from NCU) plateaus as global load throttling occurs

L1 Structure (B200)

- Memory divided up into 128B **cache lines**
- Cache lines further divided up into 32B **sectors**



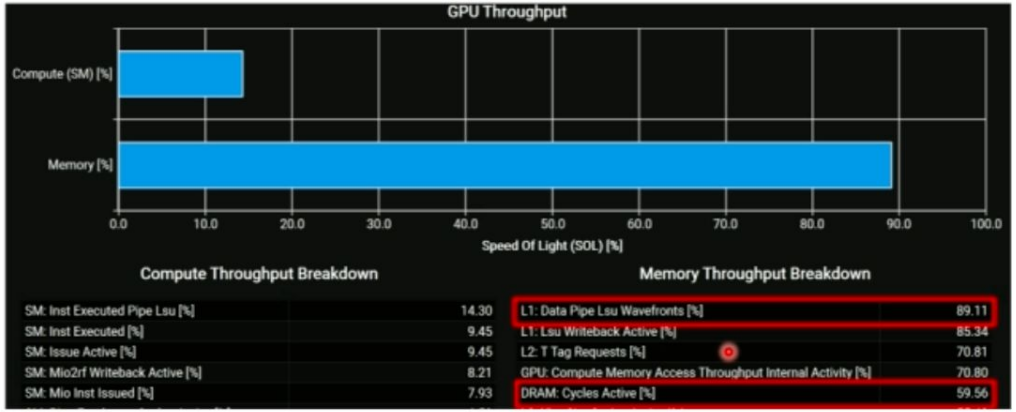
Cache line layout



L1 Wavefronts in SpMV

- NCU says the kernel is bound by L1 LSU wavefronts
  - Distinct from effective throughput/bandwidth L1
  - Wavefronts are packages of parallel work within a warp

- Increasingly a problem because:
  - DRAM bandwidth grows exponentially
  - SM count grows slowly
  - SM mem subsystem (incl. L1) scales with SM count
  - Increasing L1 bandwidth does not help



Common symptom: *high L1 Data Pipe LSU Wavefronts %*

	L1/TEX Cache			
	Instructions	Requests	Wavefronts	% Peak
Local Load	0	0	0	0
Global Load	54,343,632	54,343,632		
Global Load To Shared Store (access)	0	0	324,947,047	70.79
Global Load To Shared Store (bypass)	0	0		

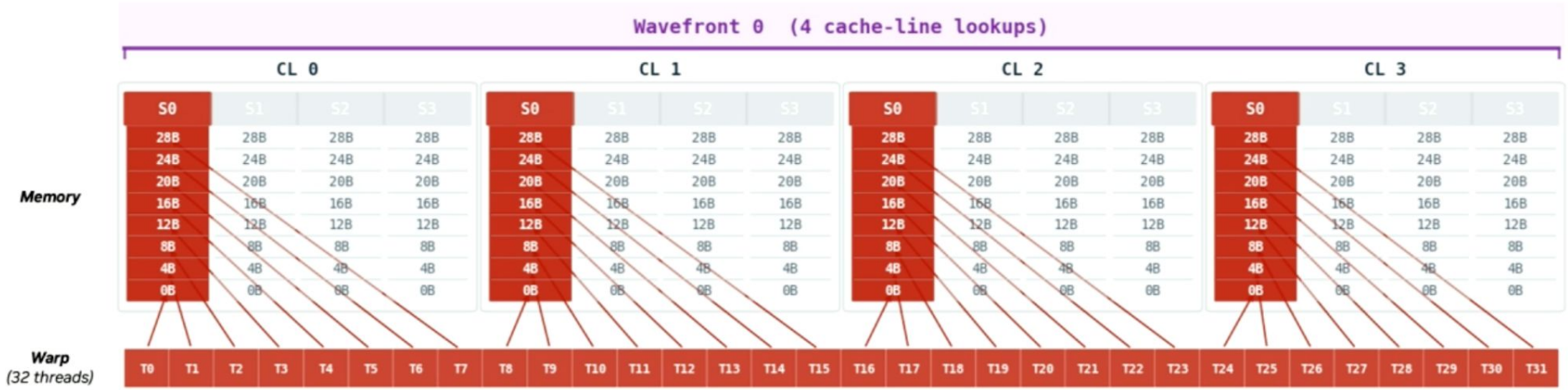
Tag-stage wavefronts

Deep dive on profiling L1 requests / wavefronts @ GTC 2021 s32089

B200 L1 Wavefronts Issued per Warp (Tag Stage)

CL = cache line
S = sector
T = thread

We can *roughly* plot the behavior of L1 wavefronts at the different stages (tag and data), with simple benchmarks accessing data by different strides

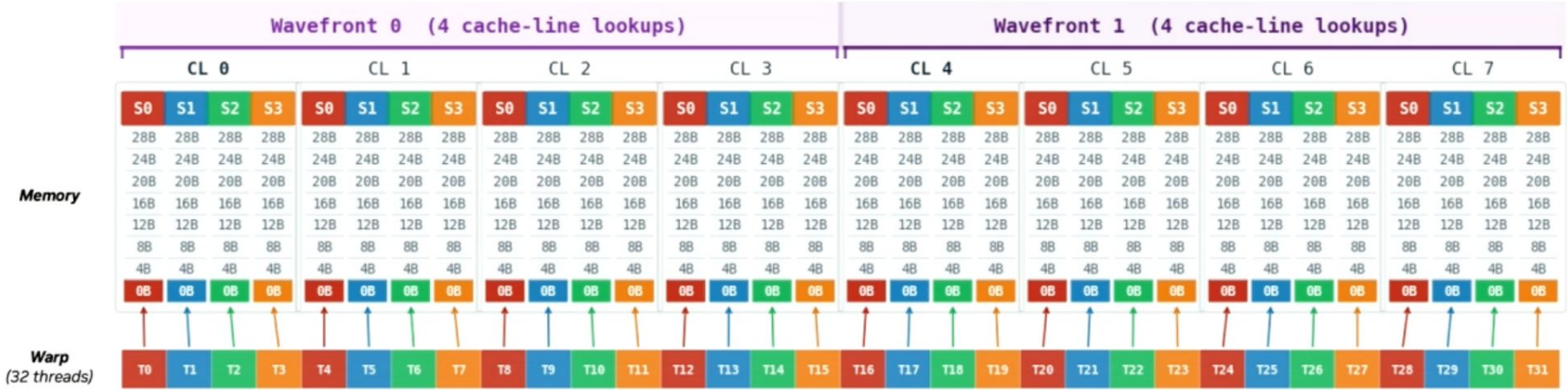


Same Sector

(threads access slot N of unique sectors of 4 cache lines, 16 sectors, 1 wavefront)

B200 L1 Wavefronts Issued per Warp (Tag Stage)

CL = cache line
S = sector
T = thread

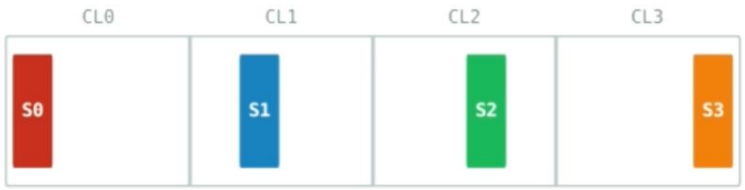


32B Stride Perspective

(threads access first element of unique sectors, 8 cache lines total, 32 sectors, 2 wavefronts)

B200 L1 Wavefronts (Data Stage)

CL = cache line
S = sector
WF = wavefront



Data-stage Wavefronts = 1

Tag-stage Wavefronts = 1

Threads of 1 warp access distinct sectors of 4 caches lines
(Data-stage and tag-stage both require only 1 wavefront)

Data-stage Wavefronts = 4

Tag-stage Wavefronts = 1

Threads of 1 warp access Sector 0 of 4 caches lines
(4 data-stage wavefronts are required)

NVIDIA

B200 L1 Wavefronts (Data Stage)

CL = cache line
S = sector
WF = wavefront

	WF1	WF2	WF3	WF4
S0	CL0 T0–1	CL1 T8–9	CL2 T16–17	CL3 T24–25
S1	CL0 T2–3	CL1 T10–11	CL2 T18–19	CL3 T26–27
S2	CL0 T4–5	CL1 T12–13	CL2 T20–21	CL3 T28–29
S3	CL0 T6–7	CL1 T14–15	CL2 T22–23	CL3 T30–31

Data-stage Wavefronts = 4

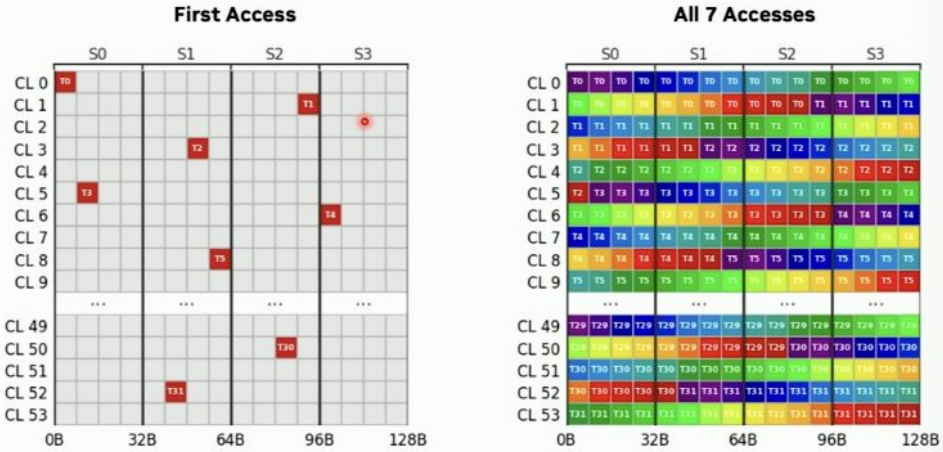
Tag-stage Wavefronts = 1

Threads perform 4B accesses to all sectors of 4 cache lines

NVIDIA

L1 Wavefronts in SpMV

- Matrix streaming with a single thread per row
  - Accesses are un-coalesced, ideally perfect reuse in L1
- Multiple options to fix this
  - Data structure changes
  - Vectorize loads
    - Alignment impact efficiency for unstructured data
  - Algorithmic changes…



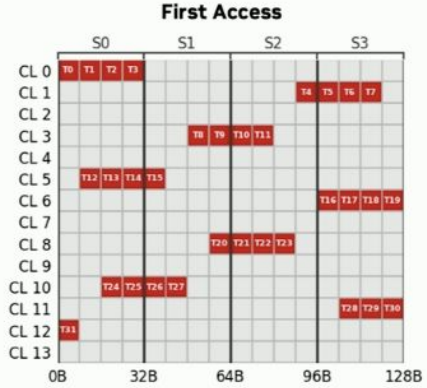
27 pt stencil matrix warp access pattern

A warp instruction to A touches 32 distinct cache lines (one per thread)

Row Data Parallel

- In SpMV, parallelism exists within rows
  - Improves L1 access patterns (coalescing)
  - Potentially causes some inactive threads
- Rows can be small
  - Leverage sub-warp parallelism (referred to as group)
    - 1-32 threads cooperate
    - shfl instructions share between cooperating threads
- 27pt stencil example with 4 threads per rows
  - Tag-stage wavefronts reduced by 2.9x
  - Data-stage wavefronts reduced by 1.9x...
  - Even best kernel still hits 85% L1 wavefronts

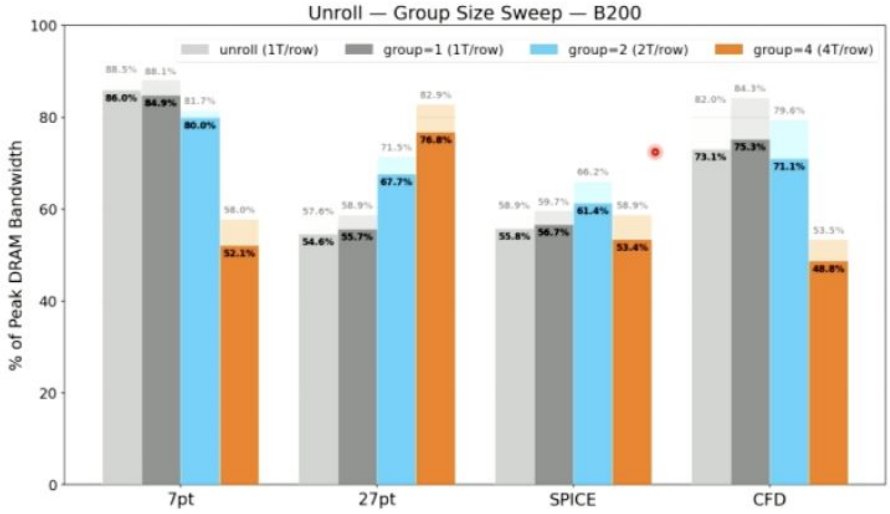
```
  if (row < nrows) {
    // stride by group_size*unroll_len each iteration
#pragma unroll 1
    for (int nz = beg + group_lane; nz < end; nz += group_size * unroll_len) {
#pragma unroll
      for (int u = 0; u < unroll_len; u++)
        if (nz + u*group_size < end)
          sum += vals[nz + u*group_size] * x[cols[nz + u*group_size]];
    }
  }

  // warp-shuffle reduction across the group
#pragma unroll
  for (int i = group_size/2; i > 0; i /= 2)
    sum += __shfl_down_sync(0xFFFFFFFF, sum, i);
```



27 pt stencil matrix access patterns with data parallel
Now only 9 cache lines per instruction, down from 32

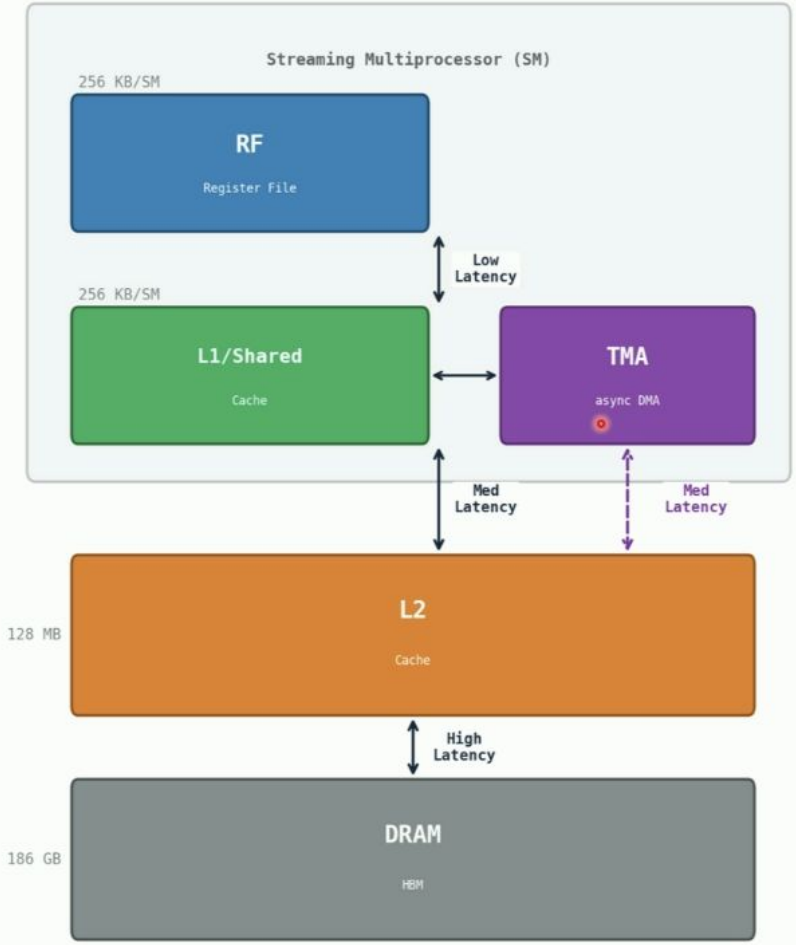
Row Data Parallel Results

- Data parallelism helped close the gap in many cases
- DRAM throughput from NCU in faint colors
  - Gaps between results indicate DRAM over fetch
- The best unroll factor and group size depends on the matrix so requires tuning
- SPICE matrix is small (only 1GB)



Async Data Copies

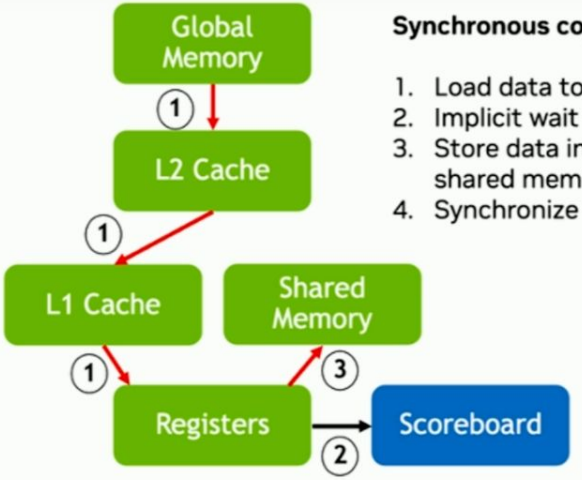
- **LDGSTS**
  - Elementwise DRAM to shared
- **TMA**
  - Tensor Memory Accelerator (hardware unit)
  - Single-thread DMA from DRAM to shared



Async Data Copies

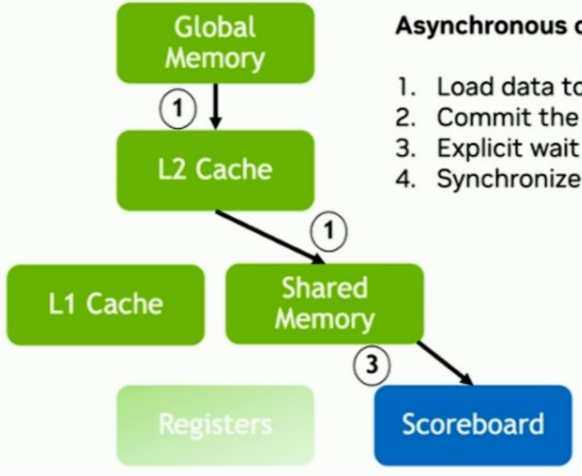
- Benefits
  - Reduced register utilization (TMA / LDGSTS)
  - L1 data stage overhead reduced (TMA / LDGSTS)
  - Gathering x may be interleaved via pipelining (TMA)

- Challenges
  - Independent alignment of cols / vals / shared / global
  - Shared mem used to stage data (may not have enough)
    - Less L1, and lower occupancy
  - No compute to hide
  - LDGSTS and global loads interfere
    - Matrix and indirect x access won't overlap



Synchronous copy

1. Load data to registers
2. Implicit wait for scoreboard
3. Store data in registers to shared memory
4. Synchronize threads



Asynchronous copy

1. Load data to shared memory
2. Commit the copy to scoreboard
3. Explicit wait for scoreboard
4. Synchronize threads

L2 Prefetch

- **`cp.async.bulk.prefetch.L2`** used to warm the L2 cache with the next block's data
  - Concurrently compute on the current block
  - Accesses to dense vector interleaved with prefetch
  - No synchronization required
  - No shared memory required
- L2 prefetch delivers 7% more perf on the SPICE matrix
- Expected to be even more beneficial on R100
  - BW / SM ratio increases again...

```
// Prefetch [ptr, ptr+nbytes) into L2, 16-byte alignment with minor overfetch
__device__ __forceinline__ void prefetch_to_L2(const void* ptr, int nbytes) {
    uintptr_t start = (uintptr_t)ptr & ~15ULL;
    int total = (int)((uintptr_t)ptr - start) + nbytes;
    total = (total + 15) & ~15;
    asm volatile (
        "cp.async.bulk.prefetch.L2.global [%0], %1;\n"
        :: "l"((const void*)start), "r"(total)
    );
}
```

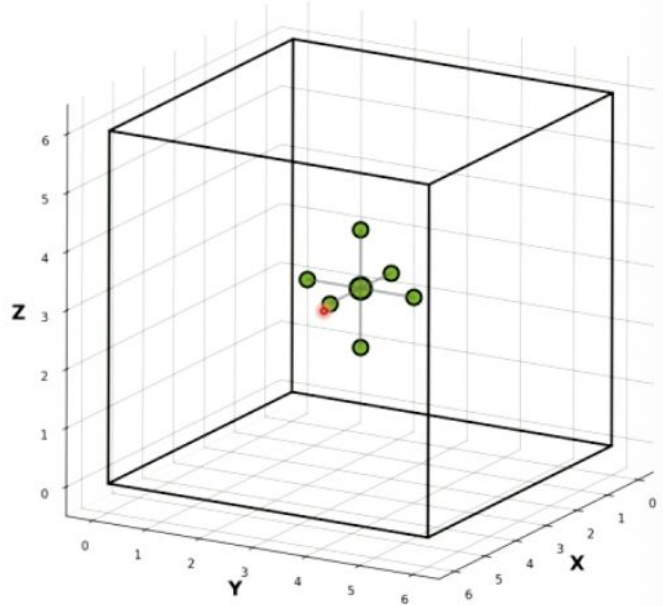
Helper function to prefetch into L2 and fix alignment

```
// Prefetch the very first block
if thread 0:
    prefetch_to_L2(vals[block_0_range], cols[block_0_range])

// Grid-stride loop
for each block of rows assigned to this CTA:

    // Prefetch next block's data into L2 (while we compute current)
    if thread 0 and next block exists:
        prefetch_to_L2(vals[next_block_range], cols[next_block_range])

    // Compute on current block (data hopefully in L2 by now)
    for each row assigned to this warp/group:
        SpMV_inner(vals[current_block_range], cols[current_block_range])
```

Pseudo-code to prefetch to L2 in a two-stage pipeline

Laplace stencil

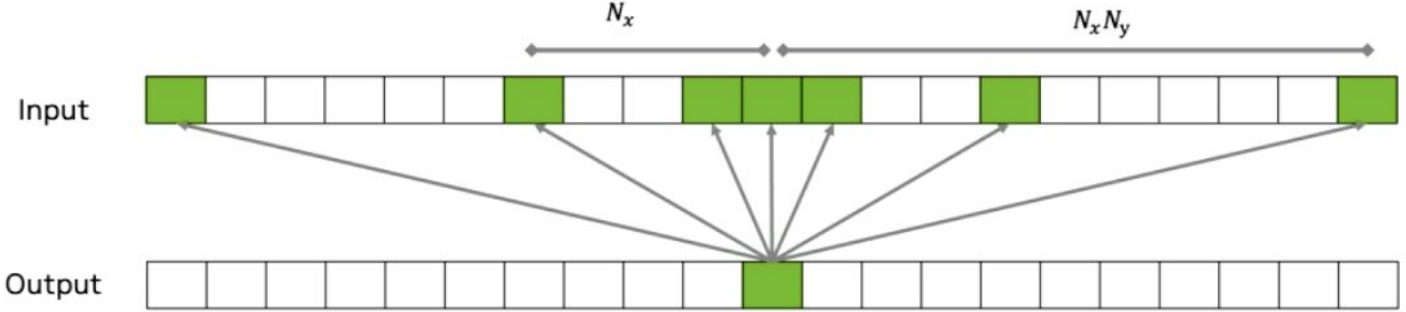
7-point stencil in 3D



- Second-order finite difference approximation of a Laplacian:
  $\Delta u = \text{div}(\text{grad u}) = \nabla^2 u$

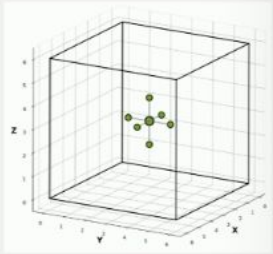
$$\Delta u \approx -c_0 u[i,j,k] + c_x(u[i-1,j,k] + u[i+1,j,k]) + c_y(u[i,j-1,k] + u[i,j+1,k]) + c_z(u[i,j,k-1] + u[i,j,k+1])$$

- Memory access pattern: Neighbor access in all dimensions



7-point Stencil

Reference implementation



- Given and input u, compute the Laplace and store it in the output v
- Each thread processes 1 element
- Flattened index computation of current and neighboring cells

- Laplacian is computed for a field of N^3 grid points in double and single precision, where $N = 1024$.

- We compute achieved bandwidth utilization as a performance measure using the computational domain (excluding halos)

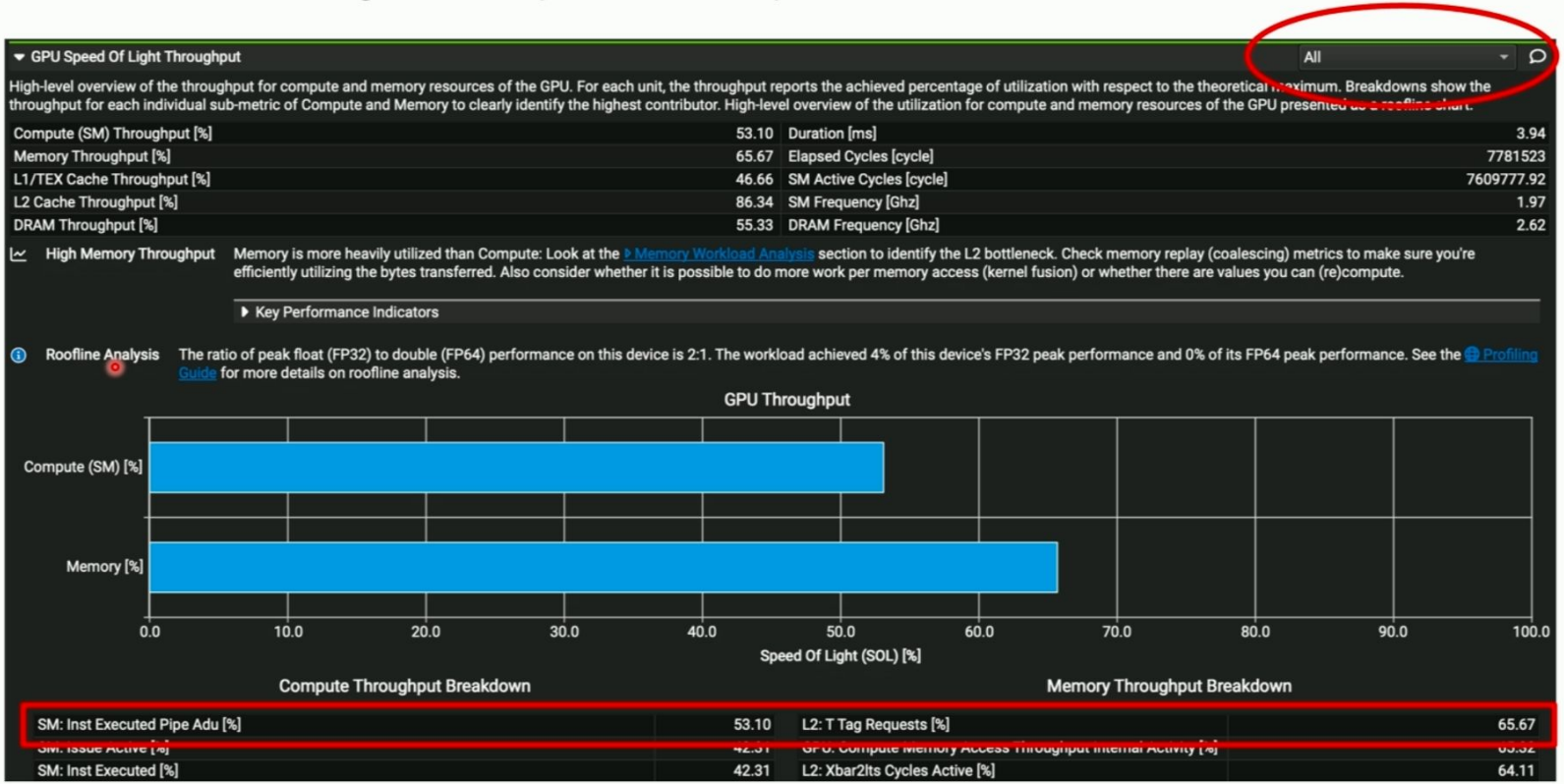
$$Bandwdith = 2\frac{N^3 sizeof(T)}{t_{mean}}$$

```
template<typename T = double>
__global__ void
laplace_ref(int nx, int ny, int nz, int stride_y, int stride_z,
            const T* __restrict__ const u, T* const __restrict__ v,
            const T coeff0, const T coeffx, const T coeffy, const T coeffz)
{
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    int j = threadIdx.y + blockIdx.y * blockDim.y;
    int k = threadIdx.z + blockIdx.z * blockDim.z;

    int tid = i + j * stride_y + k * stride_z;

    if(i>=nx || j>=ny || k>=nz) return;

    v[tid] = u[tid] * coeff0 + (u[tid - 1] + u[tid + 1]) * coeffx +
             (u[tid - stride_y] + u[tid + stride_y]) * coeffy +
             (u[tid - stride_z] + u[tid + stride_z]) * coeffz;
}
```

Laplace stencil

GH200 – Reference implementation

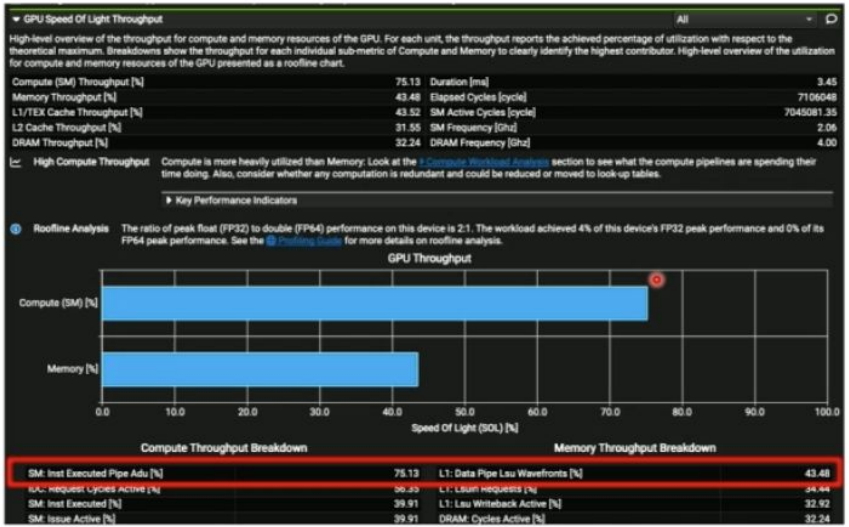
- **GH200 – float**: Memory bound (ADU + L2: Tag Requests)
  - ADU: Address divergence unit (mostly due to LDC)
  - L2: strided stencil accesses generate many cache-line lookups



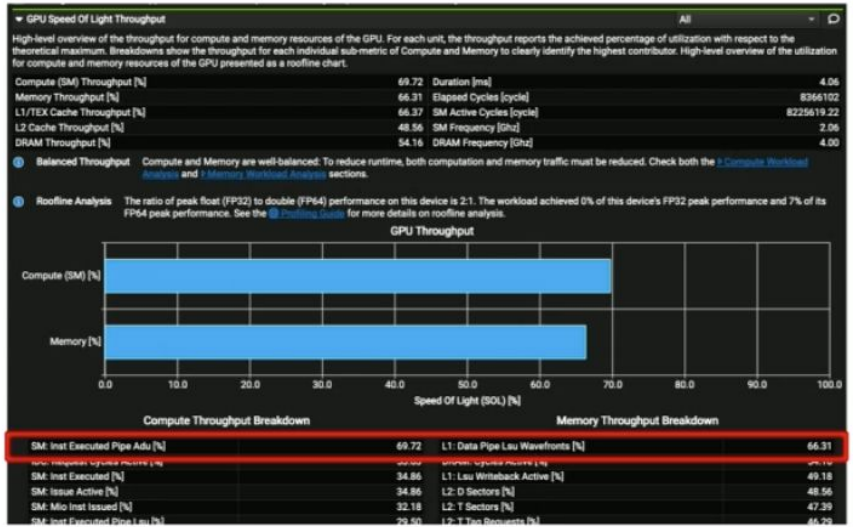
Laplace stencil

GB200 – Reference implementation

GB200 - float



GB200 - double



- Our kernel appears to become compute bound for float and double
  - High ADU and LSU wavefronts utilization

7-point Stencil

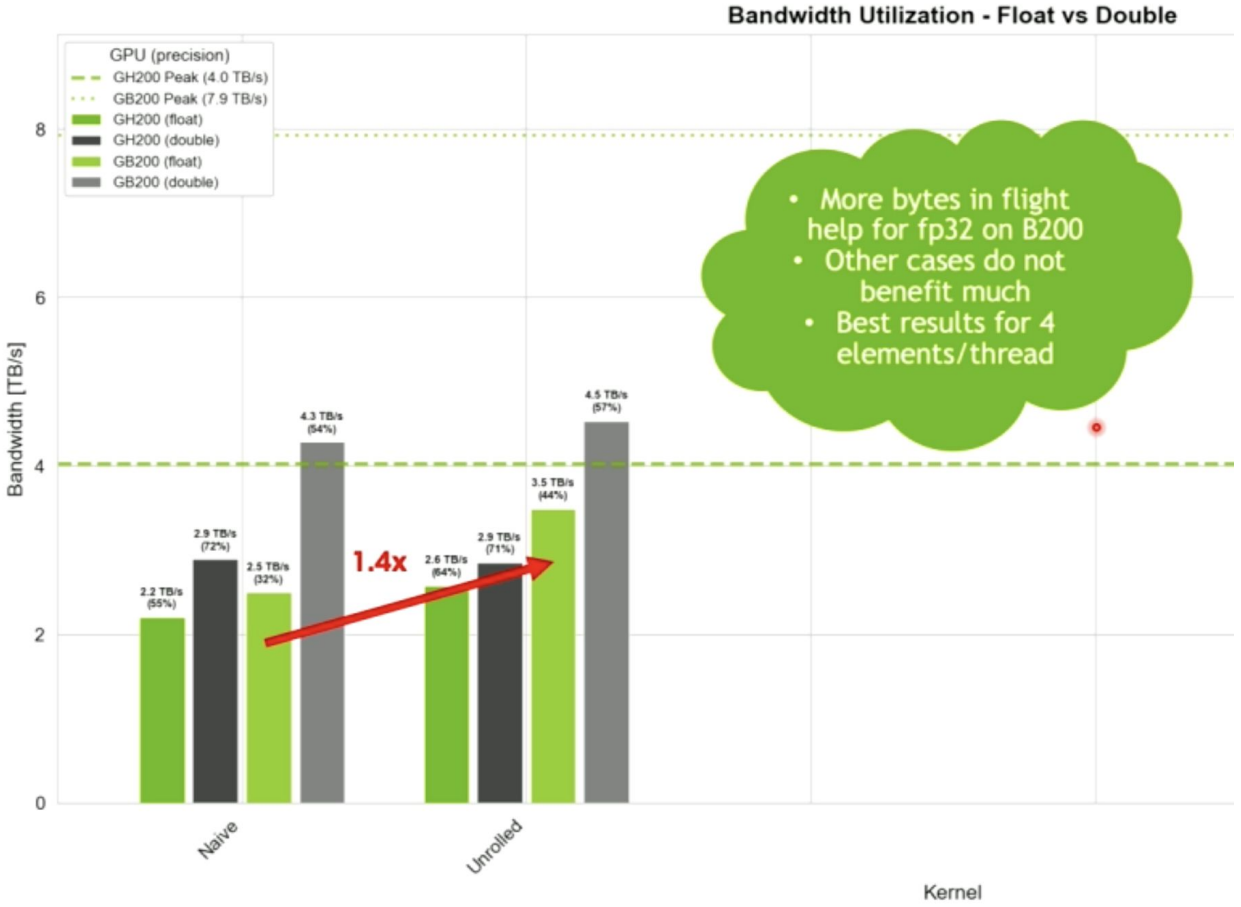
Unrolling – Increase ILP

- Let's use the same trick we used for AXPY/SpMV
  - Cluster the loads into a register queue
  - Compute multiple elements per threads to increase ILP
- More bytes in flight will surely solve our problem?!

```
template < int ELEMENTS_PER_THREAD, typename T >
__global__ void laplace_unrolled_strided(int nx, int ny, int nz,
  int stride_x, int stride_y,
  const T* __restrict__ const u, T* const __restrict__ v,
  const T coeff0, const T coeffx, const T coeffy, const T coeffz)
{
  const int tile_stride = blockDim.x * ELEMENTS_PER_THREAD;
  const int i_base = blockIdx.x * tile_stride + threadIdx.x;
  const int j = threadIdx.y + blockIdx.y * blockDim.y;
  const int k = threadIdx.z + blockIdx.z * blockDim.z;

  if (j >= ny || k >= nz) return;

  const int jk_offset = j * stride_x + k * stride_y;

  // Register arrays for preloading - 7 stencil points per element
  T reg_center[ELEMENTS_PER_THREAD];
  T reg_xm1[ELEMENTS_PER_THREAD];
  T reg_xp1[ELEMENTS_PER_THREAD];
  T reg_ym1[ELEMENTS_PER_THREAD];
  T reg_yp1[ELEMENTS_PER_THREAD];
  T reg_zm1[ELEMENTS_PER_THREAD];
  T reg_zp1[ELEMENTS_PER_THREAD];

  // Phase 1: Preload all values into registers (coalesced loads)
  #pragma unroll
  for (int n = 0; n < ELEMENTS_PER_THREAD; n++) {
    int i = i_base + n * blockDim.x;  // Strided by blockDim.x
    if (i < nx) {
      int tid = i + jk_offset;
      reg_center[n] = u[tid];
      reg_xm1[n] = u[tid - 1];
      reg_xp1[n] = u[tid + 1];
      reg_ym1[n] = u[tid - stride_x];
      reg_yp1[n] = u[tid + stride_x];
      reg_zm1[n] = u[tid - stride_y];
      reg_zp1[n] = u[tid + stride_y];
    }
  }

  // Phase 2: Compute and store (coalesced stores)
  #pragma unroll
  for (int n = 0; n < ELEMENTS_PER_THREAD; n++) {
    int i = i_base + n * blockDim.x;
    if (i < nx) {
      int tid = i + jk_offset;
      v[tid] = reg_center[n] * coeff0 +
               (reg_xm1[n] + reg_xp1[n]) * coeffx +
               (reg_ym1[n] + reg_yp1[n]) * coeffy +
               (reg_zm1[n] + reg_zp1[n]) * coeffz;
    }
  }
}
```

Laplace stencil

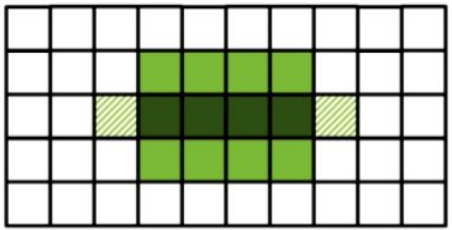
Result

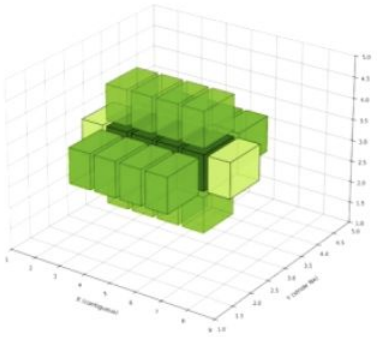


7-point Stencil

Vectorization – Increase DLP

- 4 elements/thread is optimal for unrolled, beyond that occupancy lowers performance
- Let's change the access pattern
- Vectorized loads
  - Increase DLP (float4 fetches 4 cache lines with a single instruction
  - 2 halo scalar loads in x-direction

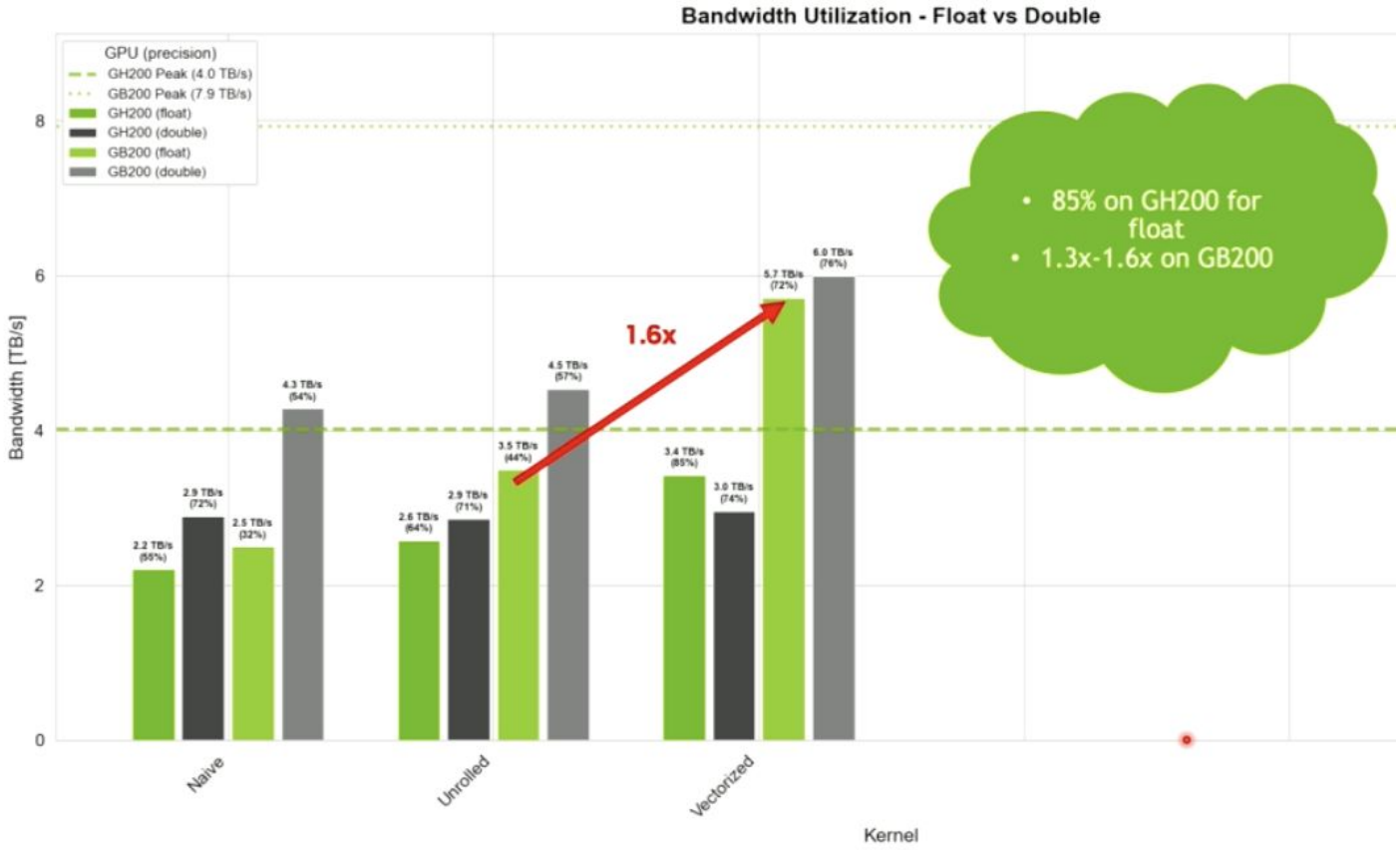




```
template < int VECTOR_WIDTH, typename T >
__global__ void laplace_vectorized(const T* __restrict__ in, T* __restrict__ out,
  int stride_x, int stride_y, int nx, int ny, int nz,
  T coeff0, T coeffx, T coeffy, T coeffz, int memory_halo)
{

  auto id = [&](int ix, int iy, int iz) {
    return ix + stride_x*iy + stride_y*iz;
  };

  int i0 = VECTOR_WIDTH * (threadIdx.x + blockIdx.x * blockDim.x);
  int j = threadIdx.y + blockIdx.y * blockDim.y;
  int k = threadIdx.z + blockIdx.z * blockDim.z;

  //Passing the full field including the halos of size 2
  if(i0+VECTOR_WIDTH>nx+4 || j>=ny+2 || k>=nz+2 || j<2 || k<2) return;

  aligned_reg_array<T, VECTOR_WIDTH> result{};
  result[0] += i0>0?coeffx * in[id(i0-1,j,k)]:0;
  result[VECTOR_WIDTH-1] += i0+VECTOR_WIDTH<nx+4?coeffx * in[id(i0+VECTOR_WIDTH,j,k)]:0;

  // only update values on the interior
  aligned_reg_array<T, VECTOR_WIDTH> center_values;
  aligned_reg_array<T, VECTOR_WIDTH> jm1_values;
  aligned_reg_array<T, VECTOR_WIDTH> jp1_values;
  aligned_reg_array<T, VECTOR_WIDTH> km1_values;
  aligned_reg_array<T, VECTOR_WIDTH> kp1_values;

  km1_values.load(&in[id(i0,j,k-1)]);
  jm1_values.load(&in[id(i0,j-1,k)]);
  center_values.load(&in[id(i0,j,k)]);
  jp1_values.load(&in[id(i0,j+1,k)]);
  kp1_values.load(&in[id(i0,j,k+1)]);

  #pragma unroll
  for (int di = 0; di < VECTOR_WIDTH; di++) {
    result[di] += coeff0 * center_values[di];
    result[di] += di>0?  coeffx * center_values[di-1] : 0;
    result[di] += di<VECTOR_WIDTH-1? coeffx * center_values[di+1] : 0;
  }

  #pragma unroll
  for(int di = 0; di < VECTOR_WIDTH; di++) {
    result[di] += (km1_values[di] + kp1_values[di]) * coeffz
                + (jm1_values[di] + jp1_values[di]) * coeffy;
  }

  result.store(&out[id(i0,j,k)]);

}
```

Laplace stencil

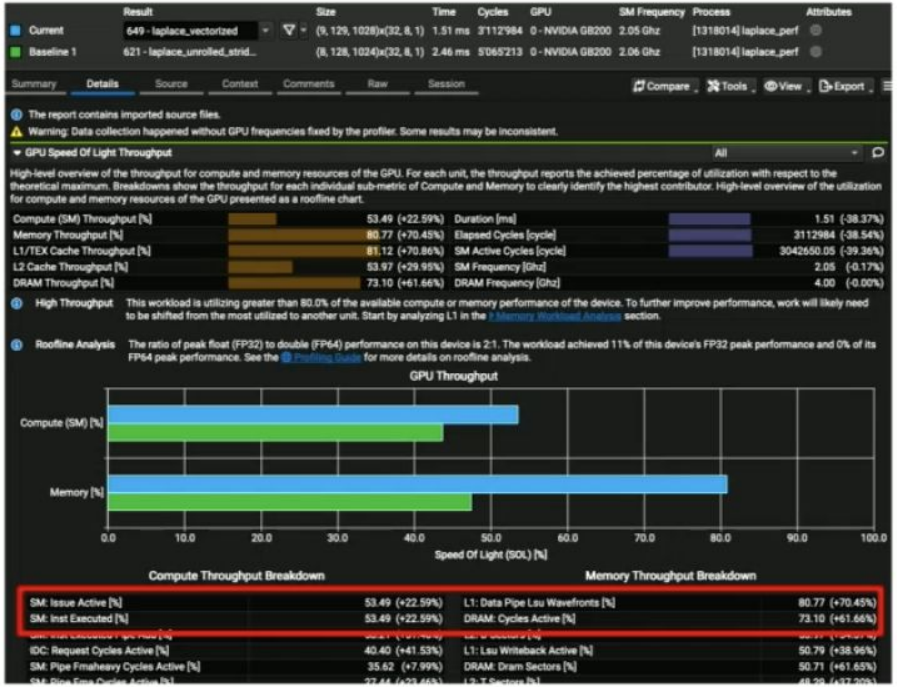
Result



Laplace stencil

GB200 - Vectorization

GB200 - float



GB200 - double

- Vectorization greatly benefits both single and double precision
- However, both kernels remain wavefront bound

Laplace stencil

GB200 - Vectorization

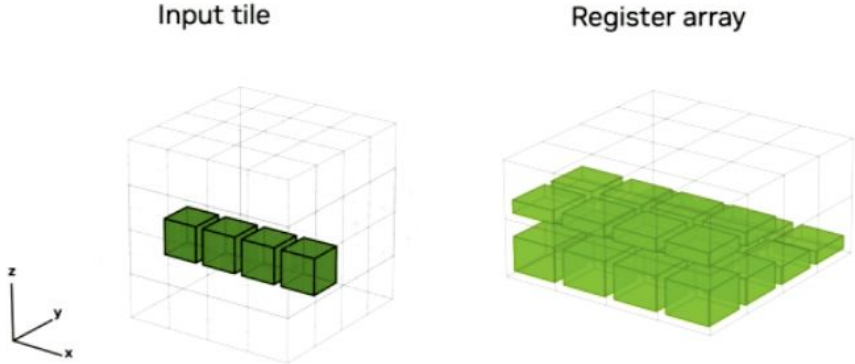
- Scalar loads are still inefficient
- We see the same inputs multiple times

Load	Cache lines	Wavefronts
Center (float4)	4	1
+y (float4)	4	1
-y (float4)	4	1
+z (float4)	4	1
-z (float4)	4	1
x-1 (scalar)	5 (1 halo spill)	2 (ceil(5/4))
x+1 (scalar)	5 (1 halo spill)	2 (ceil(5/4))
Total (loads)	**30**	**9**

7-point Stencil

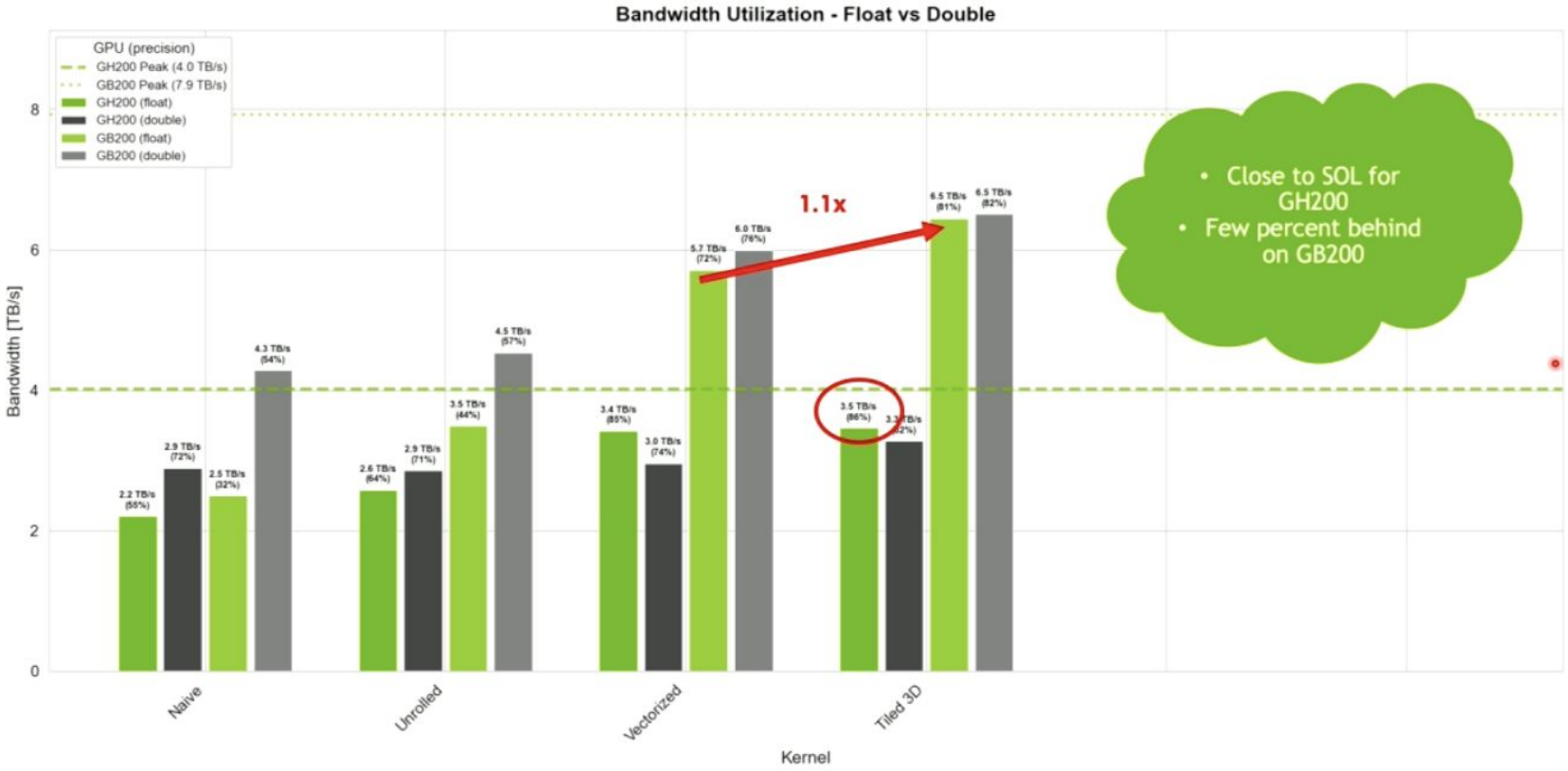
Tiling – Vectorization+Unrolling

- Tiling: Combing vectorization and unrolling
  - Each thread computes a 3D-tile.
  - Use vector loads for loading from global memory into a register queue.
  - Partial contributions are accumulated directly in the register array – input values are only seen once
    - Changes the access pattern compared to previous examples



Laplace stencil

Result



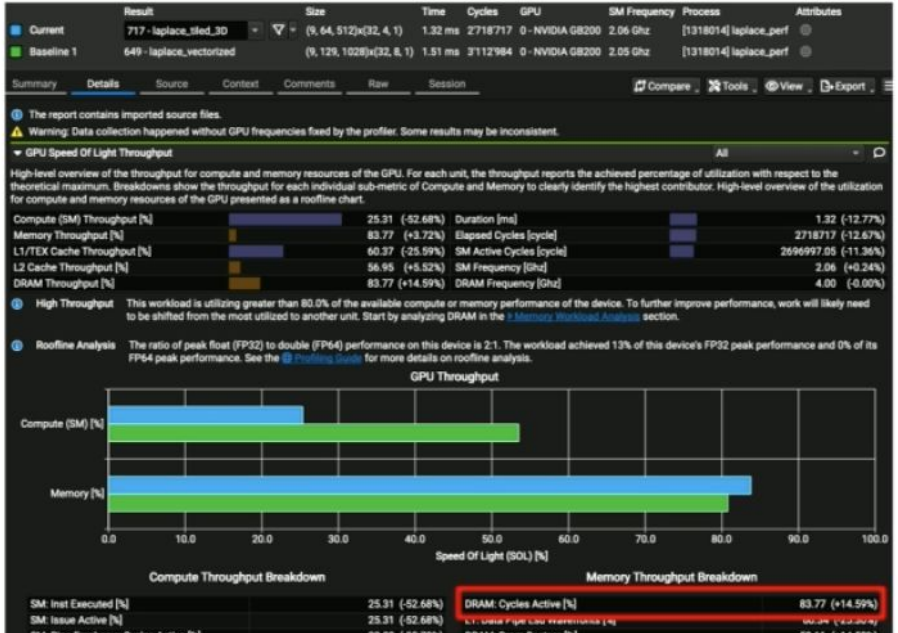
NVIDIA

Finally DRAM bound vs. L1 Wavefront bound

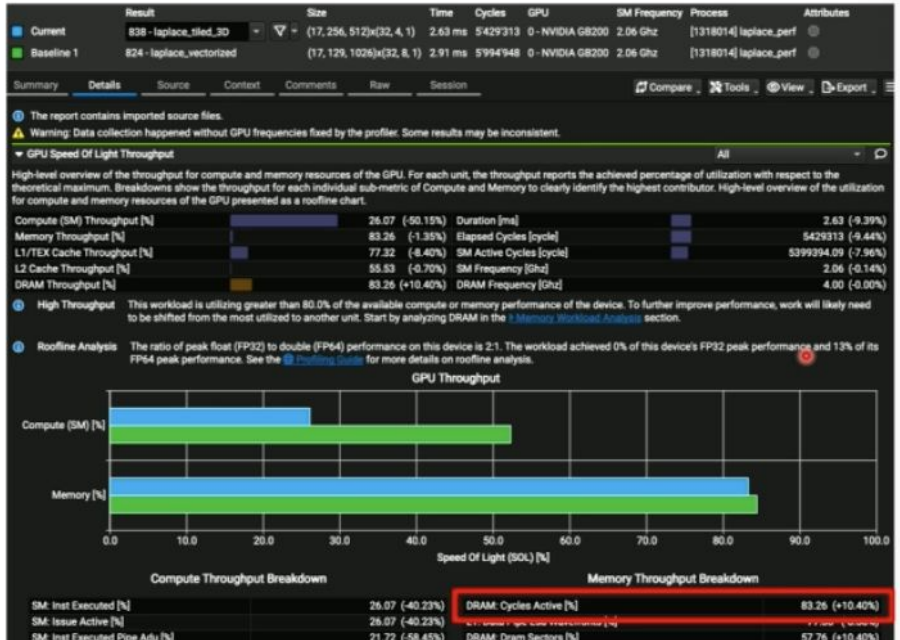
Laplace stencil

GB200 - Tiling

GB200 - float



GB200 - double



Increase ILP, data parallel, and bytes-in-flight - BUT register spilling

7-point Stencil

Asynchronous Data copies

- Increasing ILP and DLP increases bytes-in-flight at the cost of increased register usage
  - Not an issue for low compute-intensity kernels (as the Laplace kernel)
  - For compute-heavy kernels this can become a bottleneck
    - Low occupancy and register spilling into local memory

- Asynchronous copies between global and shared memory
  - Asynchronous copy (LDGSTS, Ampere++)
    - Completion: thread-local or shared memory barrier
  - Bulk-asynchronous copy (TMA 1D, TMA ND, Hopper++) – warp uniform
    - Dedicate a single thread/warp to issue TMA instructions
    - Completion: Shared memory barrier

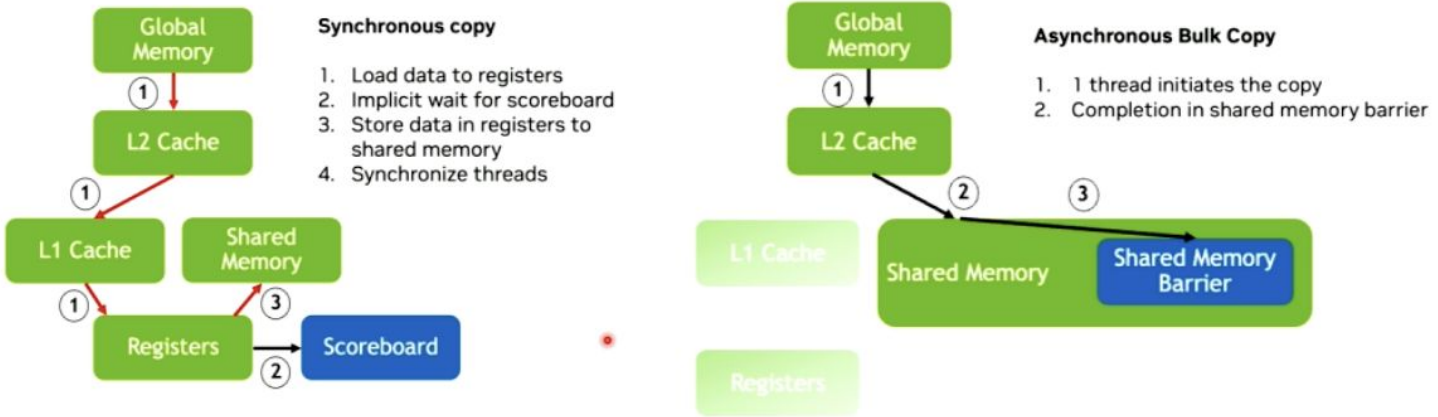
Direction		Completion Mechanism	
Source	Destination	Asynchronous Copy (LDGSTS, Ampere+)	Bulk-Asynchronous Copy (TMA, Hopper+)
GMEM	SMEM	thread-local, mbarrier	mbarrier
GMEM	DSMEM	-	mbarrier (multicast)
SMEM	GMEM	-	thread-local
SMEM	DSMEM	-	mbarrier
DSMEM	SMEM	-	mbarrier
REG	DSMEM	-	mbarrier

Async copies

7-point Stencil

Tensor Memory Accelerator (TMA)

- Asynchronous copies skip registers for compute, which frees up registers, reduces L1 traffic and MIO pressure (less instructions)



7-point Stencil

Asynchronous Data copies – TMA ND

- Single thread can issue 1D-5D block copy
- Requirements (>1D):

Address/size	Alignment
Global memory address	Must be 16B aligned
Global memory size	≥ 1B
Global memory stride	A multiple of 16B
Shared memory address	Must be 128B aligned
Shared memory barrier address	Must be 8 byte aligned
Size of transfer	Must be multiple of 16 bytes
Blocks dimension	≤ 256 elements

- Use a CUtensorMap to describe the tensor layout in global and shared memory
  - Typically created on the host and transferred to the device as a const kernel parameter annotated with __grid_constant__

```
CUtensorMap tensor_map{};
// rank is the number of dimensions of the array.
constexpr uint32_t rank = 2;
uint64_t size[rank] = {GMEM_WIDTH, GMEM_HEIGHT};
// The stride is the number of bytes to traverse from the first element of one row to the next.
// It must be a multiple of 16.
uint64_t stride[rank - 1] = {GMEM_WIDTH * sizeof(int)};
// The box_size is the size of the shared memory buffer that is used as the
// destination of a TMA transfer.
uint32_t box_size[rank] = {SMEM_WIDTH, SMEM_HEIGHT};
// The distance between elements in units of sizeof(element). A stride of 2
// can be used to load only the real component of a complex-valued tensor, for instance.
uint32_t elem_stride[rank] = {1, 1};

// Get a function pointer to the cuTensorMapEncodeTiled driver API.
auto cuTensorMapEncodeTiled = get_cuTensorMapEncodeTiled();

// Create the tensor descriptor.
CUresult res = cuTensorMapEncodeTiled(
  &tensor_map,                // CUtensorMap *tensorMap,
  CUtensorMapDataType::CU_TENSOR_MAP_DATA_TYPE_INT32,
  rank,                       // cuuint32_t tensorRank,
  tensor_ptr,                 // void *globalAddress,
  size,                       // const cuuint64_t *globalDim,
  stride,                     // const cuuint64_t *globalStrides,
  box_size,                   // const cuuint32_t *boxDim,
  elem_stride,                // const cuuint32_t *elementStrides,
  // Interleave patterns can be used to accelerate loading of values that
  // are less than 4 bytes long.
  CUtensorMapInterleave::CU_TENSOR_MAP_INTERLEAVE_NONE,
  // Swizzling can be used to avoid shared memory bank conflicts.
  CUtensorMapSwizzle::CU_TENSOR_MAP_SWIZZLE_NONE,
  // L2 Promotion can be used to widen the effect of a cache-policy to a wider
  // set of L2 cache lines.
  CUtensorMapL2promotion::CU_TENSOR_MAP_L2_PROMOTION_NONE,
  // Any element that is outside of bounds will be set to zero by the TMA transfer.
  CUtensorMapFloatOOBfill::CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE
);
```

CUDA Programming Guide - Using the Tensor Memory Accelerator (TMA)

7-point Stencil

3D-TMA

- Pad halo for fp32 to adhere to alignment requirements (1 element in addition to the halo on each side)

- Kernel structure
  - Select a TMA thread (`__shfl_sync` to convince the compiler there is only 1 thread and no divergent branch)
  - Initialized shared memory barrier
  - Dispatch TMA
  - Set expected arrival count
  - Wait for completion
  - Use smem buffer to apply stencil

Init

Dispatch TMA

Wait for completion

Compute

```
__global__ void kernel (const __grid_constant__ CUtensorMap tmap, ... ) {

  //Allocate the barrier in shared memory
  __shared__ uint64_t bar;

  //...

  //Select a TMA thread
  const int warp_id = tid/32;
  const int uniform_warp_id = __shfl_sync(~0, warp_id, 0);
  const bool elect = uniform_warp_id == 0 && ptx::elect_sync(~0);

  // Initialize the barrier
  if(elect) ptx::mbarrier_init(&bar, 1);
  __syncthreads();

  //Dispatch the TMA copy
  if (elect)
  {
    ptx::cp_async_bulk_tensor(ptx::space_shared, ptx::space_global,
      &smem[0],
      &tmap,
      {blck_offset_x, blck_offset_y, z_coord},
      &bar);

    // Set the arrival count
    ptx::mbarrier_arrive_expect_tx(
        ptx::sem_release, ptx::scope_cta, ptx::space_shared, &bar, sizeof(T) * tsize);
  }

  // Wait for TMA copy to complete
  while (!ptx::mbarrier_try_wait_parity(&bar, 0)) {}

  // Use shared memory for compute && and store the result in global memory
  apply_stencil(&smem[0], out, ...);

}
```

7-point Stencil

3D-TMA

- Pad halo for fp32 to adhere to alignment requirements (1 element in addition to the halo on each side)

- Kernel structure
  - Select a TMA thread (`__shfl_sync` to convince the compiler there is only 1 thread and no divergent branch)
  - Initialized shared memory barrier
  - Dispatch TMA
  - Set expected arrival count
  - Wait for completion
  - Use smem buffer to apply stencil

Init

Dispatch TMA

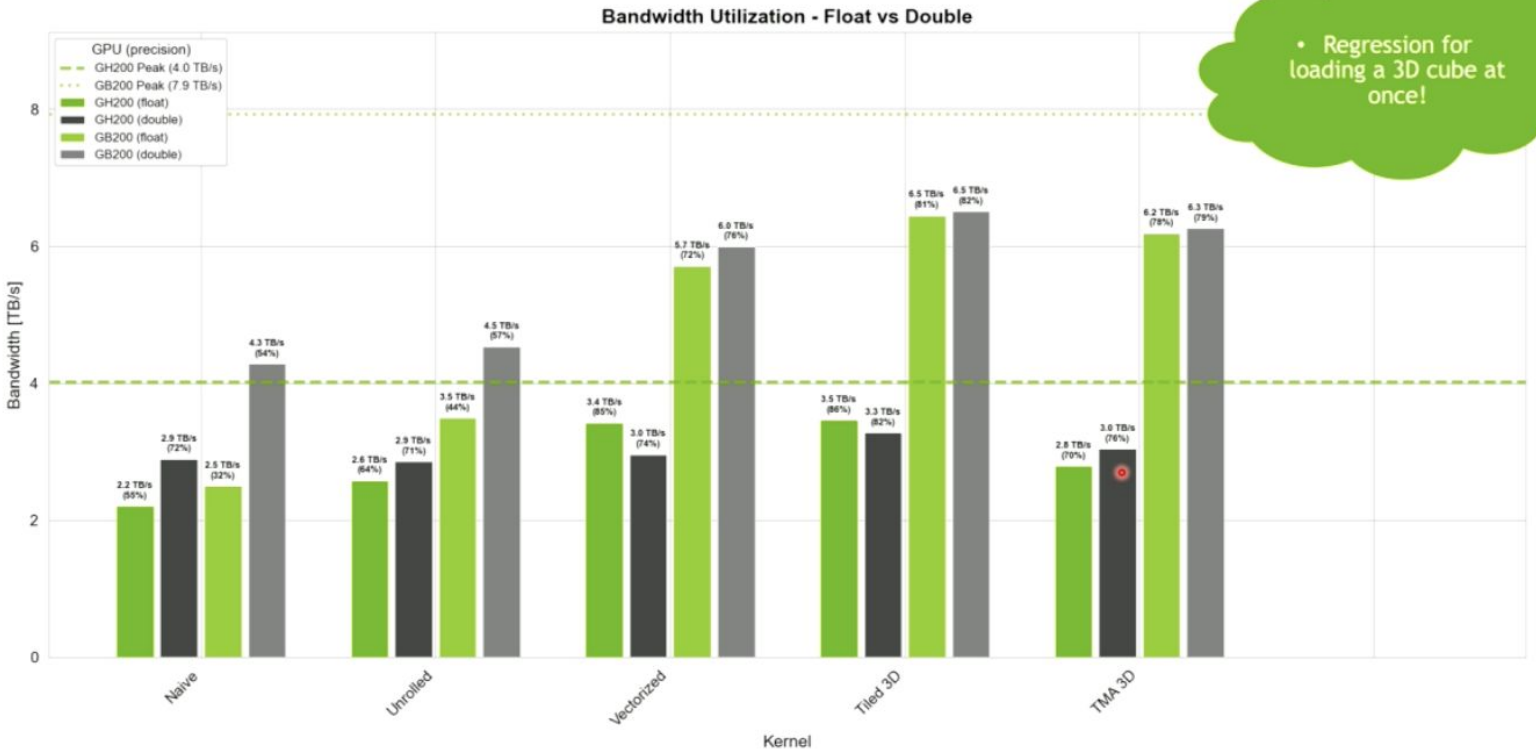
Wait for completion

Compute

```
__global__ void kernel (const __grid_constant__ CUtensorMap tmap, ... ) {

  //Allocate the barrier in shared memory
  __shared__ uint64_t bar;

  //...

  //Select a TMA thread
  const int warp_id = tid/32;
  const int uniform_warp_id = __shfl_sync(~0, warp_id, 0);
  const bool elect = uniform_warp_id == 0 && ptx::elect_sync(~0);

  // Initialize the barrier
  if(elect) ptx::mbarrier_init(&bar, 1);
  __syncthreads();

  //Dispatch the TMA copy
  if (elect)
  {
    ptx::cp_async_bulk_tensor(ptx::space_shared, ptx::space_global,
      &smem[0],
      &tmap,
      {blck_offset_x, blck_offset_y, z_coord},
      &bar);

    // Set the arrival count
    ptx::mbarrier_arrive_expect_tx(
        ptx::sem_release, ptx::scope_cta, ptx::space_shared, &bar, sizeof(T) * tsize);
  }

  // Wait for TMA copy to complete
  while (!ptx::mbarrier_try_wait_parity(&bar, 0)) {}

  // Use shared memory for compute && and store the result in global memory
  apply_stencil(&smem[0], out, ...);

}
```

Laplace stencil

Result

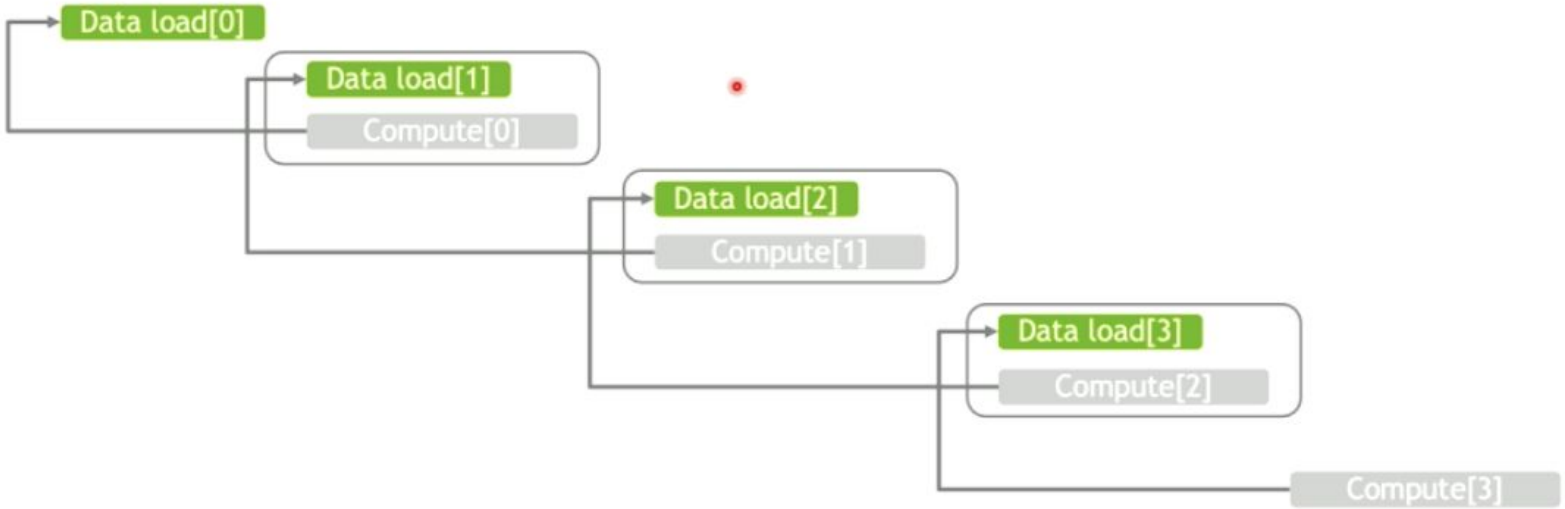


- Regression for loading a 3D cube at once!

7-point Stencil

Overlapping Compute and Memory Transfers

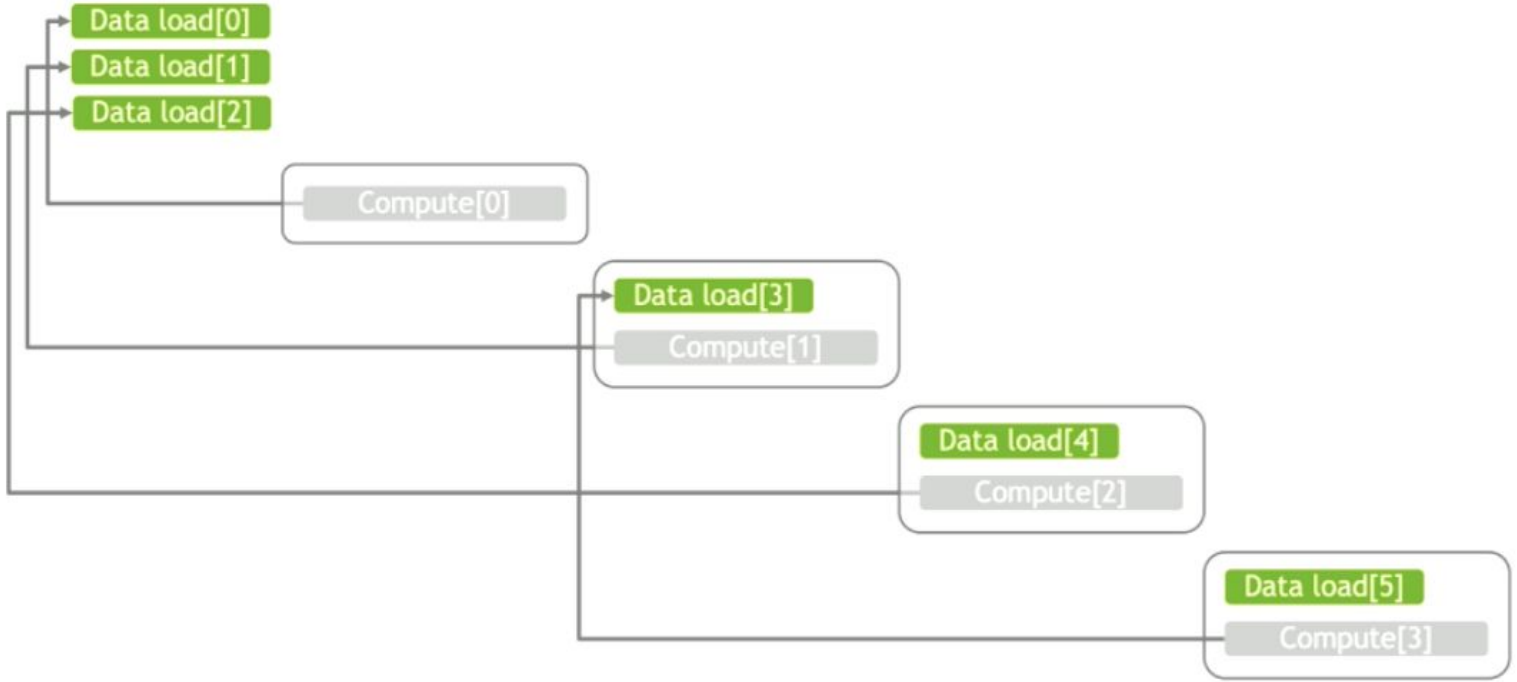
- Overlap compute and data loading



7-point Stencil

Overlapping Compute and Memory Transfers

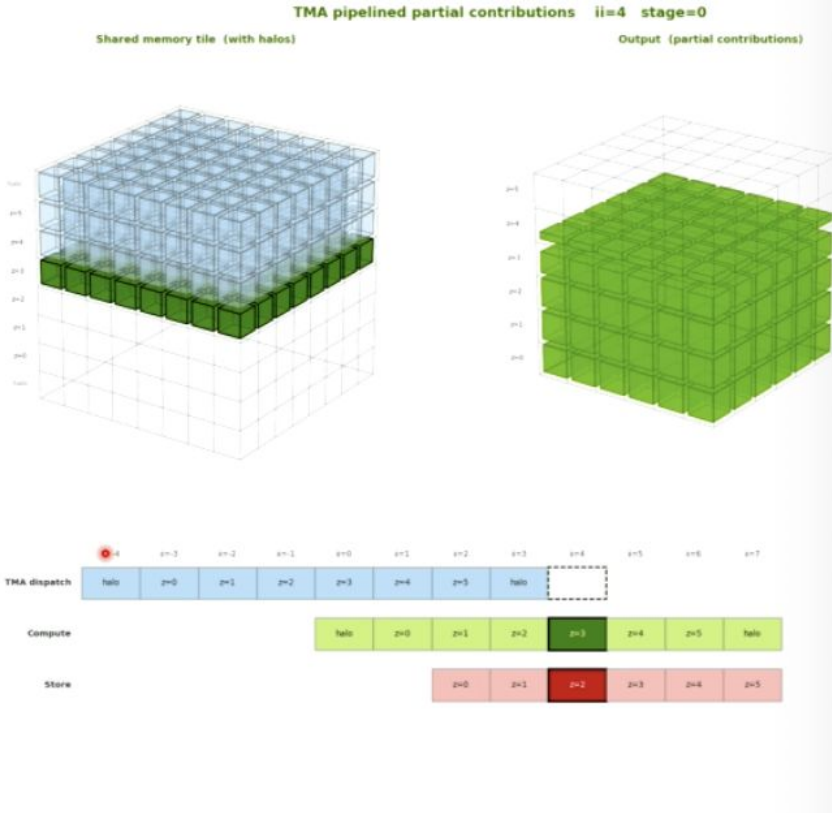
- Pipelining



7-point Stencil

Pipelined TMA

- Pipeline:
  - March in slow z direction.
  - Hold N_s stages in shared memory, each corresponding to a 2D tile (including halos).
  - Issue N_s TMA loads at the very beginning to get bytes in flight.
  - Upon completion of the 1st stage, start compute
  - Once compute finishes release the shared memory slice and issue the next TMA load.
- Compute:
  - Accumulate partial contributions, analogous to before
- Output:
  - Write the output from the registers via regular STG



7-point Stencil

Pipelined TMA

- Pipeline:
  - March in slow z direction.
  - Hold N_s stages in shared memory, each corresponding to a 2D tile (including halos).
  - Issue N_s TMA loads at the very beginning to get bytes in flight.
  - Upon completion of the 1st stage, start compute
  - Once compute finishes release the shared memory slice and issue the next TMA load.
- Compute:
  - Accumulate partial contributions, analogous to before
- Output:
  - Write the output from the registers via regular STG

Init N stages

Wait & compute

sync

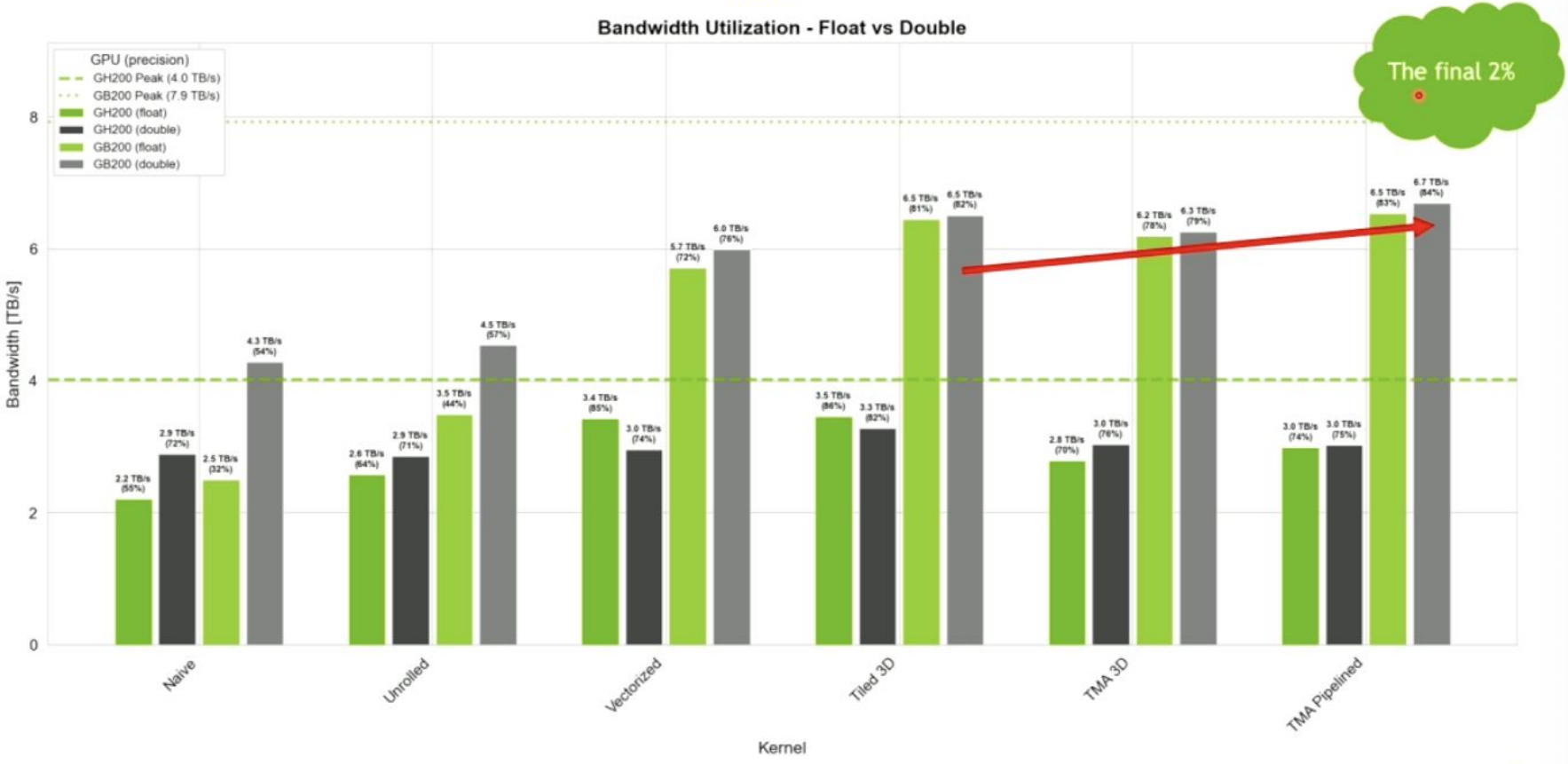
Issue TMA

Output & increment stage

```
__global__ void laplace_pipelined_tma(
  const CUtensorMap tmap, T* out, ...)
{
  //PAD smem_per_stage to 128 bytes
  __shared__ alignas(128) T smem[NUM_STAGES * smem_per_stage];
  __shared__ uint64_t bars[NUM_STAGES];

  if (tid < NUM_STAGES)
    ptx::mbarrier_init(&bars[tid], 1);
  __syncthreads();

  int stage = 0, parity = 0;
  for (int ii = -NUM_STAGES; ii < TILE_Z + 2*HALO; ++ii)
  {
    // — COMPUTE: wait for TMA, accumulate partial contributions —
    if (ii >= 0) {
      while (!ptx::mbarrier_try_wait_parity(&bars[stage], parity^1)) {}

      T* s = &smem[thread_offset + stage * stage_size];
      compute_stencil(s, out, ...);

      //Sync:
      __syncthreads();
      ptx::fence_proxy_async(ptx::space_shared);
    }


    // — TMA: async bulk load next z-plane —
    if (ii + NUM_STAGES < NUM_ITER) {
      if (elect) {
        ptx::cp_async_bulk_tensor(ptx::space_shared, ptx::space_global,
          &smem[stage * smem_per_stage],
          &tmap,
          {blck_offset_x, blck_offset_y, z_coord},
          &bars[stage]);

        ptx::mbarrier_arrive_expect_tx( ptx::sem_release, ptx::scope_cta,
          ptx::space_shared, &bars[stage], sizeof(T) * smem_per_stage);
      }
    }

    // — STORE: output completed plane —
    if (2 <= ii) out[...] = res_prev;

    stage++;
    if (stage >= NUM_STAGES) { stage = 0; parity ^= 1; }
  }
}
```

Laplace stencil

Result



NVIDIA

Summary / Conclusions

Maximize Memory Bandwidth on modern GPUs

- Kernels need to issue a certain number of transactions to achieve high memory bandwidth
  - Data dependencies can lead to stalls, which makes it hard to issue enough memory transactions
  - We have several tools at our disposal to help:
    - TLP: more active warps
    - Synchronous techniques (ILP, DLP)
    - Asynchronous techniques (L2 prefetch, LDGSTS, TMA)

- Modern GPUs have much higher DRAM bandwidth than previous architectures
  - Kernels may need modification to really take advantage of it:
    - Issue more bytes in flight to saturate memory bandwidth
    - Kernels may exhibit different bottlenecks on newer GPUs

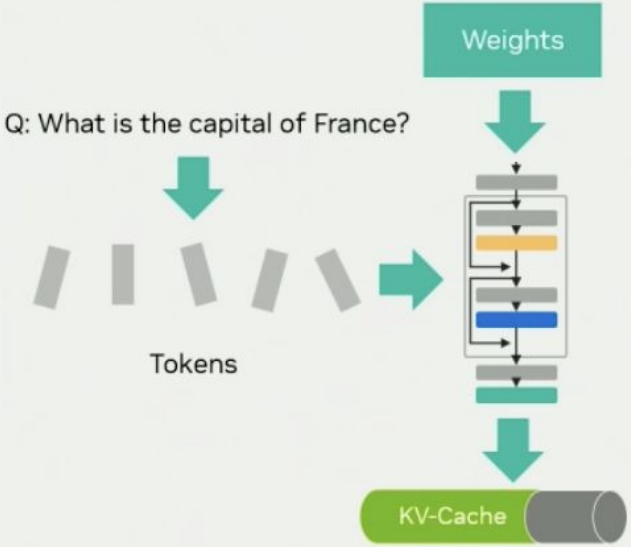
- If you have more questions, please feel free to come to our CWE:

  CWES81670 - Maximize Memory Bandwidth on Modern GPUs

Optimize Mixture-Of-Experts for Low-Latency Inference Decode on Blackwell [S81518]

Context phase and generation phase

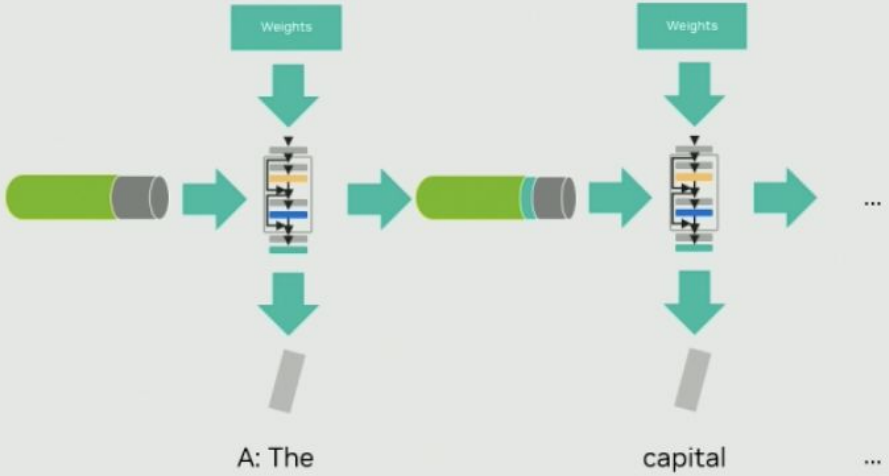
Listening and answering



Context phase

Processes all input tokens in **parallel**

Reads weights **once**



Generation phase

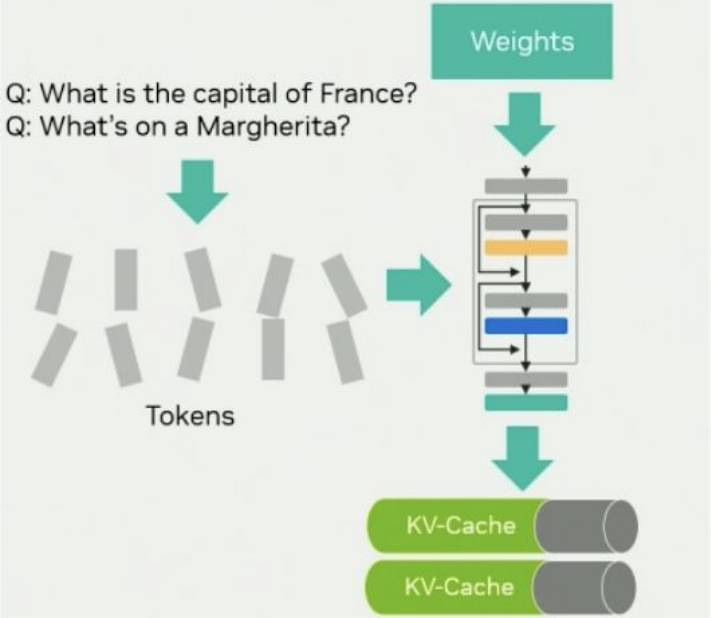
Generates output tokens **sequentially**

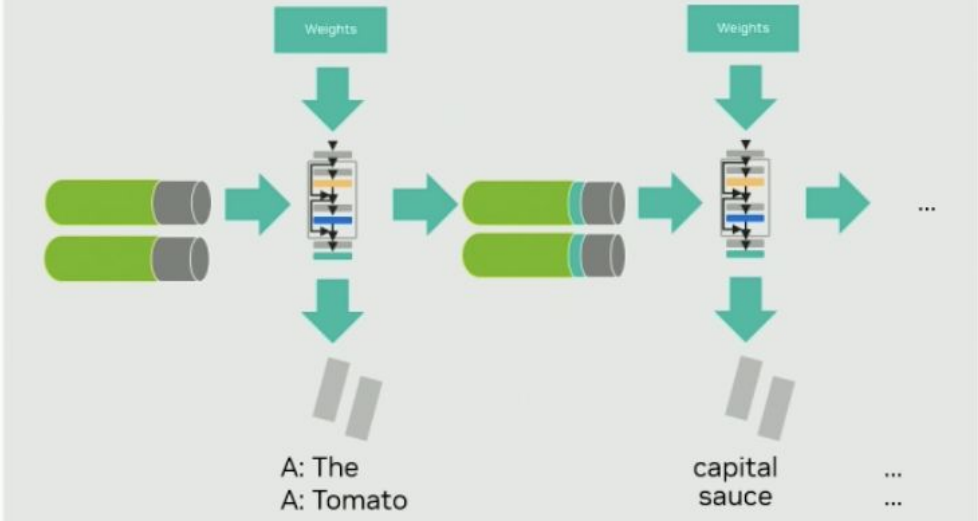
Reads weights **and** KV-Cache **in each step**

NVIDIA

Batching

Process multiple queries in parallel



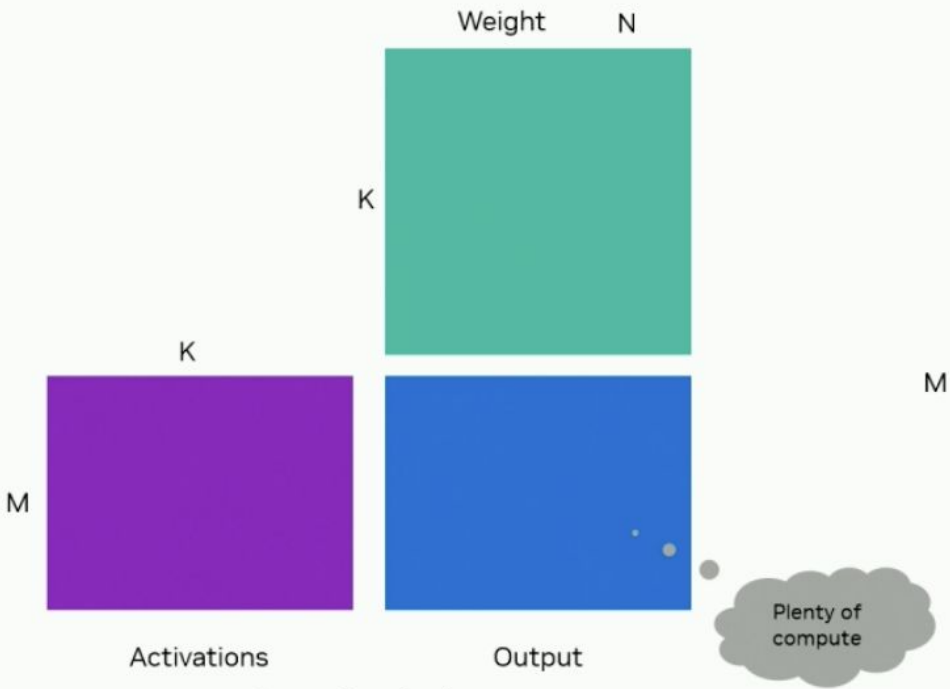


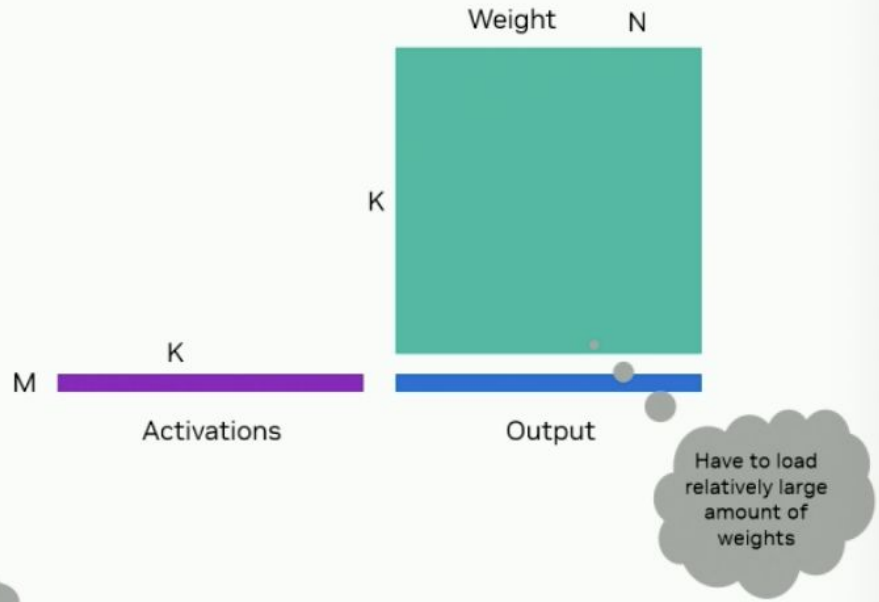
Batching

- Increases # of generated tokens per generation step
- Decreases bandwidth pressure of weight loads

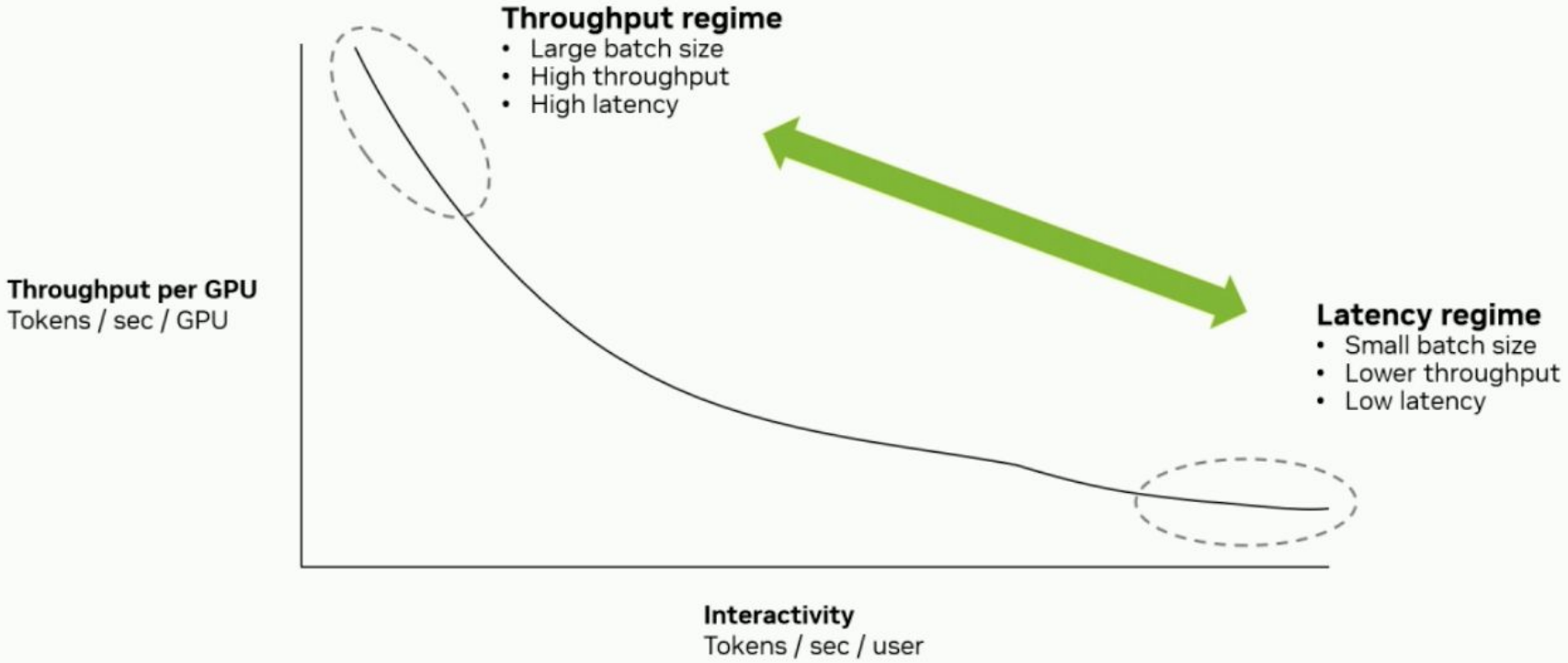
NVIDIA

Impact of batch size on GEMM





Throughput versus latency



Anatomy of Mixture-of-Experts (MoE)

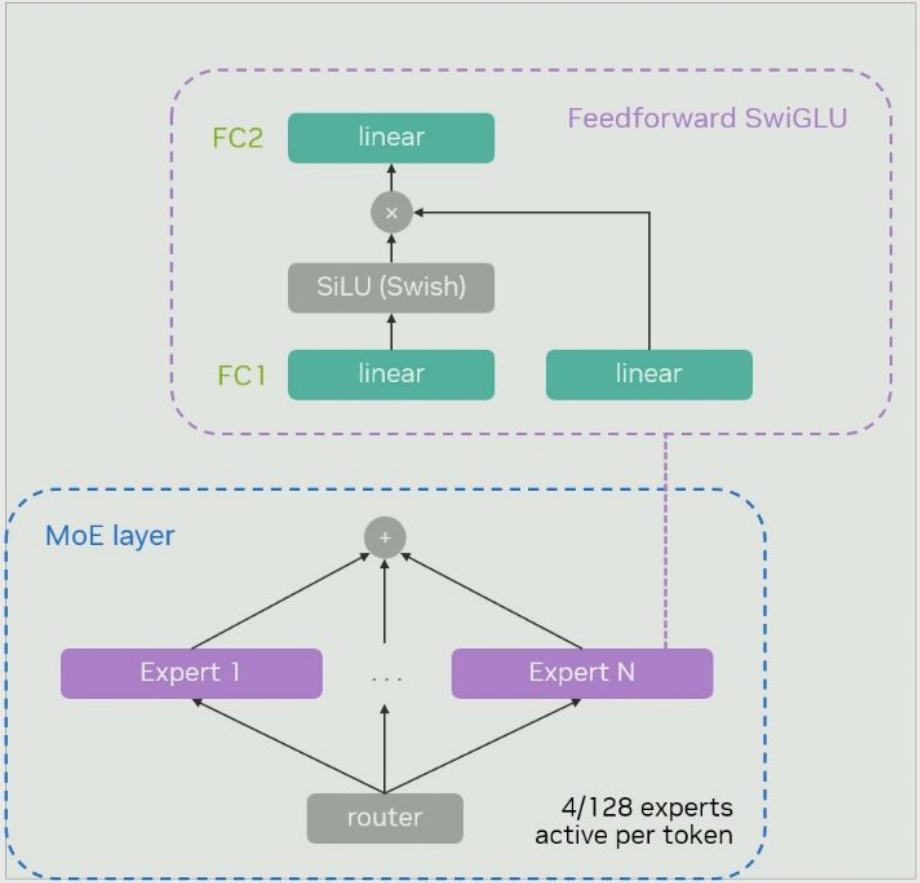
An MoE layer consists of hundreds of experts

Each token gets “routed” to a small subset of experts (4)

Each expert consists of

- A gated linear layer – 2 GEMMs + activation (FC1)
- A linear layer (FC2)

Output is weighted sum of experts



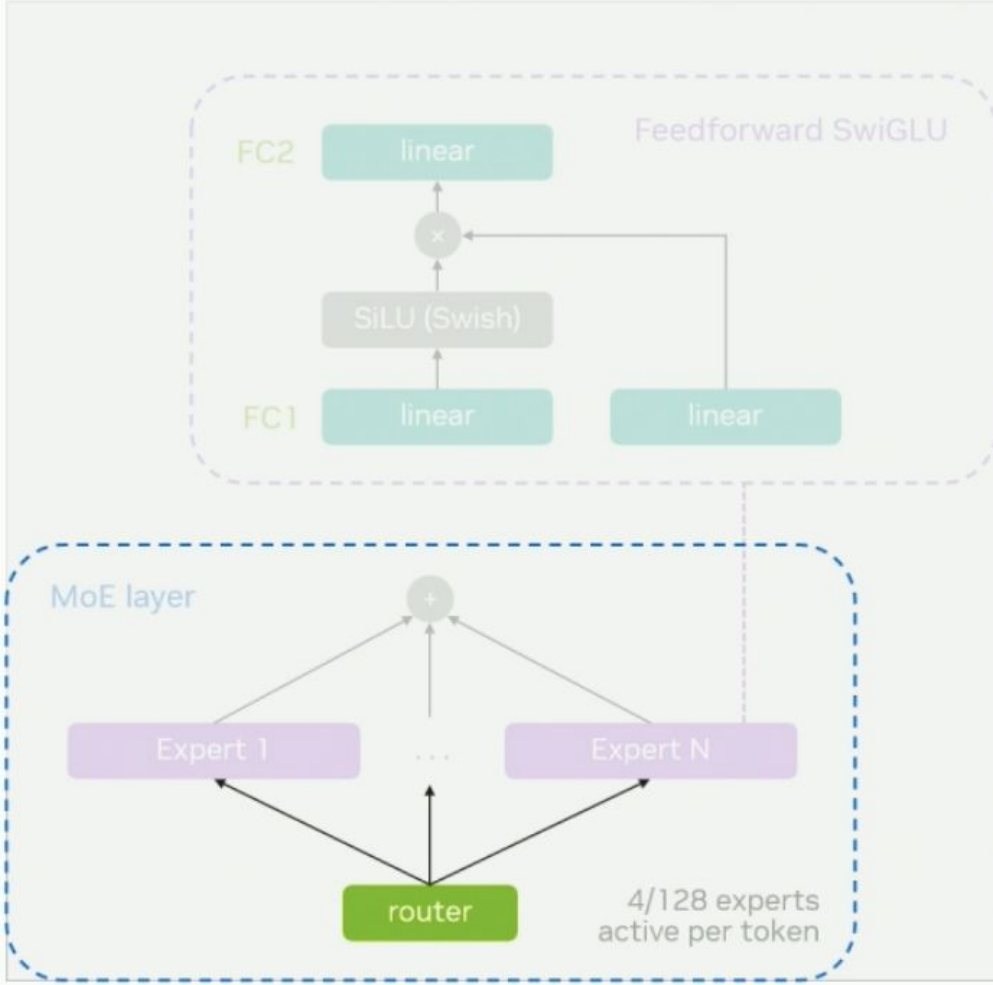
Courtesy of Louis Sugy

Routing

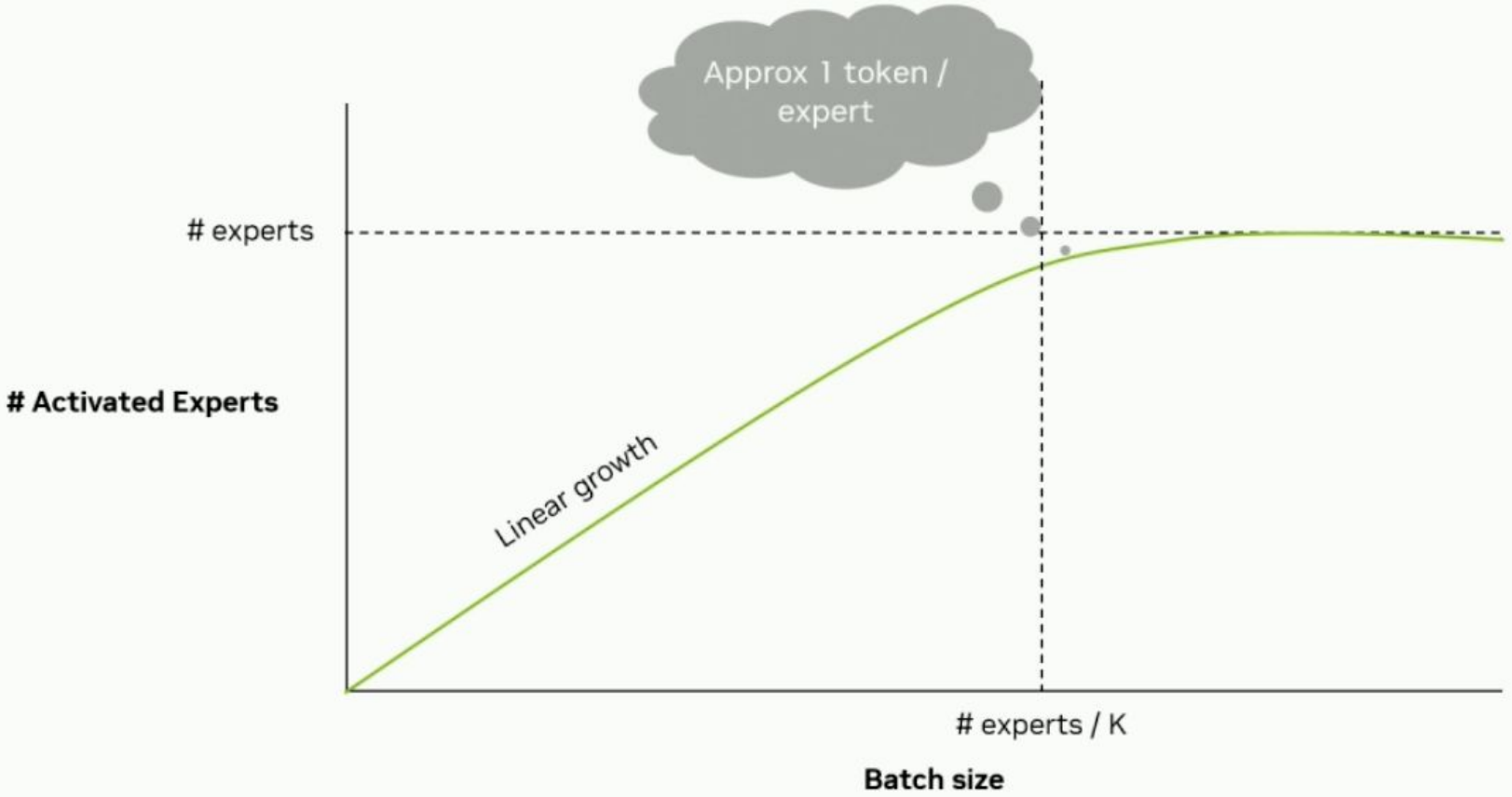
Each (token, expert) combination gets a score

Score is computed by a small GEMM

Per token: top-K scores determine K activated experts



Number of activated experts



Many kernels...

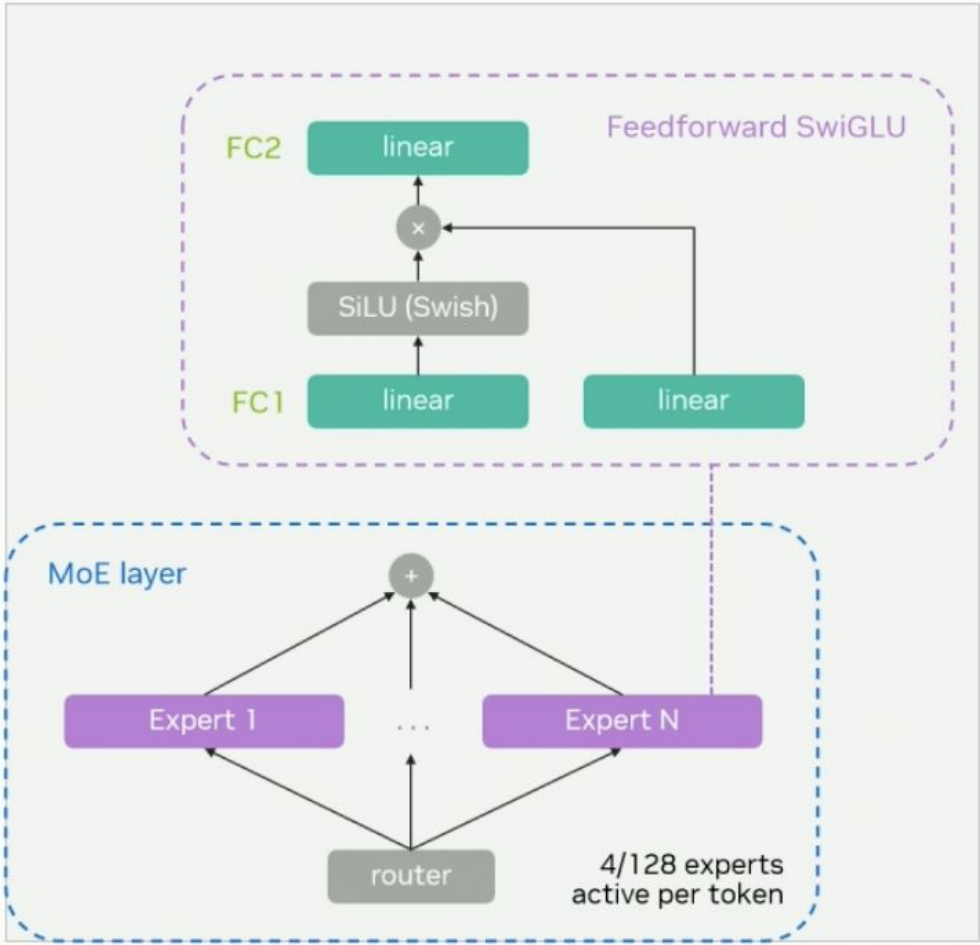
100 experts * 3 GEMMs / Expert → 300 GEMMs / MoE

1 us / kernel → 300 us per MoE

100 layers → 30ms / per LLM step

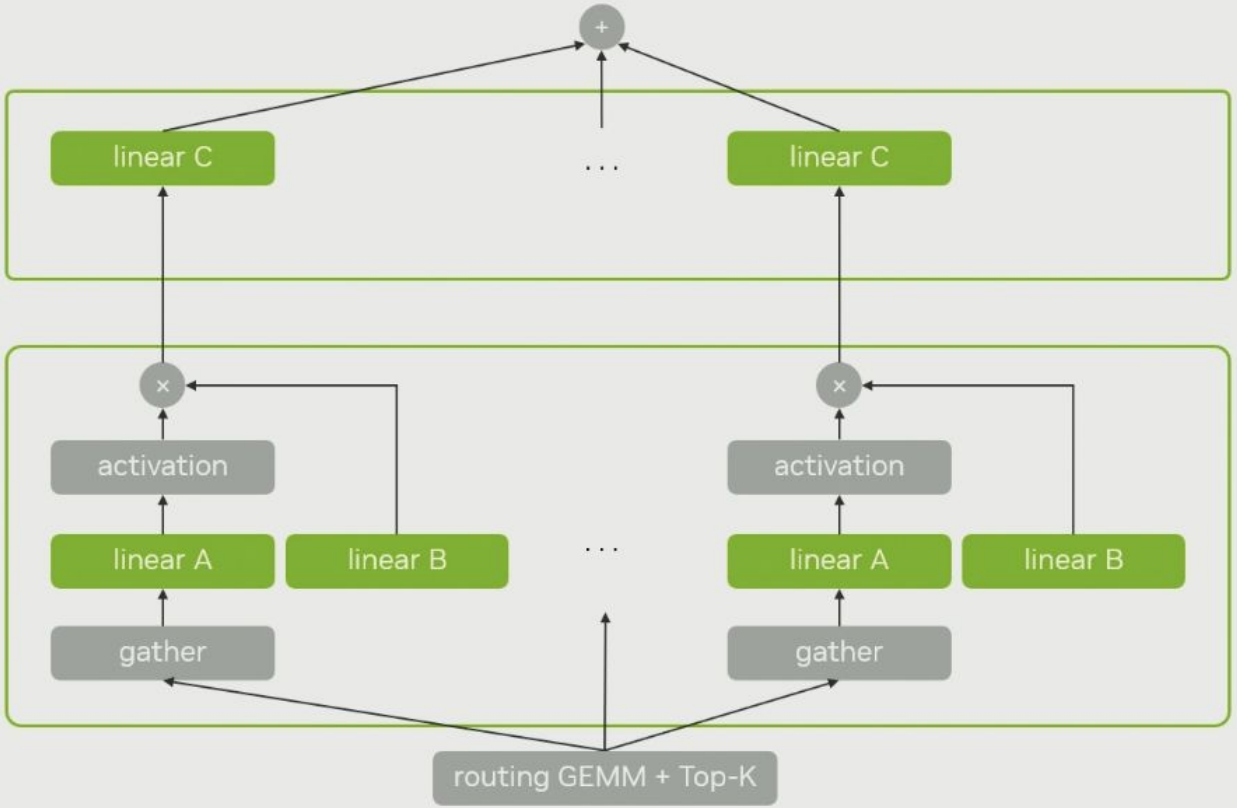
❌ Maximum of **30 tokens / second**

→ We need to **fuse** kernels



Fusing Mixture of Experts

2 kernels



Batched GEMM FC2

- Grouped by expert
- All linear layers C are computed in the same kernel

Batched GEMM FC1 with fused gather and gated activation

- Grouped by expert
- Gathers tokens routed to this expert
- Linear layers A and B are computed in the same kernel
- Activation is computed in the epilogue

Low-latency Mixture-of-Experts

Challenges

Memory bound

- Read weights every generation step
- Small batch size

Non-square GEMM

- Small batch size
- Split per expert
  - Even smaller M dimension

Short kernels

- Kernel runtimes become shorter
- Amdahl's Law: sequential latencies start to dominate
  - Launch latency
  - Prologue latency

Non-contiguous data

- Want to avoid redundant data movement
- Incorporate gather/scatter into GEMM kernel

Blackwell Tensor Core

tcgen05.mma family with Tensor Memory

Data type: TF32, BF16, FP16, {MX}FP8, {MX}FP6, {MX}FP4, NVFP4

Accumulators are in TMEM.

Operand A in TMEM or SMEM.

Operand B in SMEM

Scaling factors are in TMEM

INST_N = [8, 256],

INST_K = 32B (sm_100f), 48B(sm_103a, {MX,NV}FP4)

eg: FP16/BF16 INST_K = 16;

FP8. INST_K = 32

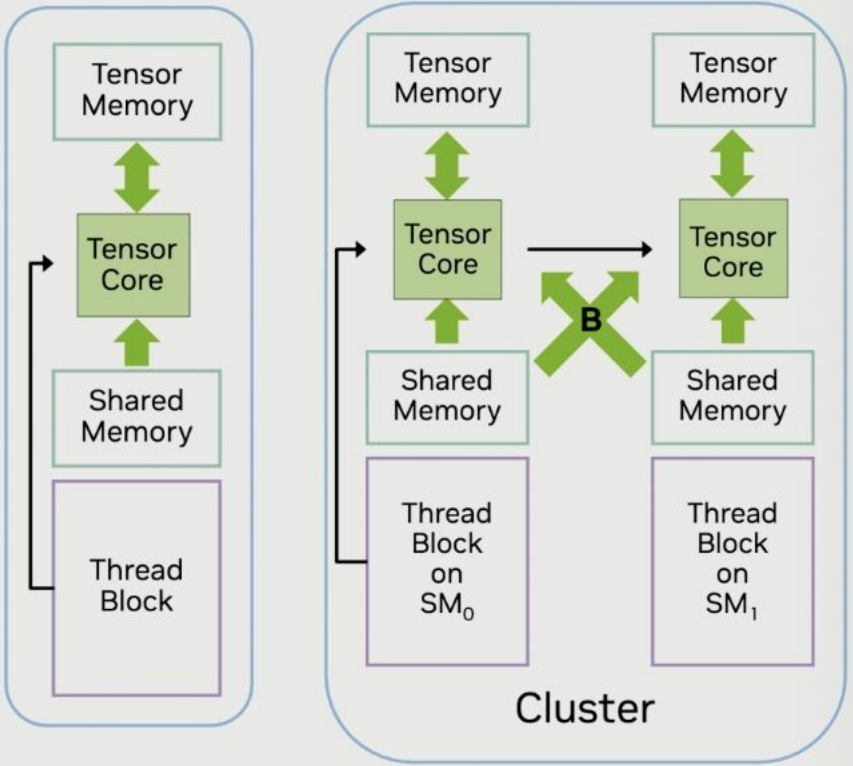
1 thread in CTA/CTA-pair submits the tensor core async execution.

1CTA mode:

- INST_M = 64, 128

2CTA mode:

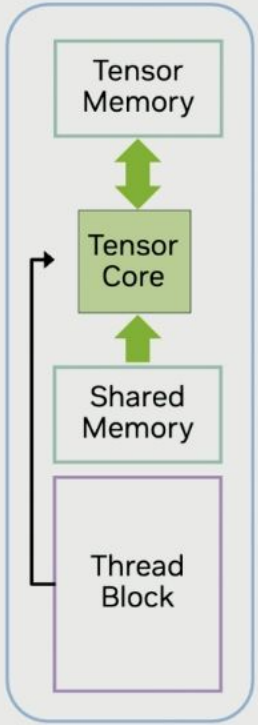
- INST_M = 128, 256

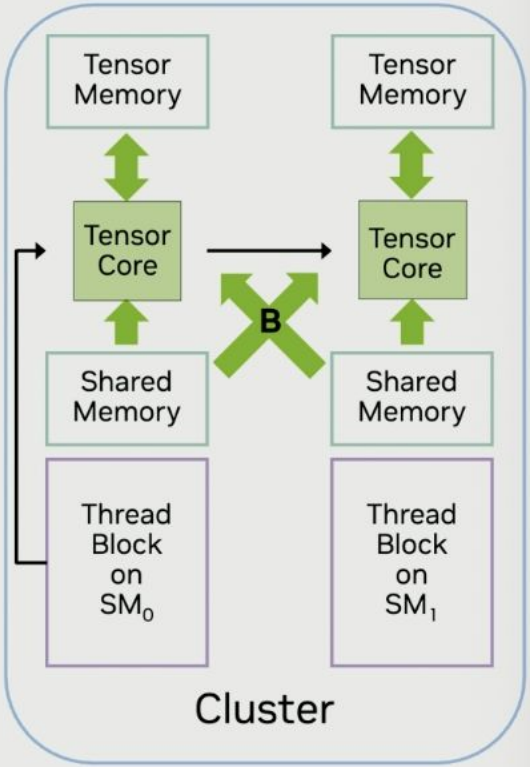


Blackwell Tensor Core

tcgen05.mma family with Tensor Memory

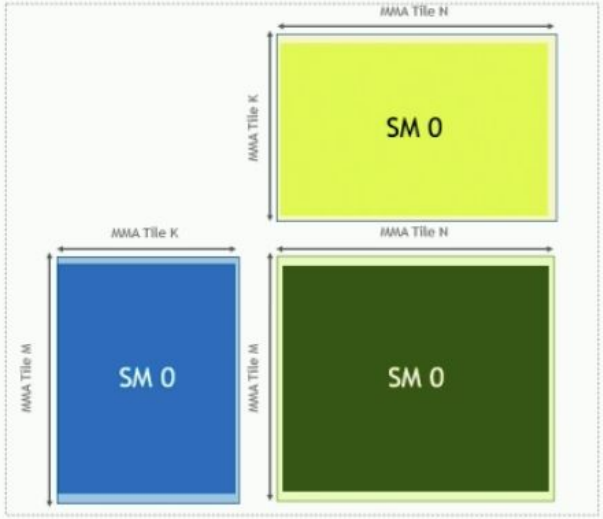
```
// 1. Floating-point type without block scaling:

tcgen05.mma.cta_group.kind   [d-tmem],  a-desc,  b-desc, idesc,
                             { disable-output-lane }, enable-input-d {, scale-input-d};

tcgen05.mma.cta_group.kind   [d-tmem], [a-tmem], b-desc, idesc,
                             { disable-output-lane }, enable-input-d {, scale-input-d};

.kind      = { .kind::f16, .kind::tf32, .kind::f8f6f4 }
.cta_group = { .cta_group::1, .cta_group::2 }

-------------------------------------------------------------------------------

// 2. Floating-point type with block scaling:

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}
                                     [d-tmem],  a-desc,  b-desc, idesc,
                                     [scale-A-tmem], [scale-B-tmem], enable-input-d;

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}
                                     [d-tmem], [a-tmem], b-desc, idesc,
                                     [scale-A-tmem], [scale-B-tmem], enable-input-d;

.kind = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }
.cta_group      = { .cta_group::1,   .cta_group::2 }
.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .block32 }

-------------------------------------------------------------------------------
```





Blackwell Tensor Core

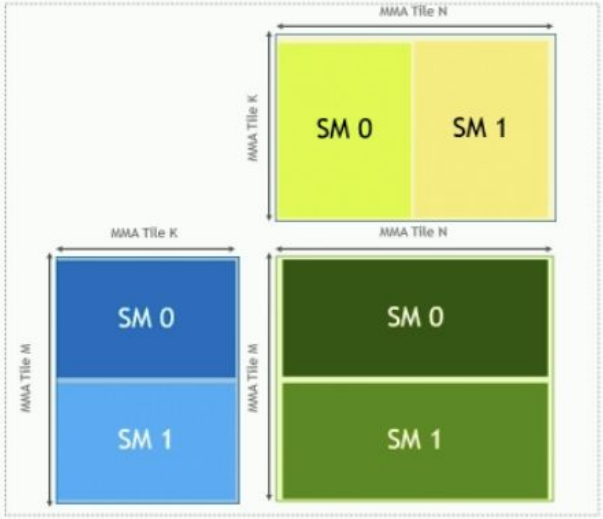
Data distribution in 1CTA & 2CTA mode



1 CTA mode:

CTA holds the whole matrixes:

A(Smem or Tmem), B(Smem) and C(Tmem)



2CTA mode:

Each CTA holds the partial matrixes:

Half A(Smem or Tmem). Each holds M/2 x K

Half B(Smem). Each holds N/2 x K, will be exchanged between SMs

Half C(Tmem). Each holds M/2 x N

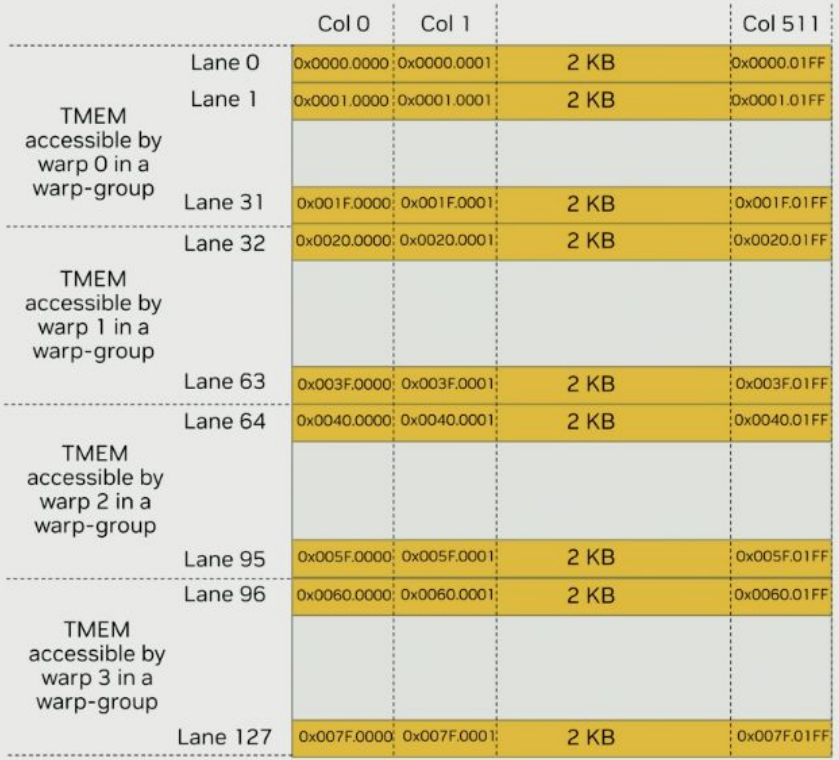
NVIDIA

Blackwell TMEM and PTX

- TMEM is word-addressed memory.

bits	31… 23	22 …16	15 … 9	8 … 0
encoding	require = 0	TMEM lane	require = 0	TMEM column

- Addresses consist of 128 "lanes" and 512 "columns".
- Each warp within a warp-group can access only 32 lanes.
- TMEM is explicitly allocated.
- CTAs allocate using tcgen05.alloc PTX
  - Allocation granularity is 128 lanes x 32 columns = 16 KB
  - On allocation, receive a warp-local TMEM address.
- CTAs deallocate using tcgen05.dealloc PTX

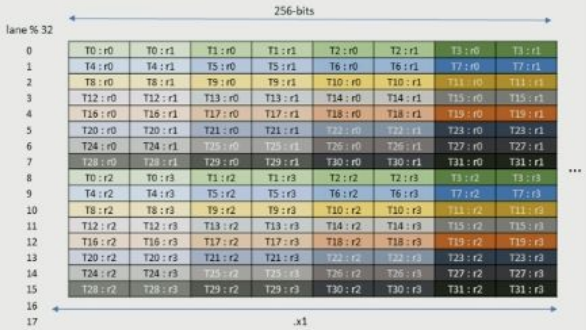


Blackwell TMEM Tensors and Ops

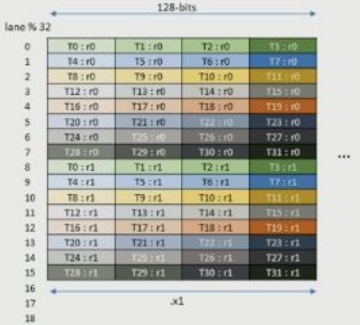
The restrictions of accessing Tmem

- TMEM addresses can NOT be dereferenced!
- Issue warp-group wide instructions to move data collectively.
  - TMEM ←→ RMEM using tcgen05.load and tcgen05.store PTX.
- 1 thread leads the CTA/CTA-Pair wide data movements.
  - TMEM ← SMEM using tcgen05.cp PTX.
- Access patterns of load/store are fixed by pre-defined layouts.

- **32dp32b2x** instruction
- Each warp access all 32 lanes
- [N]x allows to select num cols
- Good for row major direct stores



- **16dp256b1x** instruction
- Each warp accesses only 16 lanes.
- Good for 32b accumulator ->16b output stores when paired with stmatrix.m8n8



- **16dp128b1x** instruction
- Each warp accesses only 16 lanes.

NVIDIA

Leader thread of the warp

All threads in the warp

Blackwell pipelined TC instruction

tcgen05.cp and tcgen05.mma are implicitly pipelined from same thread & same execution scope(1CTA or 2CTA)

A in SMEM, B in SMEM

mbarrier.try_wait(++mbar_smem_full)

tcgen05.mma tmem_accum, ++A_smem, ++B_smem

tcgen05.mma tmem_accum, ++A_smem, ++B_smem

tcgen05.mma tmem_accum, ++A_smem, ++B_smem

tcgen05.mma tmem_accum, ++A_smem, ++B_smem

tcgen05.commit ++mbar_smem_empty

Loop over all the K iterations

tcgen05.commit mbar_tmem_full

A in TMEM, B in SMEM

mbarrier.try_wait(++mbar_smem_full)

tcgen05.cp tmem_A[0], ++A_smem,

tcgen05.cp tmem_A[1], ++A_smem,

tcgen05.cp tmem_A[2], ++A_smem,

tcgen05.cp tmem_A[3], ++A_smem,

tcgen05.mma tmem_accum, tmem_A[0], ++B_smem

tcgen05.mma tmem_accum, tmem_A[1], ++B_smem

tcgen05.mma tmem_accum, tmem_A[2], ++B_smem

tcgen05.mma tmem_accum, tmem_A[3], ++B_smem

tcgen05.commit ++mbar_smem_empty

Loop over all the K iterations

tcgen05.commit mbar_tmem_full

More pipelined instruction descriptions in PTX ISA.

NVIDIA

Blackwell NVFP4 Tensor Core

tcgen05.mma with block scaling factors, take NVFP4 as example

A&B data type: SE2M1

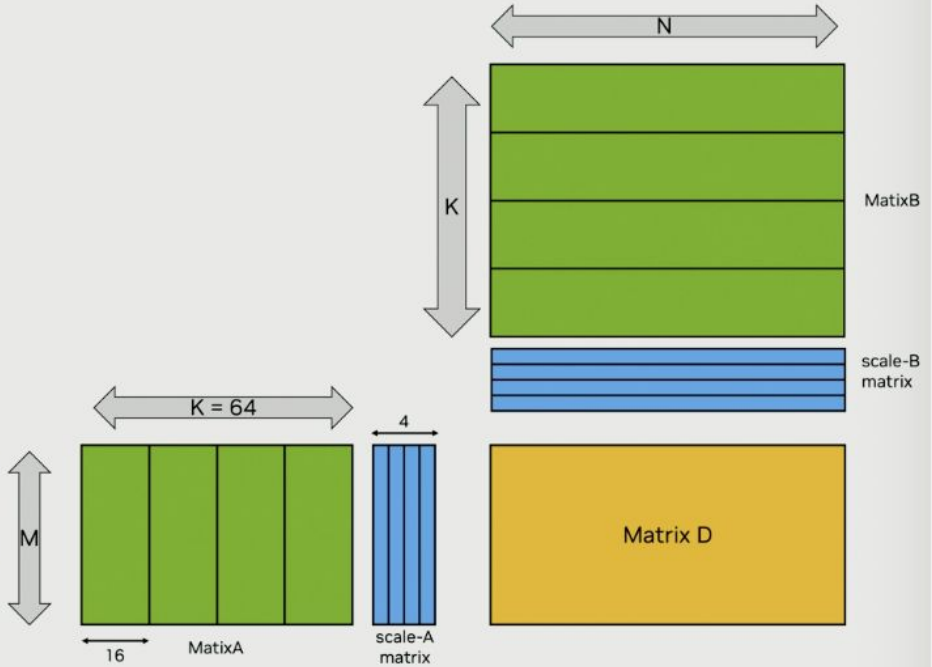
Operand A in TMEM or SMEM.

Operand B in SMEM

Scale factors data type: UE4M3

Operand A-sfs & B-sfs in TMEM

D[M][N] += (Asfs[M][0] x A[M][0:15]) x (Bsfs[0][N] x B[0:15][N])
+ (Asfs[M][1] x A[M][16:31]) x (Bsfs[1][N] x B[16:31][N])
+ (Asfs[M][2] x A[M][32:47]) x (Bsfs[2][N] x B[32:47][N])
+ (Asfs[M][3] x A[M][48:63]) x (Bsfs[3][N] x B[48:63][N])



Additional scale factors movement (SMEM->TMEM)is required.

Leader thread of the warp

All threads in the warp

Blackwell NVFP4 TensorC pipeline

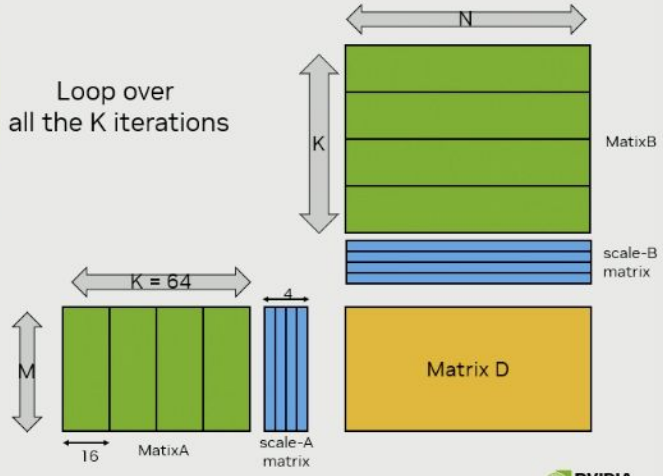
tcgen05.cp and tcgen05.mma are implicitly pipelined from same thread & same execution scope(1CTA or 2CTA)

A in SMEM, B in SMEM, Asfs in TMEM, Bsfs in TMEM

```
mbarrier.try_wait(++mbar_smem_full)

tcgen05.cp tmem_Asfs[0], ++Asfs_smem
tcgen05.cp tmem_Bsfs[0], ++Bsfs_smem
⋮
tcgen05.cp tmem_Asfs[3], ++Asfs_smem
tcgen05.cp tmem_Bsfs[3], ++Bsfs_smem

tcgen05.mma tmem_accum, ++A_smem, ++B_smem, tmem_Asfs[0], tmem_Bsfs[0]
tcgen05.mma tmem_accum, ++A_smem, ++B_smem, tmem_Asfs[1], tmem_Bsfs[1]
tcgen05.mma tmem_accum, ++A_smem, ++B_smem, tmem_Asfs[2], tmem_Bsfs[2]
tcgen05.mma tmem_accum, ++A_smem, ++B_smem, tmem_Asfs[3], tmem_Bsfs[3]

tcgen05.commit ++mbar_smem_empty

tcgen05.commit mbar_tmem_full
```

Loop over
all the K iterations

4x Tensor Core instructions for Ktile_128B

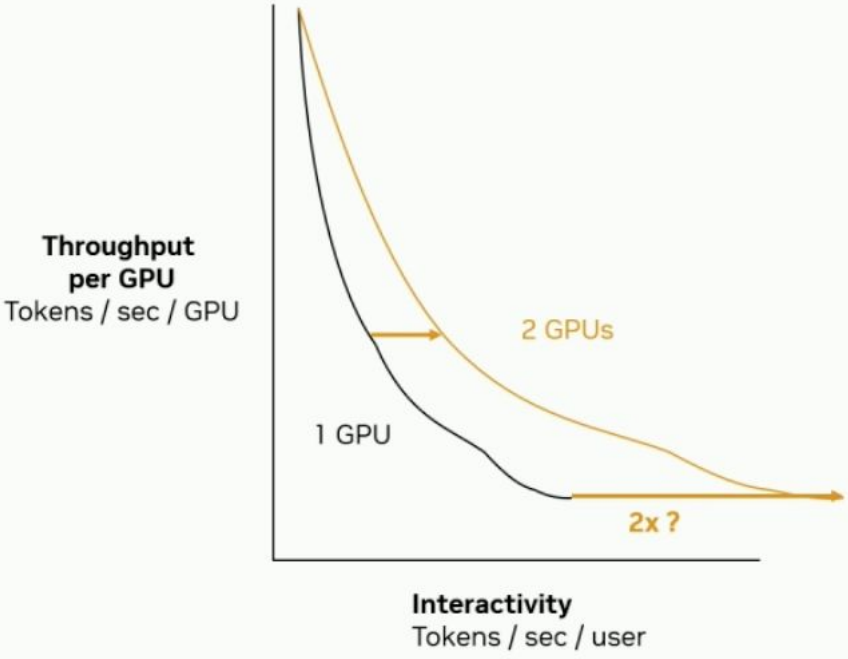


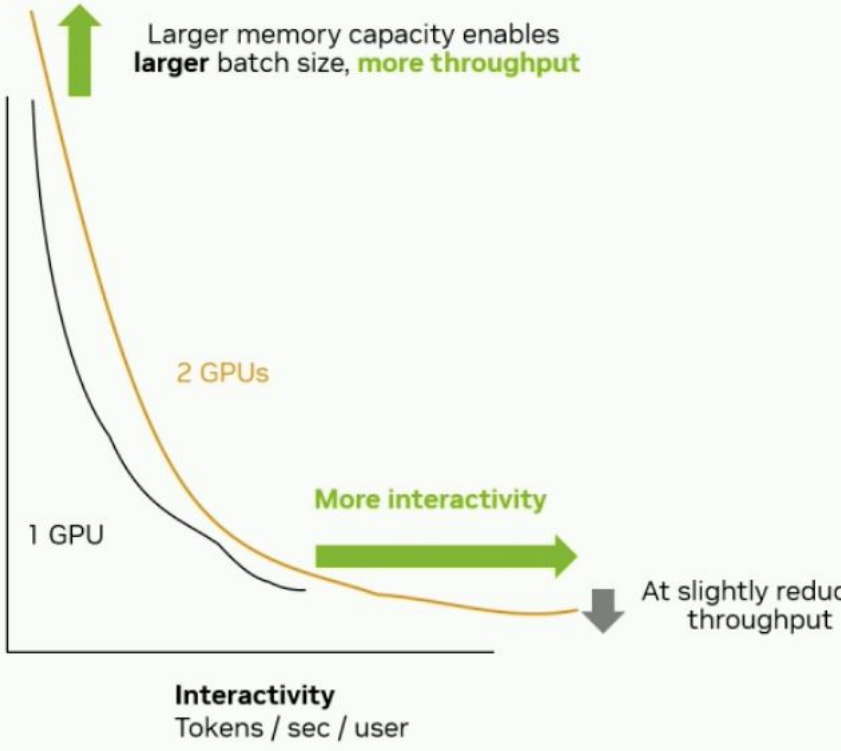
NVIDIA

Using more GPUs

Effect on interactivity and throughput

Does doubling the GPUs double the interactivity?





Using more GPUs

Distributing the work across GPUs

Expert parallelism (EP)

Each GPU gets **half the experts**

May suffer from expert imbalance

- One GPU could get all tokens – other GPU none

Less NVLink communication

- Communicates 1 token per expert token

Tensor parallelism (TP)

Each GPU gets **half of each expert**

Always balanced

- Each GPU processes same number of tokens

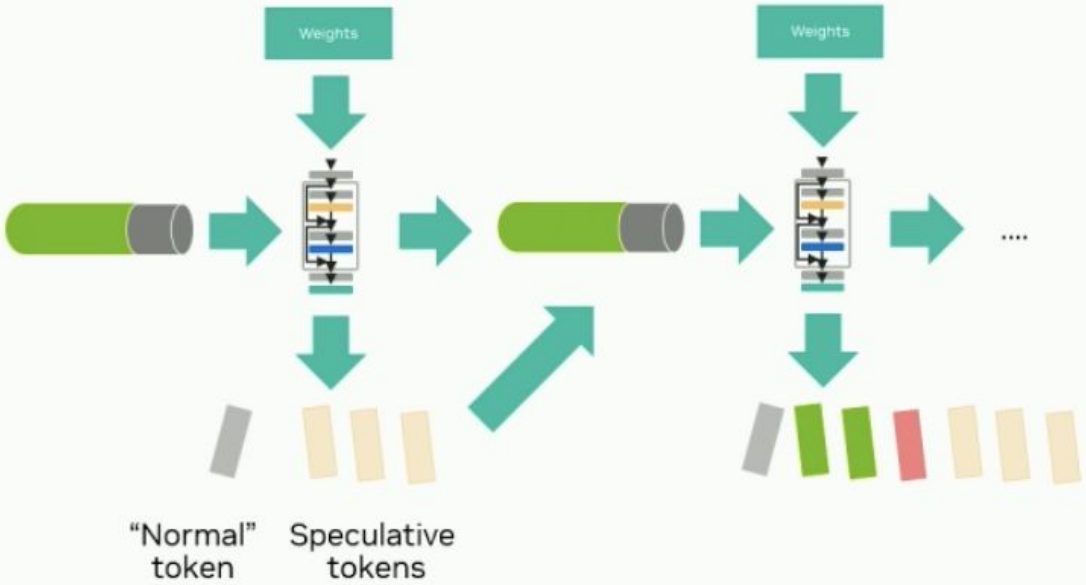
More NVLink communication

- Communicates 2 tokens per expert output token

More suited to **throughput** regime

More suited to **latency** regime

Multi-token prediction / Speculative decoding



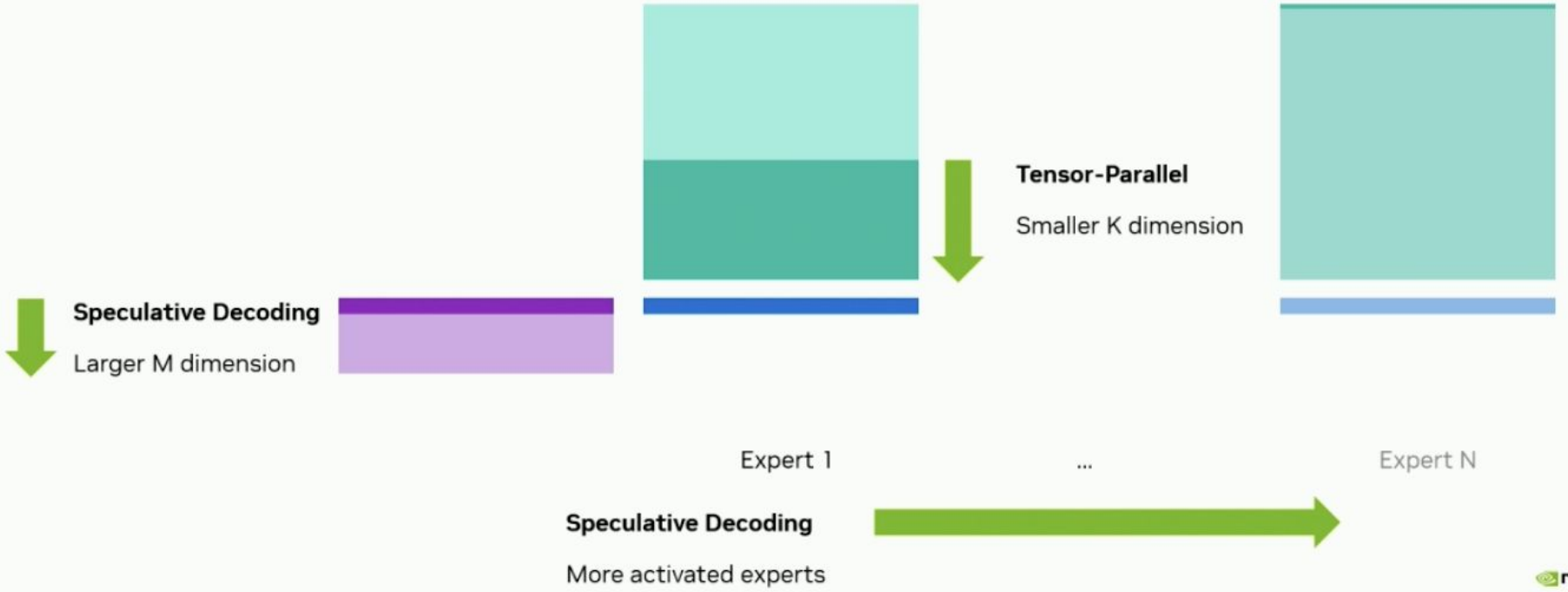
- Generate one token and **several speculative tokens**
- Speculative tokens can be be generated by
  - A dedicated smaller model running parallel
  - Extra network layers in the main LLM
- The extra tokens must be **validated**

Implications

- The batch size for the Mixture-of-Experts is higher
- The model can generate 1-4 tokens per iteration

Impact of system-level optimizations on MoE kernel

Summary

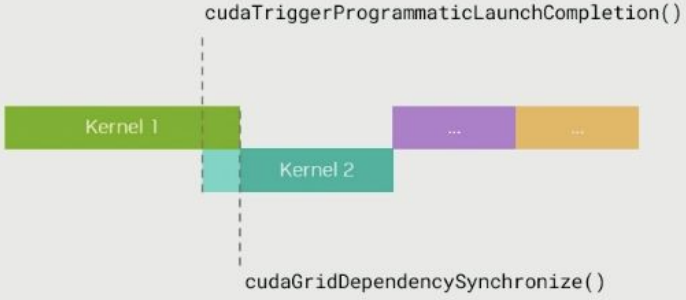


NVIDIA

System-level optimizations for short-running kernels

Reload this page

Programmatic Dependent Launch (PDL)



Normal kernel launches

- Kernels in same stream are launched back-to-back

PDL Launch

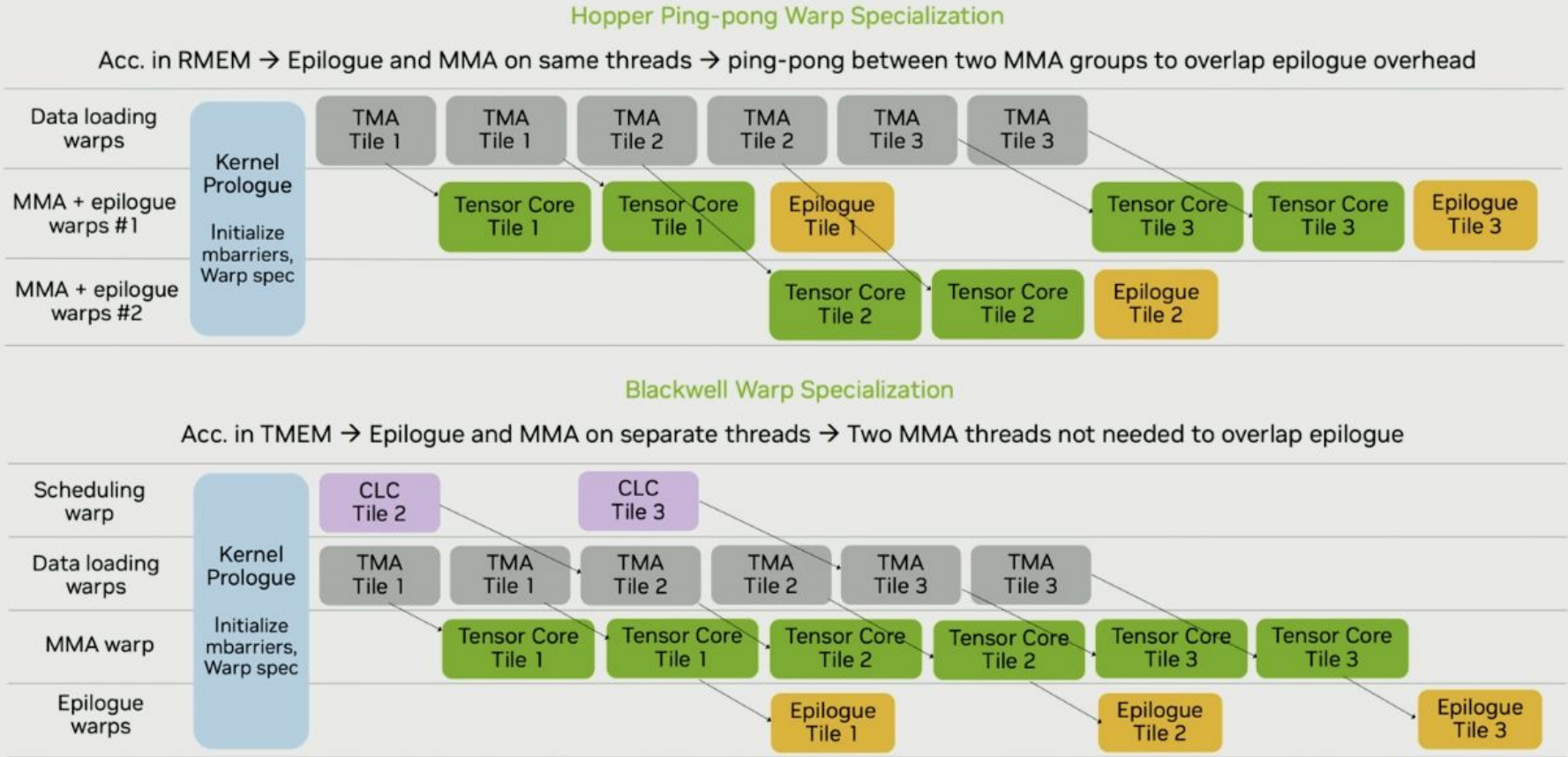
- Kernels in same stream may overlap
- Launch subsequent grid earlier using
  - `cudaTriggerProgrammaticLaunchCompletion()`
- Wait for data from previous grid using
  - `cudaGridDependencySynchronize()`
- Allows independent work to overlap
- Can give speedups even without independent work

For more information: S72686 CUDA Techniques to Maximize Concurrency and System Utilization

NVIDI

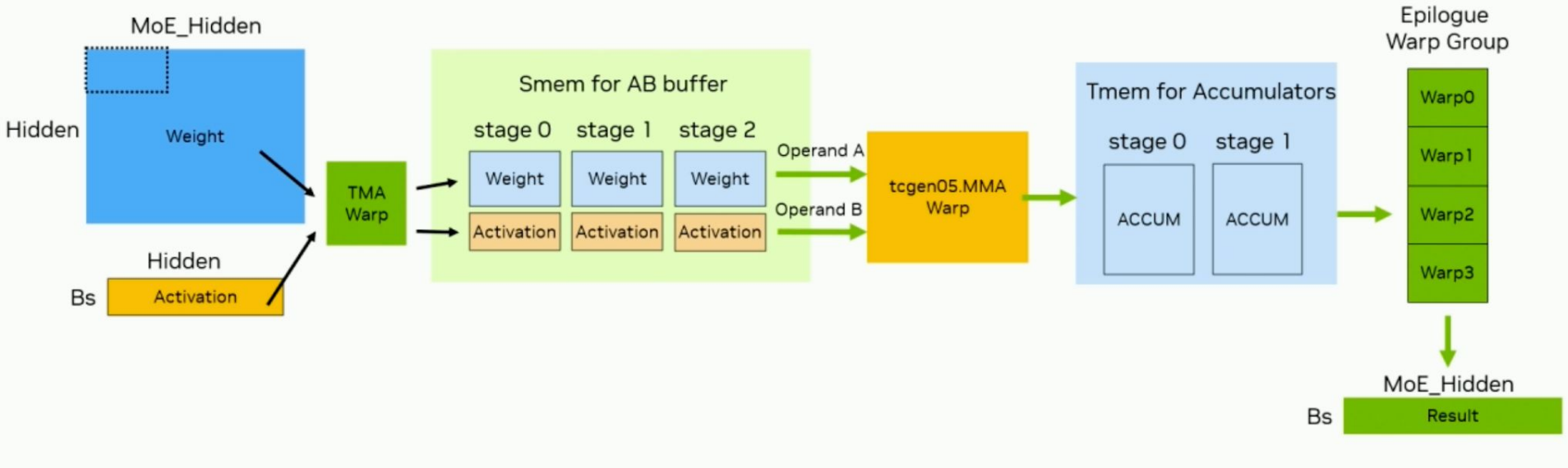
Overall Warp-Specialized Kernel Design

Evolution Hopper to Blackwell



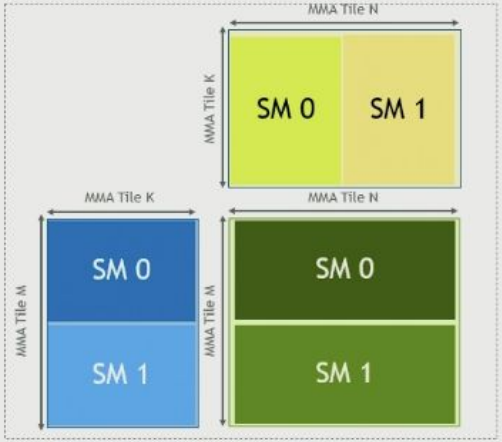
Blackwell Warp-Specialized Kernel Design

Core data movement



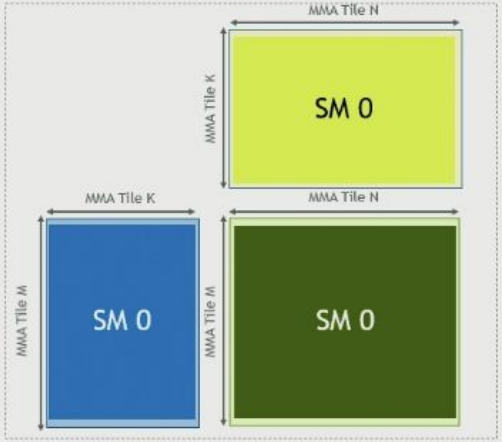
Tensor Core Warp

When to use 1CTA versus 2CTA?



- 2CTA mode
  - ✓ Better for large N
  - X More synchronization in MMA mainloop
    - ➢ But synchronization can be hidden
  - ✓ Smaller B in shared memory
    - ✓ More stages in shared memory
    - ✓ Can hide more memory latency

Better suited to **throughput regime**

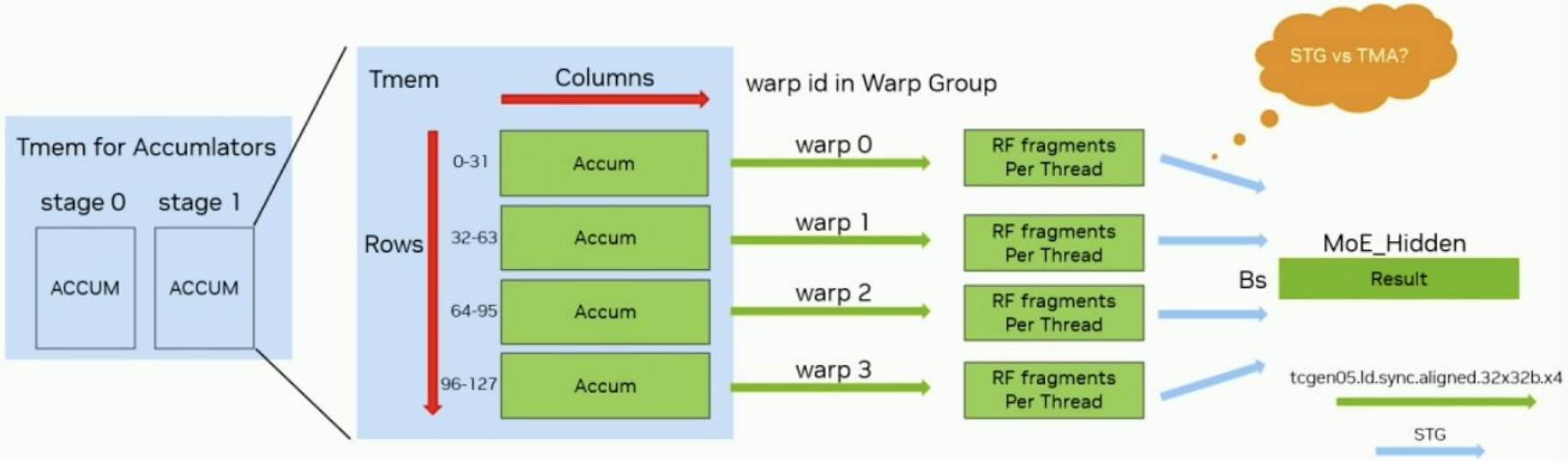


- 1CTA mode
  - ✓ Better for small N
  - ✓ Less synchronization in MMA mainloop
  - X Duplicates B in shared memory
    - ➢ **Activation is relatively small in low-latency**

Better suited to **latency regime**

Epilogue

4 warps access TMEM across all sub-partitions



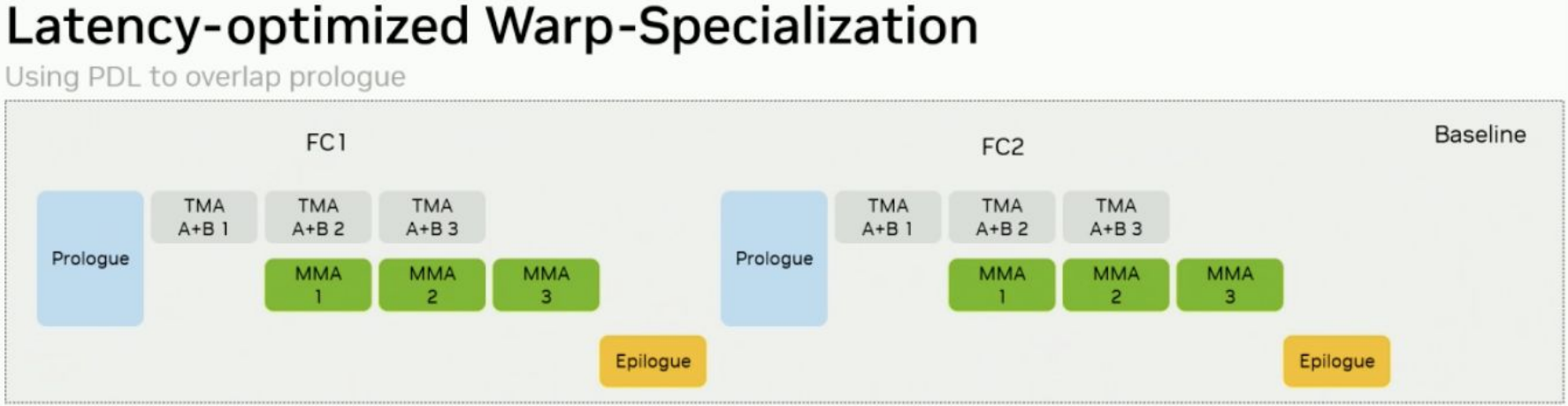
TMA

- ✗ Requires additional shared memory
- ✗ Extra shared memory store + synchronization latency
- ✓ Save instruction issue bandwidth
- ✓ Supports any output layout (M, N-major)

STG (thread store to global)

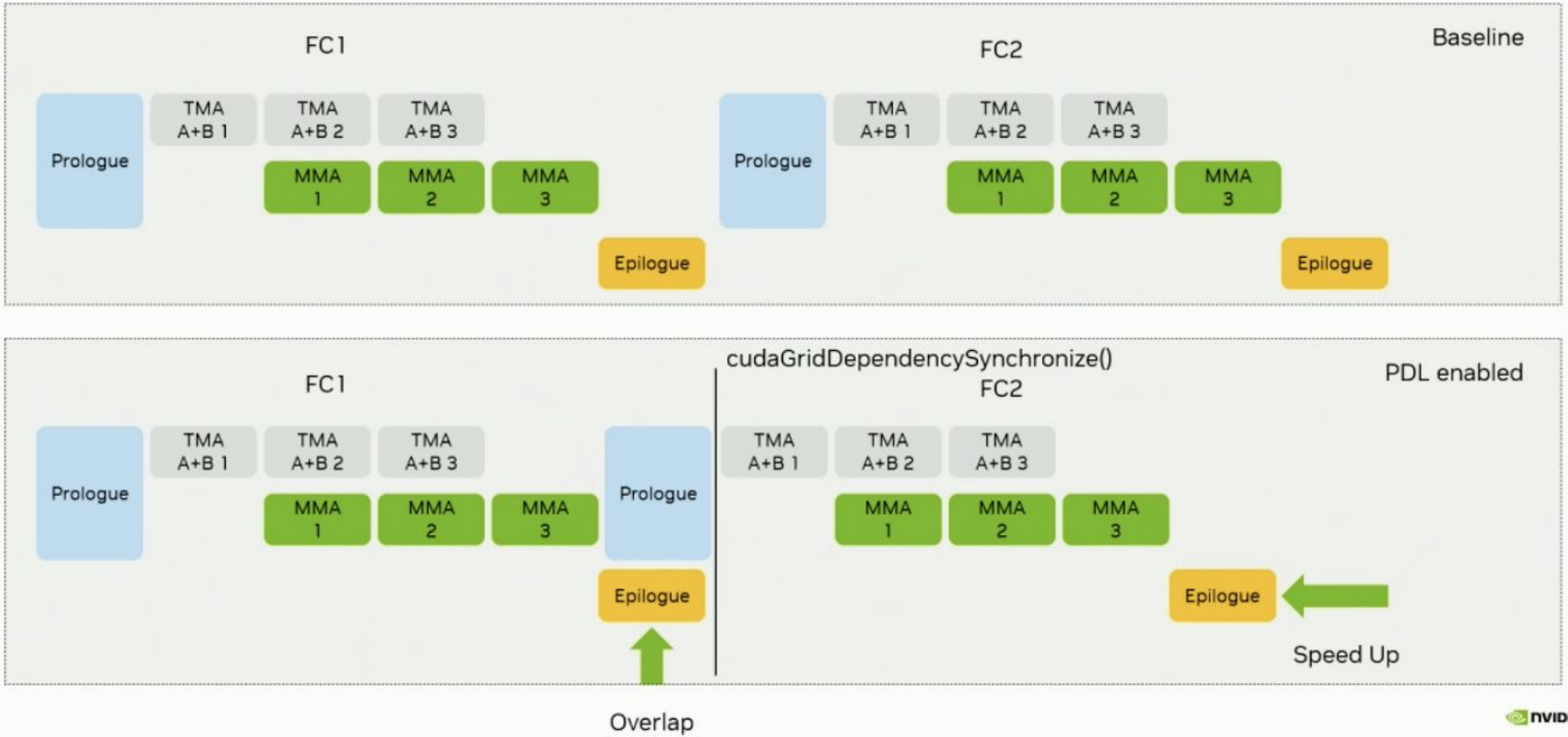
- ✓ Low-overhead direct register to memory stores
- ✓ Avoids Smem + synchronization latency
- ✗ Lots of instructions when output is large
- ✗ Requires care to ensure memory coalescing

Latency-aware warp specialization



Latency-optimized Warp-Specialization

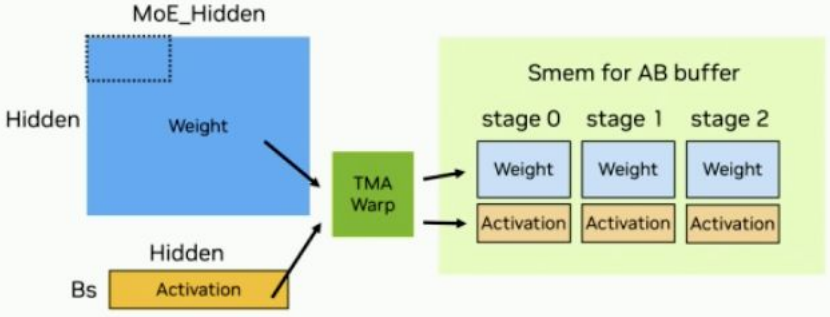
Using PDL to overlap prologue

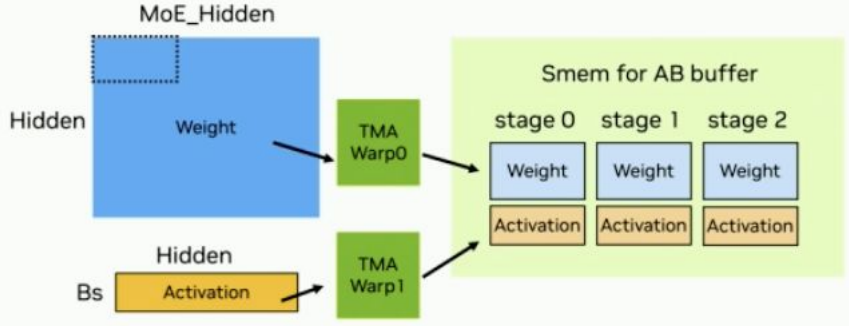


NVIDIA

TMA Warp(s) in Latency-optimized Warp-Specialization

1 TMA warp vs 2 TMA warps





Single TMA warp

- Loads of A and B must happen at the same time
- Fewer barrier updates

Two TMA warps

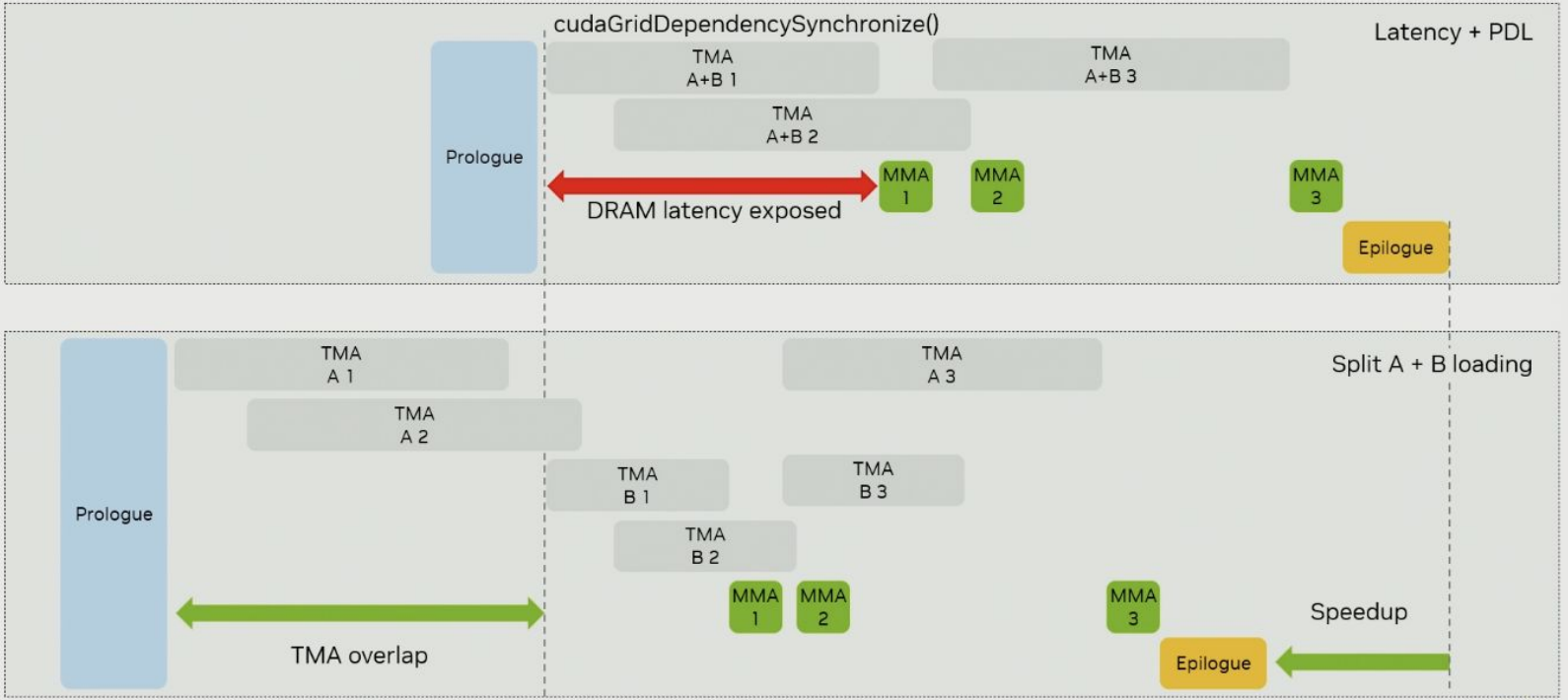
- Loads of A and B are decoupled
  - One warp won't block the other from issuing TMA requests.
  - With PDL, one warp can start loading early
- More barrier updates

More suited to **throughput** regime ⟷ More suited to **latency** regime

NVIDIA

Latency-optimized Warp-Specialization

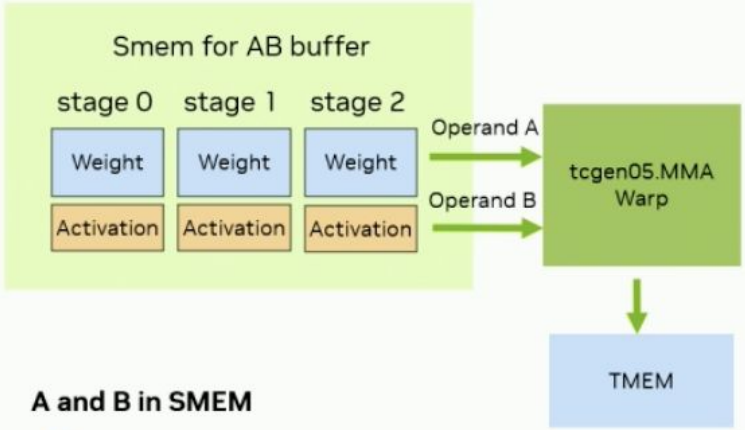
Overlap weights loading



NVIDIA

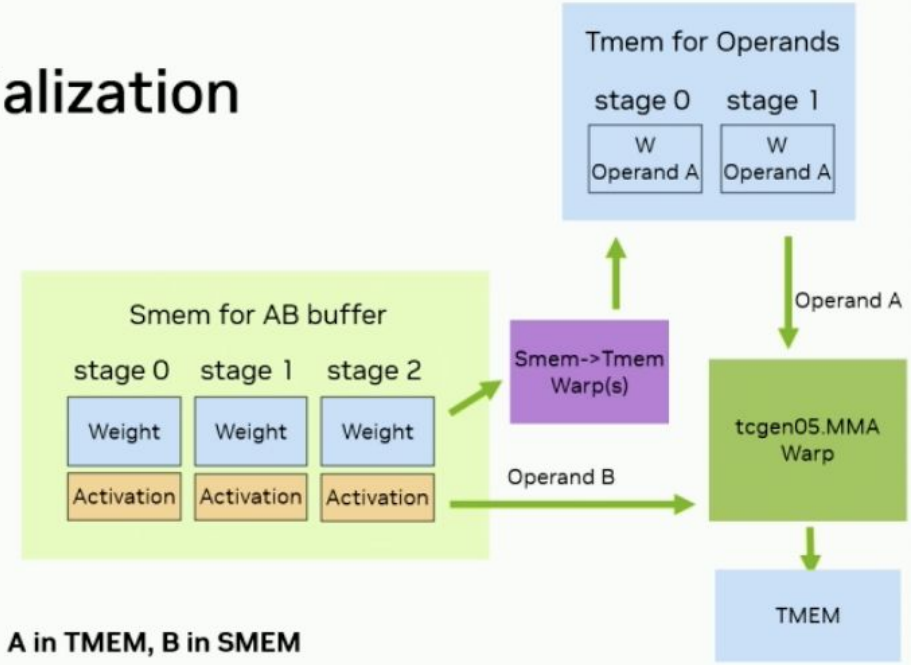
Latency-optimized Warp-Specialization

A in TMEM



A and B in SMEM

- Fewer data copies
- Fewer warps
- ✗ Fewer A stages

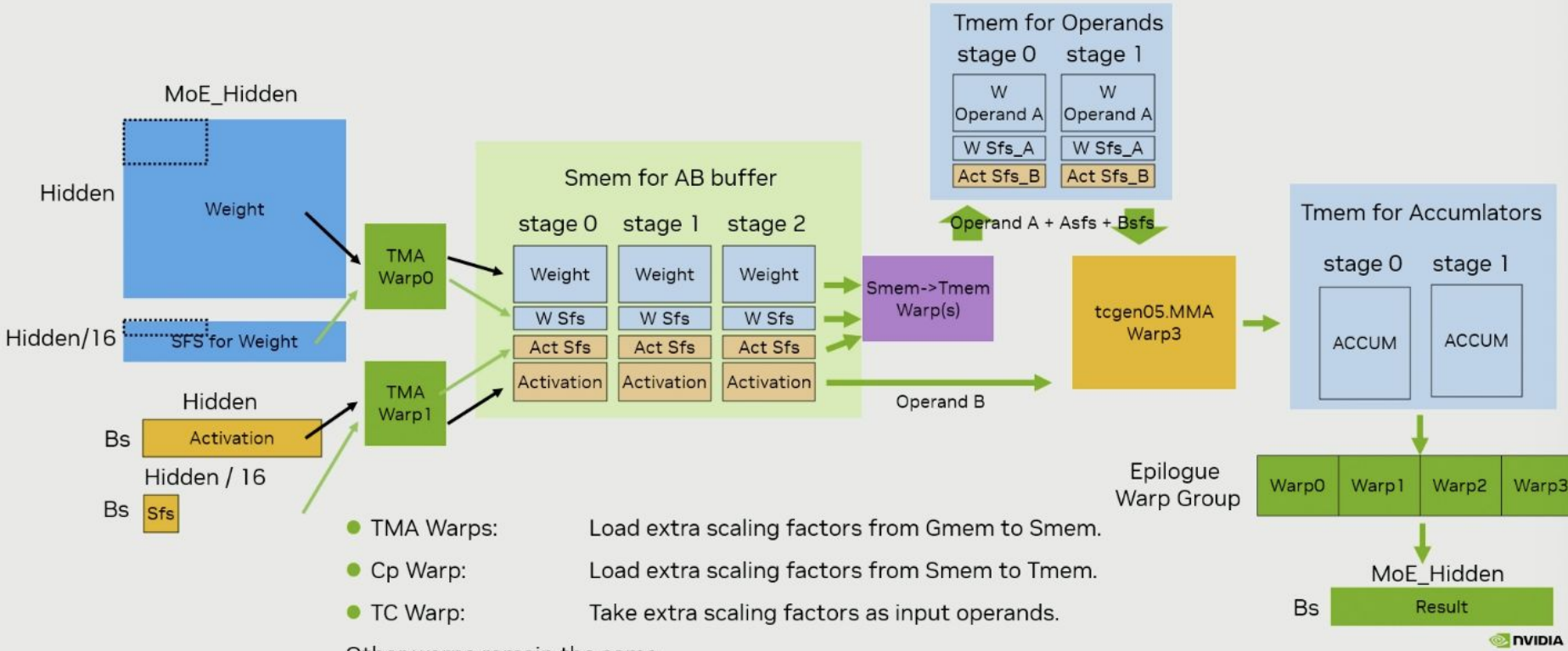


A in TMEM, B in SMEM

- ✗ More data copies (SMEM -> TMEM)
- ✗ More warps
- ✗ More TMEM usage
- More A stages – Higer tolerance for DRAM Latency
- Can refill SMEM quicker – less TMA bubbles

Latency-optimized Warp-Specialization

Scaling factors key data movment

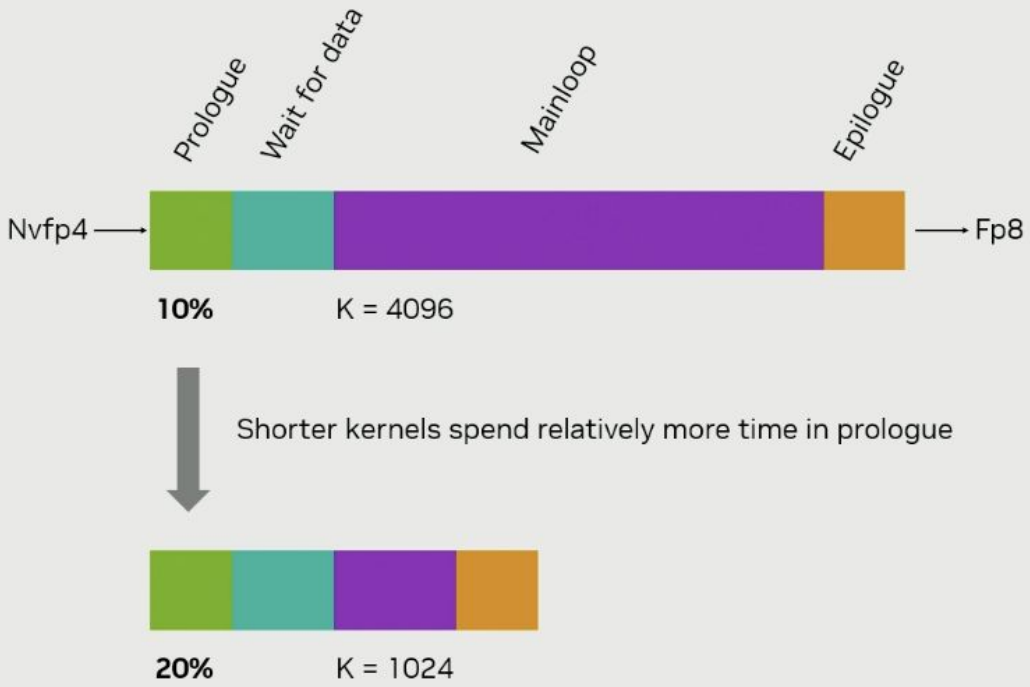


- TMA Warps: Load extra scaling factors from Gmem to Smem.
- Cp Warp: Load extra scaling factors from Smem to Tmem.
- TC Warp: Take extra scaling factors as input operands.

Other warps remain the same.

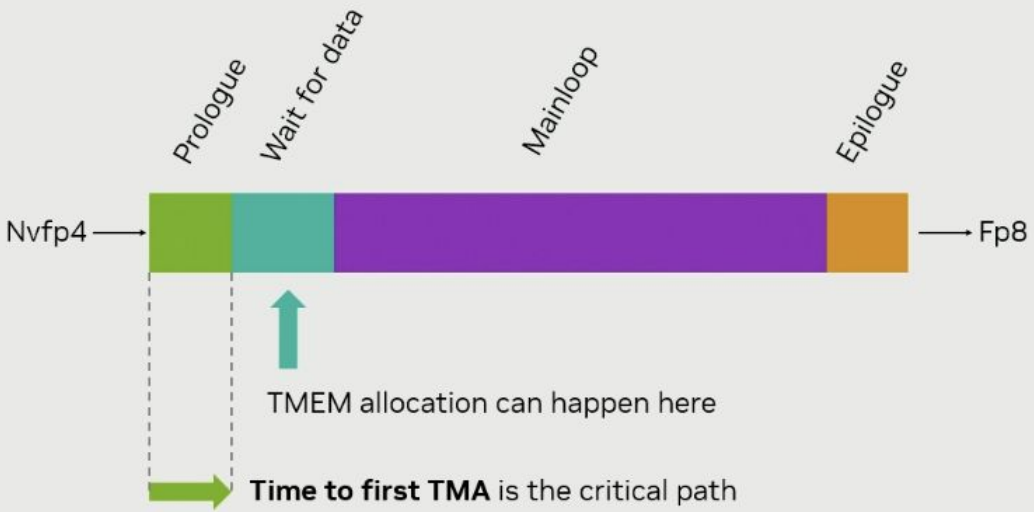
Prologue

- What do we need to start the mainloop?
  - Initialize shared memory barriers
  - Sync cluster (ensure visibility of shared memory barriers)
  - Issue TMA to fetch A and B
  - Allocate TMEM



Prologue

- What do we need to start the mainloop?
  - Initialize shared memory barriers
  - Sync cluster (ensure visibility of shared memory barriers)
  - Issue TMA to fetch A and B
  - Allocate TMEM



Prologue

Faster barrier initialization

Serialized mbarrier initialization

- **One** thread initializes **multiple** barriers

```
for (int i = 0; i < num_barriers; ++i) {
  ptx::mbarrier_init(barrier_array + i, arrival_count);
}
```

Hopper
- Requires multiple instructions

Vectorized mbarrier initialization

- **Several** threads initialize a **single** barrier

```
if (threadIdx.x < num_barriers) {
  ptx::mbarrier_init(barrier_array + threadIdx.x, arrival_count);
}
```

New in Blackwell
- Initialize multiple barriers **in a single instruction**

Prologue

Faster cluster sync time

```
// Initialize barriers
if (threadIdx.x < num_barriers) {
  ptx::mbarrier_init(barrier_array + threadIdx.x, arrival_count);
}

// Make barrier visible


ptx::barrier_cluster_arrive(ptx::sem_release);
ptx::barrier_cluster_wait(ptx::sem_acquire);
```

```
// Initialize barriers
if (threadIdx.x < num_barriers) {
  ptx::mbarrier_init(barrier_array + threadIdx.x, arrival_count);
}

// Make barrier visible
ptx::fence_mbarrier_init(ptx::sem_release, ptx::scope_cluster);

ptx::barrier_cluster_arrive(ptx::sem_relaxed);
ptx::barrier_cluster_wait(ptx::sem_acquire);
```

Slow

Fast – Always preferred

For more info: S72683 - CUDA Techniques to Maximize Memory Bandwidth and Hide Latency

TMA runtime dynamic box size

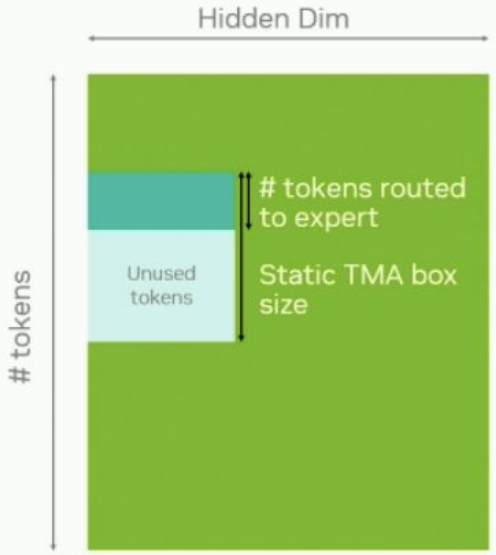
Problem statement

TMA box size is **static**

- When we use TMA, we create a tensor map on the host before launching the kernel
- The tensor map determines the box size
- The tensor map cannot easily be changed

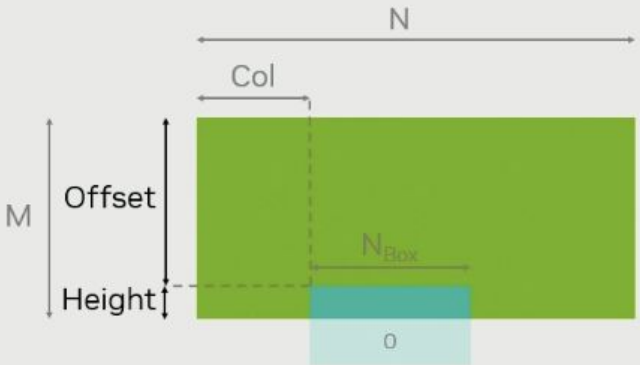
However: the number of tokens routed to an expert is **dynamic** at runtime

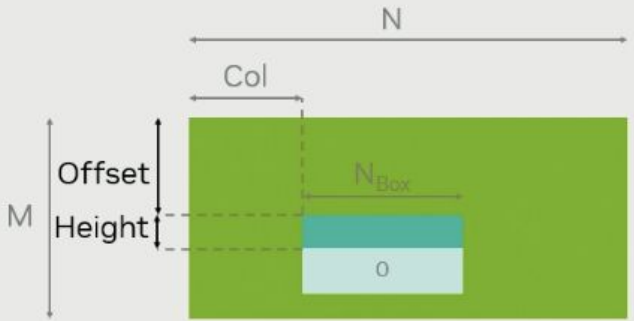
If we use a large static TMA box size, we are **loading unused tokens**



TMA runtime dynamic box size

What we want





What we can already do

- Load a box that is slightly over the edge
- Outside values get filled with zero

What we want to do

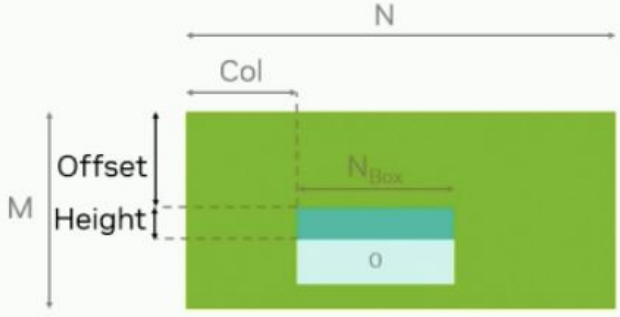
- Load a box at any offset
- With a dynamic size
- Rows beyond beyond height are filled with zero

TMA runtime dynamic box size

A peek at the solution

To get a **2-dimensional box** of Height x N_{Box} out of an M x N matrix:

- Create a **4-dimensional tensor map** with
  - `Size` = (N, M, 2^{34}, 2^{34})
  - `Stride` = (2^{34} − N, N, N, 1)
  - `Box size` = (1, 1, 256, N_{Box})
- Use coordinates:
  - `Coordinates` = (Col, 2^{30} − Height, Offset + Height, 2^{30})



Caveats

- Backing memory must be **at least 128KB(!)**
  - Otherwise: potential for page faults
- `Offset + height` must be less than `M`
  - No out-of-bounds checking by TMA anymore(!)

Thanks to OpenAI for finding and sharing this trick!

TMA runtime dynamic box size

Solve for coordinates, size, stride

Mission accomplished

Coordinates	Tensor size	Box size	Effective box size
??	??	1	1
??	??	1	1
M - Height	M	Box_M	$\min(\text{Height}, M_{Box})$
Col	N	Box_N	$\max(0, \min(N - Col, N_{Box}))$

Coordinates	Stride	Pointer offset contribution
??	??	
??	??	
M - Height	A	M * A - Height * A
Col	1	Col
Total		**Col + M * A - Height * A**

Goal: Col + Offset * N

Constraints:

- We cannot have negative coordinates
- We cannot have negative strides

TMA runtime dynamic box size

Solve for coordinates, size, stride

Coordinates	Tensor size	Box size	Effective box size
M	??	1	1
Offset + Height	??	1	1
M - Height	M	Box_M	$\min(\text{Height}, M_{Box})$
Col	N	Box_N	$\max(0, \min(N - \text{Col}, N_{Box}))$

Coordinates	Stride	Pointer offset contribution
M	X - N	- M * N + M * X
Offset + Height	N	+ Offset * N + Height * N
M - Height	N	M * N − Height * N
Col	1	Col
Total		**Col + Offset * N + 0 − 0** + M * X

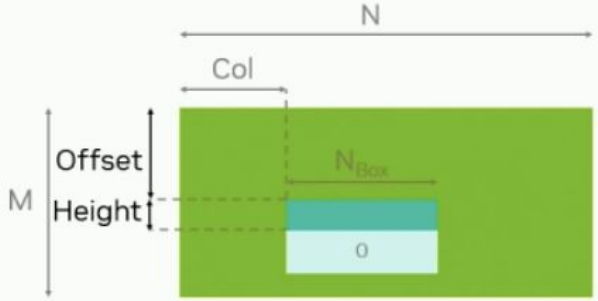
- Add Offset * N
- Correct Height * (N − A)
- Correct M * N

TMA runtime dynamic box size

The recipe

To get a box of Height x N_{Box} out of an M x N matrix:

- Create a 4-dimensional tensor map with
  - `Size` = (N, M, 2^{34}, 2^{34})
  - `Stride` = (2^{34} − N, N, N, 1)
  - `Box size` = (1, 1, 256, N_{Box})
- Use coordinates:
  - `Coordinates` = (Col, 2^{30} − Height, Offset + Height, 2^{30})



Caveats

- Backing memory must be **at least 128KB(!)**
  - Otherwise: potential for page faults
- `Offset + height` must be less than `M`
  - No out-of-bounds checking by TMA anymore(!)

Thanks to OpenAI for finding and sharing this trick!

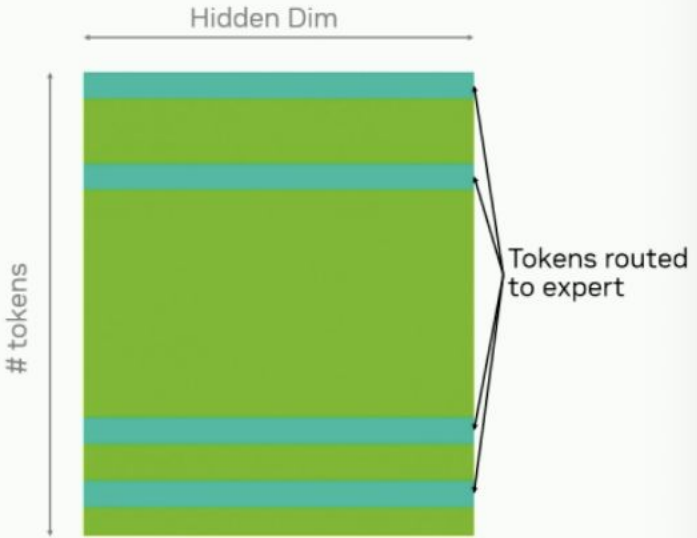
TMA GATHER4

Problem statement

TMA normally loads **contiguous** boxes of memory

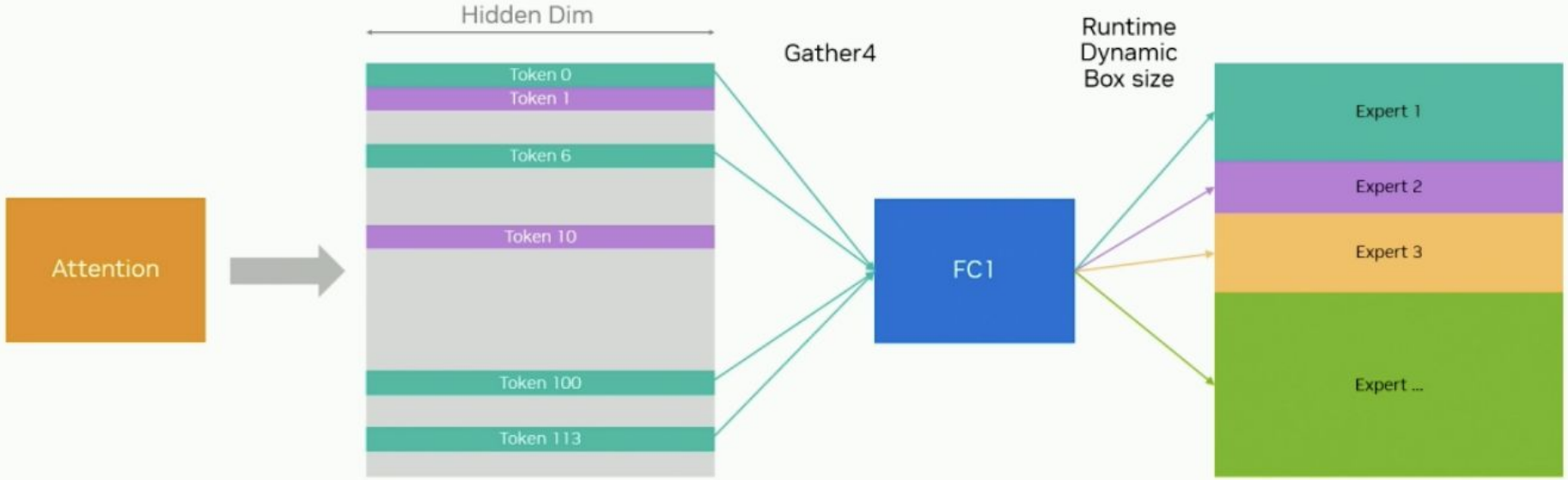
However, the tokens routed to an expert are **non-contiguous**

We **cannot** use TMA with a **large box size**



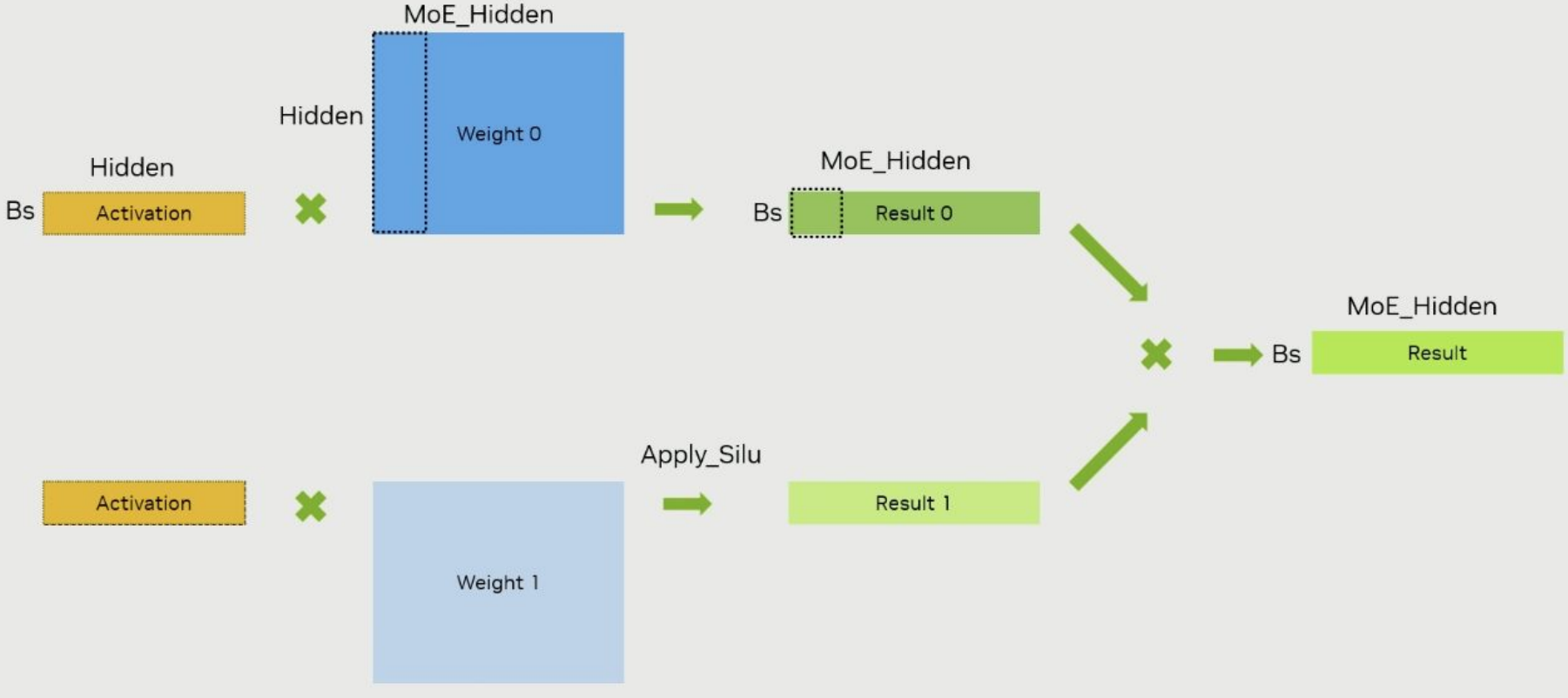
MoE FC1: Data layouts

Combining Gather4 and Runtime Dynamic box size



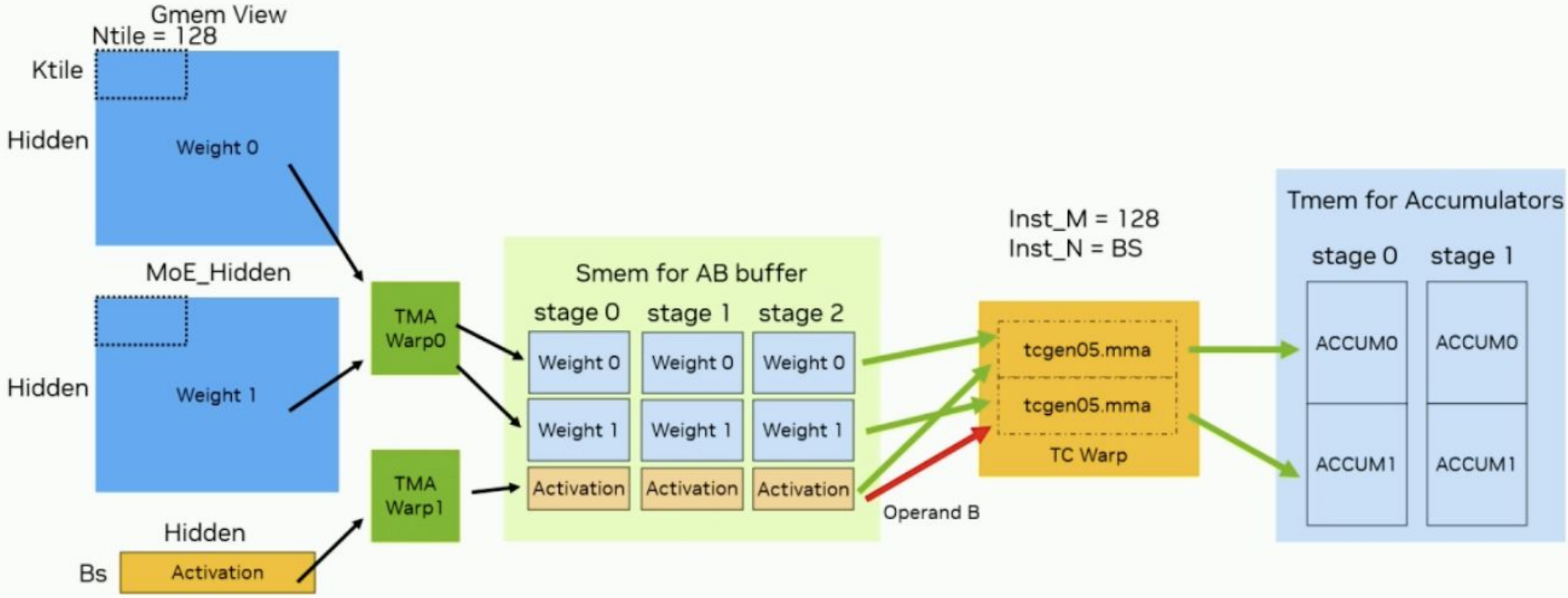
FC1 (Gated SiLU) in MoE

Per Expert View



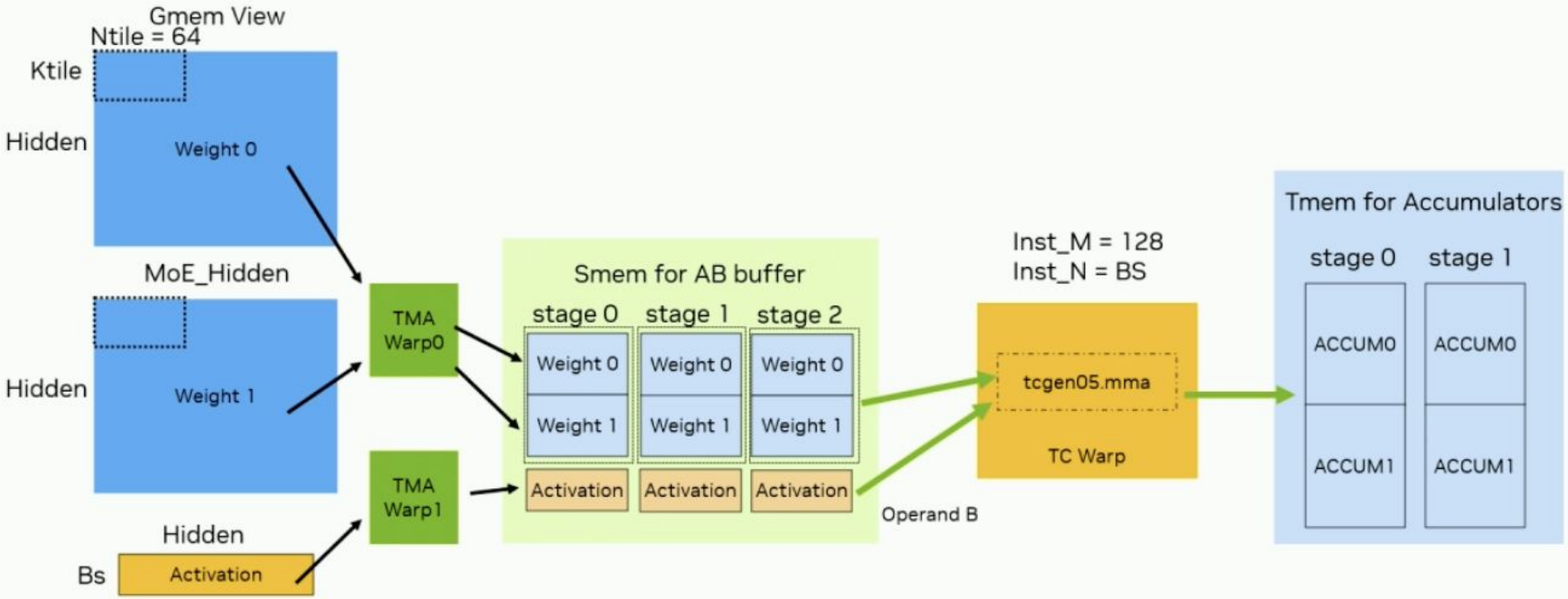
Fused FC1 (Gated SiLU)

Naïve approach.



Fused FC1 (Gated SiLU)

Concat 2 weight matrixes in SMEM

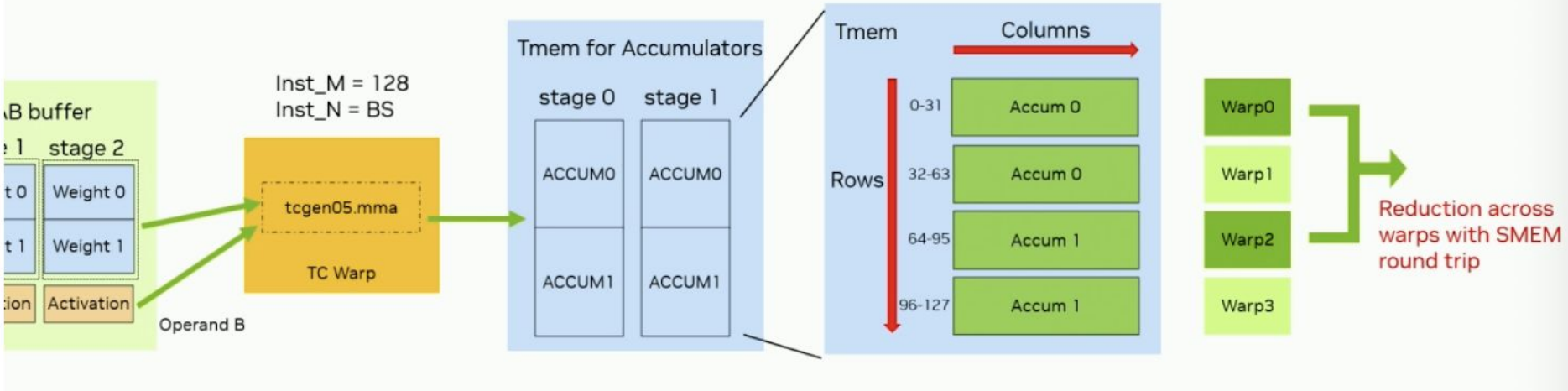


Fused FC1 (Gated SiLU)

Concat 2 weight matrixes in SMEM

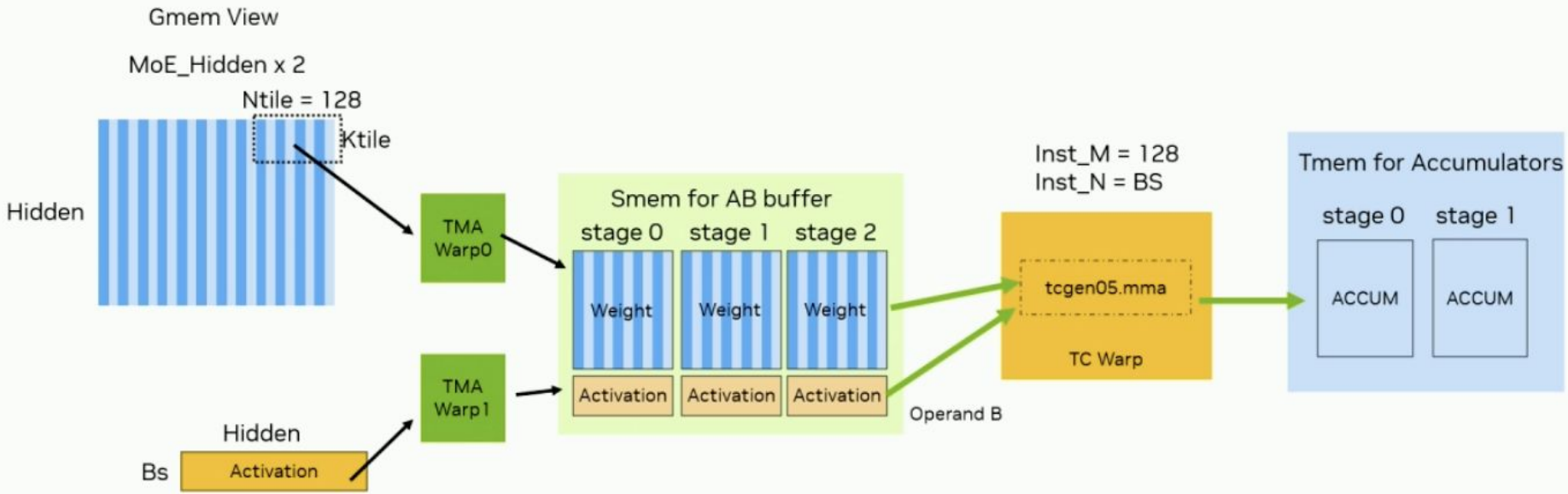
Cons:

Introduce extra latency when processing gated silu.



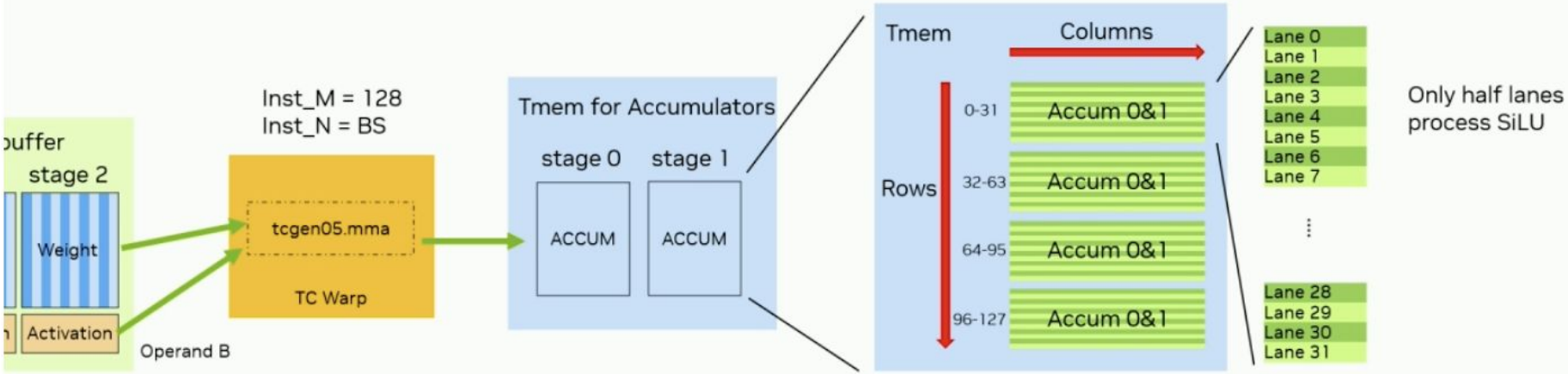
Interleaved Fused FC1 (Gated SiLU)

Interleave granularity = 1



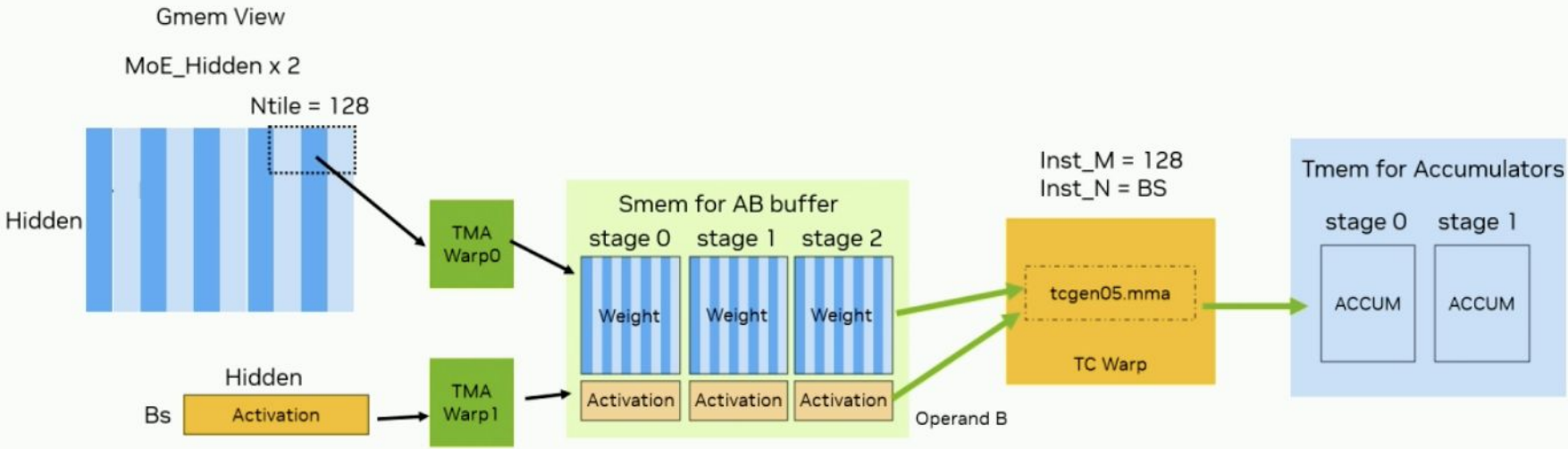
Interleaved Fused FC1 (Gated SiLU)

Interleave granularity = 1



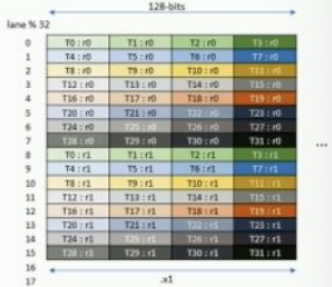
Interleaved Fused FC1 (Gated SiLU)

Interleave granularity = 8



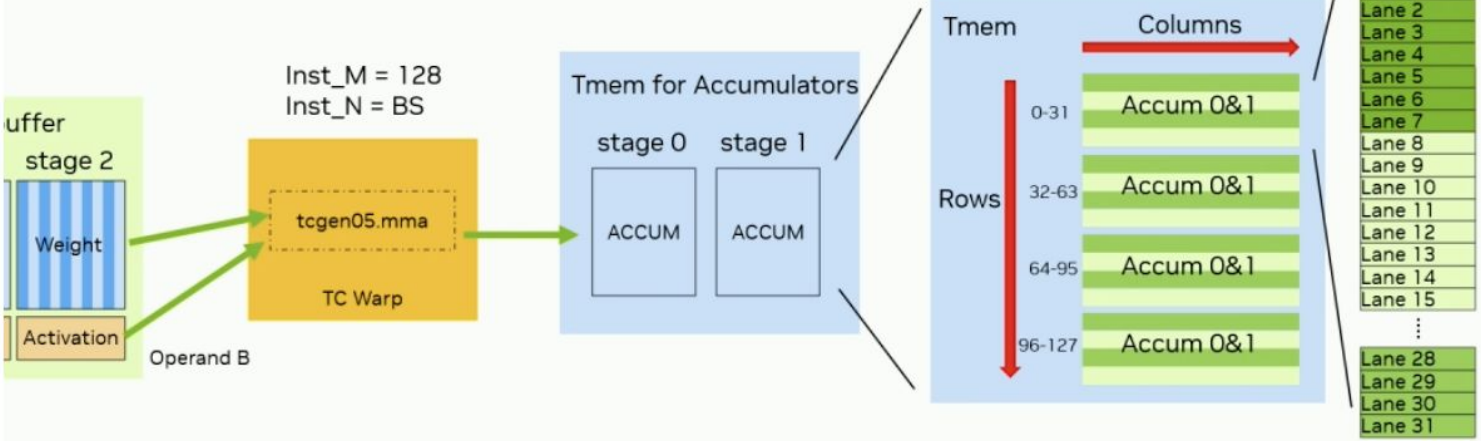
Interleaved Fused FC1 (Gated SiLU)

Interleave granularity = 8

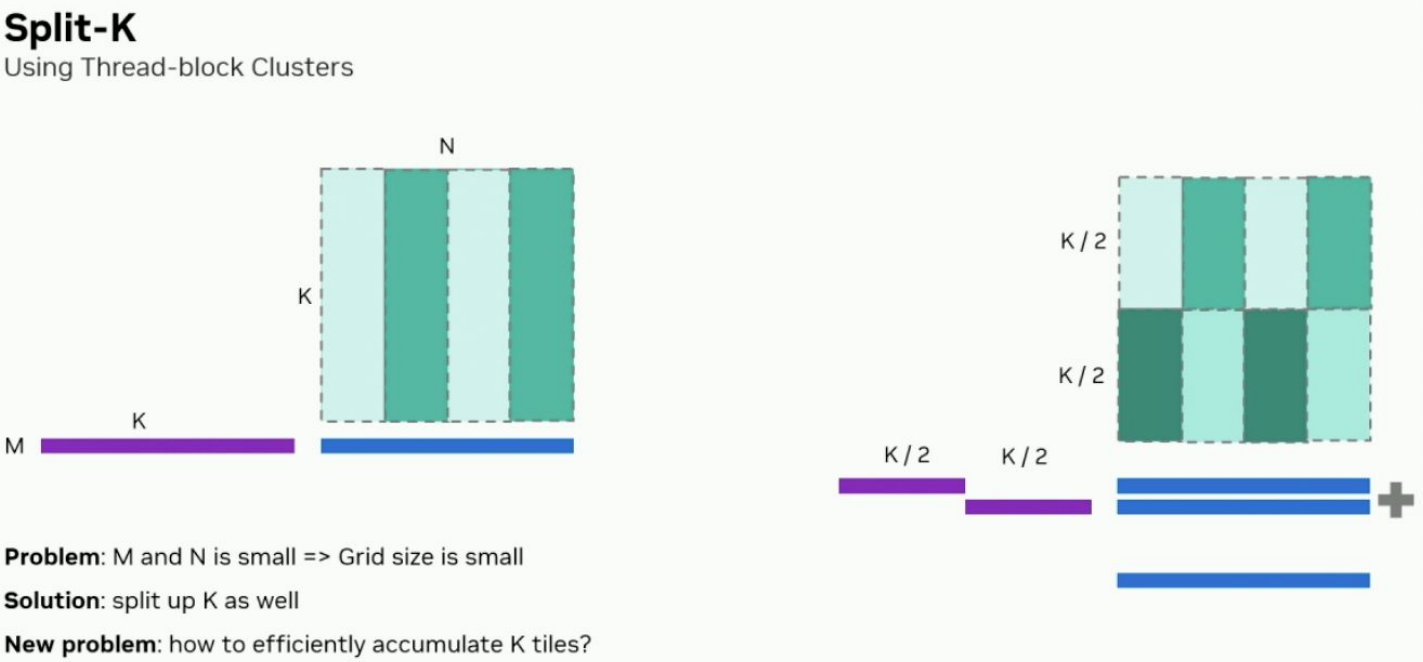


- **16dp128b1x** instruction
- Each warp accesses only 16 lanes.

T0's view

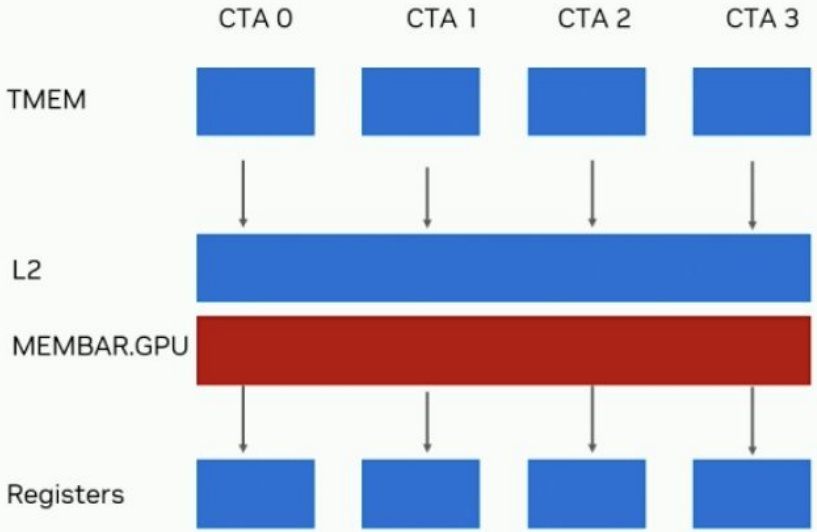


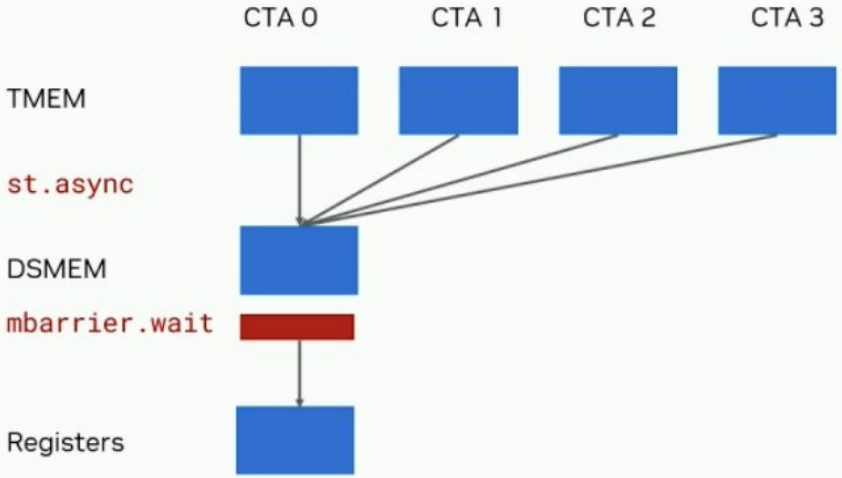
Optimize Mixture-Of-Experts for Low-Latency Inference Decode on Blackwell [S81518]



Split-K

Using Thread-block Clusters





L2-based reduction

- Higher latency

Distributed Shared Memory-based reduction

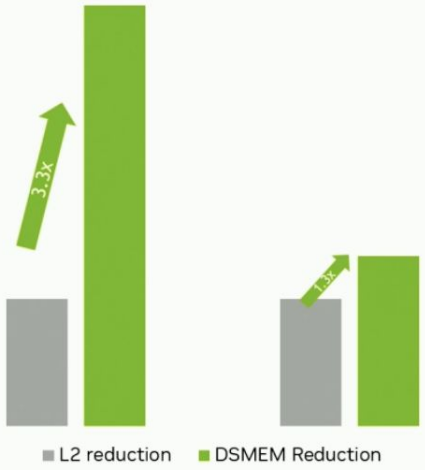
- Lower latency

Optimize Mixture-Of-Experts for Low-Latency Inference Decode on Blackwell [S81518]

Split-K

Results

- Llama 70B QKV GEMM
  - SplitK=8
  - 16x1280x8192



Optimize Mixture-Of-Experts for Low-Latency Inference Decode on Blackwell [S81518]

Summary

Blackwell architecture is well-suited for low-latency LLM inference

System-level features

- CUDA Graph
- Programmatic Dependent Launch
- Fast NVLink for Expert and Tensor Parallelism

Tensor core features

- Low-precision number formats
- Scalable tensor core hardware design

TMA features

- Sparse token loading using gather4
- Dynamic box size

Parameterized CUDA Graph Launch in PyTorch: CUDA Graphs Without the Pain [S81709]

Performance Benefits of Cuda Graphs

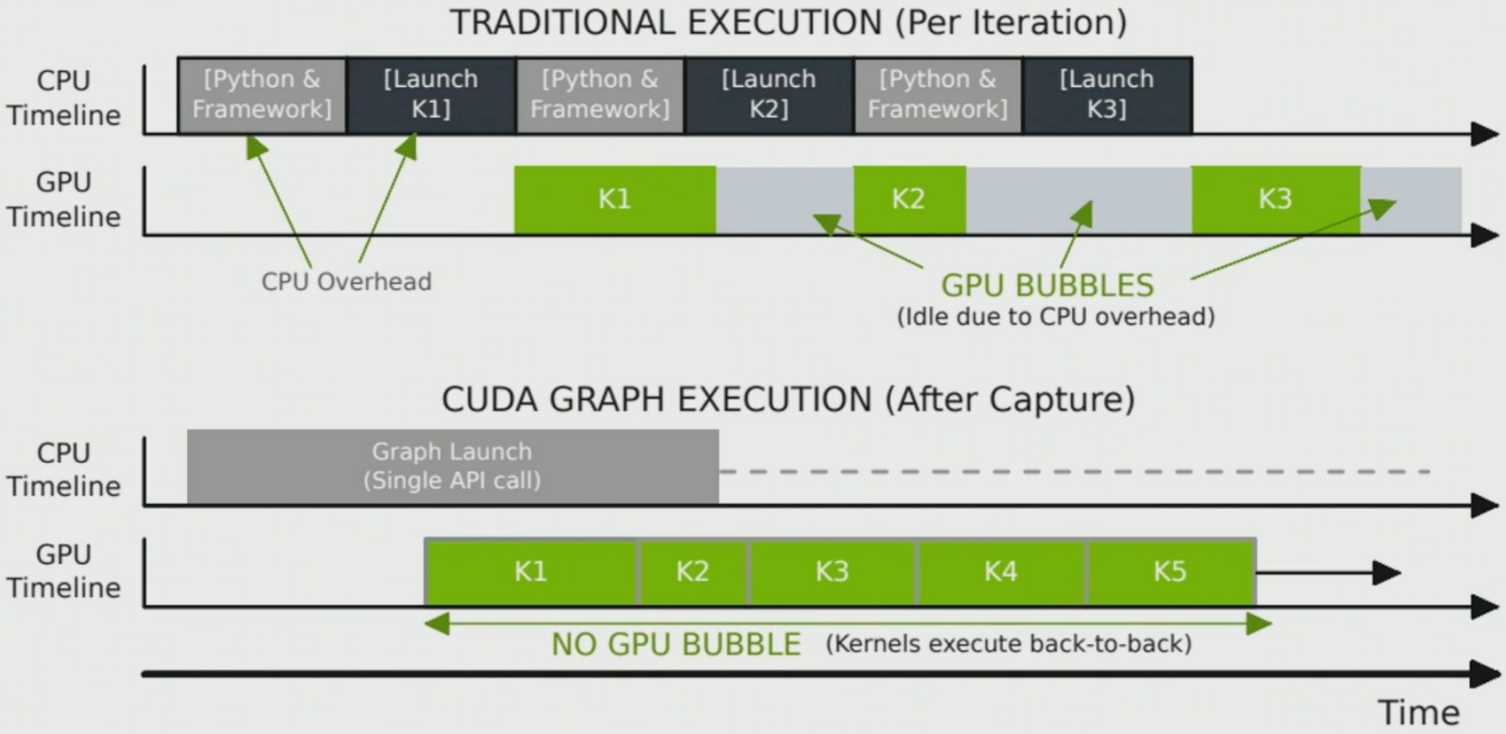


Figure 22: Traditional execution (top) versus CUDA Graph execution (bottom).

From "Scalable Training of Mixture-of-Experts Models with Megatron Core"

Why not always use CUDA Graphs?

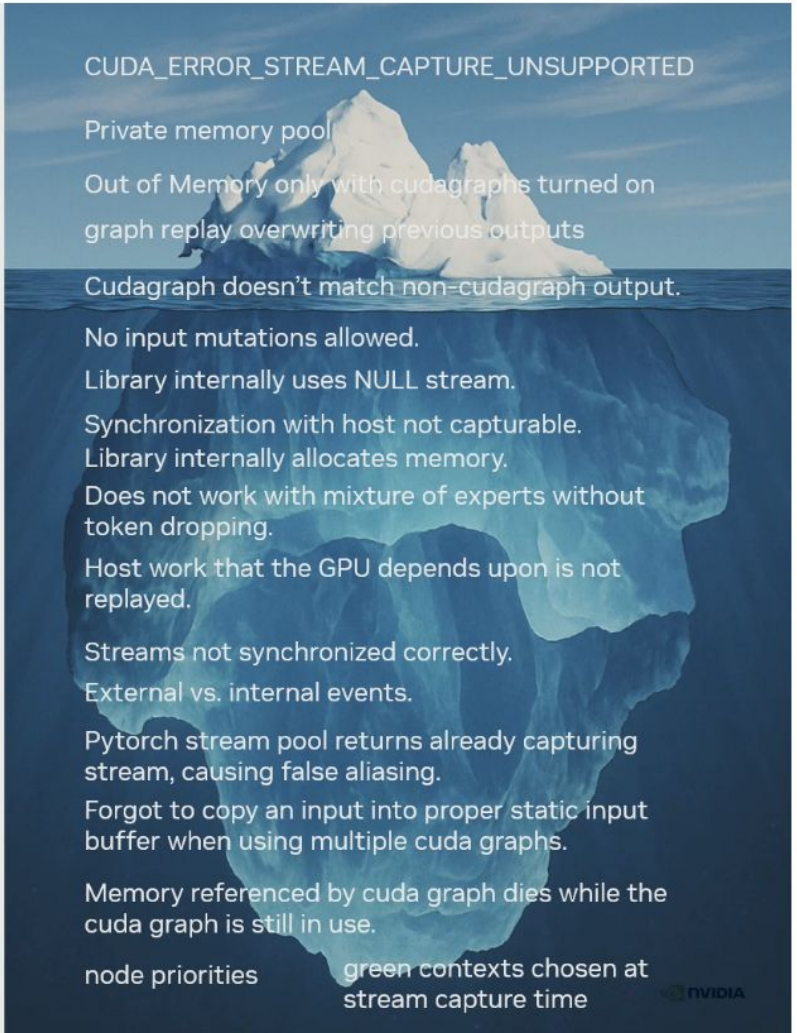
Well-formed pytorch programs may not run properly when captured as a cuda graph.

GPU code during graph capture is lazy, but CPU code during graph capture is eager.

Kernel arguments at graph capture time are reused at replay time. They cannot be changed and cannot be introspected.

1. Must copy a cuda graph's inputs into the "static input memory addresses" used at stream capture time before replay.
2. A replayed cuda graph will always write its outputs to the same memory addresses.
3. No way to introspect a cuda graph to know its input and output tensors. User responsibility to keep track.
4. Work only on fixed shape workloads

Walking through 5 common errors today.



The easiest way to use CUDA Graphs in Production

CUDA Graph Trees and Graph Partition via torch.compile()

Pros:

Just pass mode="reduce-overhead" or mode="max-autotune".

A program accepted by torch.compile() will use cuda graphs for the cuda graph compatible parts, and fall back to non cudagraph execution otherwise.

- Fxgraphs know their own inputs and outputs.
- Fxgraphs can be partitioned into cudagraph-compatible and non-cudagraph-compatible fxgraphs.

Straightforward requirement to impose on other teams.

- Successful cuda graph capture does not imply that cuda graph is correct.

Cons:

Too many graph breaks reduce benefit of cudagraphs as fxgraphs get smaller and smaller.

Many pytorch libraries introduce graph breaks or otherwise are not torch.compile() compatible.

Learn more:

CUDA Graph Trees: "CUDAGraph in a Partial Graph World" at Pytorch Conference 2024 by Elias Ellison, Meta

(FX) Graph partition: Pytorch issue #125864

Our Running Example – Training

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)

lr = 0.1

def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad)
    lr = lr * 0.9
return loss
```

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        loss = train(X.cuda(), y.cuda())
        print(loss.item())
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Error 1: Asynchronous Data Loading Breaks Stream Capture

```
torch.AcceleratorError: CUDA error: operation
failed due to a previous error
```

- Happens during stream capture at random points, but only sometimes.

Root cause: Allocating pinned host memory in a separate thread breaks stream capture because cudaHostAlloc() is an “unsupported API”

Diagnose with:

- `export CUDA_LOG_FILE=stdout`
  - Requires CUDA 12.9 user mode driver or newer.
- `export PYTHONUNBUFFERED=1`
- `export TORCH_SHOW_CPP_STACKTRACES=1`
  - Broken on ARM; see pytorch issue #168376

```
[23:57:31.242][139912224974400][CUDA][E] API call conflicts with a stream capture sequence initiated from another thread

[23:57:31.242][139912224974400][CUDA][E] Returning 900 (CUDA_ERROR_STREAM_CAPTURE_UNSUPPORTED) from cuMemHostAlloc
```

Output from running with `export CUDA_LOG_FILE=stdout`

(Potentially) Unsupported APIs

CUDA_ERROR_STREAM_CAPTURE_UNSUPPORTED

By default, unsupported APIs called from any thread will break any current stream capture.

What makes a particular CUDA API unsupported during stream capture?

- Anything that synchronously creates or deletes a resource
  - cudaMalloc(), cudaFree(), cudaHostAlloc(), etc.
- Anything that makes GPU state visible to the CPU
  - cudaStreamSynchronize(), cudaEventSynchronize(), cudaEventQuery(), etc.

When should we permit an unsupported API?

- The cuda graph never references the synchronously created or deleted resource.
- OR
- The cuda graph is guaranteed to die before the synchronously created resource dies.
- OR
- The cuda graph doesn't depend upon the newly CPU-visible state.

How do we permit an unsupported API?

Library developers:

Permit via setting current thread capture mode to relaxed via cudaThreadExchangeStreamCaptureMode before calling unsupported API.

Pytorch users:

Make sure stream capture spans only one thread and is invalidated by unsupported APIs on only the current thread by passing "thread_local" to torch.cuda.graph().

Error 1: fix

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = 0.1

def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with
  torch.autograd.grad_mode.
set_multithreading_enabled(Fals
e):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad * inv_scale)
    lr = lr * 0.9
return loss
```

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        loss = train(X.cuda(), y.cuda())
        print(loss.item())
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g,
capture_error_mode=
"thread_local" ):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Error 2: cuda stream capture joins with the NULL stream

```
UserWarning: The AccumulateGrad node's stream does
not match the stream of the node that produced the
incoming gradient. This may incur unnecessary
synchronization and break CUDA graph capture if the
AccumulateGrad node's stream is the default stream.
This mismatch is caused by an AccumulateGrad node
created prior to the current iteration being kept
alive. …
```

```
File “01_vocabulary_sample_capture.py", line 30, in
train
    loss.backward()
    ^^^^^^^^^^^^

torch.AcceleratorError: CUDA error: operation would
make the legacy stream depend on a capturing
blocking stream
```

Error 2: fix

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        loss = train(X.cuda(), y.cuda())
        print(loss.item())
        del loss
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g, capture_error_mode="thread_local"):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Deleting loss will remove autograd references from warmup iteration.

Error 2: cuda stream capture joins with the NULL stream

```
UserWarning: The AccumulateGrad node's stream does
not match the stream of the node that produced the
incoming gradient. This may incur unnecessary
synchronization and break CUDA graph capture if the
AccumulateGrad node's stream is the default stream.
This mismatch is caused by an AccumulateGrad node
created prior to the current iteration being kept
alive. …
```

```
File “01_vocabulary_sample_capture.py", line 30, in
train

    loss.backward()

    ^^^^^^^^^^^^

torch.AcceleratorError: CUDA error: operation would
make the legacy stream depend on a capturing
blocking stream
```

Error 2: Alternative fix

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        with torch.cuda.stream(
        ):    torch.cuda.Stream()
            loss = train(X.cuda(), y.cuda())
        print(loss.item())
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g, capture_error_mode="thread_local"):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Use a stream other than the default (NULL)

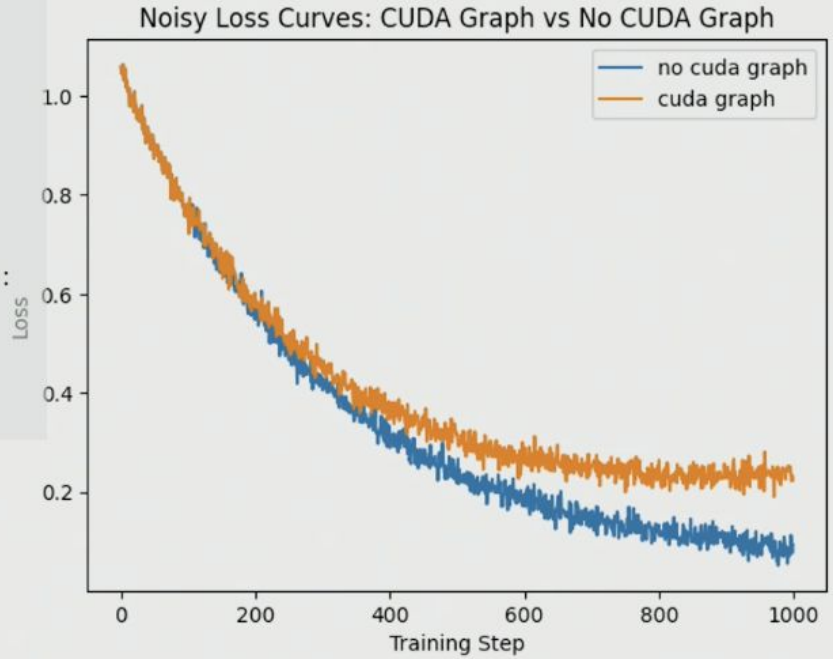
stream for warmup.

Can reduce memory fragmentation if the same stream is used for both warmup and graph capture.

- CUDA Graph Trees does this for you.

Error 3: Host Device Inconsistency

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = 0.1

def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad * inv_scale)
    lr = lr * 0.9
return loss
```



lr is a scalar python float, not a torch.Tensor.

`lr *= 0.9` is host code; it is run eagerly during stream capture. It won't rerun during cuda graph replay.

Therefore, lr will not be updated during each replay of the cudagraph.

Error 3: initial fix

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = torch.tensor(0.1,
      device="cuda")
def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad)
    lr = lr * 0.9
    return loss
```

Possible to detect this by running two stream captures with a TorchDispatchMode.

If any of the non-tensor inputs to a torch op are different, then you likely will encounter this issue.

Have not made such a debug tool yet.

See pytorch issue #167885 for updates.

If the erroneous code were to run via torch.compile() CUDAGraph Trees, a new cudagraph would be specialized for every new value of lr.

This will be slow because cuda graphs are never reused, but a warning will trigger after enough graphs are made.

But it will never compute incorrect values.

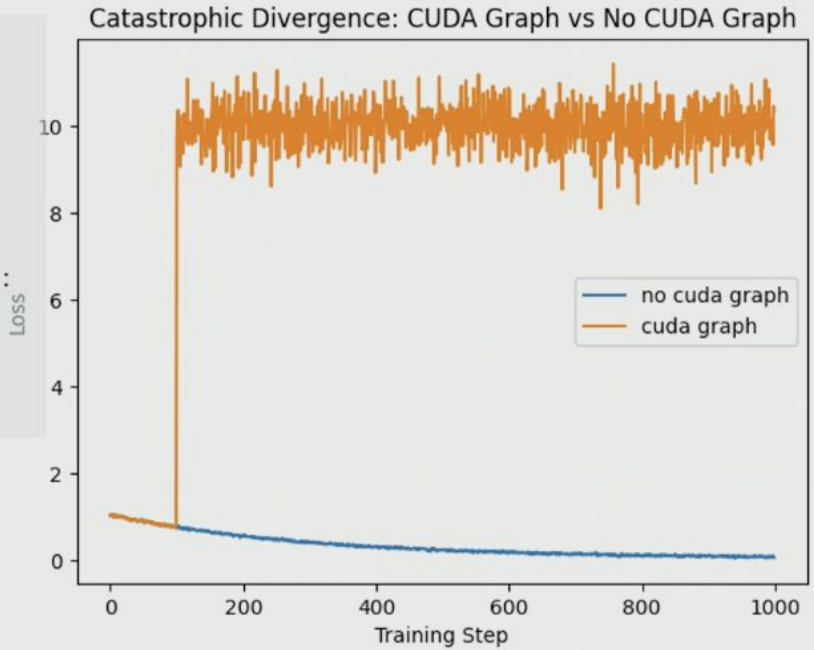
Error 4: Use after free

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = torch.tensor(0.1, device="cuda")
def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad)
    lr = lr * 0.9
    return loss
```

lr tensor is overwritten with a new lr tensor, with a different underlying memory allocation.

But the cuda graph's SGD kernels reference the old tensor.

That old tensor has no more references, so its backing memory dies and that memory can be assigned to a new tensor.



Error 4: fix

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)

lr = torch.tensor(0.1,device="cuda")

def train(X, y):
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad)

        lr.mul_(0.9)

    return loss
```

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        loss = train(X.cuda(), y.cuda())
        print(loss.item())
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g,
              capture_error_mode="thread_local",
check_input_liveness=True
              ):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Works via TorchDispatchMode. While cuda graph nodes don't know their inputs and outputs, the ops which launch these kernels do know.

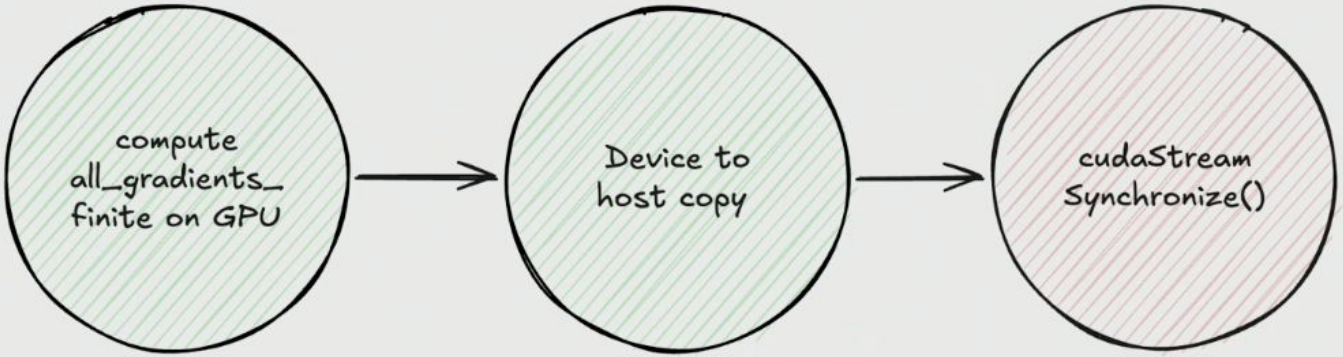
See pytorch PR #174649 for details.

What if we wanted to apply a gradient NaN check?

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)

lr = torch.tensor(0.1,device="cuda")

def train(X, y):
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        all_gradients_finite = torch.all(
                                param.grad.isfinite().all()
                                for param in model.parameters())
        if all_gradients_finite:
            for param in model.parameters():
                param.sub_(lr * param.grad)
        lr.mul_(0.9)
    return loss
```

Error 5: Data-dependent Control Flow Breaks Stream Capture

```
[12:40:35.713][140433506940736][CUDA][E] Returning 900 (CUDA_ERROR_STREAM_CAPTURE_UNSUPPORTED) from cuStreamSynchronize
 File "03_null_stream_join_autograd.py", line 40, in train
    if all_gradients_finite:
    ^^^^^^^^^^^^^^^^^^
torch.AcceleratorError: CUDA error: operation not permitted when stream is capturing
```



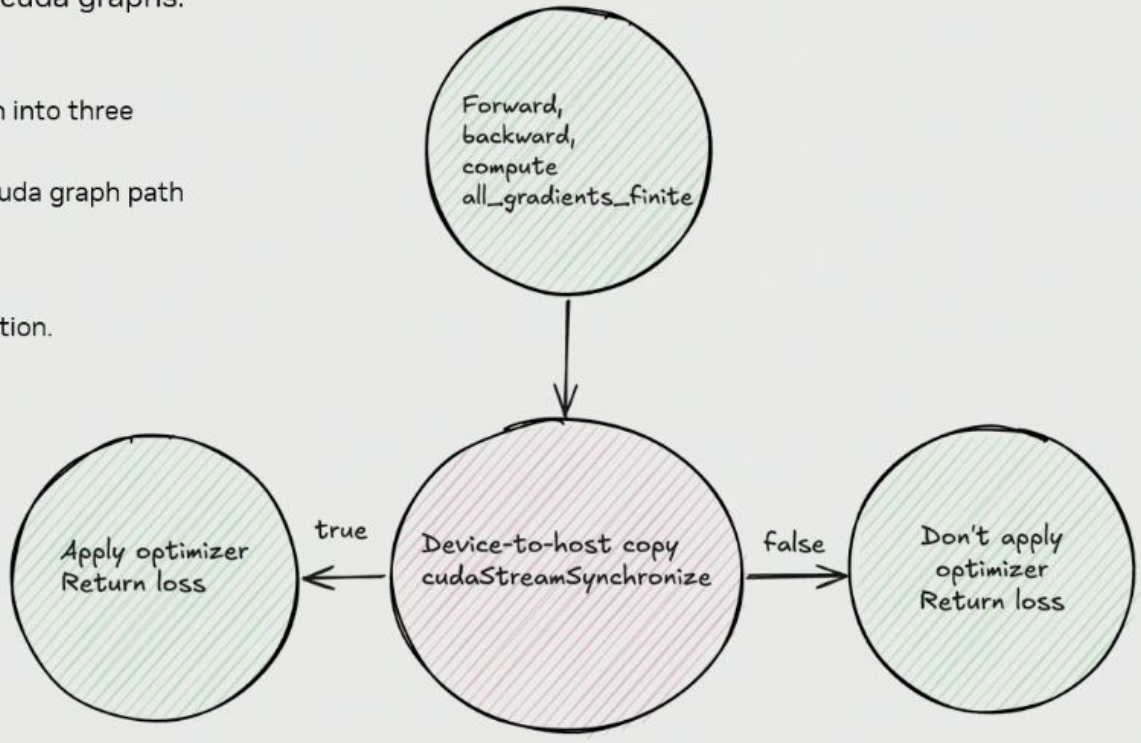
Error 5: Intrusive Fix

Partition the training workload into three cuda graphs.

Con:

- Intrusive. Need to split previous train() function into three functions
- Need to duplicate control flow logic between cuda graph path and non cuda graph path.

torch.compile() will do this for you via graph partition.



Error 5: Non-Intrusive Fix

CUDA Graph Conditional Node via torch.cond()

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)

lr = torch.tensor(0.1,device="cuda")

def train(X, y):
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        all_gradients_finite = torch.all(param.grad.isfinite().all() for param in model.parameters())

        def true_fn():
            for param in model.parameters():
                param.sub_(lr * param.grad)

        def false_fn():
            pass

        torch.cond(all_gradients_finite,true_fn,false_fn)
    lr.mul_(0.9)
    return loss
```

Error 5: Non-Intrusive Fix

Pros:

Non-intrusive, have just one cuda graph.

No repeated logic.

Cons:

Introduced into pytorch in February 2026.

Will be in pytorch 2.12.

Works only for if-else statements.

torch.while_loop() and torch.switch() cuda graph support on the way.

- Pytorch issue #175001
- Pytorch issue #175891

Conclusion

TL;DR

torch.compile() is the best way to enable cuda graphs incrementally.

We are constantly making more operations cudagraph compatible.

We are constantly making more tools for diagnosing errors.

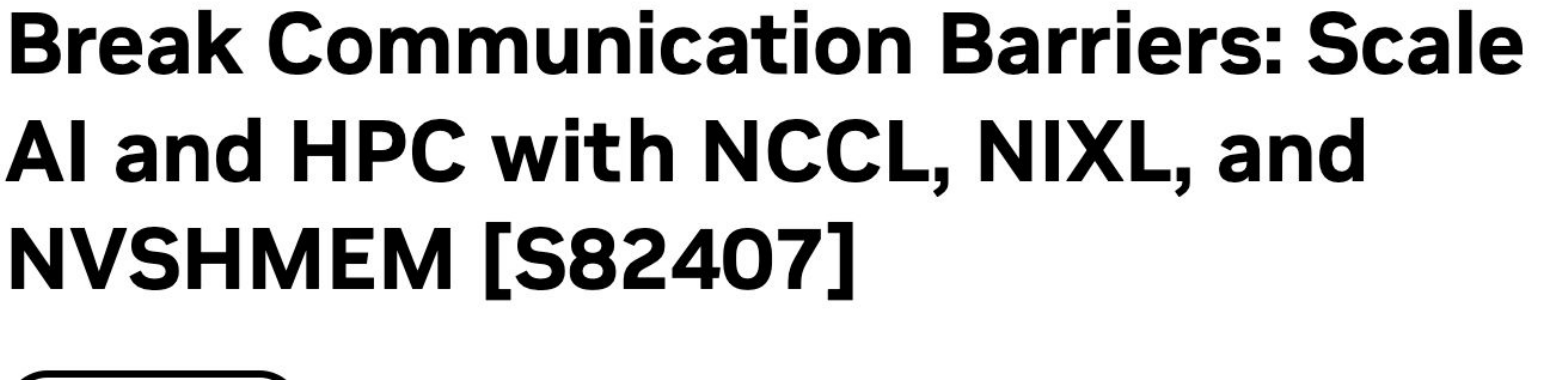
- Follow pytorch issue #167885

Questions:

- Pytorch Github issues. Tag @galv (me).
- Email: dgalvez@nvidia.com
- x.com/memorypaladin

Many more errors not covered today. Learn more at https://docs.nvidia.com/dl-cuda-graph/latest/

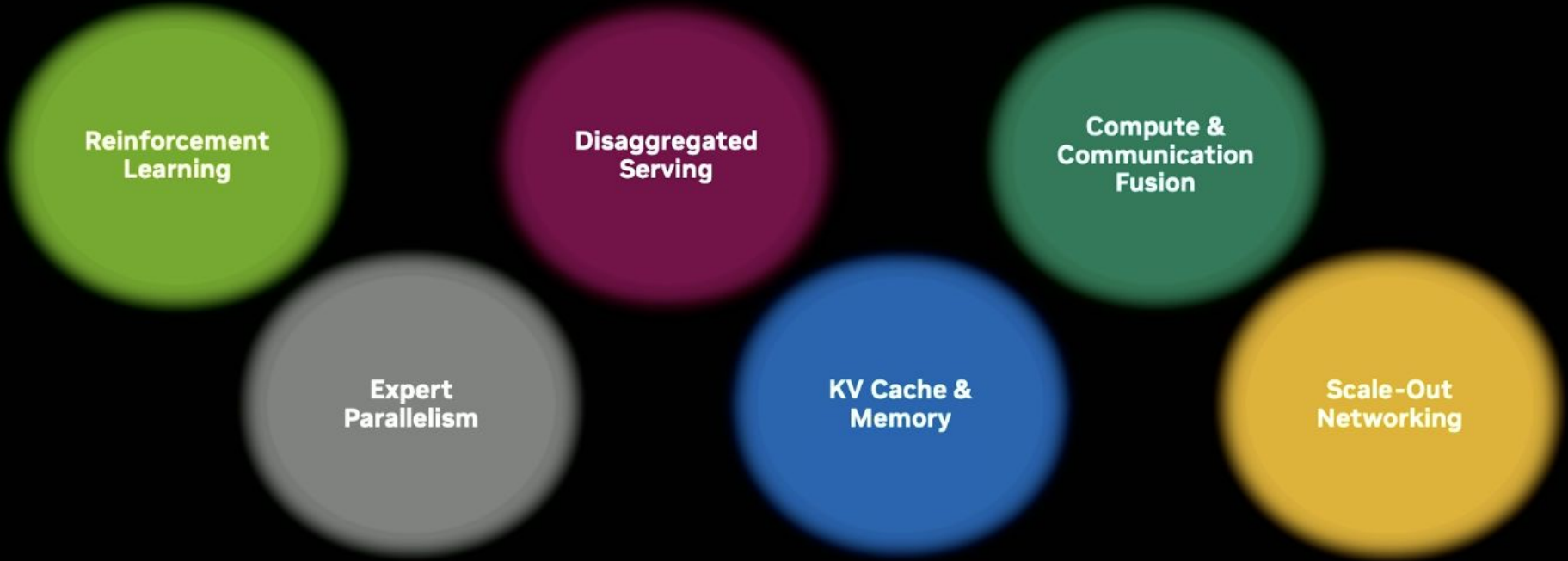
Break Communication Barriers: Scale AI and HPC with NCCL, NIXL, and NVSHMEM



Learn how to eliminate communication bottlenecks with NVIDIA Communication stack. Over the past year, each library has evolved to address specific bottlenecks: NCCL introduced Symmetric Memory support and Copy Engine collectives, enabling zero-SM communication. NVSHMEM enhanced GPU-initiated operations in the Python ecosystem. NIXL emerged as the inference-focused solution, providing async, noncontiguous transfers across memory tiers (GPU HBM, CPU DRAM, local/networked storage) with backends including GDS, UCX, and S3—essential for disaggregated inference architectures. Explore developer optimizations: custom collectives in NCCL, KV cache and storage innovations in NIXL, and expert parallelism patterns. Learn how Spectrum-X plugins extend capabilities across multi-site deployments for scale-across connectivity.

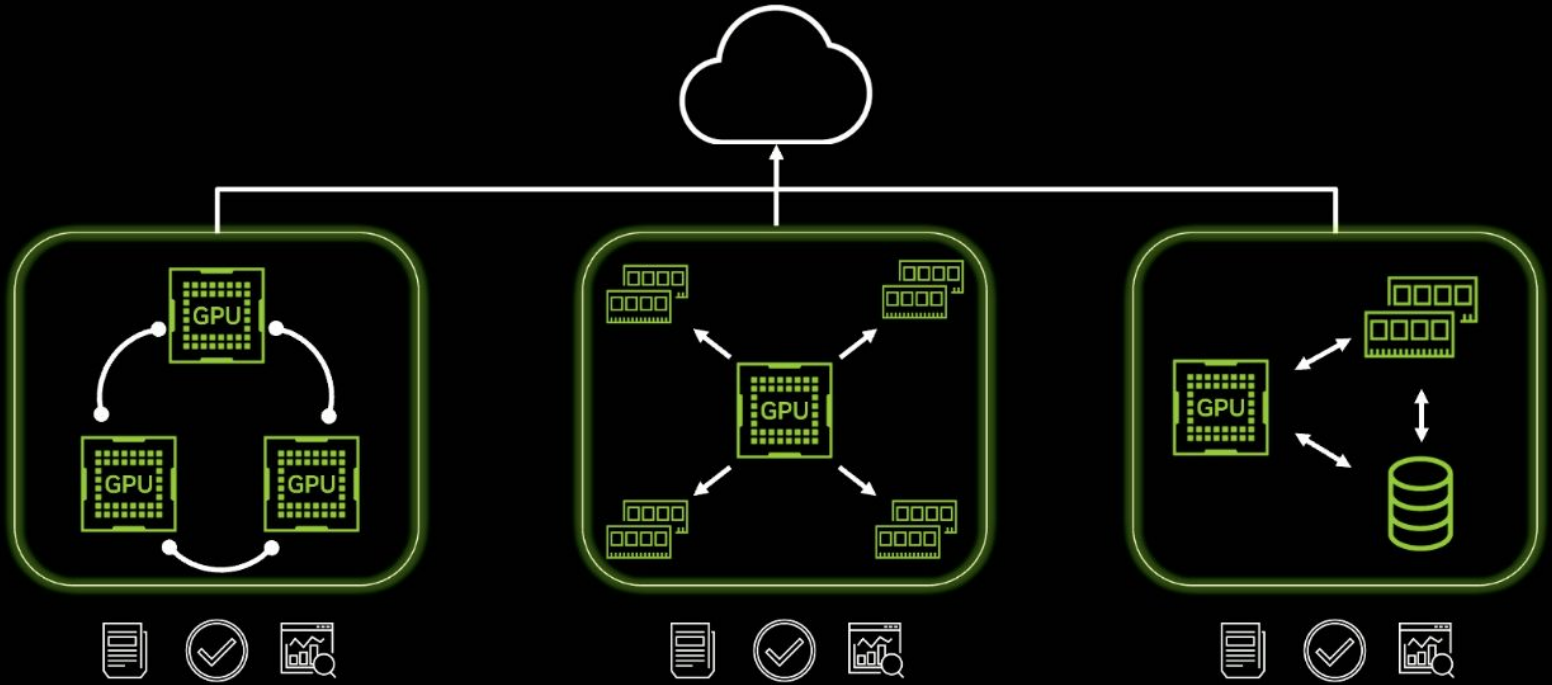
Communication Libraries at Scale

As workloads scale, communication is the critical path



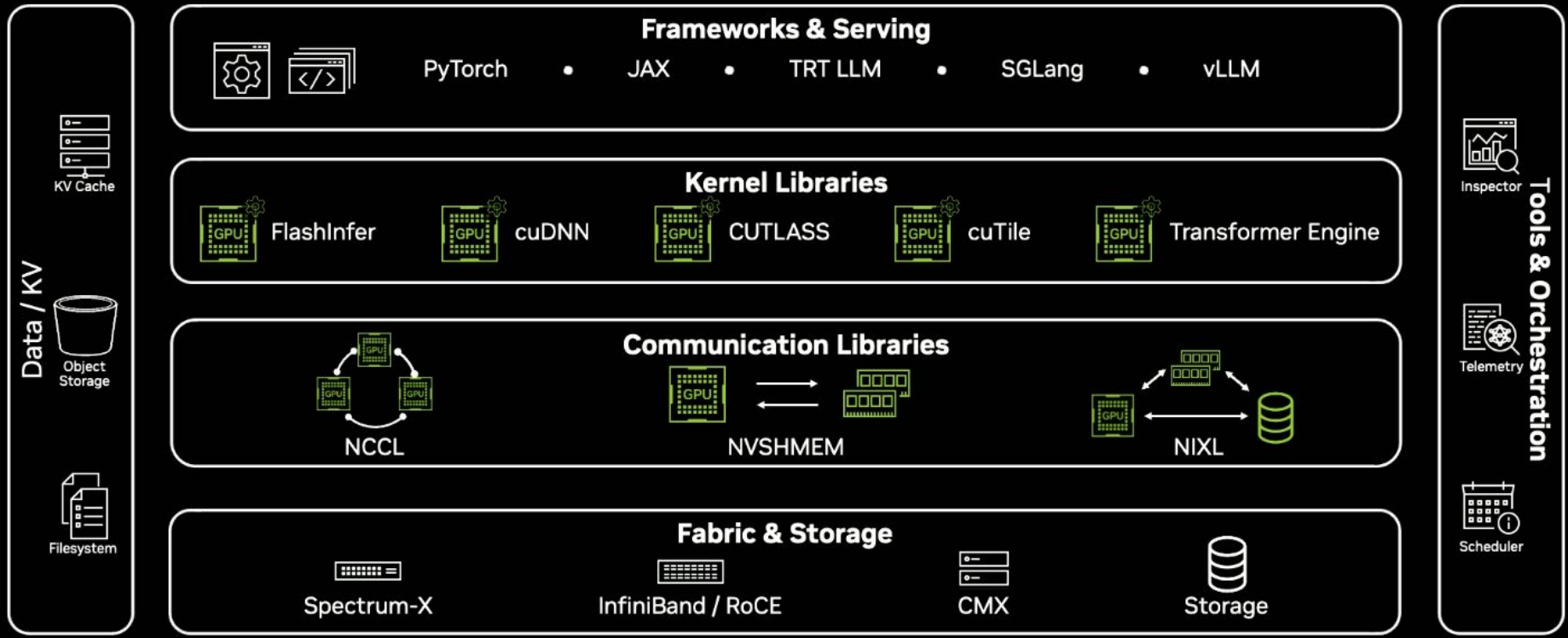
GitHub First: Where to Find Things

Public code, docs, and examples for all three libraries



Ecosystem Comms Stack

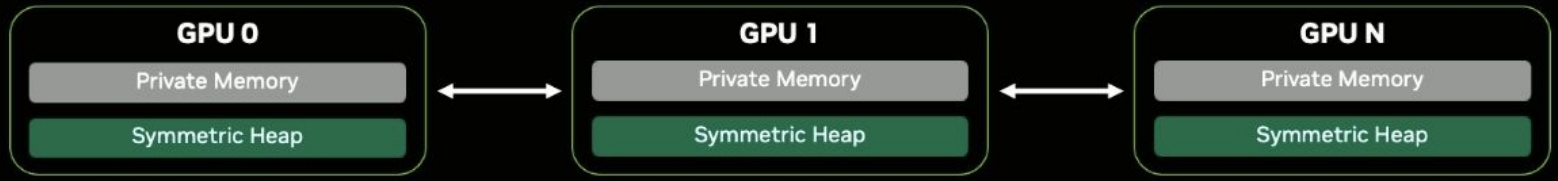
From frameworks to fabric: where NCCL, NIXL, and NVSHMEM fit



NVSHMEM

GPU-initiated communication for HPC

Programming Model



Shared address space – GPUs read/write each other's memory directly via `nvshmem_put()` / `nvshmem_get()`

How Data Moves

[GPU Kernel NVLink/PCIe → Peer GPU (same node)]
[GPU Kernel IBGDA (no CPU) → Remote GPU (another node)]

IBGDA = GPU sends directly over InfiniBand – no CPU proxy needed

Result: Lower latency in real workloads (e.g., GROMACS)

GROMACS: Molecular Dynamics

Simulating molecules for drug discovery & disease research

What It Does

- Simulate proteins & drug molecules
- Track millions of atoms
- Computes forces using Newton's laws
- Generates thermodynamic data

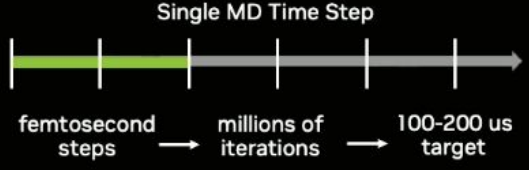
Why It Matters

Drug Discovery – protein binding, virtual screening

Disease Treatment – enzyme & antibody research

Biological Research – protein folding, membranes

The Challenge



- Needs faster steps, not bigger systems
- Multi-GPU coordination is critical
- Communication latency is the bottleneck

45k – 46M atoms | 100-200 us iterations target | Strong scaling = performance goal

GROMACS: Halo Exchange

GPU-initiated communication eliminates CPU bottlenecks

MPI

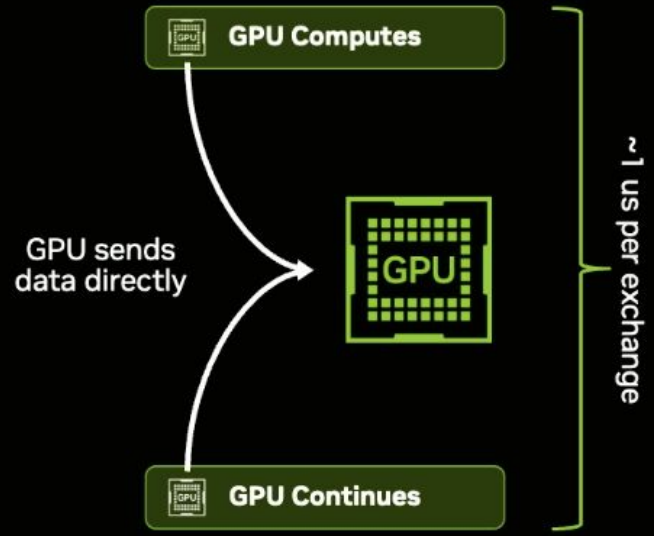
~10 us per exchange

- GPU Computes
- CPU Takes Over
  1. Pack kernel launch
  2. Synchronize
  3. MPI_Isend/Irecv
  4. Unpack kernel launch
  5. Synchronize
- GPU Resumes

- Multiple CPU↔GPU Synchronizations
- Sequential pulse forwarding (Z→ Y→ X)
- Launch overhead on critical path
- Poor compute-communication overlap

→ SOLUTION

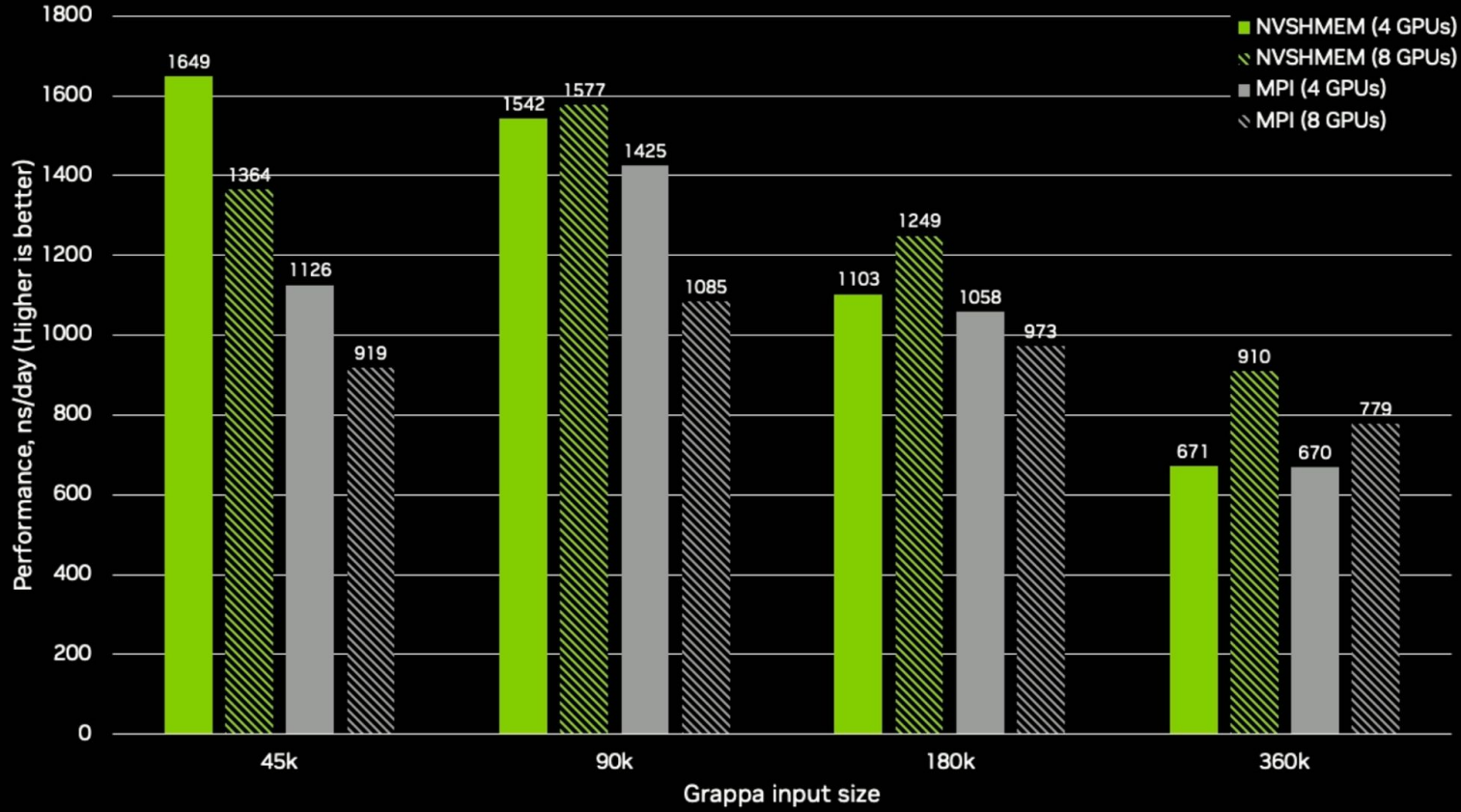
NVSHMEM



- Zero CPU↔GPU Synchronizations
- Fused kernels (pack + communication)
- Full GPU-resident control flow
- Maximum compute-communication overlap

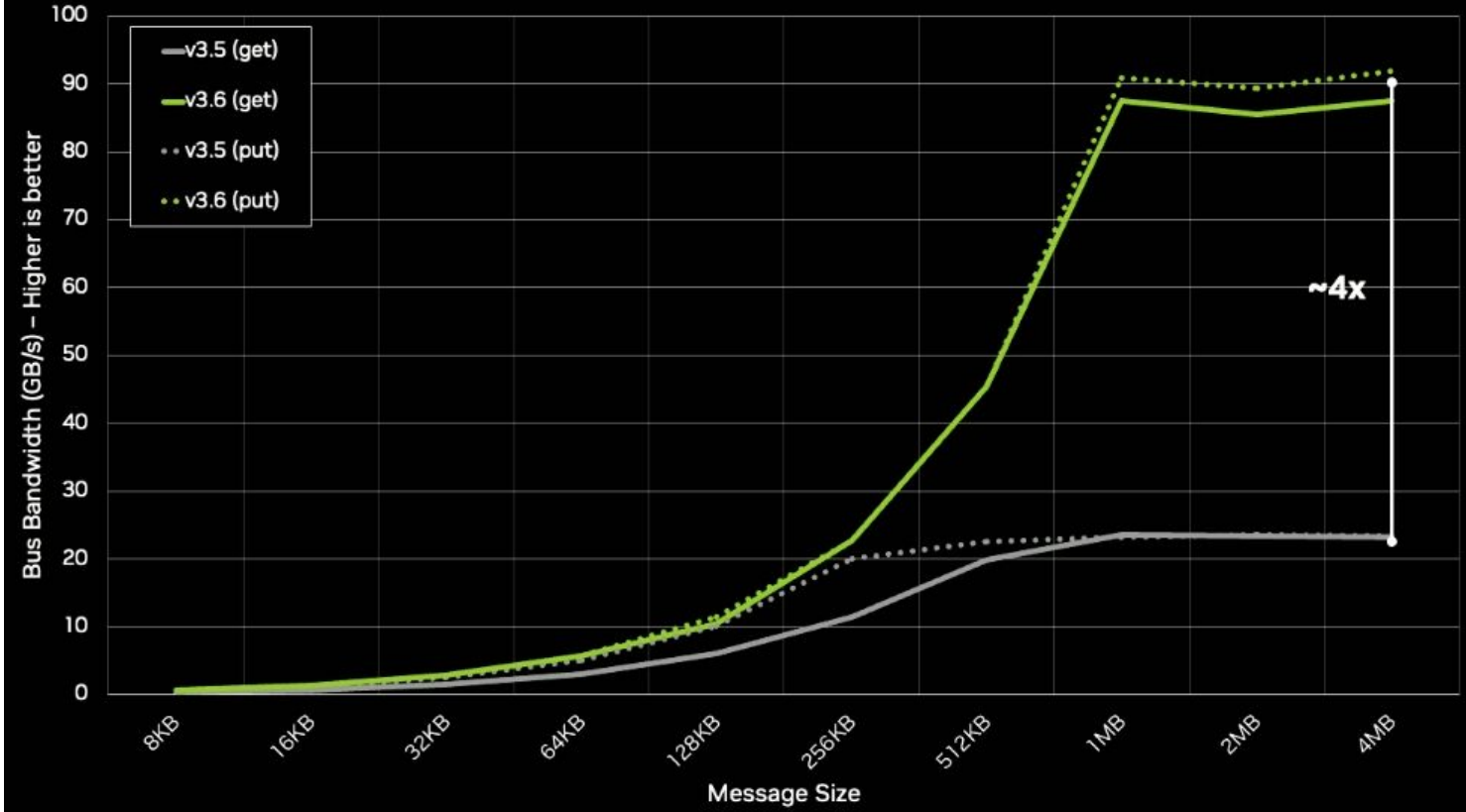
GROMACS: Performance

MPI vs. NVSHMEM on DGX H100



NVSHMEM + AWS EFA

AWS P5: Bidirectional put()/get() Improvements



NCCL Overview

It's much more than a library... It's an ecosystem

Apps & Frameworks

DL Frameworks

PyTorch | JAX/XLA | TRT-LLM | FlashInfer | SGLang | vLLM | ...

Tools

NCCL RAS | Chakra Profiler

NCCL Core

NCCL Core

Bootstrap/Init | Topo Detection | Algorithms (Ring, Tree, Hierarchical, PAT) | Protocols (Simple, LL, LL128) | Cost Model

Symmetric Memory | CE Collectives | Device APIs | Internal Network plugins (Sockets, IB verbs) | Debug System

NCCL Plugins

Network Plugin APIs

SPCX | AWS EFA | Google TCPx

Tuner Plugin APIs

AWS EFA | Google TCPX | Meta RoCE | OCI RoCE

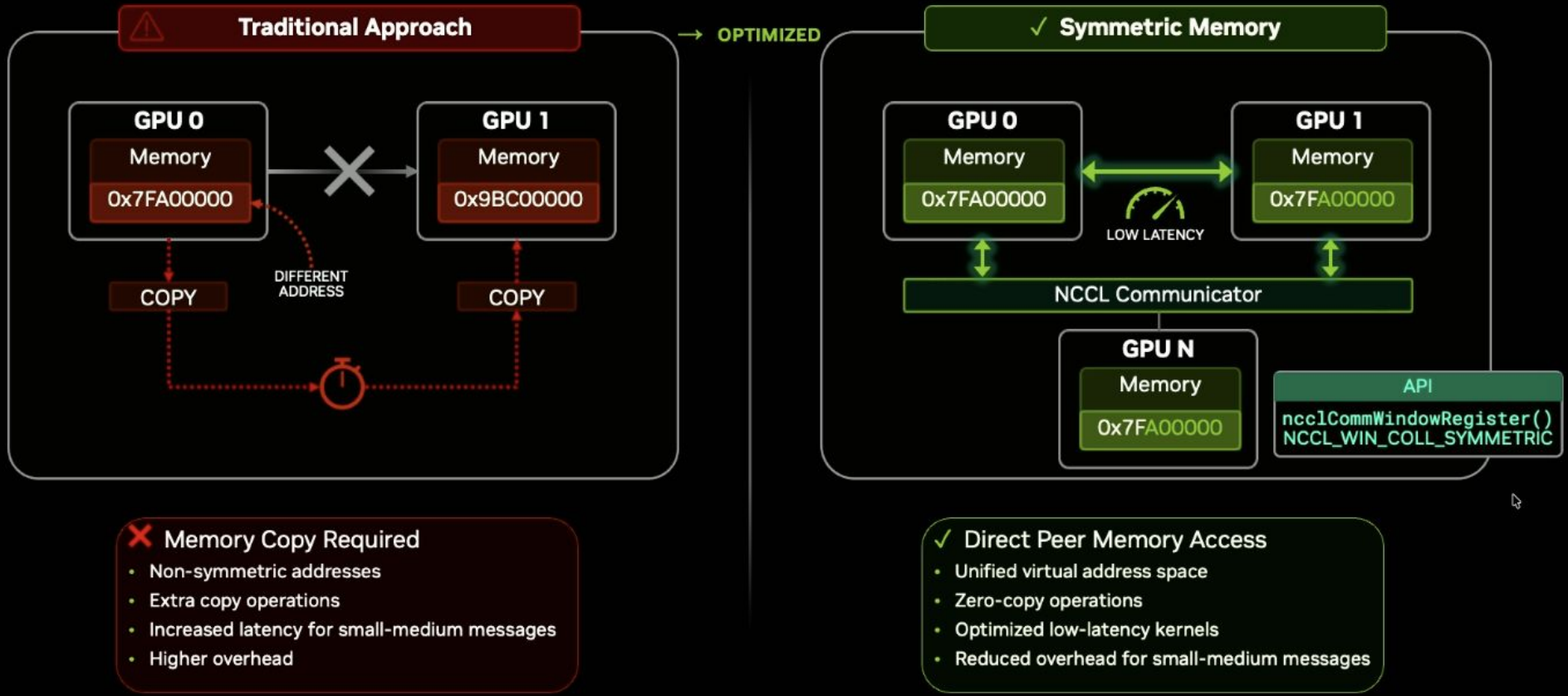
Profiler Plugin APIs

PyTorch Kineto | NCCL Inspector

NV | External

Symmetric Memory

Eliminating memory copy overhead for low-latency collectives



✗ Memory Copy Required

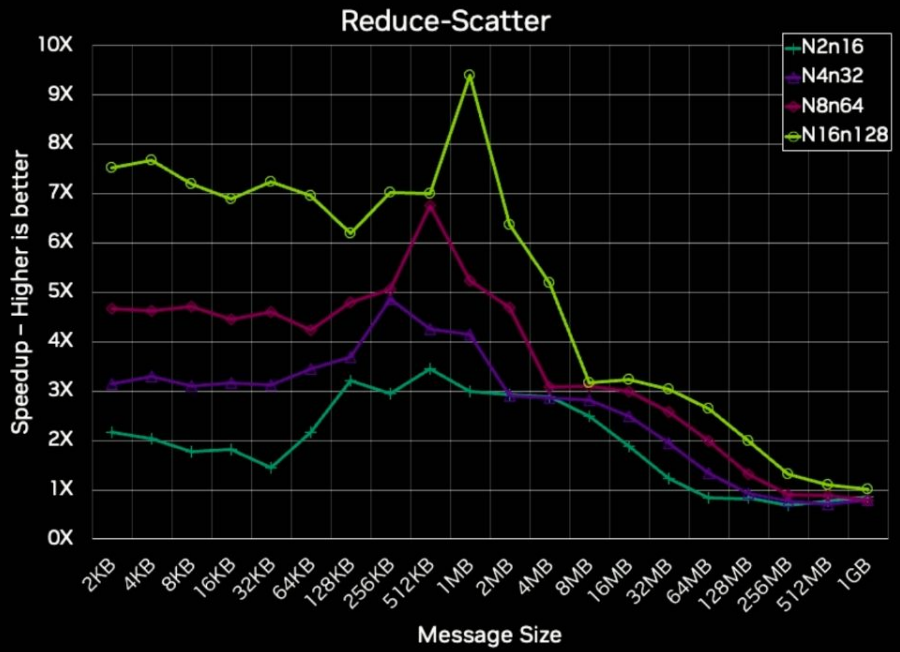
- Non-symmetric addresses
- Extra copy operations
- Increased latency for small-medium messages
- Higher overhead

✓ Direct Peer Memory Access

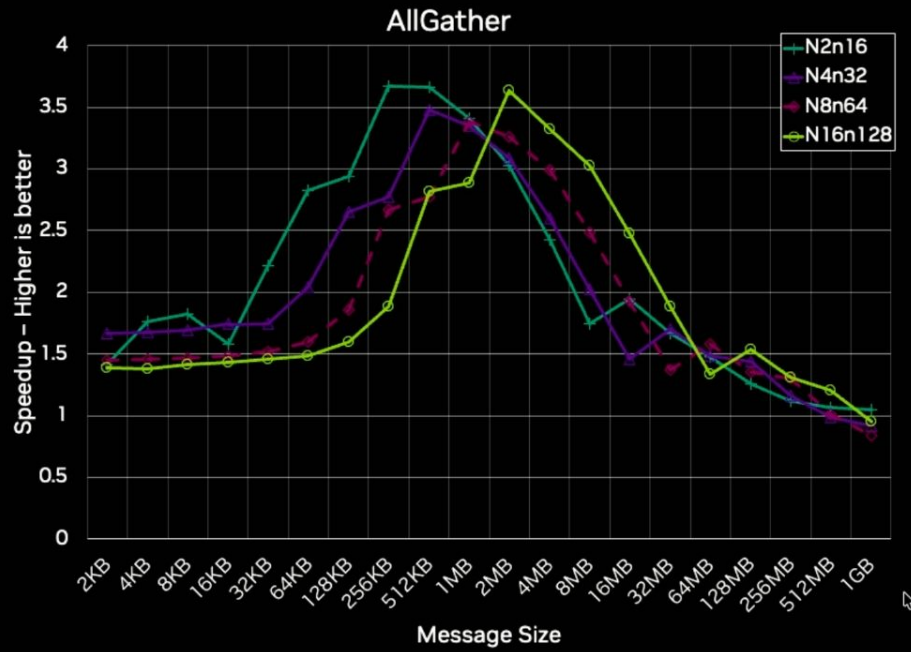
- Unified virtual address space
- Zero-copy operations
- Optimized low-latency kernels
- Reduced overhead for small-medium messages

Symmetry Memory + GPU-Initiated Networking

Speedups: GPU-Initiated Networking vs. Traditional



* NCCL 2.29 | B300 | CX8 | Spectrum-X

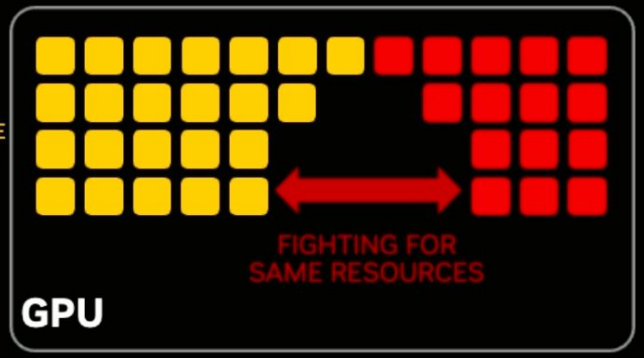


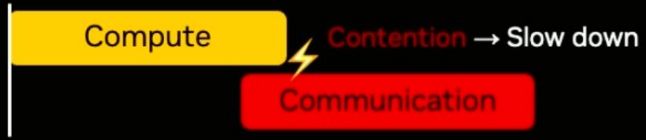
* NCCL 2.29 | B300 | CX8 | Spectrum-X

Copy Engine Collectives

Zero-SM Communication – Eliminate resource contention

⚠ SM-Based Collective (Traditional)

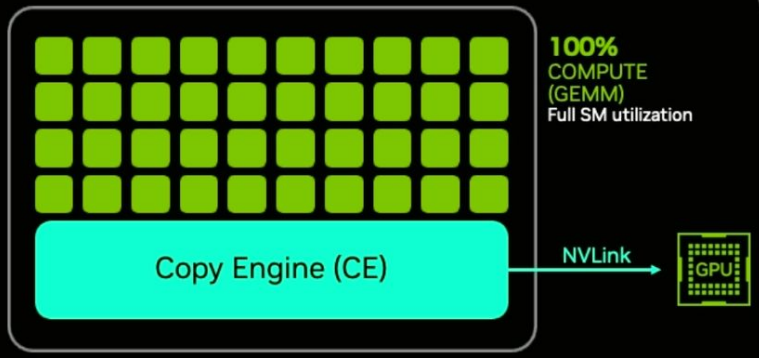


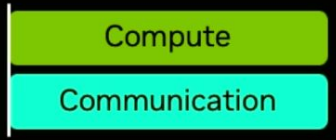


- Compute & communication compete for SMs
- Compute throughput drops with overlap
- More SMs = ⬆ NVL bandwidth, ⬇ compute

→ SOLUTION

✓ CE Collective – Zero-SM Mode





✓ True parallel execution

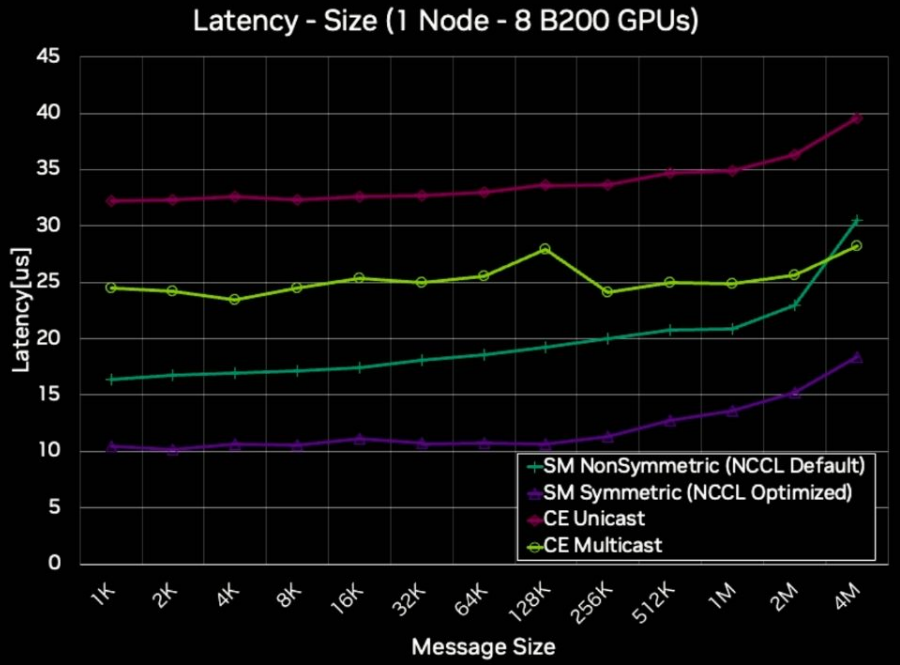
- Dedicated hardware copy
- Engines drive NVLink
- Higher bandwidth

API

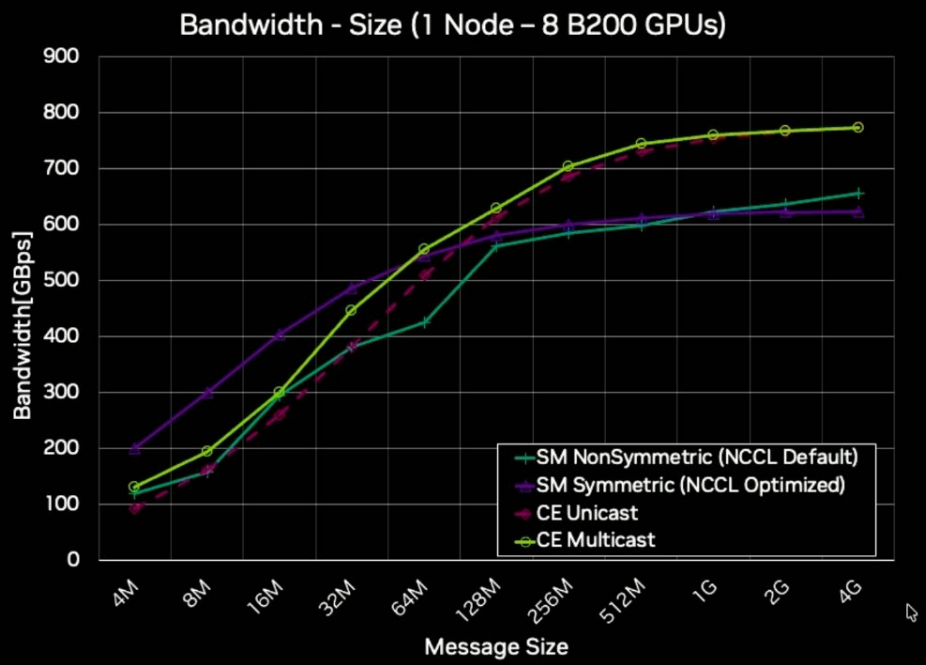
NCCL_CTA_POLICY_ZERO
Applies to AlltoAll, AllGather, Scatter, Gather

Zero-SM Performance

AllGather: Latency & Bandwidth



* NCCL 2.28 | B200 | CX7 | IB



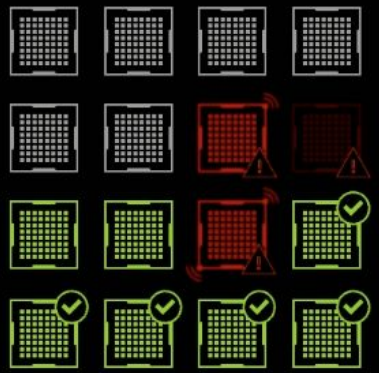
* NCCL 2.28 | B200 | CX7 | IB

Fault Tolerance: Two Complementary Options

Not mutually exclusive – choose based on your workload and failure scenario

Communicator Elasticity

Failure Detection



RAS Detects: NIC Failures, Stragglers, Link Errors

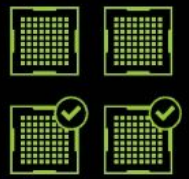
- Timeouts & retries
- Error propagated to framework
- Developer writes recovery logic

Response Flow

Fast Restart + Checkpoint

What NCCL Can Automate

- Load balancing
- Redundant port failover
- RAS: source of information for application decisions



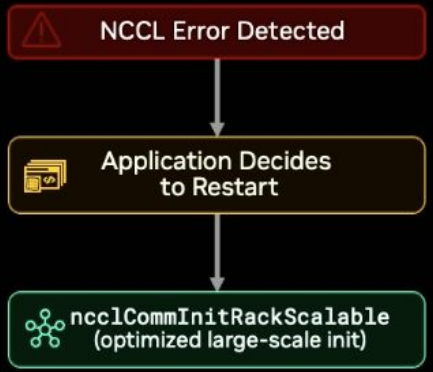
COMING SOON

CUDA Communicator Checkpoint

Model State + NCCL Comm State = Single Checkpoint File → Faster Restart

Seconds, not minutes

Fast Restart Flow



- Some application changes needed to decide when to checkpoint
- Reduce restart overhead significantly
- RAS provides state information to aid decisions

NIXL Overview

Cross-vendor communication framework optimized for high-performance AI inference data transfers

Apps & Frameworks

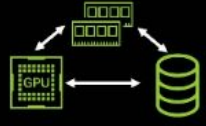
DL Frameworks

- Dynamo KVBM
- TRT-LLM
- LMCache
- SGLang
- vLLM
- Anyscale Ray

Tools

- KVBench
- NIXLBench

NIXL Core



NIXL Core

- Auto Topology Detection
- Cost Model
- Device APIs
- Resiliency
- Telemetry
- Path Optimization
- Multi-threading
- Batching
- Load Balancing
- Elasticity

NIXL Plugins

Network

- UCX
- SPC-X
- RDMA
- AWS libfabric
- Mooncake
- UCCL

Memory Types

- DRAM
- HBM

Storage

- GUSLI
- File
- S3
- S3/RDMA
- CMX
- DOCA Memos
- Azure Blob
- DeepSeek 3FS

Storage Types

- Block
- File
- Object

Metadata

- etcd
- Sockets

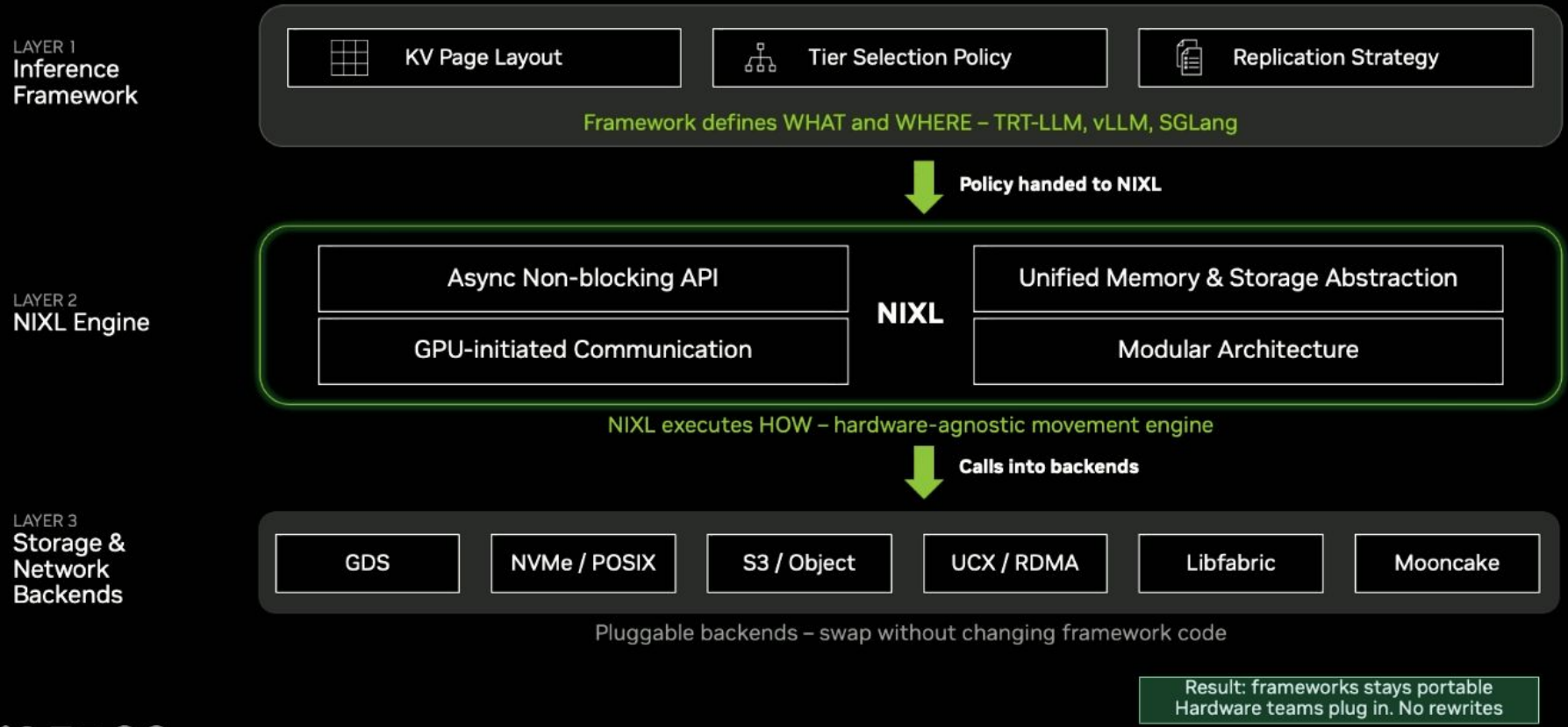
- NV
- External

SPC-X Plugin coming in 2026!

NVIDIA

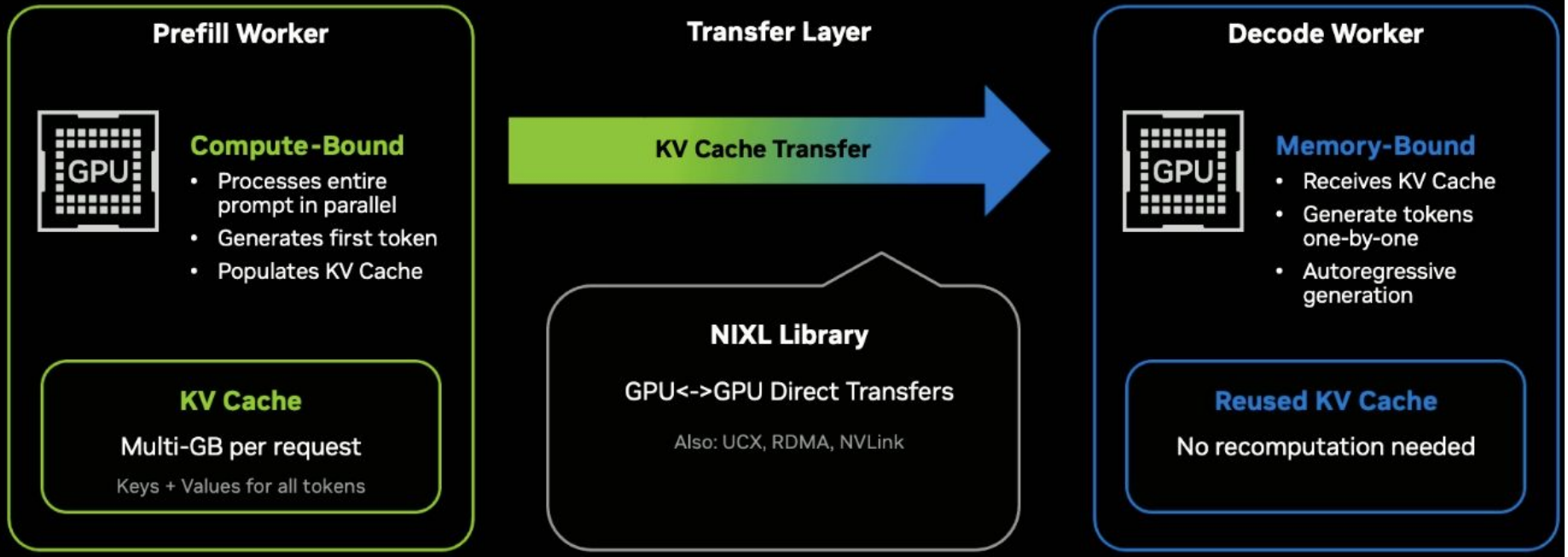
How NIXL Solves It

Decoupling KV layout policy from data movement execution



Disaggregated Serving

Prefill -> Decode | Memory-to-memory Communication



Key Benefits

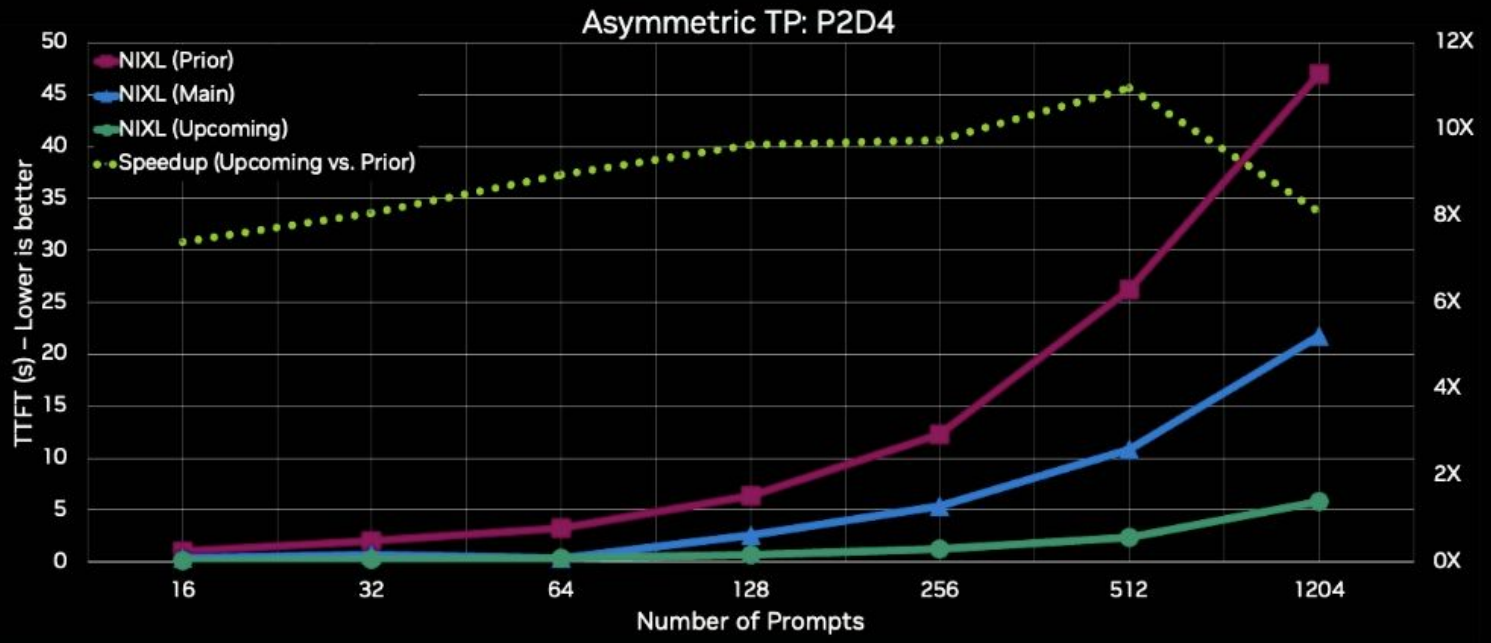
Better GPU Utilization
Each worker optimized for its workload

Improved TCO
Independent scaling of prefill vs decode

Lower TTFT
Prefill doesn't block decode

Prefill-Decode (P/D) Disaggregation Performance

TTFT Improvements coming to SGLang

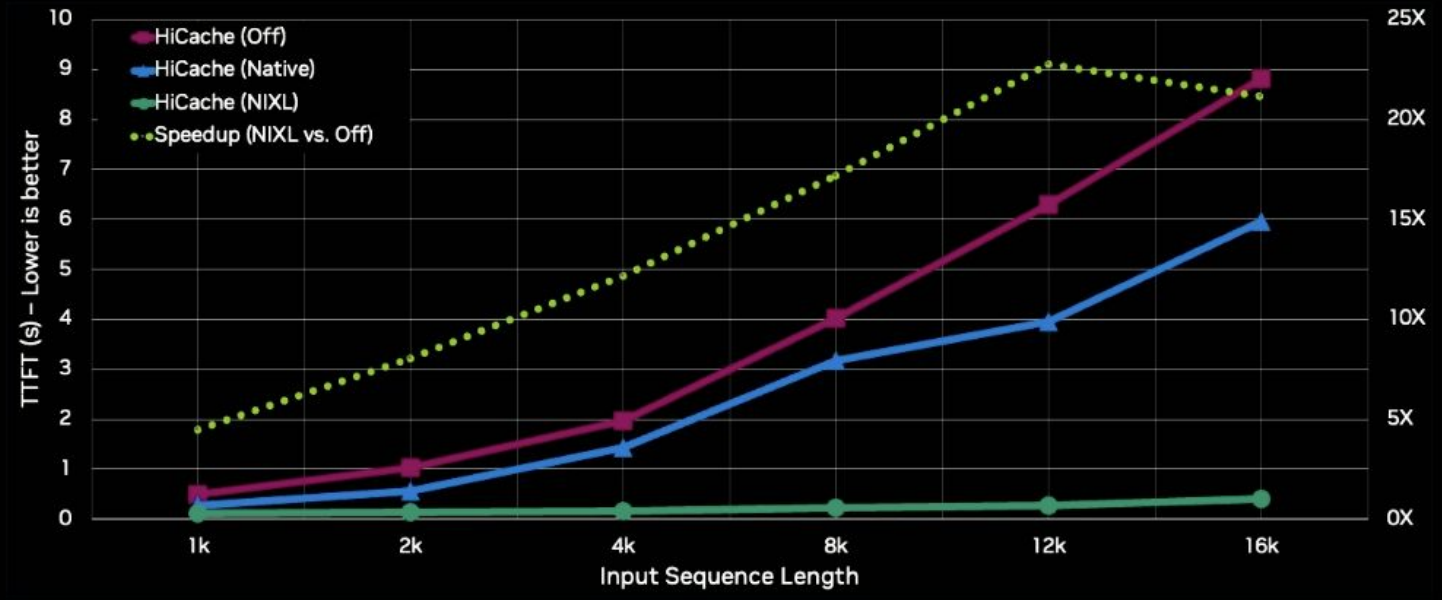


- HW setup
  - GPUs: 8xH100 80GB
  - CPU: Intel(R) Xeon(R) Platinum 8462Y+, 128 cores
  - RAM: 2T total, 256G / GPU
  - Network: 8xConnectX-7 IB 400Gbs

	Setup & Parameters
Common Options	--model-path Llama-3.1-8B-Instruct --tp 2/4
NIXL	UCX_TLS=^cuda_ipc

KV Offloading Performance

TTFT Improvements coming to HiCache

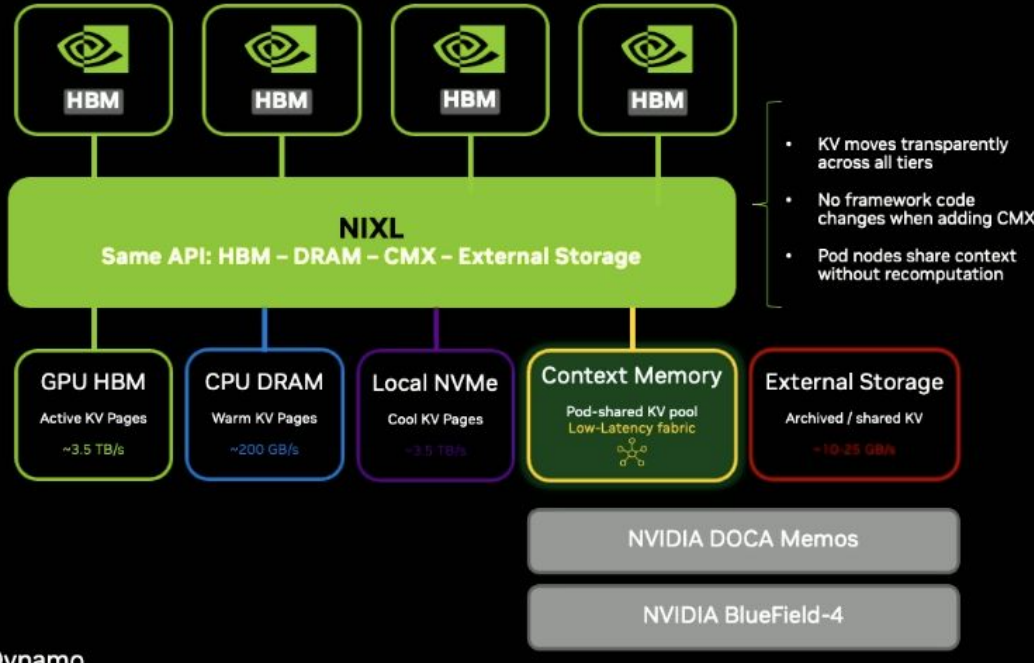


- HW setup
  - GPUs: L4 24GB
  - CPU: Intel(R) Xeon(R) Gold 6438Y+, 128 cores
  - RAM: 512G total
  - Network: 8xConnectX-7 IB 400Gbs

	Setup & Parameters
Common Options	--model-path Qwen3-14B-FP8 --page-size 64 --enable-hierarchical-cache **--hicache-storage-prefetch-policy wait_complete**
NIXL	POSIX uses the URING Plugin: --hicache-storage-backend nixl --hicache-storage-backend-extra-config '{"use_uring": "true"}'

CMX: Pod-Level Context Memory

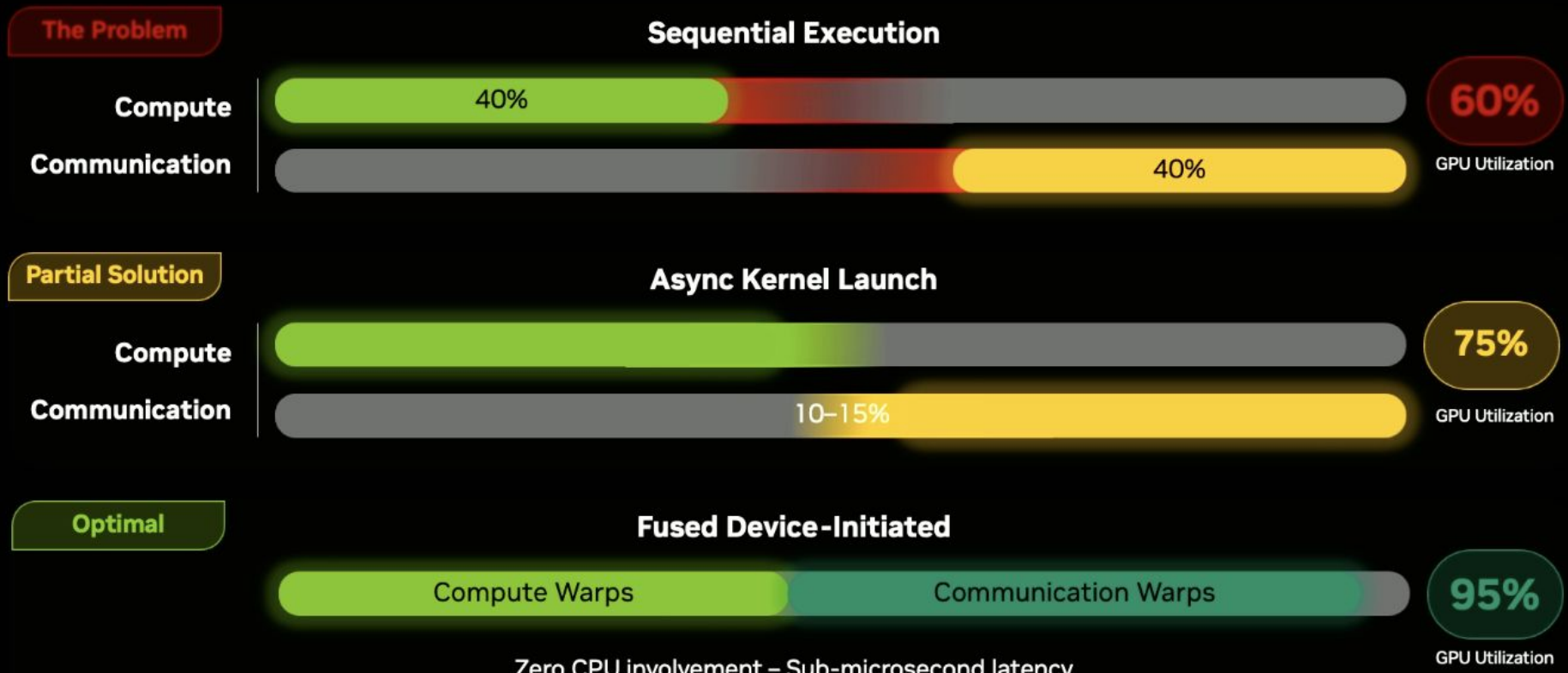
NVIDIA BlueField-4-powered storage infrastructure



- AI-native, pod-level context storage tier
- Shared across GPUs for efficient context reuse
- Complements GPU memory, local, and networked storage
- Accelerated by BlueField-4, Spectrum-X Ethernet, DOCA, NIXL, and Dynamo
- Designed by NVIDIA and built by storage industry leaders

Communication Device APIs

Developing compute-communication kernels to lower latency



NCCL Device API

Custom Collective: AllToAll using GIN – GPU-to-GPU RDMA

AlltoAll

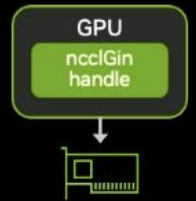
```
// Host: one-time setup
ncclMemAlloc(&buf, N* sizeof (float));
ncclCommwindowRegister(comm, but, N, &win, SYMMETRIC);
ncclDevCommCreate(comm, &regs, &devComm);

// DEVICE: custom collective kernel
__global__ void allToAllKernel(ncclDevComm devComm,
       ncclwindow_t win) {

    // ① Init GIN handle
    ncclin gin { devComm, ginct};
    auto sigVal = gin.readSignal (sigIdx);

    // ② Barrier: all ranks ready
    ginBarrier.sync(...);

    // Step 3 - RDMA put to each peer
    for (int peer = tid; peer < nRanks; peer += nTh) {
        gin.put(team, peer, win, dstoffset, win, srcoffset, size,
                ...,ncclGin_SignalInc{sigIdx}) ;
    }
    // ③ Wait for all inbound data
    gin.waitSignal(ncclCoopCta(), sigidx, sigVal + nRanks);
    gin.flush(ncclCoopCta());
}
```

1 Init Gin

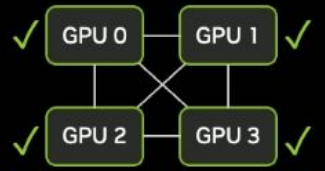
2 Barrier Sync

3 RDMA Put

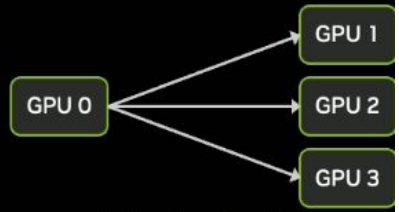
4 Wait Signal



GPU gets direct access to NIC



All ranks confirm they are ready to sen



GPU writes directly to remote memory – no CPU

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ (nRanks)

Completion tracked on-device, no `cudaStreamSync()`

* Pseudocode

NIXL Device API

Prefill + Decode: Both GPUs drive KV movement directly – load, compute, send, persist

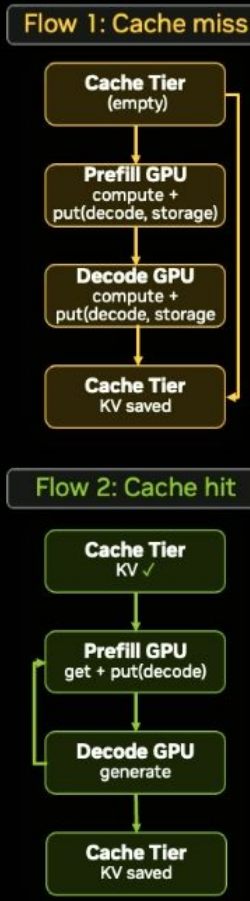
Prefill GPU – prefill_kv_kernel

```
// Prefill GPU: load-or-compute KV, send + persist
__global__ void prefill_kv_kernel(nixlMemViewH mv_local,
           nixlMemViewH mv_store, bool cache_hit, int
tiles, size_t tile_bytes) {

  nixlMemViewElem src  { mv_local,  0, 0 };
  nixlMemViewElem dstD { mv_decode, 0, 0 };
  nixlMemViewElem dstS { mv_store,  0, 0 };

  for (int t = 0; t < tiles; ++t) {
    size_t off = t * tile_bytes;
    src.offset  = off; dstD.offset = off; dstS.offset = off;

    if (cache_hit) {                                          1
      // ① CACHE HIT: KV tile already in mv_local
      // (loaded earlier via support devices)
    } else {
      // ② CACHE MISS: compute KV into mv_local
      compute_kv_tile((half*)nixlGetPtr(mv_local, 0), t);     2
      __syncthreads();
    }

    if (warpId == 0) {
      // ③ Send tile to decode GPU (RDMA)
      nixlPut<>(src, dstD, tile_bytes);                       3

      // ④ Persist tile to storage (GDS / 3FS / S3)
      nixlPut<>(src, dstS, tile_bytes);                       4
    }
  }
}
```

Decode GPU – decode_kv_kernel

```
// Decode GPU: wait for KV, generate, persist new KV
__global__ void decode_kv_kernel(nixlMemViewH mv_kv_in,
           nixlMemViewH mv_kv_out, nixlMemViewH mv_store,
           nixlMemViewH mv_signal, int expected_tiles,
size_t     all_bytes) {

  // ① Wait until producer signals all tiles
  if (threadIdx.x == 0) {
    auto* ctr = (uint64_t*)nixlGetPtr(mv_signal, 0);          1
    while (*ctr < (uint64_t)expected_tiles) {
      __nanosleep(100);
    }
  }
  __syncthreads();

  // ② Decode using received KV and build new KV
  auto* kv_in  = (half*)nixlGetPtr(mv_kv_in,  0);
  auto* kv_out = (half*)nixlGetPtr(mv_kv_out, 0);             2
  generate_tokens_and_kv(kv_in, kv_out);
  __syncthreads();

  // ③ Persist updated KV for future turns
  if (warpId == 0) {
    nixlMemViewElem src { mv_kv_out, 0, 0 };
    nixlMemViewElem dst { mv_store,  0, 0 };                  3
    nixlPut<>(src, dst, all_bytes);
  }
}
```

* Pseudocode



Expert Parallelism

Many specialists, each token goes to the right one

What is a MoE Layer?

Dense Model – one big Feed-Forward Network, every token goes here

MoE replace one big network with many small specialists

MoE Model

Each token -> best matching expert

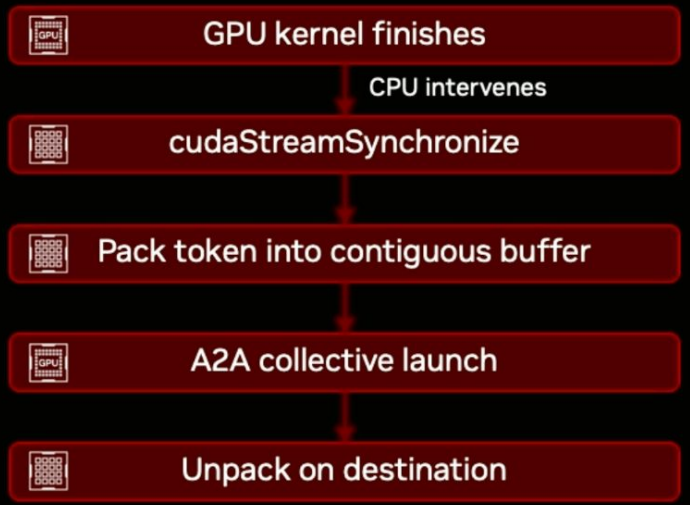
Spreading Experts Across GPUs

Small number of experts activate per token -> huge model, low compute cost

Expert Parallelism

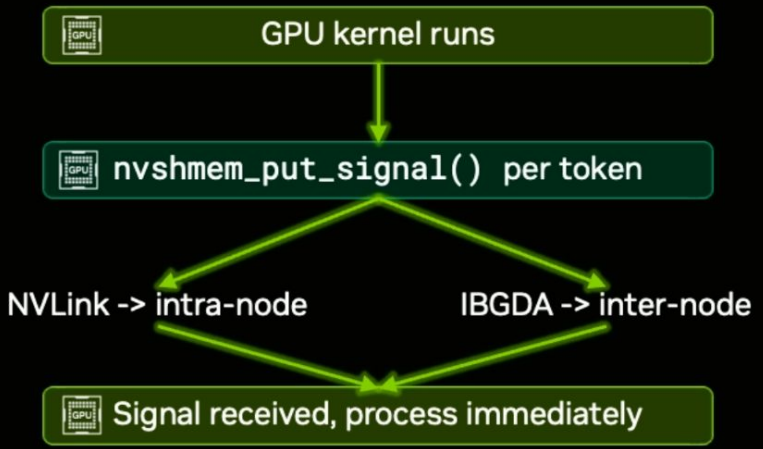
Why did Deepseek use NVSHMEM?

Multiple Host Calls



- Multiple kernel launches
- Pack/unpack overhead
- Topology-blind routing
- ~480us end-to-end

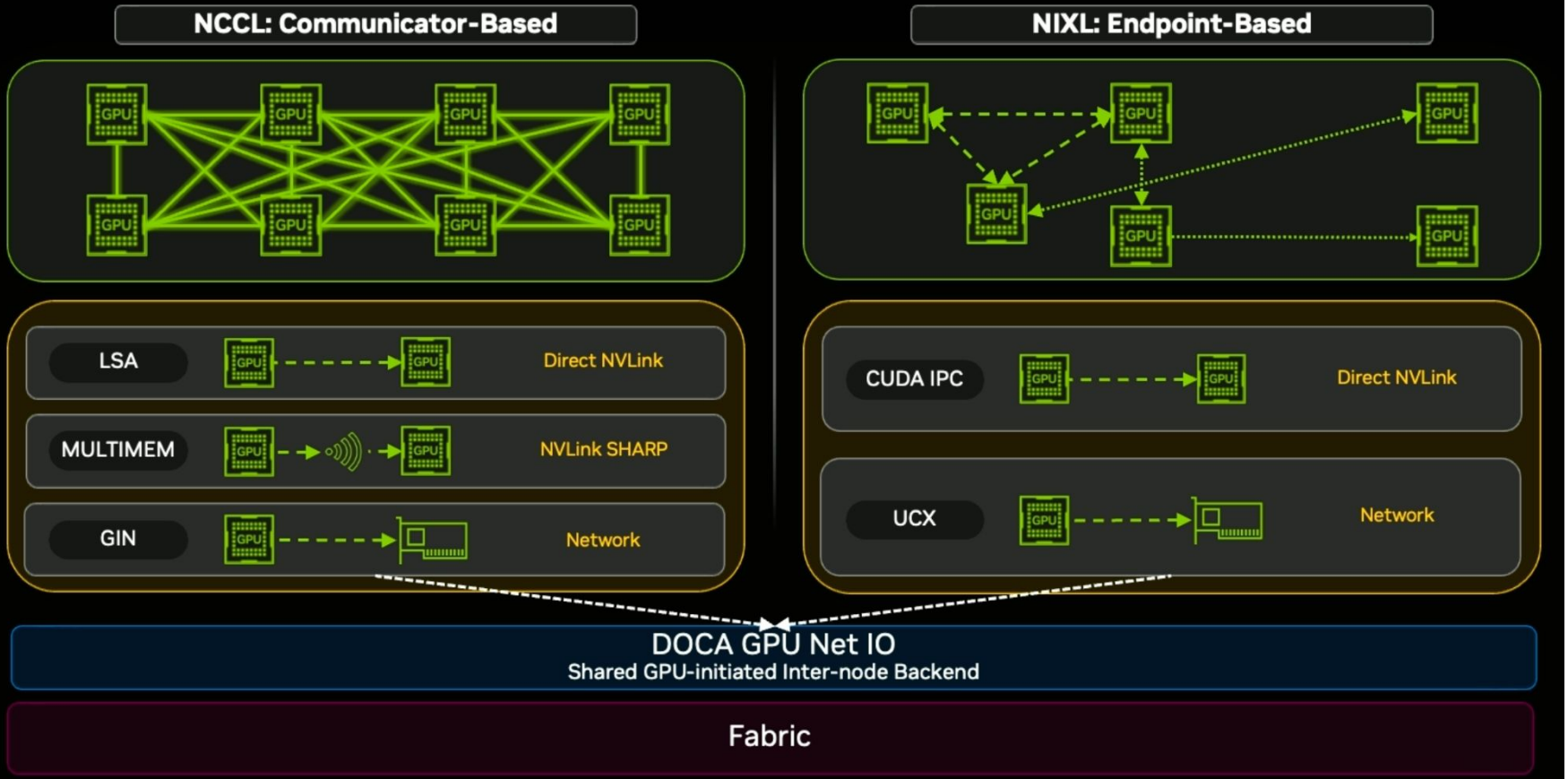
GPU-Initiated Kernel



- Single fused kernel
- Zero pack/unpack
- Topology-aware (NVLink + IB)
- ~50us end-to-end

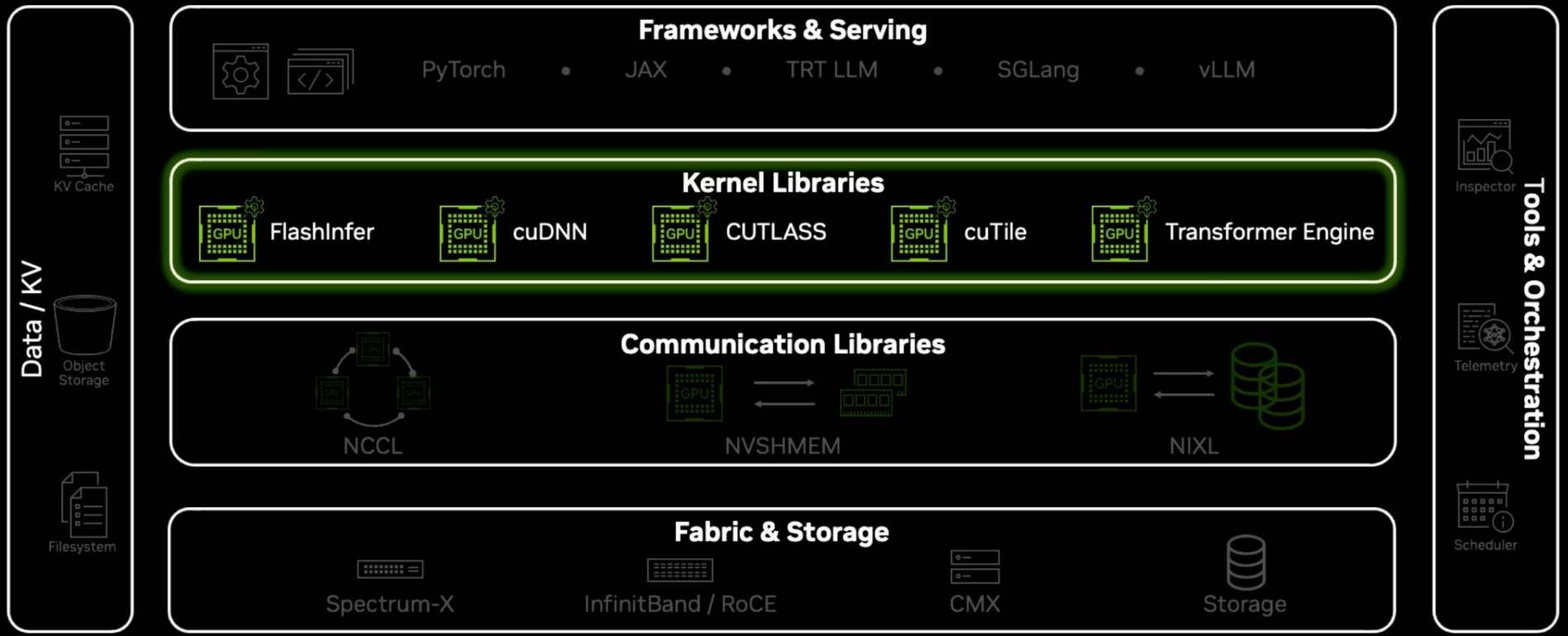
Expert Parallelism

Two new solutions to address a wider problem space



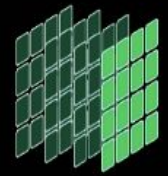
Delivering advanced kernels to developers

Minimizing new dependencies



cuTile + Communication

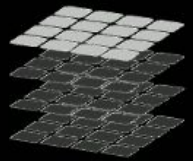
```python
import nvshmem.core.device.tile as nvshmem_tile
from nvshmem.core import tile_callback_helper
@ct.kernel(lto=True, link_lto=True)
def fft_kernel_y(input_data: ct.Array, output_data: ct.Array):
    x, z = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (x, 0, z, 0), shape=(1, FFT_SIZE, FFTS_PER_BLOCK, 2))
    fft_result = fft_y(data)
    ct.store(output_data, (x, 0, z, 0), fft_result)

@ct.kernel(lto=True, link_lto=True, host_callback=tile_callback_helper)
def fft_kernel_z(input_data: ct.Array, output_data: ct.Array):

    y, x = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (x, y, 0, 0), shape=(1, FFTS_PER_BLOCK, FFT_SIZE, 2))
    fft_result = fft_z(data)
    my_pe = nvshmem_tile.my_pe()

    dist_FFT_SIZE = FFT_SIZE // num_pes
    tile_size = FFTS_PER_BLOCK * FFT_SIZE * 2
    local_x = x
    local_y = y * FFTS_PER_BLOCK
    dst_pe =  local_y // dist_FFT_SIZE
    dst_y =   local_y %  dist_FFT_SIZE
    dst_x =   my_pe *  dist_FFT_SIZE + local_x

    dest = ct.array_view(output_data, (dst_x, dst_y, 0, 0), shape=(1, FFTS_PER_BLOCK, FFT_SIZE, 2))
    nvshmem_tile.put(dest, fft_result, dst_pe)

@ct.kernel(lto=True, link_lto=True)
def fft_kernel_x(input_data: ct.Array, output_data: ct.Array):
    y, z = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (0, y, z, 0), shape=(FFT_SIZE, 1, FFTS_PER_BLOCK, 2))
    fft_result = fft_x(data)
    ct.store(output_data, (0, y, z, 0), fft_result)
```



1 XZ FFT in the Y plane

2 XY FFT in the Z plane

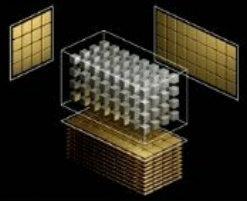
3 Exchange data between nodes using NVSHMEM



4 YZ FFT in the X plane

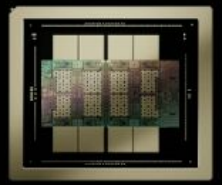
CUTLASS Python + Communication

CUTLASS 4.0



High-performance abstractions for

- Matrix-Matrix Multiplication (GEMM)
- Convolution Operations
- Mixed-Precision Computing
- Tensor Core Programming



Architecture support

Day-0 support for new architectures such as NVIDIA Rubin

Dual-Programming Interface

- C++ Templates
- Python DSLs

```python
import nvshmem.core.cute as nvshmem_cute
import nvshmem.core

@cute.kernel
def stencil_kernel(data: cute.Tensor, tmp_left: cute.Tensor, signals: cute.Tensor):
    n_pes = nvshmem_cute.n_pes()
    my_pe = nvshmem_cute.my_pe()
    left_pe = (my_pe - 1) % n_pes()
    right_pe = (my_pe + 1) % n_pes()

    # COMMUNICATION
    # Put our right edge to the tmp array of the right neighbor
    # extract right face by slicing
    right_face = data[:, -1]
    nvshmem_cute.put_signal_block(tmp_left, right_face, signals[right_pe], 1,
                                  nvshmem_cute.SIGNAL_SET, right_pe)
    nvshmem_cute.signal_wait(signals[my_pe], nvshmem_cute.CMP_EQ, 1)

    cute.arch.sync_threads()

    # Average the left and current face`
    left_val = tmp_left[idx-1]
    current_val = data[idx]
    new_val = (left_val + current_val) // 2
    data[idx] = new_val

# Launcher code
@cute.jit
def stencil_launcher(data: cute.Tensor, tmp_left: cute.Tensor, tmp_right: cute.Tensor,
                     signals: cute.Tensor):
    cute.arch.launch(stencil_kernel, grid=(data.shape[0], data.shape[1]),
                     block=(1, 1, 1), args=(data, tmp_left, tmp_right, signals))
```

Spectrum-X

World's first Ethernet platform purpose-built for AI

Performance

- **Adaptive Routing**

  Fine-grained per-packet routing eliminates flow collisions
- **RoCE Congestion Control**

  95% effective bandwidth at 100k+ GPU scale
- **Network-Aware NCCL Tuning**

  Collective-aware algorithm selection for optimal throughput

95%

Effective bandwidth vs. ~60% standard Ethernet

Resiliency & Observability

- **Automatic Failover**

  Self-healing from link failures – uninterrupted AI training
- **Dynamic Load Balancing**

  Redistributes flows across NICs based on real-time patterns
- **In-Band Telemetry**

  Hardware-accelerated, real-time congestion visibility

1.7x

AI performance improvement for LLM workloads at scale

Enhanced Capabilities

- **SuperNIC Integration**

  Bluefield-3 SuperNIC + switch tight coupling for peak AI
- **Extended NCCL Plugin APIs**

  Communication context, topology injection, transport control
- **Spectrum-XGS Scale-Across**

  Connect data centers across cities and continents as on super-factory

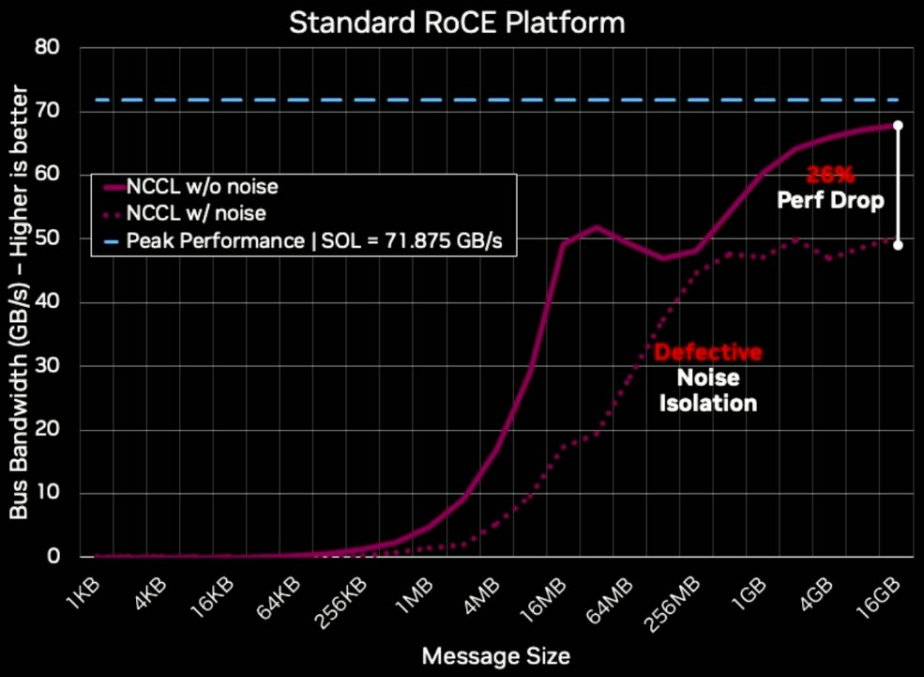
1.9x

NCCL Performance in cross-data center environments

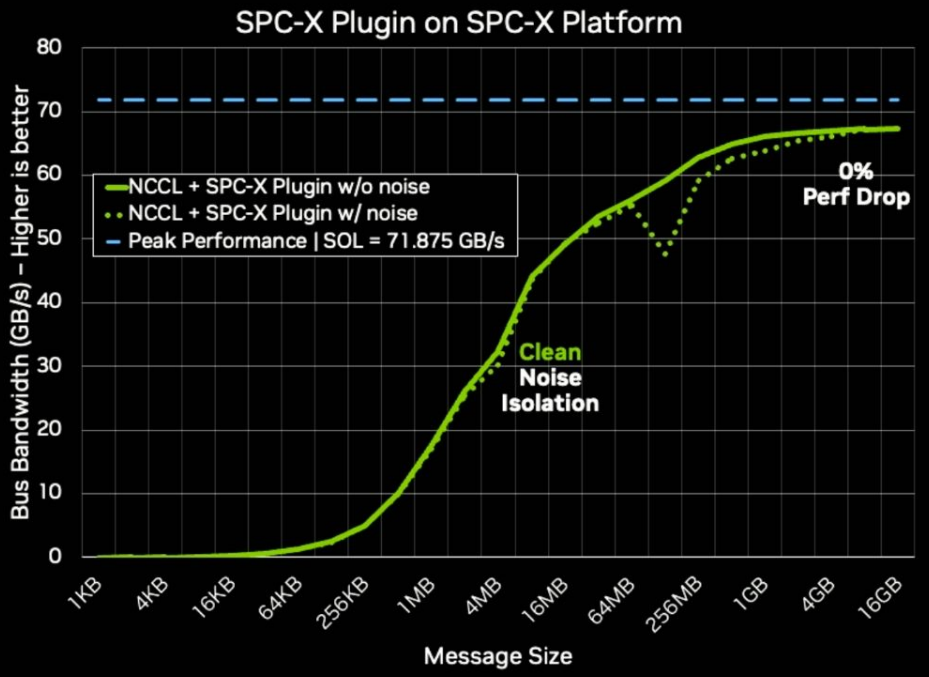
Adopted By: CoreWeave Oracle Tesla xAI

NCCL Spectrum-X Plugin

NCCL Performance: A2A (Noise Isolation)



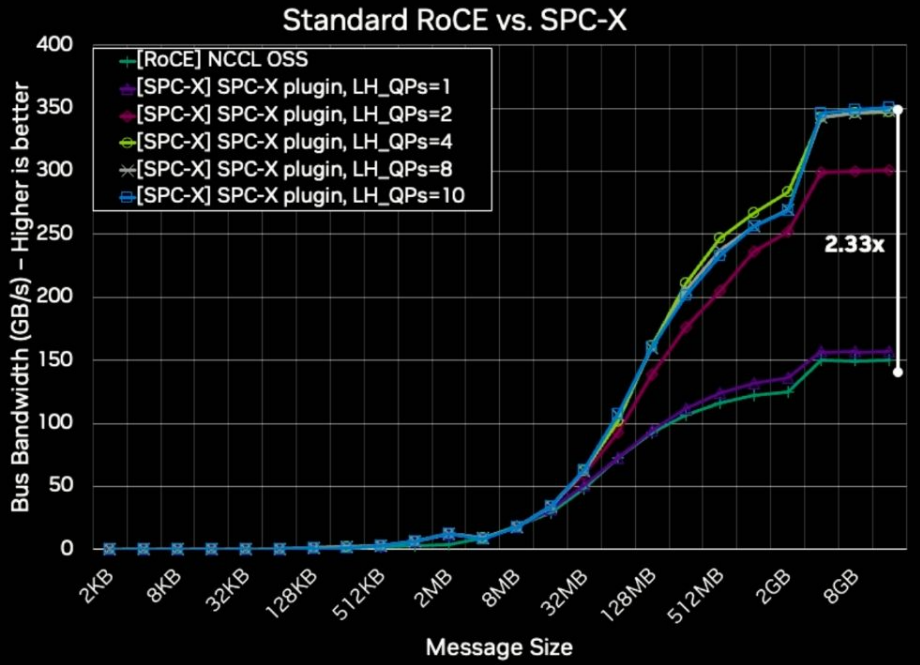
* 3N Alltoall | SPLIT_MASK = 0x0 | User-Buf Reg = On | Out-of-place



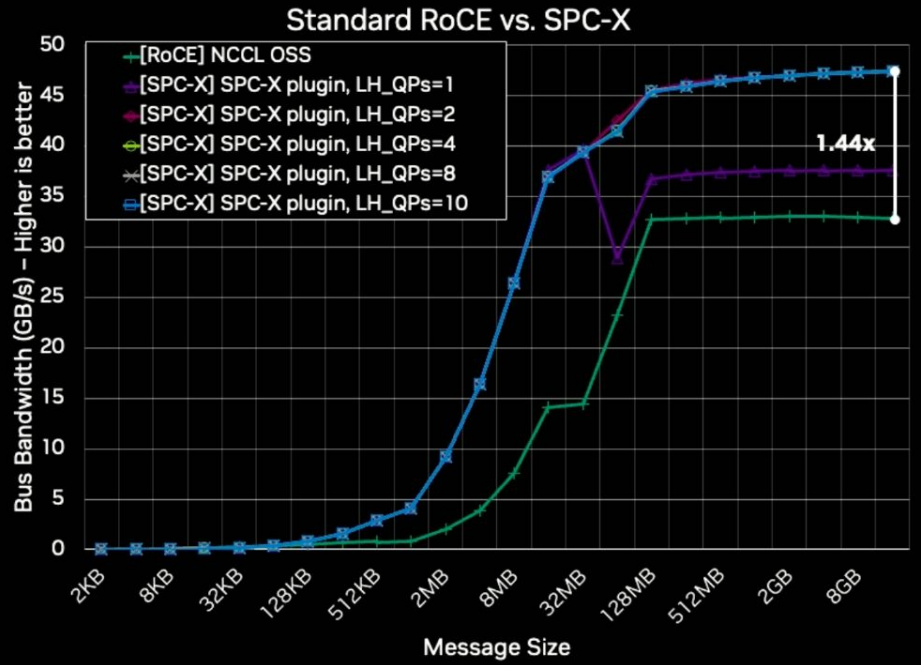
* 3N Alltoall | SPLIT_MASK = 0x0 | User-Buf Reg = On | Out-of-place

NCCL Spectrum-X Plugin

NCCL Performance: Long Haul AllReduce



* AllReduce | SPLIT_MASK = 0x0 | 8N | 64 GPUs | 10KM | Out-of-place



* AllReduce | SPLIT_MASK = 0x7 | 8N | 64 GPUs | 10KM | Out-of-place

Roadmap H1'26

Feedback & Prioritization Welcome

NCCL	NIXL	NVSHMEM
Elastic Buffer	**DFLW Containers Support**	**DSL support**
MPS Support	POSIX per engine I/O queues support	User Buffer Registration Enhancements
NCCL on Windows	**DSL support**	TMA-Accelerated Put/Get
Port Recovery	Dell S3 RDMA Support	**Improved AWS EFA performance**
Extending Device APIs	CMX Enablement	IB Nonblocking RMA Completion
Improved Inspector support	**Load balance and Failover**	Expose RoCE/IBGDA Counters
DSL Support	KV compression service	
TMA Support	RL / Weight transfer API	

Break Communication Barriers: Scale AI and HPC with NCCL, NIXL, and NVSHMEM [S82407]

Accelerate AI Inference Using DOCA for Storage [S81773]

Cutting-Edge Molecular Dynamics on the Latest Multi-Node NVLink Technology [S81542]

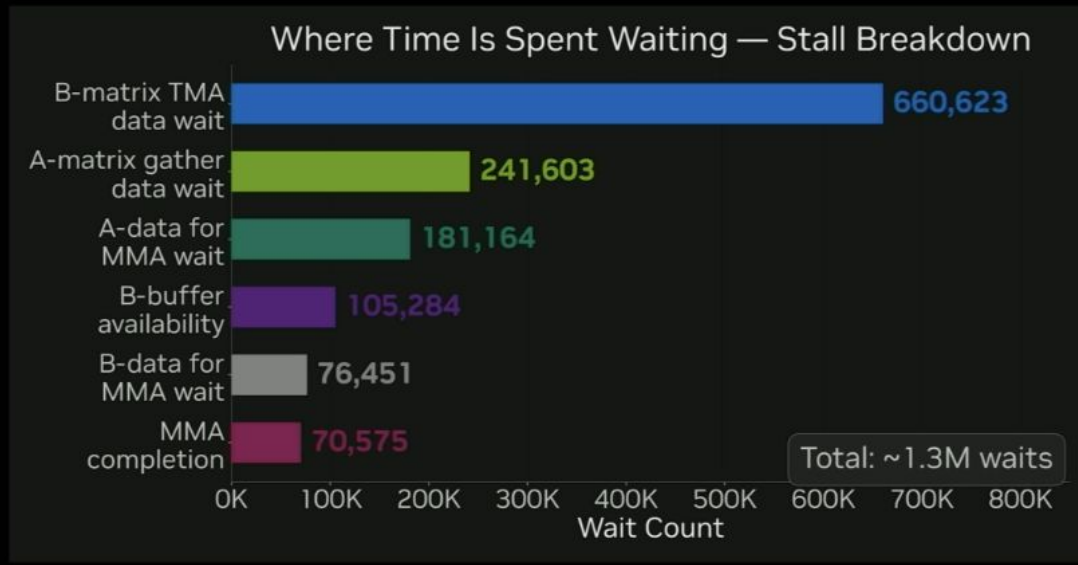
Inter-GPU Communication Techniques and Libraries for HPC and AI [CWES81615]

Subject to Change

Wait Times: Where Time Is Spent Stalling

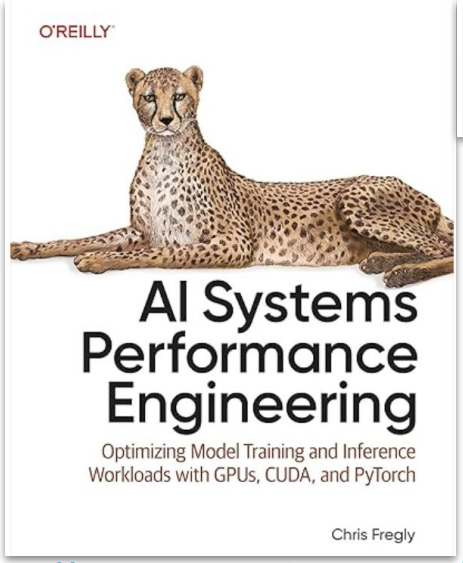
SASS execution counts reveal the true bottlenecks

- **B-matrix TMA is the #1 bottleneck** — triple-buffer can't fully hide latency
- **A-matrix scatter-gather** second largest — scattered loads inherently slower than contiguous
- **~1.3M waits** (32 ns each) — B-matrix starvation caps TC at 54.5%



#1 O’Reilly Book - AI Systems Perf - Updated NVIDIA GTC 2026!!

https://www.amazon.com/Systems-Performance-Engineering-Optimizing-Algorithms/dp/B0F47689K8/



https://www.meetup.com/ai-performance-engineering/

https://github.com/cfregly/ai-performance-engineering/tree/main/code